statistics-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `statistics_ai999x.about()` for the same answer.

Every capability statistics-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import statistics_ai999x

result = statistics_ai999x.solve('mean', x='2, 4, 9')
result.show()
# mean = 5, n = 3, sum = 15
```

By topic

topic	capabilities
Analysis of variance	One-way ANOVA
Categorical data	Fisher's exact test (2 x 2), McNemar's test for paired proportions
Confidence intervals	Confidence interval for a mean (sigma known, z-interval), Confidence interval for a mean from data (t-interval), Confidence interval for a mean from summary statistics (t-interval), Confidence interval for a proportion (Wald), Wilson score interval for a proportion, Confidence interval for the difference of two means, Confidence interval for a variance and standard deviation
Confidence intervals and tests	Critical value of z, Critical value of Student's t, Critical value of chi-square, Critical value of F
Descriptive statistics	Arithmetic mean, Median, Mode, Range, Population variance, Sample variance, Population standard deviation, Sample standard deviation, Z-score of a value, Weighted mean, Five-number summary and interquartile range, Percentile of a data set, Coefficient of variation, Geometric mean, Harmonic mean, Mean absolute deviation, Median absolute deviation, Standard error of the mean, Mean and standard deviation of a frequency table, Trimmed mean, Outliers by the 1.5 IQR rule, Skewness and kurtosis
Effect sizes	Cohen's d and Hedges' g
Experimental design	Randomized complete block design (two-way ANOVA without replication), Cronbach's alpha (internal consistency), 2^2 factorial design: main effects and interaction, 2^3 factorial design: main effects and interactions
Hypothesis tests	One-sample z-test for a mean (sigma known), One-sample t-test, Two-sample t-test (pooled variances), Welch's two-sample t-test (unequal variances), Paired t-test, Two-sample t-test from summary statistics, One-proportion z-test, Two-proportion z-test, Chi-square goodness-of-fit test, Chi-square test of independence, Chi-square test for a variance, F-test for equality of two variances, Levene's test for equal variances, Bartlett's test for equal variances

topic	capabilities
Nonparametric tests	Mann-Whitney U test (large-sample), Mann-Whitney U test (exact), Wilcoxon signed-rank test (large-sample), Wilcoxon signed-rank test (exact), Kruskal-Wallis H test, Friedman test, Sign test
Power and sample size	Sample size to estimate a proportion, Sample size to estimate a mean, Power of a one-sample z-test, Power of a two-sample z-test, Sample size per group to compare two means (z-test power)
Probability	Probability of at least one occurrence, Probability rules: complement, addition and multiplication, Joint, marginal and conditional probabilities from a two-way table
Probability distributions	Expected value and variance of a discrete random variable, Binomial probability: exactly k successes, Binomial probability: at most k successes, Binomial probability: at least k successes, Mean and standard deviation of a binomial distribution, Poisson probability: exactly k events, Poisson probability: at most k events, Poisson probability: at least k events, Geometric distribution: first success on trial k, Hypergeometric probability (sampling without replacement), Negative binomial: r-th success on trial n, Normal probability below a value, Normal probability above a value, Normal probability between two values, Inverse normal (normal percentile), Exponential probability below a value, Exponential probability above a value
Quality control	X-bar and R control chart limits, X-bar and S control chart limits, Individuals and moving range (I-MR) chart limits, P chart limits (fraction defective), C chart limits (defects per unit), Process capability (Cp, Cpk) and expected nonconforming
Regression and correlation	Pearson correlation coefficient, Significance test for a correlation coefficient, Confidence interval for a correlation (Fisher z), Spearman rank correlation, Kendall's tau-b rank correlation, Covariance, Simple linear regression (least squares line), Prediction from a least squares line, Multiple linear regression, Durbin-Watson statistic
Reliability and survival	Weibull reliability, hazard rate and MTTF, Exponential reliability (constant failure rate), Reliability of a series system, Reliability of a parallel system, K-out-of-n system reliability, Kaplan-Meier survival estimate, Log-rank test comparing two survival curves
Sampling distributions	Probability that a sample mean is below a value, Probability that a sample mean is above a value

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
mean	Arithmetic mean	x	mean, n, sum
median	Median	x	median, n
mode	Mode	x	modes, frequency, has_mode
range	Range	x	range, minimum, maximum
population_variance	Population variance	x	variance, mean, n, standard_deviation
sample_variance	Sample variance	x	variance, mean, n, standard_deviation
population_standard_deviation	Population standard deviation	x	standard_deviation, mean, n, variance
sample_standard_deviation	Sample standard deviation	x	standard_deviation, mean, n, variance
z_score	z-score of a value	x, mu, sigma	z
weighted_mean	Weighted mean	x, w	weighted_mean, total_weight
five_number_summary	Five-number summary and interquartile range	x, method	minimum, q1, median, q3, maximum, iqr

id	operation	inputs	outputs
percentile	Percentile of a data set	x, p, method	percentile
coefficient_of_variation	Coefficient of variation	x	cv, cv_percent, mean, standard_deviation
probability_at_least_one	Probability of at least one occurrence	p, n	probability, none
geometric_mean	Geometric mean	x	geometric_mean
harmonic_mean	Harmonic mean	x	harmonic_mean
mean_absolute_deviation	Mean absolute deviation	x	mean_absolute_deviation, mean
median_absolute_deviation	Median absolute deviation	x	median_absolute_deviation, median
standard_error_of_mean	Standard error of the mean	x	standard_error, standard_deviation, n
frequency_table_statistics	Mean and standard deviation of a frequency table	x, f	mean, sample_standard_deviation, population_standard_deviation, n
trimmed_mean	Trimmed mean	x, proportion	trimmed_mean, removed_each_end
iqr_outliers	Outliers by the 1.5 IQR rule	x	lower_fence, upper_fence, outliers, q1, q3, iqr
discrete_random_variable	Expected value and variance of a discrete random variable	x, p	expected_value, variance, standard_deviation
skewness_kurtosis	Skewness and kurtosis	x	skewness, excess_kurtosis, skewness_biased, excess_kurtosis_biased
binomial_exactly	Binomial probability: exactly k successes	n, k, p	probability
binomial_at_most	Binomial probability: at most k successes	n, k, p	probability
binomial_at_least	Binomial probability: at least k successes	n, k, p	probability
binomial_mean_sd	Mean and standard deviation of a binomial distribution	n, p	mean, variance, standard_deviation
poisson_exactly	Poisson probability: exactly k events	lam, k	probability
poisson_at_most	Poisson probability: at most k events	lam, k	probability
poisson_at_least	Poisson probability: at least k events	lam, k	probability
geometric_probability	Geometric distribution: first success on trial k	p, k	probability, cumulative
hypergeometric_probability	Hypergeometric probability (sampling without replacement)	N, K, n, k	probability, cumulative

id	operation	inputs	outputs
negative_binomial_probability	Negative binomial: r-th success on trial n	r, p, n	probability
normal_probability_below	Normal probability below a value	x, mu, sigma	probability, z
normal_probability_above	Normal probability above a value	x, mu, sigma	probability, z
normal_probability_between	Normal probability between two values	a, b, mu, sigma	probability, z_a, z_b
inverse_normal	Inverse normal (normal percentile)	p, mu, sigma	x, z
z_critical_value	Critical value of z	alpha, tails	critical_value
t_critical_value	Critical value of Student's t	df, alpha, tails	critical_value
chi_square_critical_value	Critical value of chi-square	df, alpha	critical_value, left_critical_value
f_critical_value	Critical value of F	df1, df2, alpha	critical_value
exponential_probability_below	Exponential probability below a value	x, rate, mean	probability
exponential_probability_above	Exponential probability above a value	x, rate, mean	probability
sample_mean_probability_below	Probability that a sample mean is below a value	x, mu, sigma, n	probability, z, standard_error
sample_mean_probability_above	Probability that a sample mean is above a value	x, mu, sigma, n	probability, z, standard_error
ci_mean_known_sigma	Confidence interval for a mean (sigma known, z-interval)	xbar, sigma, n, confidence	lower, upper, margin_of_error, critical_value
ci_mean_t	Confidence interval for a mean from data (t-interval)	x, confidence	lower, upper, mean, margin_of_error, critical_value, df
ci_mean_t_summary	Confidence interval for a mean from summary statistics (t-interval)	xbar, s, n, confidence	lower, upper, margin_of_error, critical_value, df
ci_proportion	Confidence interval for a proportion (Wald)	x, n, confidence	proportion, lower, upper, margin_of_error
wilson_interval	Wilson score interval for a proportion	x, n, confidence	proportion, lower, upper, center
ci_two_means	Confidence interval for the difference of two means	x, y, confidence, variance	difference, lower, upper, df, standard_error
ci_variance	Confidence interval for a variance and	x, confidence	variance, lower, upper, sd_lower, sd_upper

id	operation	inputs	outputs
	standard deviation		
z_test_mean	One-sample z-test for a mean (sigma known)	xbar, mu0, sigma, n, alternative	z, p_value
t_test_one_sample	One-sample t-test	x, mu0, alternative	t, df, p_value, mean, standard_deviation
t_test_two_sample_pooled	Two-sample t-test (pooled variances)	x, y, alternative	t, df, p_value, difference
welch_t_test	Welch's two-sample t-test (unequal variances)	x, y, alternative	t, df, p_value, difference
paired_t_test	Paired t-test	x, y, alternative	t, df, p_value, mean_difference
t_test_summary_statistics	Two-sample t-test from summary statistics	m1, s1, n1, m2, s2, n2, variance, alternative	t, df, p_value, difference
proportion_z_test	One-proportion z-test	x, n, p0, alternative	z, p_value, proportion
two_proportion_z_test	Two-proportion z-test	x1, n1, x2, n2, alternative	z, p_value, pooled_proportion, difference
chi_square_goodness_of_fit	Chi-square goodness-of-fit test	observed, proportions	chi_square, df, p_value, expected
chi_square_independence	Chi-square test of independence	table	chi_square, df, p_value, expected
chi_square_variance_test	Chi-square test for a variance	x, sigma0, alternative	chi_square, df, p_value, sample_variance
f_test_variances	F-test for equality of two variances	x, y, alternative	f, df1, df2, p_value
pearson_correlation	Pearson correlation coefficient	x, y	r, r_squared, p_value, n
correlation_t_test	Significance test for a correlation coefficient	r, n, alternative	t, df, p_value
correlation_confidence_interval	Confidence interval for a correlation (Fisher z)	x, y, confidence	r, lower, upper
spearman_correlation	Spearman rank correlation	x, y	rho, p_value, n
kendall_tau	Kendall's tau-b rank correlation	x, y	tau, p_value, concordant, discordant
covariance	Covariance	x, y	sample_covariance, population_covariance
simple_linear_regression	Simple linear regression (least squares line)	x, y	slope, intercept, r, r_squared, p_value, slope_se, intercept_se, residual_se
regression_prediction	Prediction from a least squares line	x, y, x0	y_hat, slope, intercept
multiple_linear_regression	Multiple linear regression	X, y	coefficients, r_squared, adj_r_squared, f, f_p_value, residual_se

id	operation	inputs	outputs
durbin_watson	Durbin-Watson statistic	residuals	durbin_watson
one_way_anova	One-way ANOVA	g1, g2, g3, g4, g5	f, p_value, df_between, df_within, ss_between, ss_within, ms_between, ms_within, eta_squared
randomized_block_anova	Randomized complete block design (two-way ANOVA without replication)	data	f_treatment, p_treatment, f_block, p_block, ss_treatment, ss_block, ss_error, df_treatment, df_block, df_error
levene_test	Levene's test for equal variances	g1, g2, g3, g4, g5, center	w, p_value, df1, df2
bartlett_test	Bartlett's test for equal variances	g1, g2, g3, g4, g5	statistic, df, p_value
mann_whitney_u	Mann-Whitney U test (large-sample)	x, y, alternative	u, p_value
mann_whitney_exact	Mann-Whitney U test (exact)	x, y, alternative	u, p_value
wilcoxon_signed_rank	Wilcoxon signed-rank test (large-sample)	x, y, alternative	statistic, p_value, r_plus, n
wilcoxon_signed_rank_exact	Wilcoxon signed-rank test (exact)	x, y, alternative	statistic, p_value, r_plus, n
kruskal_wallis	Kruskal-Wallis H test	g1, g2, g3, g4, g5	h, df, p_value
friedman_test	Friedman test	data	chi_square, df, p_value
sign_test	Sign test	x, y, alternative	positives, n, p_value
fisher_exact_test	Fisher's exact test (2 x 2)	table, alternative	p_value, odds_ratio
mcnemar_test	McNemar's test for paired proportions	table	chi_square, p_value, exact_p_value, discordant
cohens_d	Cohen's d and Hedges' g	x, y	d, hedges_g, pooled_sd
cronbach_alpha	Cronbach's alpha (internal consistency)	data	alpha, items, respondents
factorial_2x2_effects	2^2 factorial design: main effects and interaction	y	grand_mean, A, B, AB
factorial_2x2x2_effects	2^3 factorial design: main effects and interactions	y	grand_mean, A, B, C, AB, AC, BC, ABC
sample_size_proportion	Sample size to estimate a proportion	margin, confidence, p	n, n_exact
sample_size_mean	Sample size to estimate a mean	sigma, margin, confidence	n, n_exact
power_one_sample_z	Power of a one-sample z-test	d, n, alpha, alternative	power

id	operation	inputs	outputs
power_two_sample_z	Power of a two-sample z-test	d, n, alpha, alternative	power
sample_size_two_means	Sample size per group to compare two means (z-test power)	d, alpha, power, alternative	n_per_group, n_exact
xbar_r_chart	X-bar and R control chart limits	data	center_xbar, mean_range, xbar_ucl, xbar_lcl, r_ucl, r_lcl, A2, D3, D4
xbar_s_chart	X-bar and S control chart limits	data	center_xbar, mean_sd, xbar_ucl, xbar_lcl, s_ucl, s_lcl, A3, B3, B4
individuals_moving_range_chart	Individuals and moving range (I-MR) chart limits	x	center, mean_moving_range, x_ucl, x_lcl, mr_ucl, mr_lcl
p_chart	p chart limits (fraction defective)	defectives, n	center, ucl, lcl, sigma
c_chart	c chart limits (defects per unit)	counts	center, ucl, lcl
process_capability	Process capability (Cp, Cpk) and expected nonconforming	mu, sigma, lsl, usl	cp, cpk, cpu, cpl, ppm_out
weibull_reliability	Weibull reliability, hazard rate and MTTF	beta, eta, t	reliability, unreliability, hazard_rate, mttf
exponential_reliability	Exponential reliability (constant failure rate)	t, rate, mtbf	reliability, unreliability, mttf, failure_rate
series_system_reliability	Reliability of a series system	r	reliability
parallel_system_reliability	Reliability of a parallel system	r	reliability
k_out_of_n_reliability	k-out-of-n system reliability	n, k, r	reliability
kaplan_meier	Kaplan-Meier survival estimate	times, events, t	survival, event_times, survival_curve
log_rank_test	Log-rank test comparing two survival curves	times1, events1, times2, events2	chi_square, p_value, observed_1, expected_1
probability_rules	Probability rules: complement, addition and multiplication	p_a, p_b, p_a_and_b, p_a_or_b, relation	p_not_a, p_not_b, p_a_and_b, p_a_or_b, p_neither, p_a_only, p_b_only, p_a_given_b, p_b_given_a
two_way_table_probabilities	Joint, marginal and conditional probabilities from a two-way table	table, row, column	p_row_given_column, p_column_given_row, p_row_and_column, p_row, p_column, p_row_or_column, total, joint, row_marginals, column_marginals, row_given_column, column_given_row

Arithmetic mean, `mean`

The arithmetic mean (average) of a list of numbers.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

input	meaning	condition
`x`	the data values	

output	meaning
`mean`	arithmetic mean
`n`	number of values
`sum`	sum of the values

- **Method:** sum of the values divided by their number (compensated summation)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 4, 9 as dots on a number line (equal values stacked); marked: the returned mean 5 = sum 15 / n 3, the point the dots balance on.

```
statistics_ai999x.solve('mean', x='2, 4, 9')
# -> mean = 5, n = 3, sum = 15   (SOLVED)
```

```
statistics_ai999x.solve('mean', x=[12, 15, 11, 18, 14])
# -> mean = 14, n = 5, sum = 70   (SOLVED)
```

```
statistics_ai999x.solve('mean', x='-3.5, 0, 7.25, 10')
# -> mean = 3.4375, n = 4, sum = 13.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mean of [2, 4, 9].	SOLVED	mean = 5, n = 3, sum = 15
What is the average of [12, 15, 11, 18, 14]?	SOLVED	mean = 14, n = 5, sum = 70
Compute the arithmetic mean of [1.5, 2.5, 3.5, 10].	SOLVED	mean = 4.375, n = 4, sum = 17.5
Find the mean of [1, two, 3].	INVALID_INPUT	x: 'two' is not a number.
Find the mean.	NEEDS_INPUT	mean needs x (the data values). Example: solve('mean', x='2, 4, 9').

Median, `median`

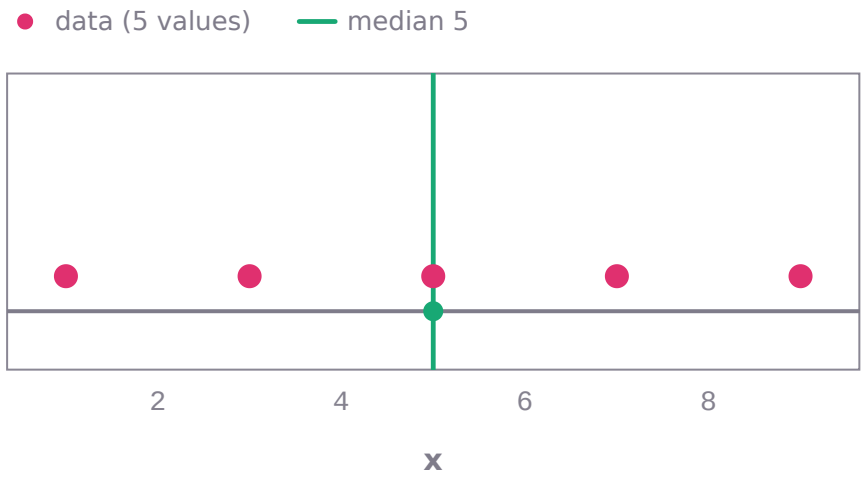
The median (middle value) of a list of numbers; the mean of the two middle values when n is even.

$$\tilde{x} = x_{((n+1)/2)} \text{ or } \tfrac{1}{2}\left(x_{(n/2)} + x_{(n/2+1)}\right)$$

input	meaning	condition
`x`	the data values	

output	meaning
`median`	median
`n`	number of values

- **Method:** sort the values; take the middle one (odd n) or the mean of the two middle ones (even n)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 7, 1, 5, 3, 9 as dots on a number line; marked: the returned median 5 of the n = 5 values, with as many values below as above.

```
statistics_ai999x.solve('median', x='7, 1, 5, 3, 9')
# -> median = 5, n = 5   (SOLVED)
```

```
statistics_ai999x.solve('median', x=[4, 8, 15, 16, 23, 42])
# -> median = 15.5, n = 6   (SOLVED)
```

```
statistics_ai999x.solve('median', x='3.2')
# -> median = 3.2, n = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the median of [7, 1, 5, 3, 9].	SOLVED	median = 5, n = 5
What is the median of [4, 8, 15, 16, 23, 42]?	SOLVED	median = 15.5, n = 6
Find the median.	NEEDS_INPUT	median needs x (the data values). Example: solve('median', x='7, 1, 5, 3, 9').

Mode, `mode`

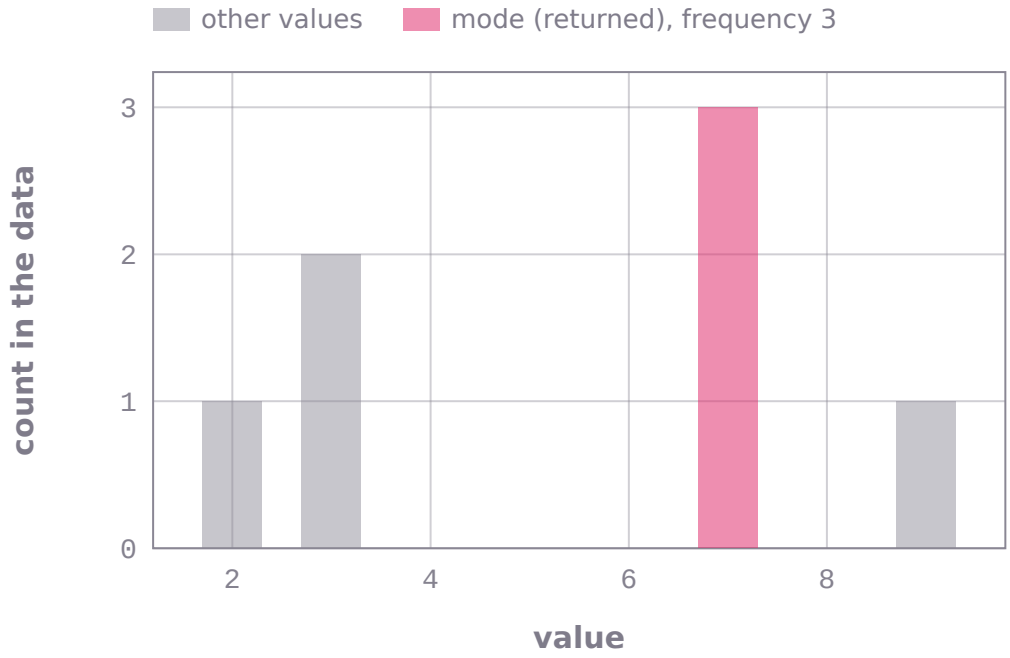
The mode(s) of a list of numbers: every value that occurs most often (no mode when all values occur equally often).

$$\text{mode} = \arg\max_{v} \#\{i : x_i = v\}$$

input	meaning	condition
`x`	the data values	

output	meaning
`modes`	the most frequent value(s), ascending; empty if there is no mode
`frequency`	how often the mode occurs
`has_mode`	yes or no

- **Method:** count each value; report all values with the largest count
- **Assumes:** no mode is reported when every distinct value occurs equally often
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



How often each value occurs in x = 3, 7, 7, 2, 9, 7, 3 (counts tallied from the input); the returned mode 7 with frequency 3 is in the first colour.

```
statistics_ai999x.solve('mode', x='3, 7, 7, 2, 9, 7, 3')
# -> modes = [7.0], frequency = 3, has_mode = yes   (SOLVED)
```

```
statistics_ai999x.solve('mode', x=[1, 2, 2, 3, 3, 4])
# -> modes = [2.0, 3.0], frequency = 2, has_mode = yes   (SOLVED)
```

```
statistics_ai999x.solve('mode', x='1, 2, 3')
# -> modes = [], frequency = 1, has_mode = no   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mode of [3, 7, 7, 2, 9, 7, 3].	SOLVED	modes = [7.0], frequency = 3, has_mode = yes
What is the mode of [1, 2, 2, 3, 3, 4]?	SOLVED	modes = [2.0, 3.0], frequency = 2, has_mode = yes
Find the mode of [5, 6, 7].	SOLVED	modes = [], frequency = 1, has_mode = no

Range, `range`

The range (largest minus smallest value) of a list of numbers.

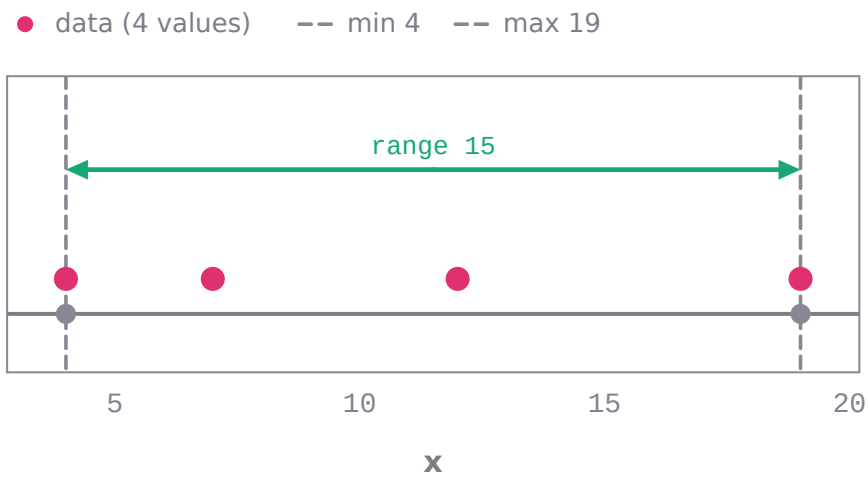
$$R = x_{\max} - x_{\min}$$

input	meaning	condition
`x`	the data values	

output	meaning
`range`	maximum minus minimum
`minimum`	smallest value

output	meaning
`maximum`	largest value

- **Method:** maximum minus minimum
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 12, 4, 19, 7; marked: the returned minimum 4, maximum 19 and the range 15 between them.

```
statistics_ai999x.solve('range', x='12, 4, 19, 7')
# -> range = 15, minimum = 4, maximum = 19   (SOLVED)
```

```
statistics_ai999x.solve('range', x=[3.5, 9.25, 1, 6])
# -> range = 8.25, minimum = 1, maximum = 9.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the range of [12, 4, 19, 7].	SOLVED	range = 15, minimum = 4, maximum = 19
What is the range of the data [3.5, 9.25, 1, 6]?	SOLVED	range = 8.25, minimum = 1, maximum = 9.25

Population variance, `population_variance`

The population variance of a list of numbers (divisor n).

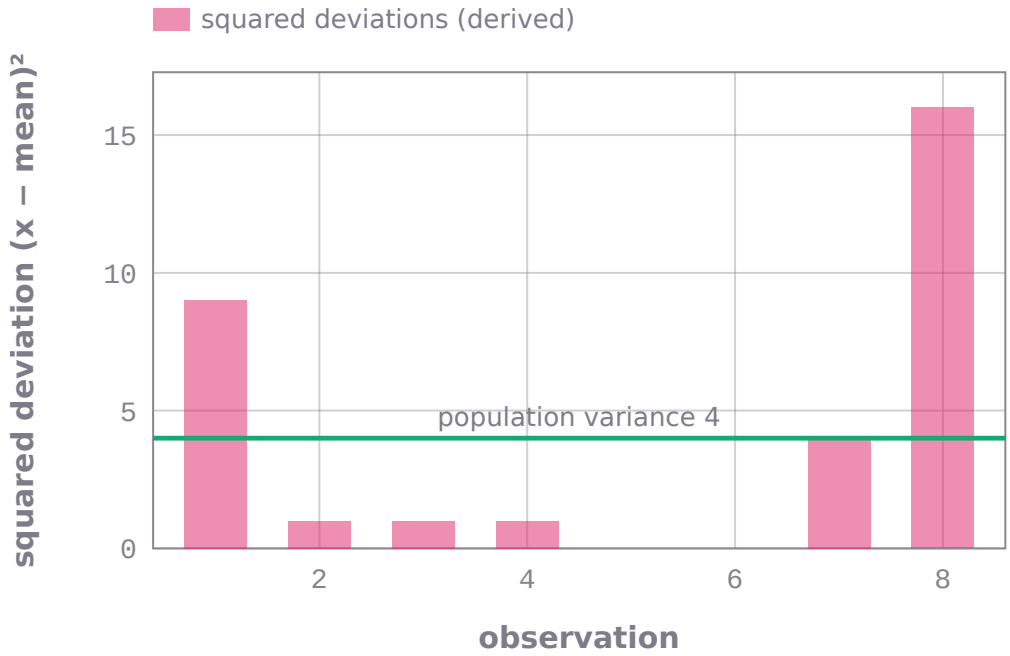
$$\sigma^2 = \frac{1}{n}\sum(x_i - \mu)^2$$

input	meaning	condition
`x`	the data values	

output	meaning
`variance`	population variance
`mean`	arithmetic mean

output	meaning
`n`	number of values
`standard_deviation`	the companion measure

- **Method:** two-pass: mean first, then the sum of squared deviations divided by n
- **Assumes:** the data are the whole population (divisor n)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



For x = 2, 4, 4, 4, 5, 5, 7, 9: each observation's squared distance from the returned mean 5 (derived from the inputs and the mean); the returned population variance 4 is their sum divided by n (standard deviation 2).

```
statistics_ai999x.solve('population_variance', x='2, 4, 4, 4, 5, 5, 7, 9')
# -> mean = 5, n = 8, variance = 4, standard_deviation = 2   (SOLVED)
```

```
statistics_ai999x.solve('population_variance', x=[10, 12, 23, 23, 16, 23, 21, 16])
# -> mean = 18, n = 8, variance = 24, standard_deviation = 4.89898   (SOLVED)
```

```
statistics_ai999x.solve('population_variance', x='1.5, 2.5')
# -> mean = 2, n = 2, variance = 0.25, standard_deviation = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the population variance of [2, 4, 4, 4, 5, 5, 7, 9].	SOLVED	mean = 5, n = 8, variance = 4, standard_deviation = 2
Compute the population variance of [10, 12, 23, 23, 16, 23, 21, 16].	SOLVED	mean = 18, n = 8, variance = 24, standard_deviation = 4.89898
What is the population variance of [600, 470, 170, 430, 300]?	SOLVED	mean = 394, n = 5, variance = 21704, standard_deviation = 147.323
Find the population variance.	NEEDS_INPUT	population_variance needs x (the data values). Example: solve('population_variance', x='2, 4, 4, 4, 5, 5, 7, 9').

Sample variance, `sample_variance`

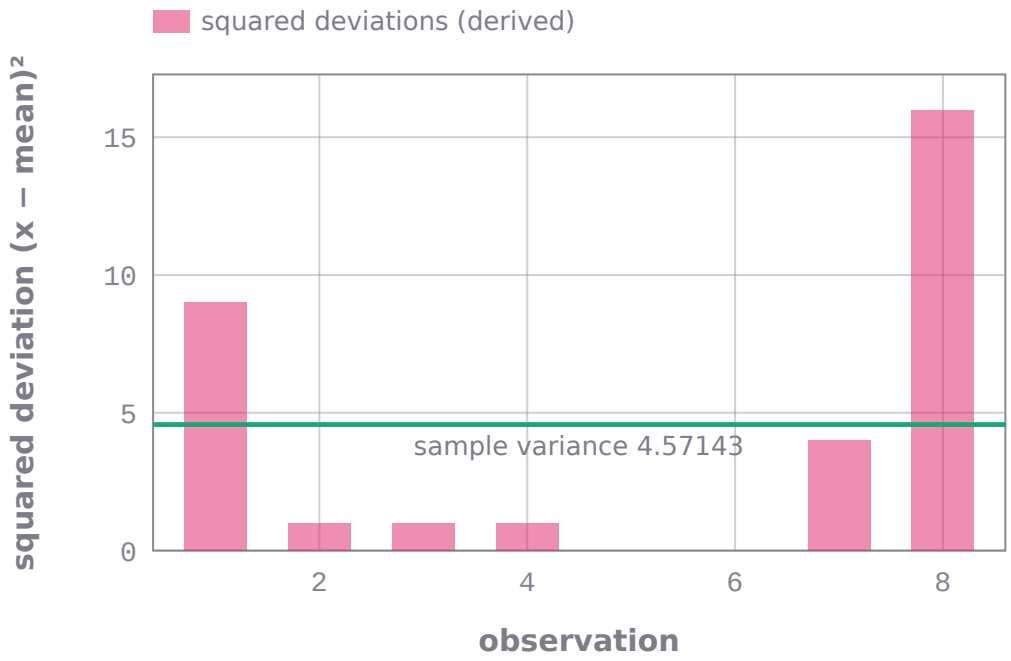
The sample variance of a list of numbers (divisor n - 1).

$$s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$$

input	meaning	condition
`x`	the data values (at least two)	

output	meaning
`variance`	sample variance
`mean`	arithmetic mean
`n`	number of values
`standard_deviation`	the companion measure

- **Method:** two-pass: mean first, then the sum of squared deviations divided by $n - 1$
- **Assumes:** the data are a sample (divisor $n - 1$, Bessel's correction); at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



For x = 2, 4, 4, 4, 5, 5, 7, 9: each observation's squared distance from the returned mean 5 (derived from the inputs and the mean); the returned sample variance 4.57143 is their sum divided by n − 1 (standard deviation 2.13809).

```
statistics_ai999x.solve('sample_variance', x='2, 4, 4, 4, 5, 5, 7, 9')
# -> mean = 5, n = 8, variance = 4.57143, standard_deviation = 2.13809   (SOLVED)
```

```
statistics_ai999x.solve('sample_variance', x=[10, 12, 23, 23, 16, 23, 21, 16])
# -> mean = 18, n = 8, variance = 27.4286, standard_deviation = 5.23723   (SOLVED)
```

```
statistics_ai999x.solve('sample_variance', x='1.5, 2.5')
# -> mean = 2, n = 2, variance = 0.5, standard_deviation = 0.707107   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sample variance of [2, 4, 4, 4, 5, 5, 7, 9].	SOLVED	mean = 5, n = 8, variance = 4.57143, standard_deviation = 2.13809
Compute the sample variance of [10, 12, 23, 23, 16, 23, 21, 16].	SOLVED	mean = 18, n = 8, variance = 27.4286, standard_deviation = 5.23723
What is the sample variance of [600, 470, 170, 430, 300]?	SOLVED	mean = 394, n = 5, variance = 27130, standard_deviation = 164.712
Find the sample variance.	NEEDS_INPUT	sample_variance needs x (the data values (at least two)). Example: solve('sample_variance', x='2, 4, 4, 4, 5, 5, 7, 9').

Population standard deviation, `population_standard_deviation`

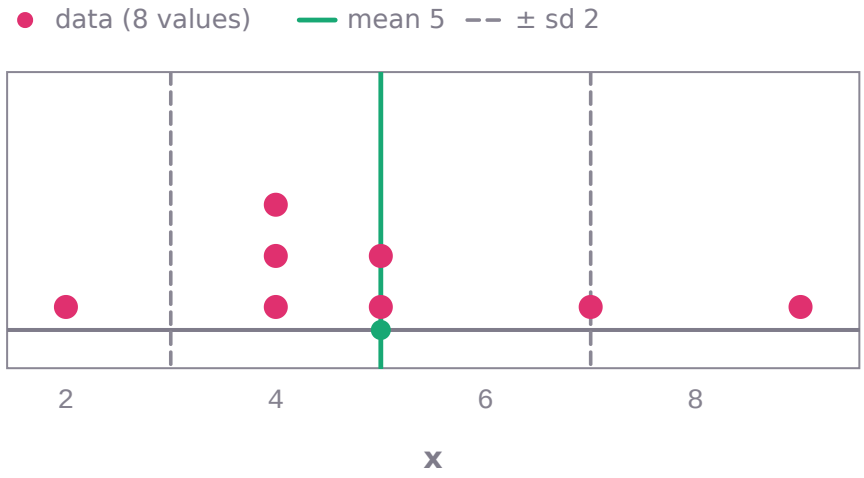
The population standard deviation of a list of numbers (divisor n).

$$\sigma = \sqrt{\frac{1}{n}\sum(x_i - \mu)^2}$$

input	meaning	condition
`x`	the data values	

output	meaning
`standard_deviation`	population standard deviation
`mean`	arithmetic mean
`n`	number of values
`variance`	the companion measure

- **Method:** two-pass: mean first, then the sum of squared deviations divided by n
- **Assumes:** the data are the whole population (divisor n)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 4, 4, 4, 5, 5, 7, 9 (n = 8); marked: the returned mean 5 and, dashed, one returned population standard deviation 2 either side (variance 4).

```
statistics_ai999x.solve('population_standard_deviation', x='2, 4, 4, 4, 5, 5, 7, 9')
# -> mean = 5, n = 8, standard_deviation = 2, variance = 4   (SOLVED)
```

```
statistics_ai999x.solve('population_standard_deviation', x=[10, 12, 23, 23, 16, 23, 21,
16])
# -> mean = 18, n = 8, standard_deviation = 4.89898, variance = 24   (SOLVED)
```

```
statistics_ai999x.solve('population_standard_deviation', x='1.5, 2.5')
# -> mean = 2, n = 2, standard_deviation = 0.5, variance = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the population standard deviation of [2, 4, 4, 4, 5, 5, 7, 9].	SOLVED	mean = 5, n = 8, standard_deviation = 2, variance = 4
Compute the population standard deviation of [10, 12, 23, 23, 16, 23, 21, 16].	SOLVED	mean = 18, n = 8, standard_deviation = 4.89898, variance = 24
What is the population standard deviation of [600, 470, 170, 430, 300]?	SOLVED	mean = 394, n = 5, standard_deviation = 147.323, variance = 21704
Find the population standard deviation.	NEEDS_INPUT	population_standard_deviation needs x (the data values). Example: solve('population_standard_deviation', x='2, 4, 4, 4,

Sample standard deviation, `sample_standard_deviation`

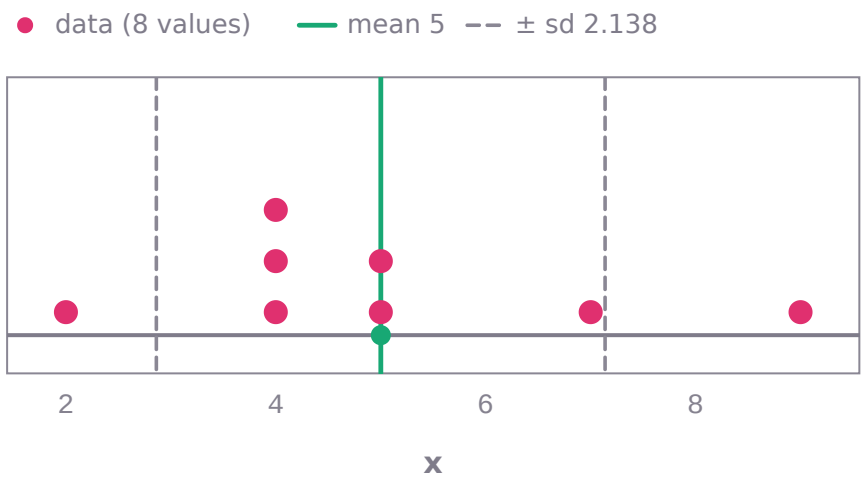
The sample standard deviation of a list of numbers (divisor n - 1).

$$s = \sqrt{\frac{1}{n-1}\sum(x_i - \bar{x})^2}$$

input	meaning	condition
x	the data values (at least two)	

output	meaning
`standard_deviation`	sample standard deviation
`mean`	arithmetic mean
`n`	number of values
`variance`	the companion measure

- **Method:** two-pass: mean first, then the sum of squared deviations divided by $n - 1$
- **Assumes:** the data are a sample (divisor $n - 1$, Bessel's correction); at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 4, 4, 4, 5, 5, 7, 9 (n = 8); marked: the returned mean 5 and, dashed, one returned sample standard deviation 2.13809 either side (variance 4.57143).

```
statistics_ai999x.solve('sample_standard_deviation', x='2, 4, 4, 4, 5, 5, 7, 9')
# -> mean = 5, n = 8, standard_deviation = 2.13809, variance = 4.57143   (SOLVED)
```

```
statistics_ai999x.solve('sample_standard_deviation', x=[10, 12, 23, 23, 16, 23, 21, 16])
# -> mean = 18, n = 8, standard_deviation = 5.23723, variance = 27.4286   (SOLVED)
```

```
statistics_ai999x.solve('sample_standard_deviation', x='1.5, 2.5')
# -> mean = 2, n = 2, standard_deviation = 0.707107, variance = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sample standard deviation of [2, 4, 4, 4, 5, 5, 7, 9].	SOLVED	mean = 5, n = 8, standard_deviation = 2.13809, variance = 4.57143
Compute the sample standard deviation of [10, 12, 23, 23, 16, 23, 21, 16].	SOLVED	mean = 18, n = 8, standard_deviation = 5.23723, variance = 27.4286
What is the sample standard deviation of [600, 470, 170, 430, 300]?	SOLVED	mean = 394, n = 5, standard_deviation = 164.712, variance = 27130
Find the sample standard deviation.	NEEDS_INPUT	sample_standard_deviation needs x (the data values (at least two)). Example: solve('sample_standard_deviation', x='2, 4,

z-score of a value, `z_score`

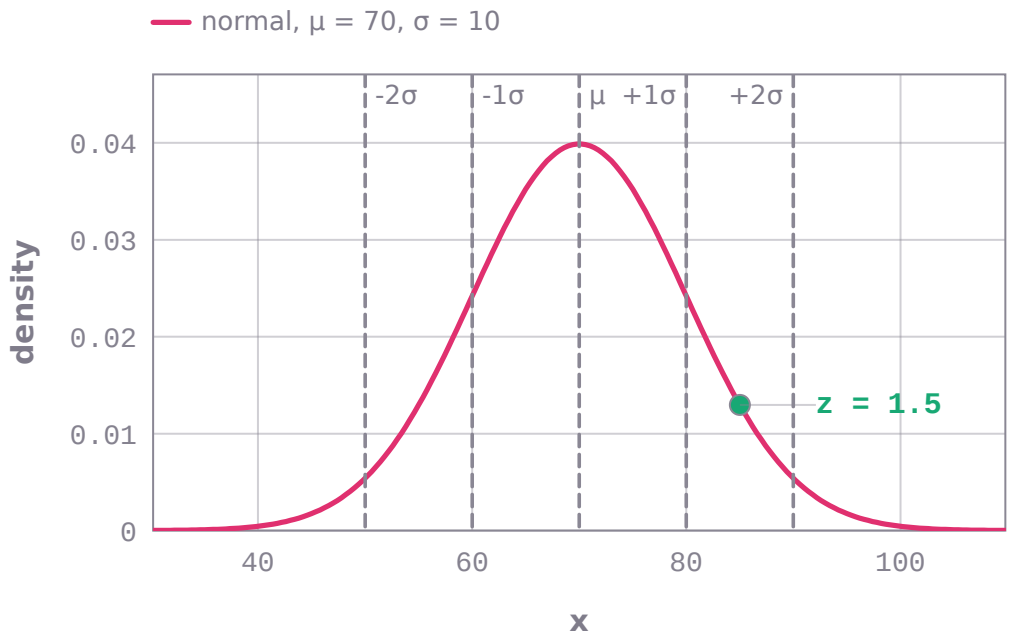
The z-score (standard score) of a value x in a distribution with mean mu and standard deviation sigma.

$$z = \frac{x - \mu}{\sigma}$$

input	meaning	condition
`x`	the value	
`mu`	the mean	
`sigma`	the standard deviation (> 0)	

output	meaning
`z`	number of standard deviations x lies above (+) or below (-) the mean

- **Method:** $z = (x - mu)/\sigma$
- **Assumes:** $\sigma > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A value x = 85 on the normal density with μ = 70, σ = 10 (density from the package's cumulative probabilities on a grid, differenced), with lines one and two standard deviations from the mean; marked: the returned z = 1.5, its distance from μ in standard deviations.

```
statistics_ai999x.solve('z_score', x=85, mu=70, sigma=10)
# -> z = 1.5   (SOLVED)
```

```
statistics_ai999x.solve('z_score', x=42, mu=50, sigma=4)
# -> z = -2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the z-score of x = 85 when the mean = 70 and the standard deviation = 10.	SOLVED	z = 1.5
What is the z-score for a value of 42 with mu = 50 and sigma = 4?	SOLVED	z = -2
Find the z-score of x = 85 when the mean = 70 and the standard deviation = 0.	INVALID_INPUT	z-score of a value: the standard deviation must be a positive number.
Find the z-score of x = 85 when the mean = 70.	NEEDS_INPUT	z_score needs sigma (the standard deviation (> 0)). Example: solve('z_score', x=85, mu=70, sigma=10).

Weighted mean, `weighted_mean`

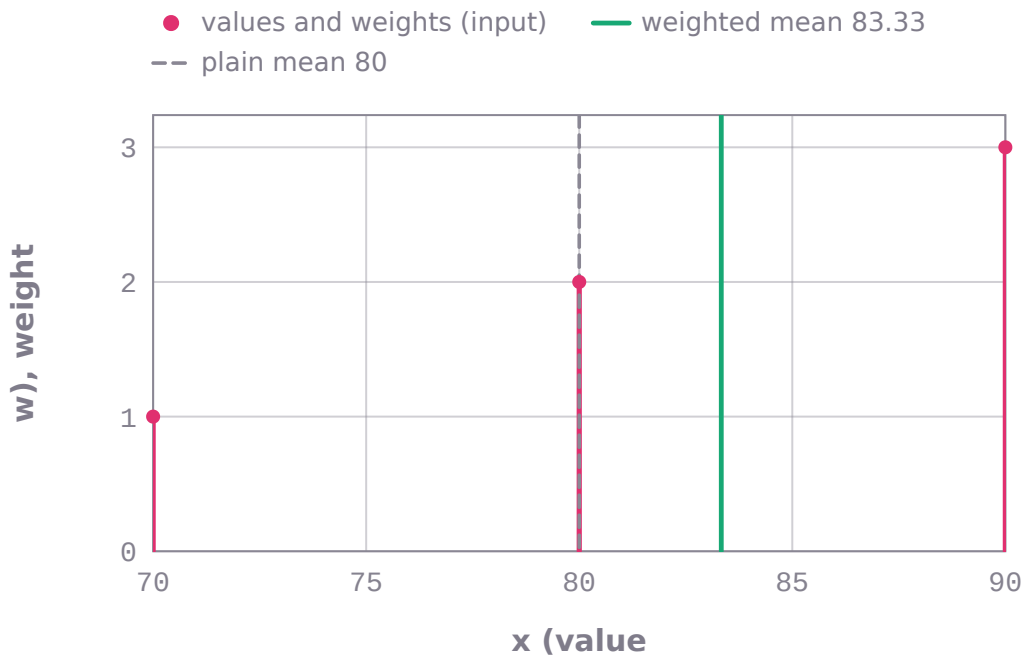
The weighted mean of values x with weights w (grade-point averages, portfolio averages).

$$\bar{x}_w = \frac{\sum w_i x_i}{\sum w_i}$$

input	meaning	condition
`x`	the values	
`w`	the weights (non-negative, same length)	

output	meaning
`weighted_mean`	sum(w x) / sum(w)
`total_weight`	sum of the weights

- **Method:** sum of weight times value divided by the sum of weights
- **Assumes:** same number of values and weights; weights >= 0 with a positive sum
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The values 90, 80, 70 with weights 3, 2, 1 (total weight 6) as stems; marked: the returned weighted mean 83.3333, pulled towards the heavily weighted values, and the unweighted mean 80 from the package.

```
statistics_ai999x.solve('weighted_mean', x='90, 80, 70', w='3, 2, 1')
# -> weighted_mean = 83.3333, total_weight = 6   (SOLVED)
```

```
statistics_ai999x.solve('weighted_mean', x=[3.7, 3.0, 4.0], w=[4, 3, 2])
# -> weighted_mean = 3.53333, total_weight = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the weighted mean of values [90, 80, 70] with weights [3, 2, 1].	SOLVED	weighted_mean = 83.3333, total_weight = 6
Compute the weighted average of [3.7, 3.0, 4.0] with weights [4, 3, 2].	SOLVED	weighted_mean = 3.53333, total_weight = 9
Find the weighted mean of values [90, 80, 70] with weights [3, 2].	INVALID_INPUT	Weighted mean: values and weights must have the same length.
Find the weighted mean of values [90, 80, 70].	NEEDS_INPUT	weighted_mean needs w (the weights (non-negative, same length)). Example: solve('weighted_mean', x='90, 80, 70', w='3, 2

Five-number summary and interquartile range, `five_number_summary`

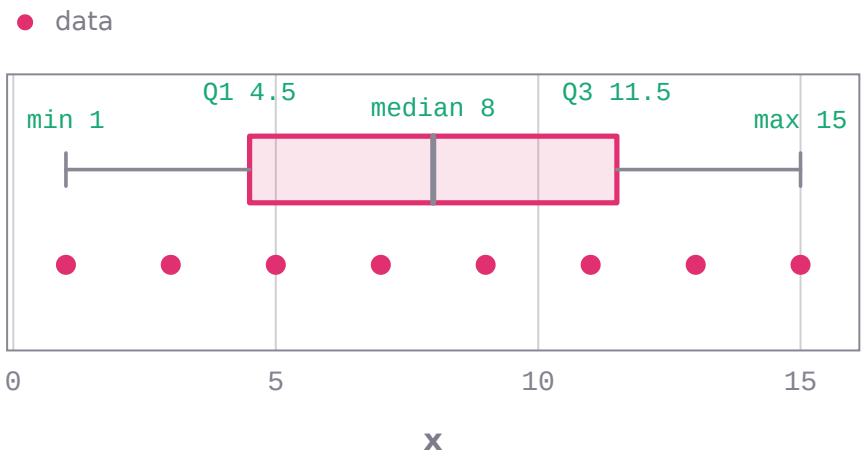
Minimum, first quartile, median, third quartile, maximum and the interquartile range of a data set.

$$Q_p = x_{(\lfloor h \rfloor)} + (h - \lfloor h \rfloor)\,(x_{(\lfloor h \rfloor+1)} - x_{(\lfloor h \rfloor)}),\ h = 1 + (n-1)p$$

input	meaning	condition
`x`	the data values (at least two)	
`method`	quantile method: linear (type 7, spreadsheet/numpy default) or weibull (type 6, p(n+1))	

output	meaning
`minimum`	smallest value
`q1`	first quartile
`median`	median
`q3`	third quartile
`maximum`	largest value
`iqr`	interquartile range $Q_3 - Q_1$

- **Method:** sort; quartile at position 1 + (n - 1)p (linear) or p(n + 1) (weibull), interpolating between neighbours
- **Assumes:** quartiles by linear interpolation between order statistics (Hyndman-Fan type 7, as in spreadsheets and numpy) unless method = weibull (type 6, as in Minitab and SPSS); textbooks that take medians of the halves can differ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A box plot of x = 1, 3, 5, 7, 9, 11, 13, 15 drawn from the returned five-number summary: minimum 1, Q1 4.5, median 8, Q3 11.5, maximum 15 (IQR 7, method default), with the data below.

```
statistics_ai999x.solve('five_number_summary', x='1, 3, 5, 7, 9, 11, 13, 15')
# -> minimum = 1, q1 = 4.5, median = 8, q3 = 11.5, maximum = 15, iqr = 7   (SOLVED)
```

```
statistics_ai999x.solve('five_number_summary', x=[6, 47, 49, 15, 42, 41, 7, 39, 43, 40,
36], method='weibull')
# -> minimum = 6, q1 = 15, median = 40, q3 = 43, maximum = 49, iqr = 28   (SOLVED)
```

```
statistics_ai999x.solve('five_number_summary', x=[6, 47, 49, 15, 42, 41, 7, 39, 43, 40,
36])
# -> minimum = 6, q1 = 25.5, median = 40, q3 = 42.5, maximum = 49, iqr = 17   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the five-number summary of [1, 3, 5, 7, 9, 11, 13, 15].	SOLVED	minimum = 1, q1 = 4.5, median = 8, q3 = 11.5, maximum = 15, iqr = 7

question	outcome	result
Find the quartiles and IQR of [6, 47, 49, 15, 42, 41, 7, 39, 43, 40, 36] using the weibull method.	SOLVED	minimum = 6, q1 = 15, median = 40, q3 = 43, maximum = 49, iqr = 28
What is the interquartile range of [2, 4, 4, 5, 6, 7, 8]?	SOLVED	minimum = 2, q1 = 4, median = 5, q3 = 6.5, maximum = 8, iqr = 2.5

Percentile of a data set, `percentile`

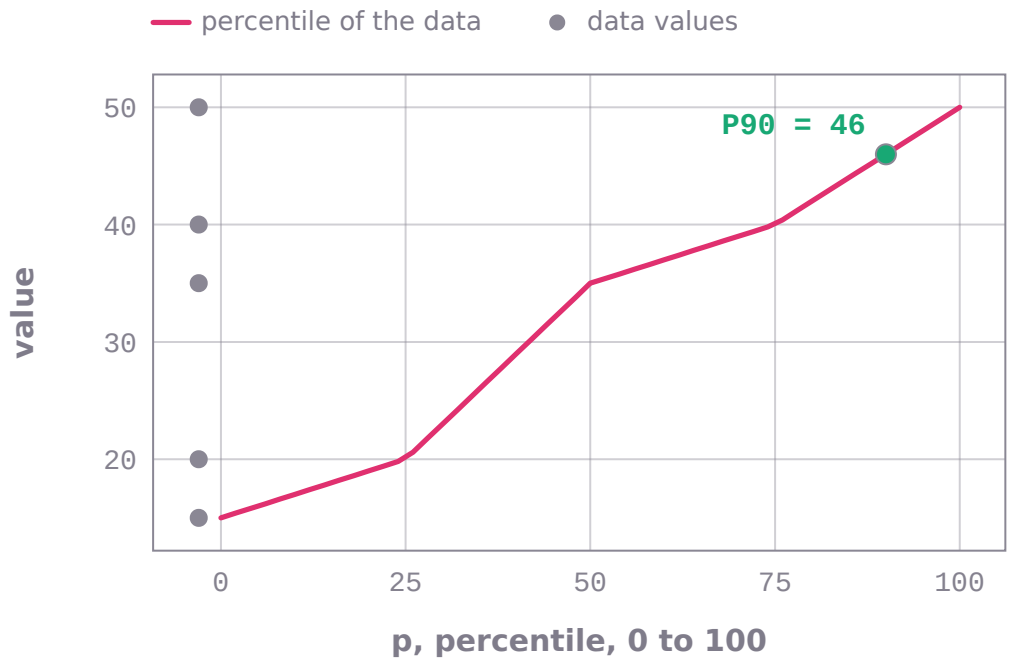
The p-th percentile of a data set (linear interpolation, or the weibull/p(n+1) method).

$$P_p = x_{(\lfloor h \rfloor)} + (h - \lfloor h \rfloor)(x_{(\lfloor h \rfloor + 1)} - x_{(\lfloor h \rfloor)})$$

input	meaning	condition
`x`	the data values (at least two)	
`p`	the percentile, 0 to 100	
`method`	quantile method: linear (type 7, spreadsheet/numpy default) or weibull (type 6, p(n+1))	

output	meaning
`percentile`	the p-th percentile

- **Method:** sort; position h = (n - 1)$p/100$ (0-based, linear) or (n + 1)$p/100 - 1$ (weibull); interpolate
- **Assumes:** $0 \le p \le 100$; linear interpolation (type 7) unless method = weibull (type 6)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The p-th percentile of x = 15, 20, 35, 40, 50 for p = 0 … 100 (one package run every 2 points), the data values shown on the left edge; between data values the percentile is interpolated. Marked: the worked example p = 90, percentile 46.

```
statistics_ai999x.solve('percentile', x='15, 20, 35, 40, 50', p=90)
# -> percentile = 46   (SOLVED)
```

```
statistics_ai999x.solve('percentile', x=[3, 6, 7, 8, 8, 10, 13, 15, 16, 20], p=25)
# -> percentile = 7.25   (SOLVED)
```

```
statistics_ai999x.solve('percentile', x=[3, 6, 7, 8, 8, 10, 13, 15, 16, 20], p=25,
method='weibull')
# -> percentile = 6.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 90th percentile of [15, 20, 35, 40, 50].	SOLVED	percentile = 46
What is the 25th percentile of [3, 6, 7, 8, 8, 10, 13, 15, 16, 20]?	SOLVED	percentile = 7.25
Find the 150th percentile of [1, 2, 3].	INVALID_INPUT	Percentile of a data set: the percentile must be between 0 and 100.

Coefficient of variation, `coefficient_of_variation`

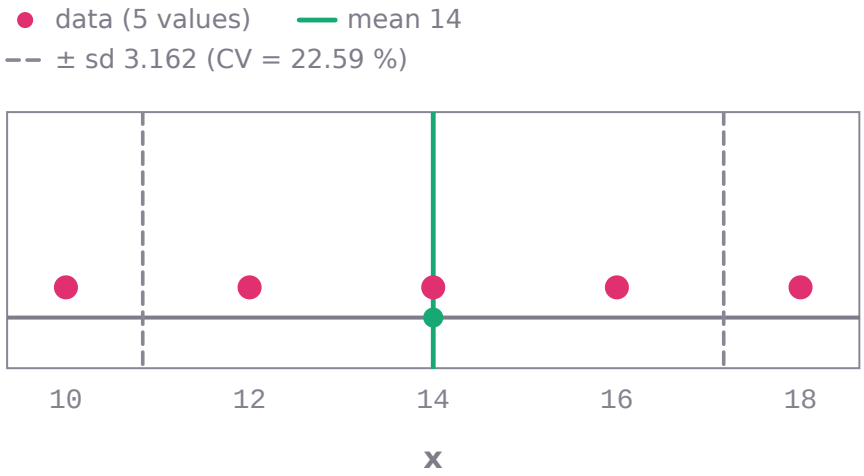
The coefficient of variation (sample standard deviation divided by the mean), as a ratio and a percentage.

$$CV = \frac{s}{\bar{x}}$$

input	meaning	condition
`x`	the data values (at least two)	

output	meaning
`cv`	s / mean
`cv_percent`	100 s / mean
`mean`	mean
`standard_deviation`	sample standard deviation

- **Method:** CV = s / mean
- **Assumes:** positive mean; sample standard deviation $(n - 1)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 10, 12, 14, 16, 18; marked: the returned mean 14 and, dashed, one standard deviation 3.16228 either side. The coefficient of variation is their ratio: 0.225877 (22.5877 %).

```
statistics_ai999x.solve('coefficient_of_variation', x='10, 12, 14, 16, 18')
# -> cv = 0.225877, cv_percent = 22.5877, mean = 14, standard_deviation = 3.16228
(SOLVED)
```

```
statistics_ai999x.solve('coefficient_of_variation', x=[52, 48, 50, 55, 45])
# -> cv = 0.0761577, cv_percent = 7.61577, mean = 50, standard_deviation = 3.80789
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the coefficient of variation of [10, 12, 14, 16, 18].	SOLVED	cv = 0.225877, cv_percent = 22.5877, mean = 14, standard_deviation = 3.16228
What is the CV of [52, 48, 50, 55, 45]?	SOLVED	cv = 0.0761577, cv_percent = 7.61577, mean = 50, standard_deviation = 3.80789
Find the coefficient of variation of [-5, -3, -1].	INVALID_INPUT	Coefficient of variation: the coefficient of variation needs a positive mean (ratio-scale data).

Probability of at least one occurrence, `probability_at_least_one`

The probability that an event of probability p occurs at least once in n independent trials.

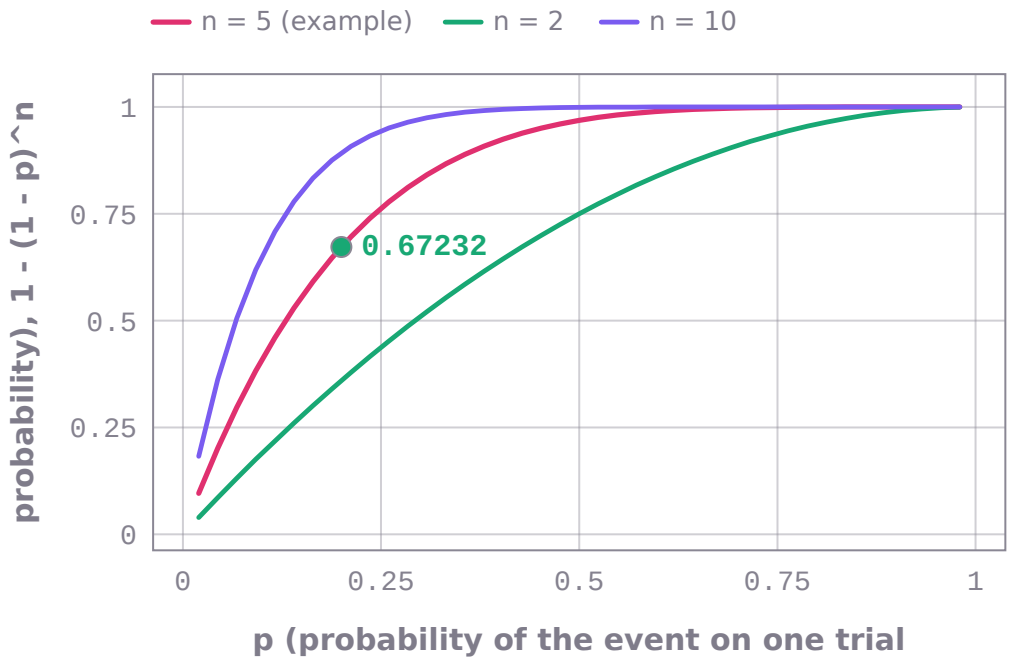
$$P(\text{at least one}) = 1 - (1 - p)^n$$

input	meaning	condition
`p`	probability of the event on one trial	
`n`	number of independent trials	

output	meaning
`probability`	$1 - (1 - p)^n$
`none`	probability that it never occurs, $(1 - p)^n$

- **Method:** complement rule: $1 - (1 - p)^n$

- **Assumes:** independent trials with the same probability p
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



probability (1 - (1 - p)^n) against p (probability of the event on one trial) from 0.02 to 0.98. One curve for each value of n (number of independent trials): 5, 2, 10. 41 points per curve computed by statistics-ai999x; marked: the worked example, p = 0.2, probability = 0.67232.

```
statistics_ai999x.solve('probability_at_least_one', p=0.2, n=5)
# -> probability = 0.67232, none = 0.32768   (SOLVED)
```

```
statistics_ai999x.solve('probability_at_least_one', p=0.1, n=10)
# -> probability = 0.651322, none = 0.348678   (SOLVED)
```

```
statistics_ai999x.solve('probability_at_least_one', p=0.16666666666666666, n=4)
# -> probability = 0.517747, none = 0.482253   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the probability of at least one success in n = 5 independent trials with probability = 0.2?	SOLVED	probability = 0.67232, none = 0.32768
Find the probability of at least one occurrence with p = 0.1 over n = 10 independent trials.	SOLVED	probability = 0.651322, none = 0.348678
Find the probability of at least one occurrence with p = 1.5 over n = 10 independent trials.	INVALID_INPUT	Probability of at least one occurrence: the probability of the event on one trial must be a probability between 0 and 1.
What is the probability of at least one success with p = 0.3?	NEEDS_INPUT	probability_at_least_one needs n (number of independent trials). Example: solve('probability_at_least_one', p=0.2, n=5).

Geometric mean, `geometric_mean`

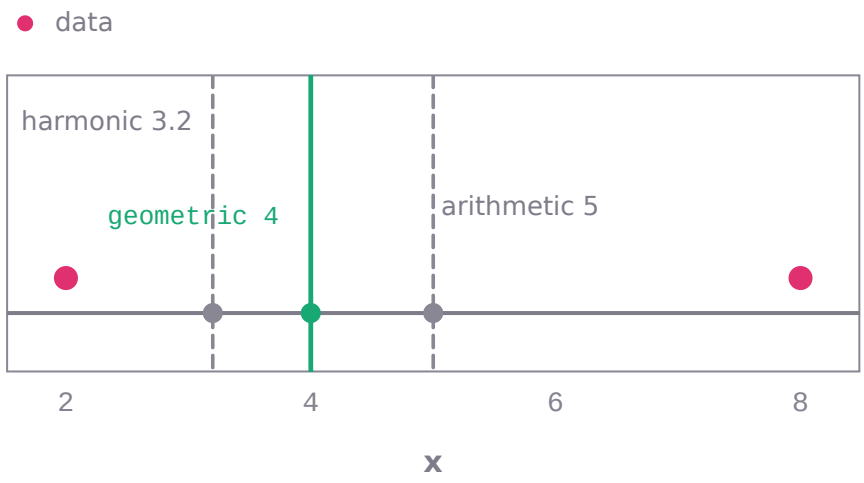
The geometric mean of positive numbers (average growth factors, ratios).

$$G = \left(\prod x_i\right)^{1/n}$$

input	meaning	condition
`x`	positive values	

output	meaning
`geometric_mean`	n-th root of the product

- **Method:** exp of the mean of the logarithms
- **Assumes:** all values positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 8 with three means from the package: arithmetic 5, geometric 4, harmonic 3.2 (always in this order, harmonic ≤ geometric ≤ arithmetic). The geometric mean of this capability is in the first colour: 4.

```
statistics_ai999x.solve('geometric_mean', x='2, 8')
# -> geometric_mean = 4   (SOLVED)
```

```
statistics_ai999x.solve('geometric_mean', x=[1.05, 1.1, 0.95, 1.2])
# -> geometric_mean = 1.0712   (SOLVED)
```

```
statistics_ai999x.solve('geometric_mean', x='4, 1, 0.03125')
# -> geometric_mean = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the geometric mean of [2, 8].	SOLVED	geometric_mean = 4
What is the geometric mean of [1.05, 1.10, 0.95, 1.20]?	SOLVED	geometric_mean = 1.0712
Find the geometric mean of [2, -8].	INVALID_INPUT	Geometric mean: all values must be positive.

Harmonic mean, `harmonic_mean`

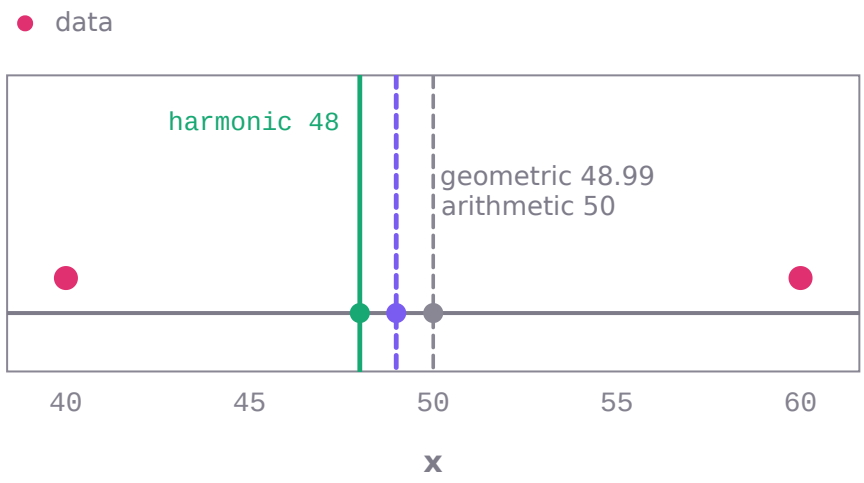
The harmonic mean of positive numbers (average speeds over equal distances, average rates).

$$H = \frac{n}{\sum 1/x_i}$$

input	meaning	condition
`x`	positive values	

output	meaning
`harmonic_mean`	n divided by the sum of reciprocals

- **Method:** n / sum(1/x)
- **Assumes:** all values positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 40, 60 with three means from the package: arithmetic 50, geometric 48.9898, harmonic 48 (always in this order, harmonic ≤ geometric ≤ arithmetic). The harmonic mean of this capability is in the first colour: 48.

```
statistics_ai999x.solve('harmonic_mean', x='40, 60')
# -> harmonic_mean = 48   (SOLVED)
```

```
statistics_ai999x.solve('harmonic_mean', x=[1, 2, 4])
# -> harmonic_mean = 1.71429   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the harmonic mean of [40, 60].	SOLVED	harmonic_mean = 48
What is the harmonic mean of [1, 2, 4]?	SOLVED	harmonic_mean = 1.71429
Find the harmonic mean of [0, 2].	INVALID_INPUT	Harmonic mean: all values must be positive.

Mean absolute deviation, `mean_absolute_deviation`

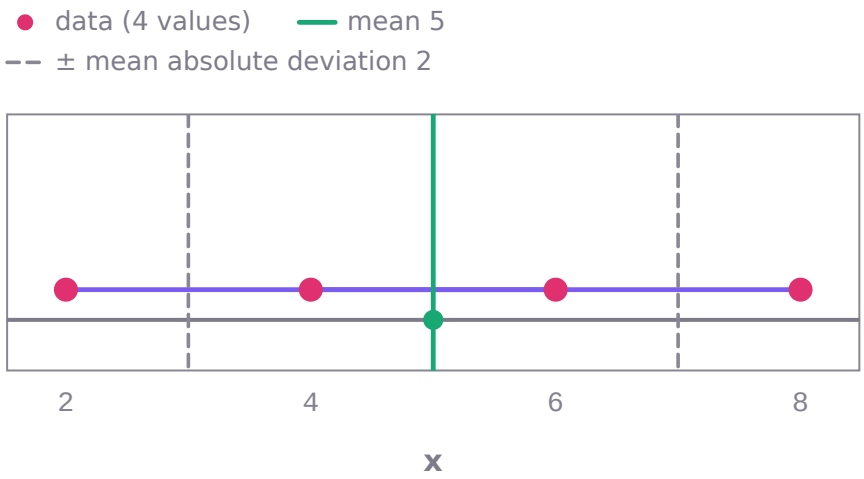
The mean absolute deviation about the mean of a list of numbers.

$$MAD = \frac{1}{n}\sum |x_i - \bar{x}|$$

input	meaning	condition
`x`	the data values	

output	meaning
`mean_absolute_deviation`	average distance from the mean
`mean`	mean

- **Method:** mean of |x - mean|
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 4, 6, 8 with each value's distance to the returned mean 5 drawn as a segment; the returned mean absolute deviation 2 is the average length of those segments, drawn dashed either side of the mean.

```
statistics_ai999x.solve('mean_absolute_deviation', x='2, 4, 6, 8')
# -> mean_absolute_deviation = 2, mean = 5   (SOLVED)
```

```
statistics_ai999x.solve('mean_absolute_deviation', x=[10, 15, 20, 30, 50])
# -> mean_absolute_deviation = 12, mean = 25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mean absolute deviation of [2, 4, 6, 8].	SOLVED	mean_absolute_deviation = 2, mean = 5
What is the mean absolute deviation of [10, 15, 20, 30, 50]?	SOLVED	mean_absolute_deviation = 12, mean = 25

Median absolute deviation, `median_absolute_deviation`

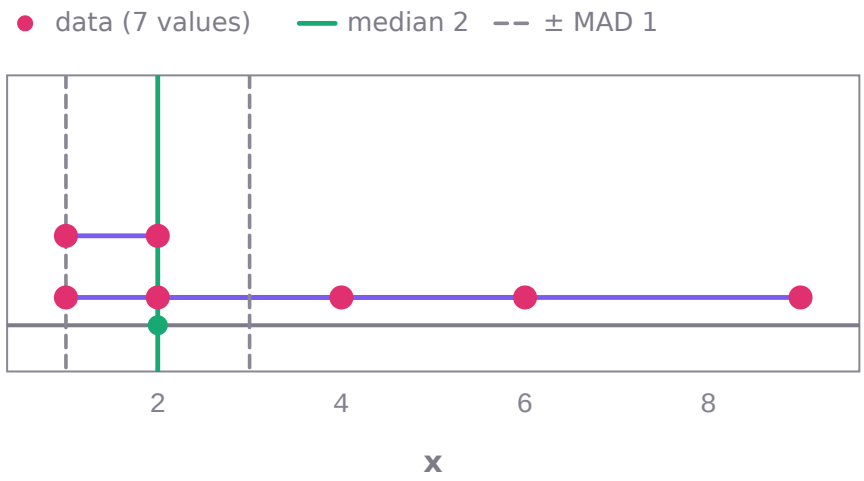
The median absolute deviation (MAD) about the median, a robust measure of spread (unscaled).

$$MAD = \text{median}\left(|x_i - \tilde{x}|\right)$$

input	meaning	condition
`x`	the data values	

output	meaning
`median_absolute_deviation`	median of |x - median|
`median`	median

- **Method:** median of the absolute deviations from the median
- **Assumes:** unscaled (multiply by 1.4826 to estimate a normal sigma)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 1, 1, 2, 2, 4, 6, 9 with each value's distance to the returned median 2 drawn as a segment; the returned median absolute deviation 1 is the median of those distances (dashed either side), robust to the long segments of extreme values.

```
statistics_ai999x.solve('median_absolute_deviation', x='1, 1, 2, 2, 4, 6, 9')
# -> median_absolute_deviation = 1, median = 2   (SOLVED)
```

```
statistics_ai999x.solve('median_absolute_deviation', x=[10, 12, 13, 15, 40])
# -> median_absolute_deviation = 2, median = 13   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the median absolute deviation of [1, 1, 2, 2, 4, 6, 9].	SOLVED	median_absolute_deviation = 1, median = 2
What is the median absolute deviation of [10, 12, 13, 15, 40]?	SOLVED	median_absolute_deviation = 2, median = 13

Standard error of the mean, `standard_error_of_mean`

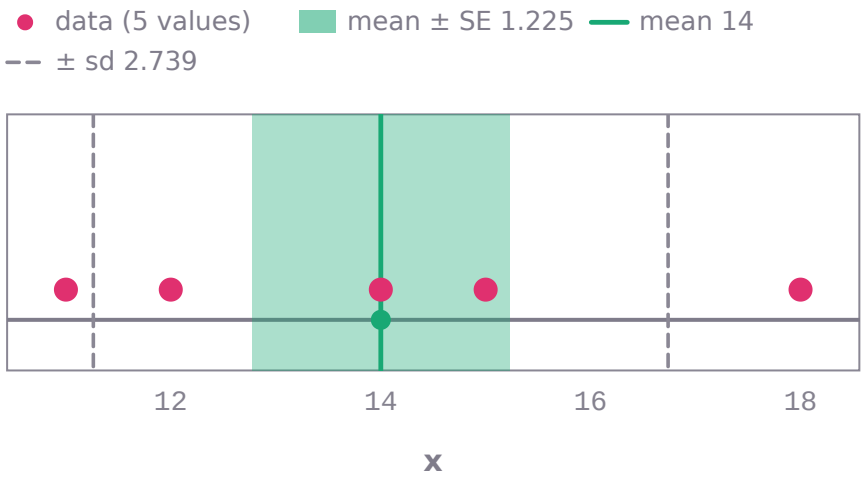
The standard error of the mean of a sample: s / sqrt(n).

$$SE_{\bar{x}} = \frac{s}{\sqrt{n}}$$

input	meaning	condition
x	the data values (at least two)	

output	meaning
standard_error	s / sqrt(n)
standard_deviation	sample standard deviation
n	sample size

- **Method:** sample standard deviation divided by the square root of n
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 12, 15, 11, 18, 14 (n = 5); marked: the mean 14 (from the package's mean), dashed: one returned standard deviation 2.73861 either side, shaded: one returned standard error 1.22474 = sd/√n either side, the uncertainty of the mean itself.

```
statistics_ai999x.solve('standard_error_of_mean', x='12, 15, 11, 18, 14')
# -> standard_error = 1.22474, standard_deviation = 2.73861, n = 5   (SOLVED)
```

```
statistics_ai999x.solve('standard_error_of_mean', x=[2.1, 2.5, 2.2, 2.8, 2.4, 2.6])
# -> standard_error = 0.105409, standard_deviation = 0.258199, n = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the standard error of the mean of [12, 15, 11, 18, 14].	SOLVED	standard_error = 1.22474, standard_deviation = 2.73861, n = 5
What is the standard error of [2.1, 2.5, 2.2, 2.8, 2.4, 2.6]?	SOLVED	standard_error = 0.105409, standard_deviation = 0.258199, n = 6
Find the standard error of the mean of [12].	INVALID_INPUT	x: needs at least 2 entries, got 1.

Mean and standard deviation of a frequency table, `frequency_table_statistics`

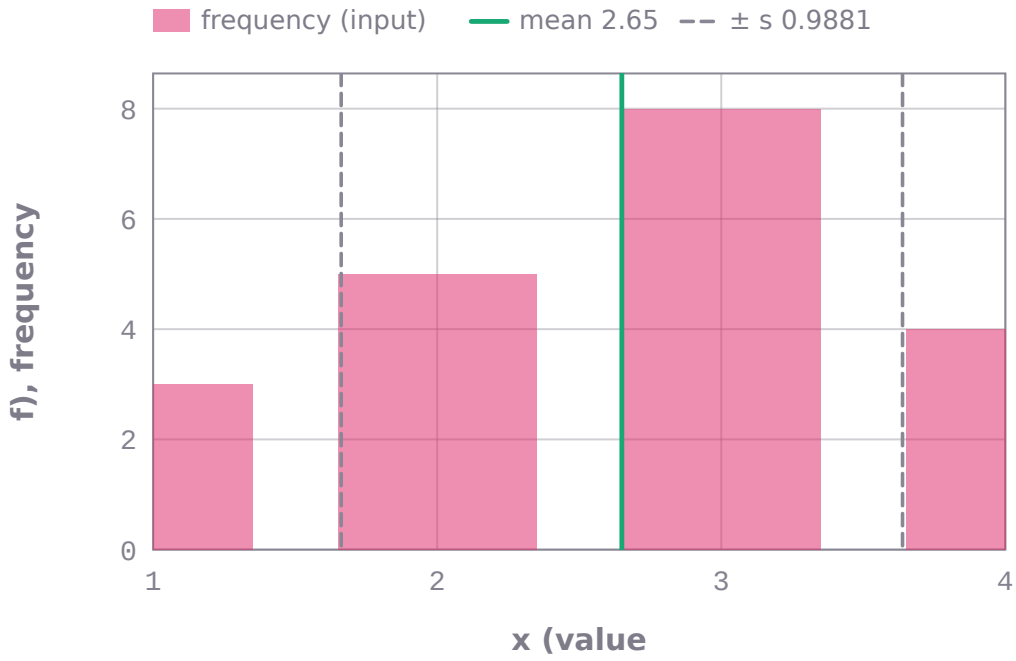
The mean and the sample and population standard deviations of data given as values with frequencies.

$$\bar{x} = \frac{\sum f_i x_i}{\sum f_i}, \quad s = \sqrt{\frac{\sum f_i (x_i - \bar{x})^2}{n - 1}}$$

input	meaning	condition
`x`	the distinct values	
`f`	their frequencies (whole numbers)	

output	meaning
`mean`	sum(f x) / n
`sample_standard_deviation`	divisor $n - 1$
`population_standard_deviation`	divisor n
`n`	total frequency

- **Method:** weighted sums: mean = sum(f x)/n, then sum(f (x - mean)^2) over $n - 1$ or n
- **Assumes:** frequencies are whole numbers; total frequency >= 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The frequency table x = 1, 2, 3, 4, f = 3, 5, 8, 4 (n = 20) as bars; marked: the returned mean 2.65 and, dashed, one returned sample standard deviation 0.988087 either side (population sd 0.963068).

```
statistics_ai999x.solve('frequency_table_statistics', x='1, 2, 3, 4', f='3, 5, 8, 4')
# -> mean = 2.65, sample_standard_deviation = 0.988087, population_standard_deviation =
0.963068, n = 20   (SOLVED)
```

```
statistics_ai999x.solve('frequency_table_statistics', x=[10, 20, 30], f=[2, 5, 3])
# -> mean = 21, sample_standard_deviation = 7.37865, population_standard_deviation = 7, n =
10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mean and standard deviation of the frequency table with values [1, 2, 3, 4] and frequencies [3, 5, 8, 4].	SOLVED	mean = 2.65, sample_standard_deviation = 0.988087, population_standard_deviation = 0.963068, n = 20
Find the mean of values [10, 20, 30] with frequencies [2, 5, 3].	SOLVED	mean = 21, sample_standard_deviation = 7.37865, population_standard_deviation = 7, n = 10
Find the mean of values [10, 20, 30] with frequencies [2, 5.5, 3].	INVALID_INPUT	Mean and standard deviation of a frequency table: frequencies must be whole numbers >= 0.

Trimmed mean, `trimmed_mean`

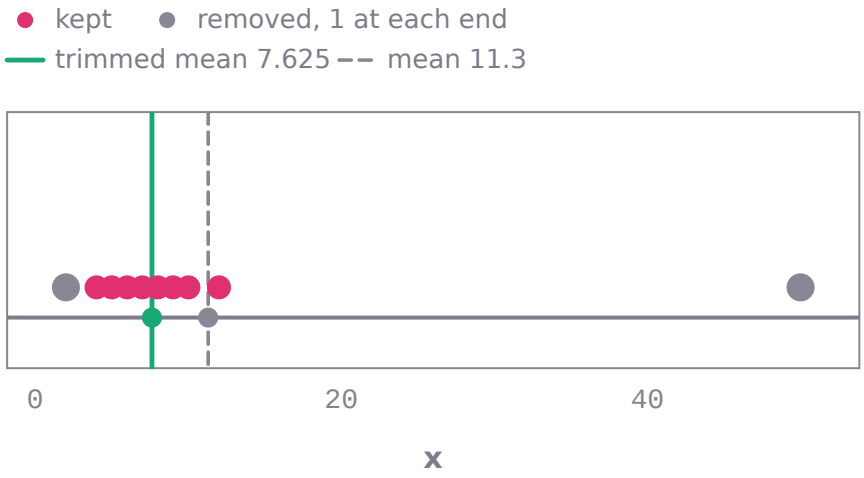
The trimmed mean: the mean after removing the given proportion of values from EACH end.

$$\bar{x}_\alpha = \frac{1}{n-2k}\sum_{i=k+1}^{n-k} x_{(i)},\ k = \lfloor \alpha n \rfloor$$

input	meaning	condition
`x`	the data values	
`proportion`	proportion cut from each end (0.1 for a 10% trimmed mean)	

output	meaning
`trimmed_mean`	mean of the remaining values
`removed_each_end`	values removed at each end

- **Method:** sort, drop floor(proportion * n) values at each end, average the rest
- **Assumes:** 0 <= proportion < 0.5; floor(proportion * n) values removed from each end
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 4, 5, 6, 7, 8, 9, 10, 12, 50 with the proportion 0.1 cut from each end: the returned 1 value(s) at each end are greyed. Marked: the returned trimmed mean 7.625 and, dashed, the untrimmed mean 11.3 from the package, pulled by the extreme value.

```
statistics_ai999x.solve('trimmed_mean', x='2, 4, 5, 6, 7, 8, 9, 10, 12, 50',
proportion=0.1)
# -> trimmed_mean = 7.625, removed_each_end = 1   (SOLVED)
```

```
statistics_ai999x.solve('trimmed_mean', x=[3, 8, 1, 9, 4, 6, 100, 5], proportion=0.25)
# -> trimmed_mean = 5.75, removed_each_end = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 10% trimmed mean of [2, 4, 5, 6, 7, 8, 9, 10, 12, 50].	SOLVED	trimmed_mean = 7.625, removed_each_end = 1
Find the trimmed mean of [3, 8, 1, 9, 4, 6, 100, 5] with proportion = 0.25.	SOLVED	trimmed_mean = 5.75, removed_each_end = 2
Find the 60% trimmed mean of [1, 2, 3, 4].	INVALID_INPUT	Trimmed mean: the proportion cut from each end must be at least 0 and below 0.5.
Find the trimmed mean of [1, 2, 3, 4].	NEEDS_INPUT	trimmed_mean needs proportion (proportion cut from each end (0.1 for a 10% trimmed mean)). Example: solve('trimmed_mean'

Outliers by the 1.5 IQR rule, `iqr_outliers`

The lower and upper fences $Q_1 - 1.5$ IQR and $Q_3 + 1.5$ IQR and the values outside them.

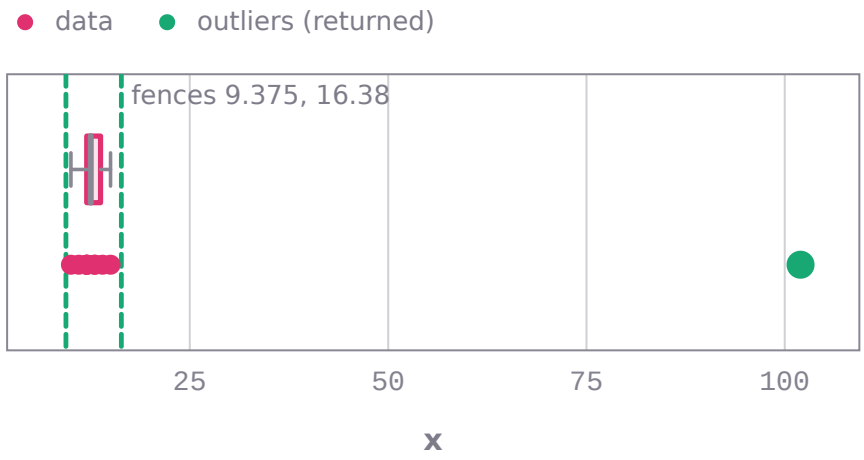
$$[Q_1 - 1.5\,IQR,\ Q_3 + 1.5\,IQR]$$

input	meaning	condition
`x`	the data values (at least four)	

output	meaning
`lower_fence`	$Q_1 - 1.5$ IQR
`upper_fence`	$Q_3 + 1.5$ IQR
`outliers`	values outside the fences

output	meaning
`q1`	first quartile
`q3`	third quartile
`iqr`	interquartile range

- **Method:** Tukey's fences from the quartiles
- **Assumes:** quartiles by linear interpolation (type 7)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x = 10, 12, 12, 13, 12, 11, 14, 13, 15, 102 with its box (Q1 12, median 12.5, Q3 13.75; whiskers to the most extreme values inside the fences) and the returned fences Q1 − 1.5·IQR = 9.375 and Q3 + 1.5·IQR = 16.375 (IQR 1.75); the returned outliers 102 lie beyond them.

```
statistics_ai999x.solve('iqr_outliers', x='10, 12, 12, 13, 12, 11, 14, 13, 15, 102')
# -> lower_fence = 9.375, upper_fence = 16.375, outliers = [102.0], q1 = 12, q3 = 13.75,
iqr = 1.75   (SOLVED)
```

```
statistics_ai999x.solve('iqr_outliers', x=[1, 3, 4, 5, 5, 6, 7, 30])
# -> lower_fence = 0, upper_fence = 10, outliers = [30.0], q1 = 3.75, q3 = 6.25, iqr = 2.5
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the outliers in [10, 12, 12, 13, 12, 11, 14, 13, 15, 102].	SOLVED	lower_fence = 9.375, upper_fence = 16.375, outliers = [102.0], q1 = 12, q3 = 13.75, iqr = 1.75
Use the 1.5 IQR rule to find outliers in [1, 3, 4, 5, 5, 6, 7, 30].	SOLVED	lower_fence = 0, upper_fence = 10, outliers = [30.0], q1 = 3.75, q3 = 6.25, iqr = 2.5
Find the outliers in [1, 2, 3].	INVALID_INPUT	x: needs at least 4 entries, got 3.

Expected value and variance of a discrete random variable, `discrete_random_variable`

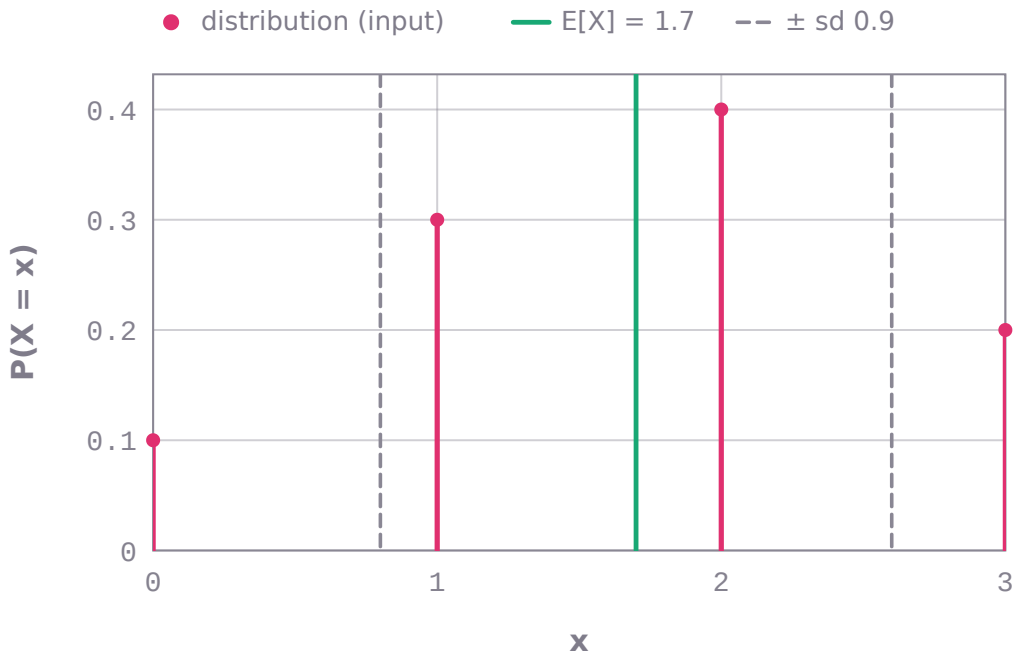
The expected value, variance and standard deviation of a discrete random variable given its values and probabilities.

$$E[X] = \sum x_i p_i, \quad \operatorname{Var}(X) = \sum (x_i - \mu)^2 p_i$$

input	meaning	condition
`x`	the values of the random variable	
`p`	their probabilities (summing to 1)	

output	meaning
`expected_value`	E[X] = sum x p
`variance`	Var[X] = sum (x - E[X])^2 p
`standard_deviation`	sqrt(Var[X])

- **Method:** E[X] = sum x p; Var[X] = sum p (x - E[X])^2
- **Assumes:** probabilities non-negative, summing to 1 (within 1e-9)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The distribution x = 0, 1, 2, 3 with probabilities 0.1, 0.3, 0.4, 0.2; marked: the returned expected value 1.7 (the balance point) and, dashed, one returned standard deviation 0.9 either side (variance 0.81).

```
statistics_ai999x.solve('discrete_random_variable', x='0, 1, 2, 3', p='0.1, 0.3, 0.4, 0.2')
# -> expected_value = 1.7, variance = 0.81, standard_deviation = 0.9   (SOLVED)
```

```
statistics_ai999x.solve('discrete_random_variable', x=[10, 20], p=[0.25, 0.75])
# -> expected_value = 17.5, variance = 18.75, standard_deviation = 4.33013   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the expected value of a random variable with values [0, 1, 2, 3] and probabilities [0.1, 0.3, 0.4, 0.2].	SOLVED	expected_value = 1.7, variance = 0.81, standard_deviation = 0.9
Find the expected value and variance of the probability distribution with values [10, 20] and probabilities [0.25, 0.75].	SOLVED	expected_value = 17.5, variance = 18.75, standard_deviation = 4.33013

question	outcome	result
Find the expected value of a random variable with values [0, 1] and probabilities [0.5, 0.6].	INVALID_INPUT	Expected value and variance of a discrete random variable: probabilities must be non-negative and sum to 1.

Skewness and kurtosis, `skewness_kurtosis`

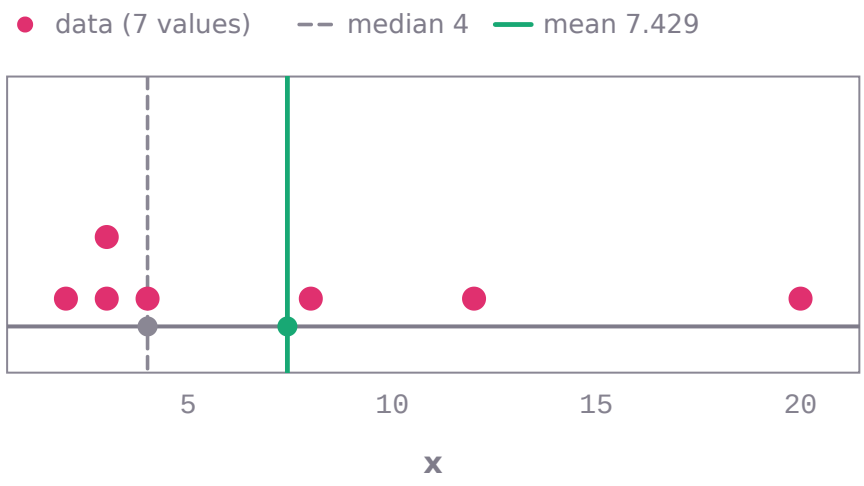
Sample skewness and excess kurtosis: the adjusted Fisher-Pearson estimates (G1, G2, as in spreadsheets and most software) and the moment estimates (g1, g2).

$$G_1 = \frac{\sqrt{n(n-1)}}{n-2}\,\frac{m_3}{m_2^{3/2}}, \qquad G_2 = \frac{(n-1)\left((n+1)g_2 + 6\right)}{(n-2)(n-3)}$$

input	meaning	condition
`x`	the data values (at least four)	

output	meaning
`skewness`	adjusted skewness G1
`excess_kurtosis`	adjusted excess kurtosis G2
`skewness_biased`	moment skewness g1
`excess_kurtosis_biased`	moment excess kurtosis g2

- **Method:** central moments m2, m3, m4; $g_1 = m_3/m_2^{1.5}$, $g_2 = m_4/m_2^2 - 3$; small-sample adjustments for G1 and G2
- **Assumes:** at least four values, not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 2, 3, 3, 4, 8, 12, 20 with its mean 7.42857 and median 4 (from the package): the long right tail pulls the mean above the median. Returned: skewness 1.39985 (biased 1.08001), excess kurtosis 1.37206 (biased -0.178307).

```
statistics_ai999x.solve('skewness_kurtosis', x='2, 3, 3, 4, 8, 12, 20')
# -> skewness = 1.39985, excess_kurtosis = 1.37206, skewness_biased = 1.08001,
excess_kurtosis_biased = -0.178307   (SOLVED)
```

```
statistics_ai999x.solve('skewness_kurtosis', x=[1, 2, 2, 3, 3, 3, 4, 10])
# -> skewness = 2.24026, excess_kurtosis = 5.66914, skewness_biased = 1.7962,
excess_kurtosis_biased = 2.03292   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the skewness and kurtosis of [2, 3, 3, 4, 8, 12, 20].	SOLVED	skewness = 1.39985, excess_kurtosis = 1.37206, skewness_biased = 1.08001, excess_kurtosis_biased = -0.178307
What is the skewness of [1, 2, 2, 3, 3, 3, 4, 10]?	SOLVED	skewness = 2.24026, excess_kurtosis = 5.66914, skewness_biased = 1.7962, excess_kurtosis_biased = 2.03292
Find the skewness of [1, 2, 3].	INVALID_INPUT	x: needs at least 4 entries, got 3.

Binomial probability: exactly k successes, `binomial_exactly`

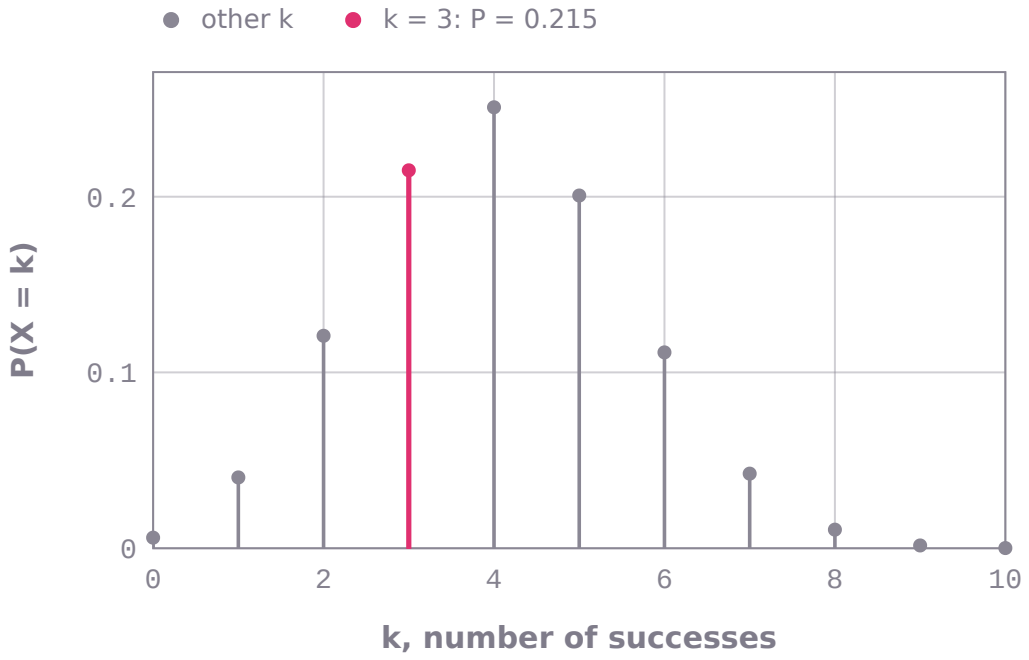
The probability of exactly k successes in n independent trials with success probability p.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

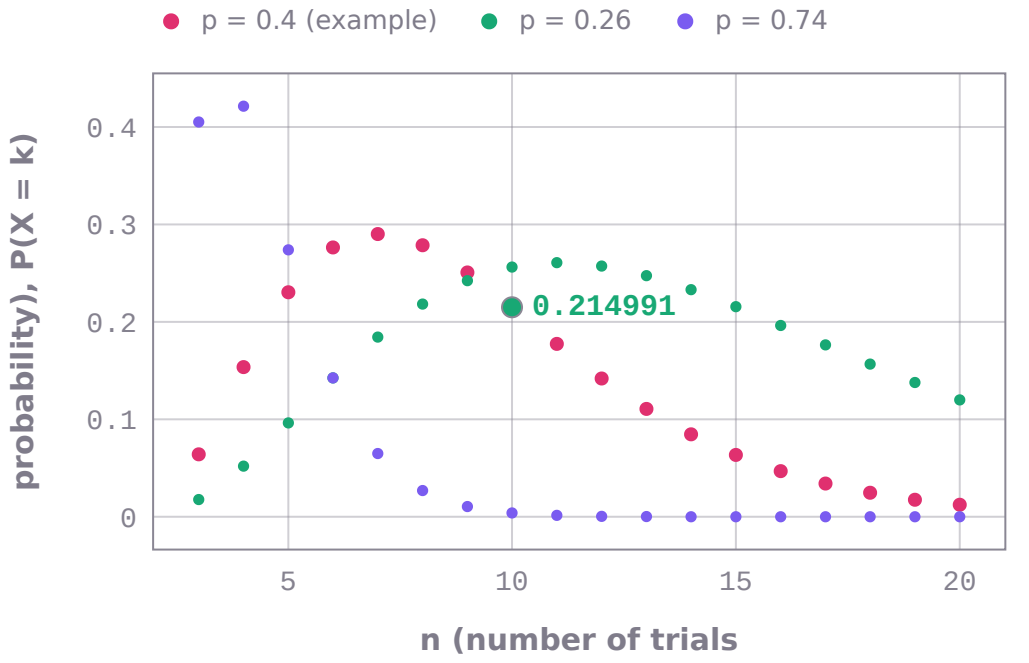
input	meaning	condition
`n`	number of trials	
`k`	number of successes	
`p`	probability of success on each trial	

output	meaning
`probability`	P(X = k)

- **Method:** exact binomial probabilities summed term by term (log-space binomial coefficients)
- **Assumes:** n independent trials, constant success probability $0 < p < 1$; $0 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The binomial distribution with n = 10 trials and p = 0.4: P(X = k) for k = 0 … 10, each from the package; the stems with k = 3 are in the first colour and carry the returned probability 0.214991.



probability (P(X = k)) against n (number of trials) from 1 to 20. One curve for each value of p (probability of success on each trial): 0.4, 0.26, 0.74. Other inputs as in the worked example: k = 3. 18 points per curve computed by statistics-ai999x; marked: the worked example, n = 10, probability = 0.214991.

```
statistics_ai999x.solve('binomial_exactly', n=10, k=3, p=0.4)
# -> probability = 0.214991   (SOLVED)
```

```
statistics_ai999x.solve('binomial_exactly', n=20, k=5, p=0.25)
# -> probability = 0.202331   (SOLVED)
```

```
statistics_ai999x.solve('binomial_exactly', n=6, k=6, p=0.5)
# -> probability = 0.015625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the binomial probability of exactly k = 3 successes in n = 10 trials with p = 0.4.	SOLVED	probability = 0.214991
In n = 20 trials with probability of success = 0.25, what is the probability of exactly k = 5 successes?	SOLVED	probability = 0.202331
Find the binomial probability of exactly k = 12 successes in n = 10 trials with p = 0.4.	INVALID_INPUT	Binomial probability: exactly k successes: k cannot exceed the number of trials n.
Find the binomial probability of exactly k = 3 successes in n = 10 trials.	NEEDS_INPUT	binomial_exactly needs p (probability of success on each trial). Example: solve('binomial_exactly', n=10, k=3, p=0.4).

Binomial probability: at most k successes, `binomial_at_most`

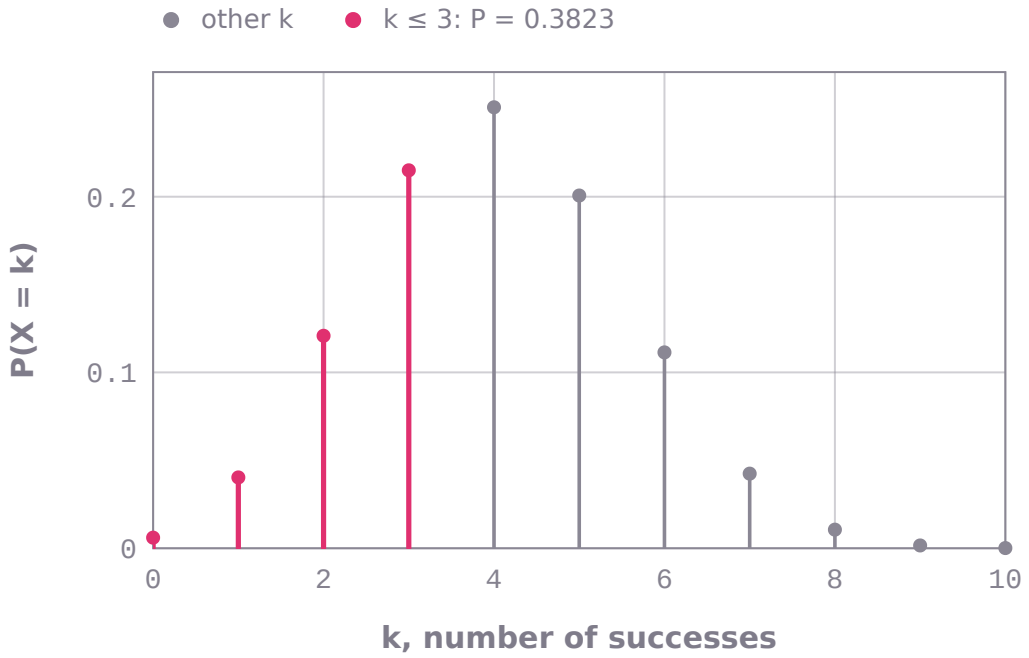
The probability of at most k successes in n independent trials with success probability p.

$$P(X \le k) = \sum_{j=0}^{k} \binom{n}{j} p^j (1-p)^{n-j}$$

input	meaning	condition
`n`	number of trials	
`k`	number of successes	
`p`	probability of success on each trial	

output	meaning
`probability`	P(X <= k)

- **Method:** exact binomial probabilities summed term by term (log-space binomial coefficients)
- **Assumes:** n independent trials, constant success probability $0 < p < 1$; $0 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The binomial distribution with n = 10 trials and p = 0.4: P(X = k) for k = 0 … 10, each from the package; the stems with k ≤ 3 are in the first colour and carry the returned probability 0.382281.

```
statistics_ai999x.solve('binomial_at_most', n=10, k=3, p=0.4)
# -> probability = 0.382281   (SOLVED)
```

```
statistics_ai999x.solve('binomial_at_most', n=20, k=5, p=0.25)
# -> probability = 0.617173   (SOLVED)
```

```
statistics_ai999x.solve('binomial_at_most', n=6, k=6, p=0.5)
# -> probability = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the binomial probability of at most k = 3 successes in n = 10 trials with p = 0.4.	SOLVED	probability = 0.382281
In n = 20 trials with probability of success = 0.25, what is the probability of at most k = 5 successes?	SOLVED	probability = 0.617173
Find the binomial probability of at most k = 12 successes in n = 10 trials with p = 0.4.	INVALID_INPUT	Binomial probability: at most k successes: k cannot exceed the number of trials n.
Find the binomial probability of at most k = 3 successes in n = 10 trials.	NEEDS_INPUT	binomial_at_most needs p (probability of success on each trial). Example: solve('binomial_at_most', n=10, k=3, p=0.4).

Binomial probability: at least k successes, `binomial_at_least`

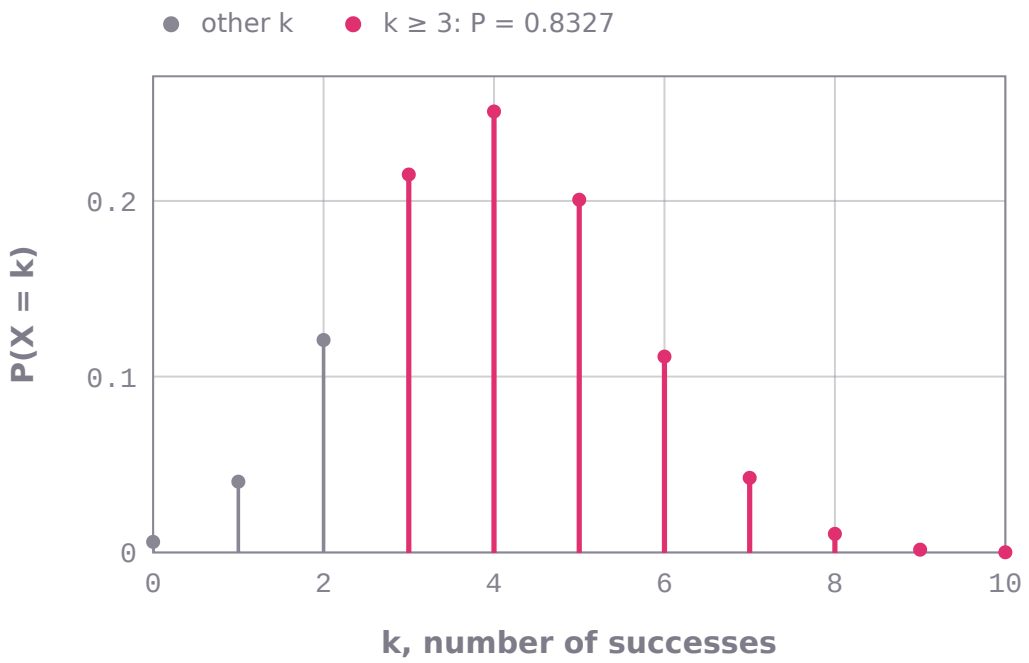
The probability of at least k successes in n independent trials with success probability p.

$$P(X \geq k) = \sum_{j=k}^{n} \binom{n}{j} p^j (1-p)^{n-j}$$

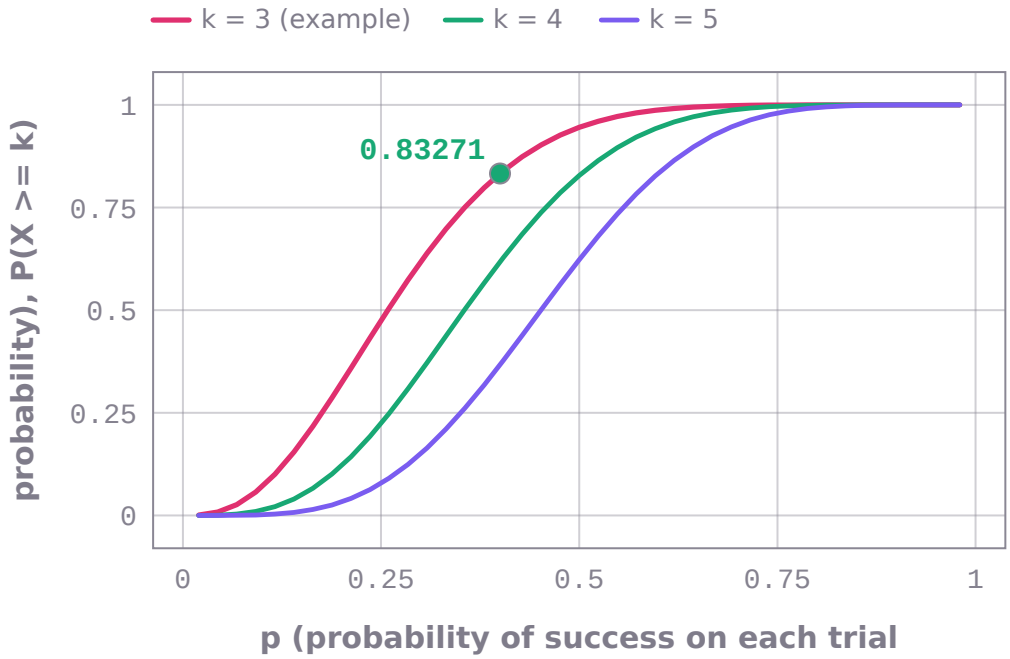
input	meaning	condition
`n`	number of trials	
`k`	number of successes	
`p`	probability of success on each trial	

output	meaning
`probability`	P(X >= k)

- **Method:** exact binomial probabilities summed term by term (log-space binomial coefficients)
- **Assumes:** n independent trials, constant success probability $0 < p < 1$; $0 \leq k \leq n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The binomial distribution with n = 10 trials and p = 0.4: P(X = k) for k = 0 … 10, each from the package; the stems with k ≥ 3 are in the first colour and carry the returned probability 0.83271.



probability (P(X >= k)) against p (probability of success on each trial) from 0.02 to 0.98. One curve for each value of k (number of successes): 3, 4, 5. Other inputs as in the worked example: n = 10. 41 points per curve computed by statistics-ai999x; marked: the worked example, p = 0.4, probability = 0.83271.

```
statistics_ai999x.solve('binomial_at_least', n=10, k=3, p=0.4)
# -> probability = 0.83271   (SOLVED)
```

```
statistics_ai999x.solve('binomial_at_least', n=20, k=5, p=0.25)
# -> probability = 0.585158   (SOLVED)
```

```
statistics_ai999x.solve('binomial_at_least', n=6, k=6, p=0.5)
# -> probability = 0.015625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the binomial probability of at least k = 3 successes in n = 10 trials with p = 0.4.	SOLVED	probability = 0.83271
In n = 20 trials with probability of success = 0.25, what is the probability of at least k = 5 successes?	SOLVED	probability = 0.585158
Find the binomial probability of at least k = 12 successes in n = 10 trials with p = 0.4.	INVALID_INPUT	Binomial probability: at least k successes: k cannot exceed the number of trials n.
Find the binomial probability of at least k = 3 successes in n = 10 trials.	NEEDS_INPUT	binomial_at_least needs p (probability of success on each trial). Example: solve('binomial_at_least', n=10, k=3, p=0.4).

Mean and standard deviation of a binomial distribution, `binomial_mean_sd`

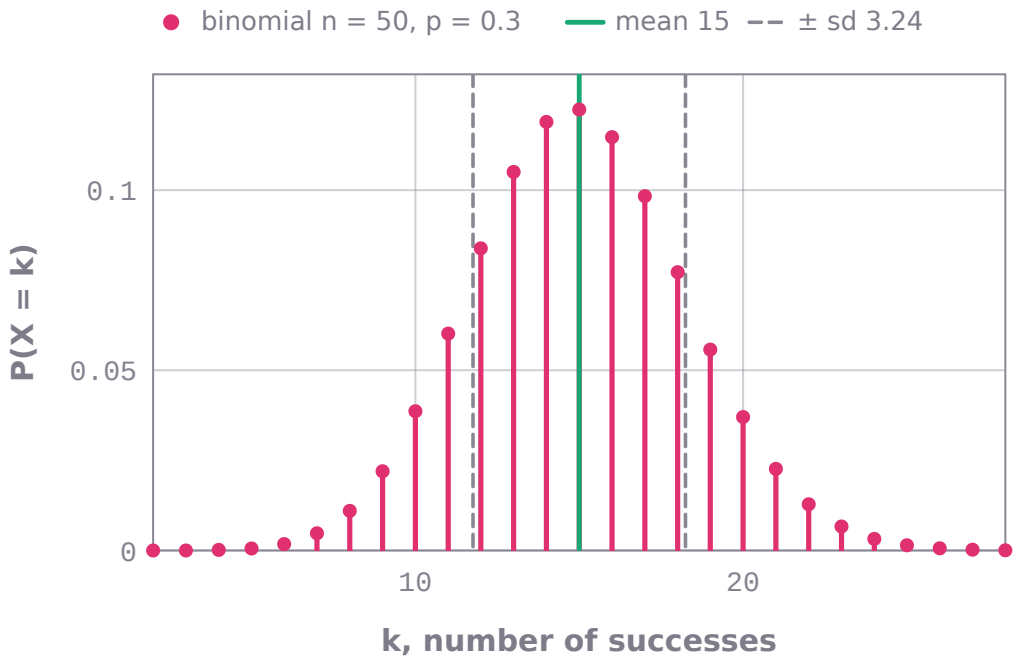
The mean np, variance np(1-p) and standard deviation of a binomial random variable.

$$\mu = np, \quad \sigma = \sqrt{np(1-p)}$$

input	meaning	condition
`n`	number of trials	
`p`	probability of success	

output	meaning
`mean`	n p
`variance`	n p $(1-p)$
`standard_deviation`	sqrt(n p $(1-p)$)

- **Method:** mean = np, variance = np(1 - p)
- **Assumes:** $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The binomial distribution with n = 50, p = 0.3 (probabilities from the package's binomial runs, k = 2 … 28); marked: the returned mean 15 and, dashed, one returned standard deviation 3.24037 either side (variance 10.5).

```
statistics_ai999x.solve('binomial_mean_sd', n=50, p=0.3)
# -> mean = 15, variance = 10.5, standard_deviation = 3.24037   (SOLVED)
```

```
statistics_ai999x.solve('binomial_mean_sd', n=12, p=0.5)
# -> mean = 6, variance = 3, standard_deviation = 1.73205   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the mean and standard deviation of a binomial distribution with n = 50 and p = 0.3.	SOLVED	mean = 15, variance = 10.5, standard_deviation = 3.24037
What is the expected value and variance of a binomial random variable with n = 12 trials and p = 0.5?	SOLVED	mean = 6, variance = 3, standard_deviation = 1.73205

question	outcome	result
Find the mean of a binomial distribution with n = 50 and p = 1.3.	INVALID_INPUT	Mean and standard deviation of a binomial distribution: p must be a probability strictly between 0 and 1.

Poisson probability: exactly k events, `poisson_exactly`

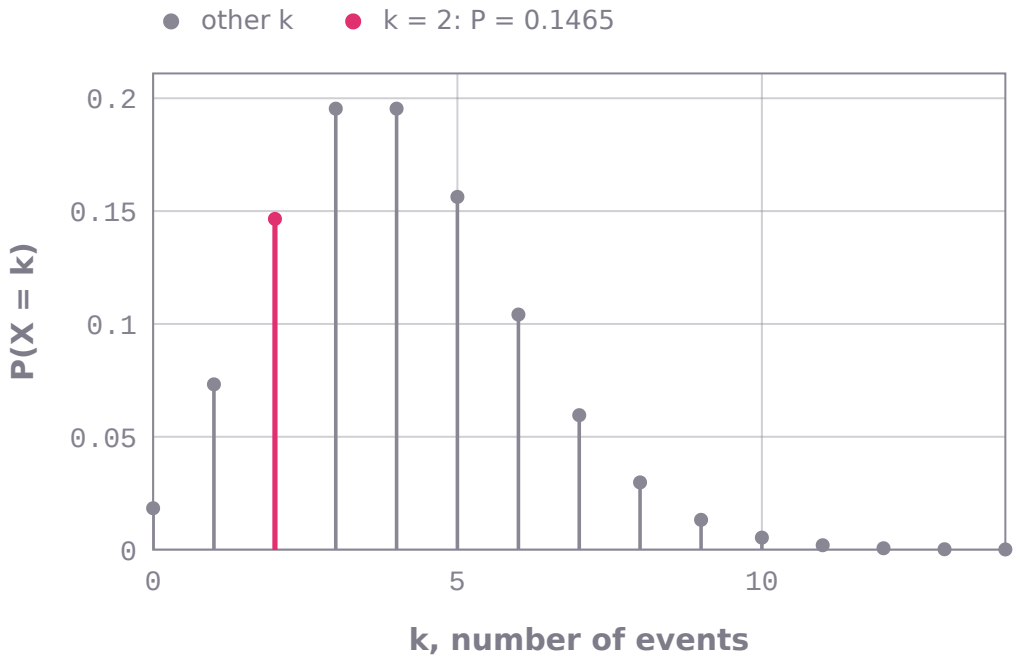
The probability of exactly k events when events occur independently at mean rate lambda per interval.

$$P(X = k) = \frac{e^{-\lambda}\lambda^{k}}{k!}$$

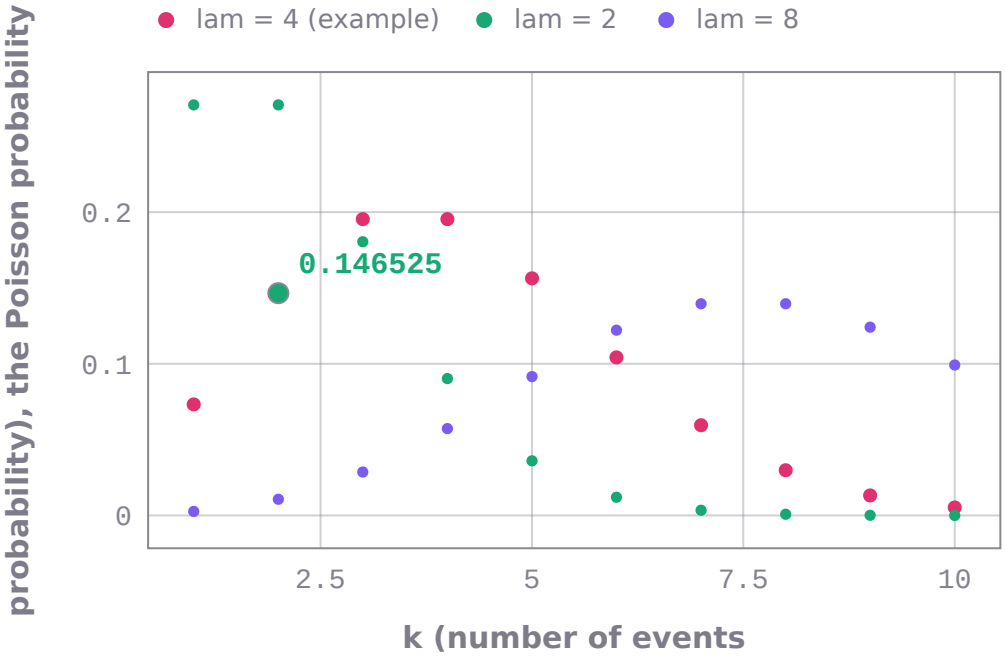
input	meaning	condition
`lam`	the mean number of events $(\lambda > 0)$	
`k`	number of events	

output	meaning
`probability`	the Poisson probability

- **Method:** Poisson probabilities summed term by term
- **Assumes:** events independent, at a constant mean rate; $\lambda > 0$, k a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Poisson distribution with mean λ = 4: P(X = k) for k = 0 … 14, each from the package; the stems with k = 2 are in the first colour and carry the returned probability 0.146525.



probability (the Poisson probability) against k (number of events) from 1 to 10. One curve for each value of lam (the mean number of events (lambda > 0)): 4, 2, 8. 10 points per curve computed by statistics-ai999x; marked: the worked example, k = 2, probability = 0.146525.

```
statistics_ai999x.solve('poisson_exactly', lam=4, k=2)
# -> probability = 0.146525   (SOLVED)
```

```
statistics_ai999x.solve('poisson_exactly', lam=3.5, k=6)
# -> probability = 0.0770983   (SOLVED)
```

```
statistics_ai999x.solve('poisson_exactly', lam=0.5, k=0)
# -> probability = 0.606531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a Poisson distribution with lambda = 4, find the probability of exactly k = 2 events.	SOLVED	probability = 0.146525
Calls arrive as a Poisson process with mean = 3.5 per hour; what is the probability of exactly k = 6 calls?	SOLVED	probability = 0.0770983
For a Poisson distribution with lambda = -4, find the probability of exactly k = 2 events.	INVALID_INPUT	Poisson probability: exactly k events: lambda must be a positive number.
For a Poisson distribution, find the probability of exactly k = 2 events.	NEEDS_INPUT	poisson_exactly needs lam (the mean number of events (lambda > 0)). Example: solve('poisson_exactly', lam=4, k=2).

Poisson probability: at most k events, `poisson_at_most`

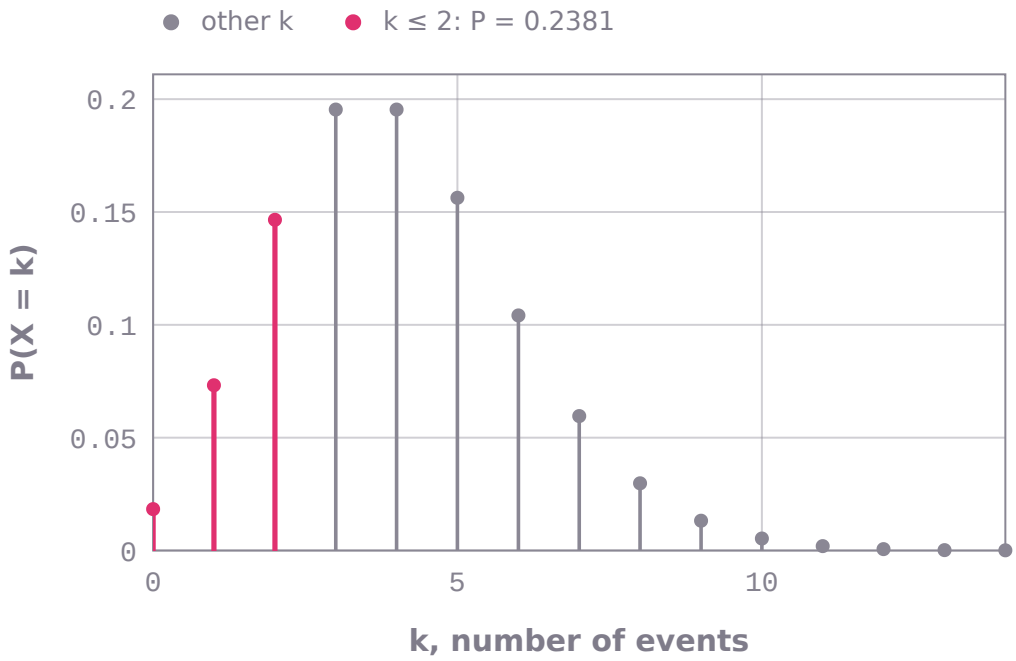
The probability of at most k events when events occur independently at mean rate lambda per interval.

$$P(X \le k) = \sum_{j=0}^{k} \frac{e^{-\lambda}\lambda^{j}}{j!}$$

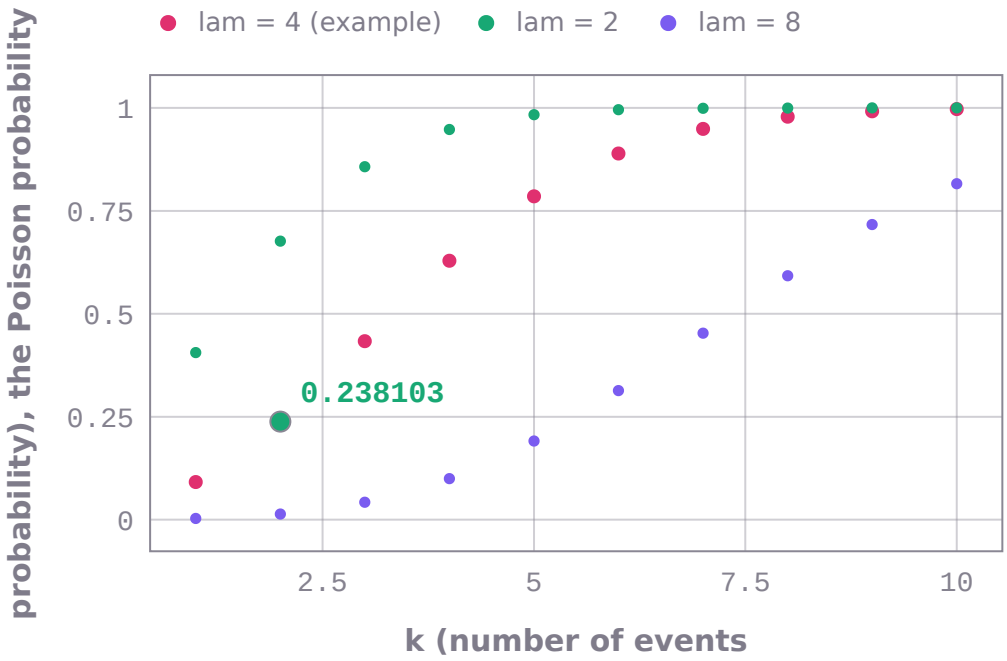
input	meaning	condition
`lam`	the mean number of events $(\lambda > 0)$	
`k`	number of events	

output	meaning
`probability`	the Poisson probability

- **Method:** Poisson probabilities summed term by term
- **Assumes:** events independent, at a constant mean rate; $\lambda > 0$, k a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Poisson distribution with mean λ = 4: P(X = k) for k = 0 … 14, each from the package; the stems with k ≤ 2 are in the first colour and carry the returned probability 0.238103.



probability (the Poisson probability) against k (number of events) from 1 to 10. One curve for each value of lam (the mean number of events (lambda > 0)): 4, 2, 8. 10 points per curve computed by statistics-ai999x; marked: the worked example, k = 2, probability = 0.238103.

```
statistics_ai999x.solve('poisson_at_most', lam=4, k=2)
# -> probability = 0.238103   (SOLVED)
```

```
statistics_ai999x.solve('poisson_at_most', lam=3.5, k=6)
# -> probability = 0.934712   (SOLVED)
```

```
statistics_ai999x.solve('poisson_at_most', lam=0.5, k=0)
# -> probability = 0.606531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a Poisson distribution with lambda = 4, find the probability of at most k = 2 events.	SOLVED	probability = 0.238103
Calls arrive as a Poisson process with mean = 3.5 per hour; what is the probability of at most k = 6 calls?	SOLVED	probability = 0.934712
For a Poisson distribution with lambda = -4, find the probability of at most k = 2 events.	INVALID_INPUT	Poisson probability: at most k events: lambda must be a positive number.
For a Poisson distribution, find the probability of at most k = 2 events.	NEEDS_INPUT	poisson_at_most needs lam (the mean number of events (lambda > 0)). Example: solve('poisson_at_most', lam=4, k=2).

Poisson probability: at least k events, `poisson_at_least`

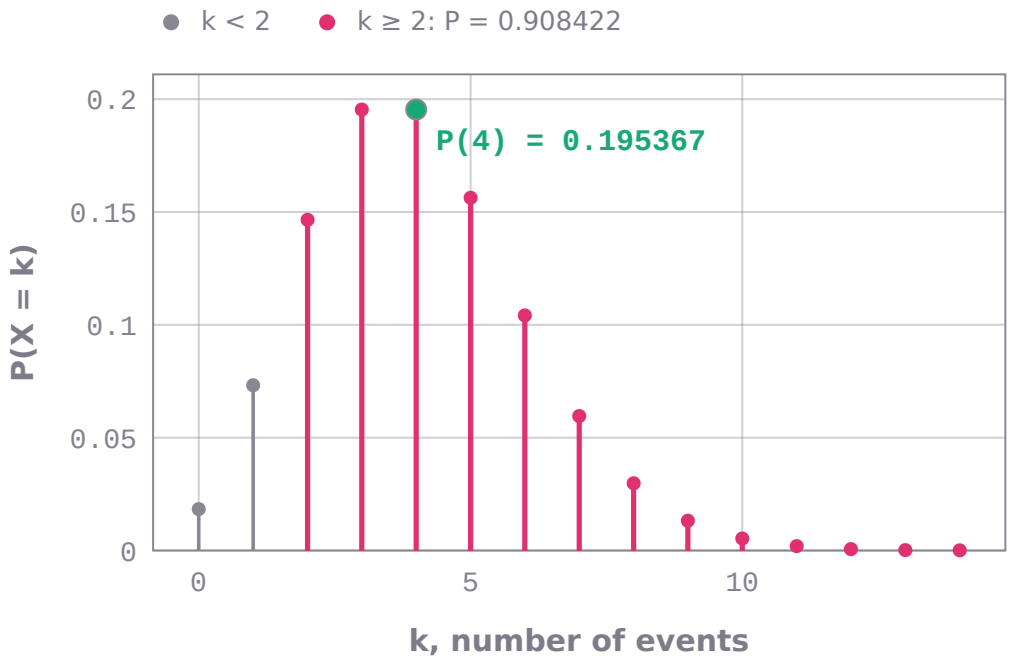
The probability of at least k events when events occur independently at mean rate lambda per interval.

$$P(X \geq k) = 1 - \sum_{j=0}^{k-1} \frac{e^{-\lambda}\lambda^{j}}{j!}$$

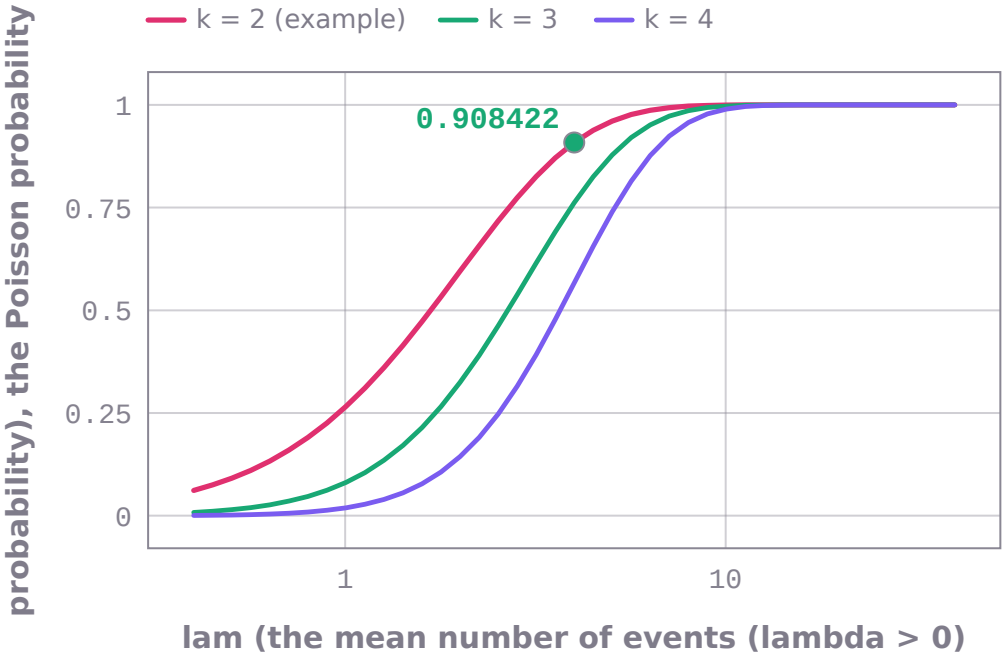
input	meaning	condition
`lam`	the mean number of events $(\lambda > 0)$	
`k`	number of events	

output	meaning
`probability`	the Poisson probability

- **Method:** Poisson probabilities summed term by term
- **Assumes:** events independent, at a constant mean rate; $\lambda > 0$, k a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Poisson distribution with mean λ = 4: P(X = k) for k = 0 … 14, each from the package; the stems with k ≥ 2 carry P(X ≥ 2) = 0.908422; marked: the most probable count k = 4, P = 0.195367.



probability (the Poisson probability) against lam (the mean number of events (lambda > 0)) from 0.4 to 40 on a logarithmic axis. One curve for each value of k (number of events): 2, 3, 4. 41 points per curve computed by statistics-ai999x; marked: the worked example, lam = 4, probability = 0.908422.

```
statistics_ai999x.solve('poisson_at_least', lam=4, k=2)
# -> probability = 0.908422   (SOLVED)
```

```
statistics_ai999x.solve('poisson_at_least', lam=3.5, k=6)
# -> probability = 0.142386   (SOLVED)
```

```
statistics_ai999x.solve('poisson_at_least', lam=0.5, k=0)
# -> probability = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a Poisson distribution with lambda = 4, find the probability of at least k = 2 events.	SOLVED	probability = 0.908422
Calls arrive as a Poisson process with mean = 3.5 per hour; what is the probability of at least k = 6 calls?	SOLVED	probability = 0.142386
For a Poisson distribution with lambda = -4, find the probability of at least k = 2 events.	INVALID_INPUT	Poisson probability: at least k events: lambda must be a positive number.
For a Poisson distribution, find the probability of at least k = 2 events.	NEEDS_INPUT	poisson_at_least needs lam (the mean number of events (lambda > 0)). Example: solve('poisson_at_least', lam=4, k=2).

Geometric distribution: first success on trial k, `geometric_probability`

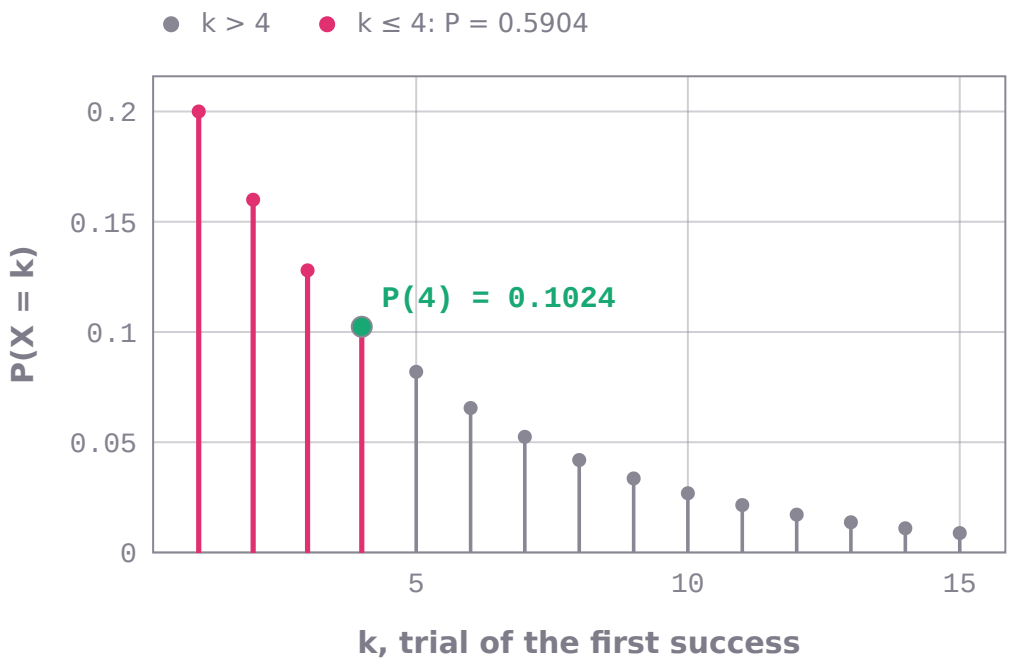
The probability that the first success occurs on trial k, and that it occurs on or before trial k.

$$P(X = k) = (1-p)^{k-1}p, \quad P(X \le k) = 1-(1-p)^k$$

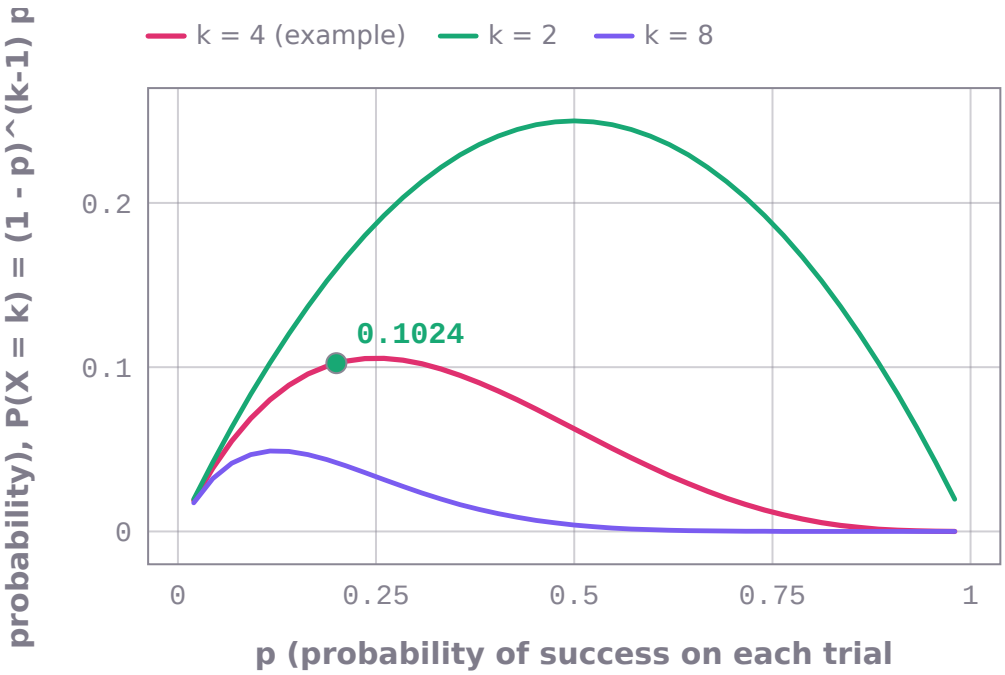
input	meaning	condition
`p`	probability of success on each trial	
`k`	the trial number (1, 2, ...)	

output	meaning
`probability`	P(X = k) = (1 - p)^(k-1) p
`cumulative`	P(X <= k) = 1 - (1 - p)^k

- **Method:** $(1-p)^{k-1}$ p, and the complement of k failures
- **Assumes:** independent trials, constant p; X counts trials up to and including the first success
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The geometric distribution with success probability p = 0.2: P(first success on trial k) for k = 1 … 15, each from the package; marked: k = 4, P = 0.1024; the stems k ≤ 4 carry the returned cumulative probability 0.5904.



probability (P(X = k) = (1 - p)^(k-1) p) against p (probability of success on each trial) from 0.02 to 0.98. One curve for each value of k (the trial number (1, 2, ...)): 4, 2, 8. 41 points per curve computed by statistics-ai999x; marked: the worked example, p = 0.2, probability = 0.1024.

```
statistics_ai999x.solve('geometric_probability', p=0.2, k=4)
# -> probability = 0.1024, cumulative = 0.5904   (SOLVED)
```

```
statistics_ai999x.solve('geometric_probability', p=0.5, k=3)
# -> probability = 0.125, cumulative = 0.875   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a geometric distribution with p = 0.2, find the probability that the first success is on trial k = 4.	SOLVED	probability = 0.1024, cumulative = 0.5904
What is the probability that the first success occurs on trial k = 3 when p = 0.5?	SOLVED	probability = 0.125, cumulative = 0.875
What is the probability that the first success occurs on trial k = 0 when p = 0.5?	INVALID_INPUT	k: must be at least 1, got 0.

Hypergeometric probability (sampling without replacement), `hypergeometric_probability`

The probability of exactly k successes, and of at most k, in n draws without replacement from a population of N items of which K are successes.

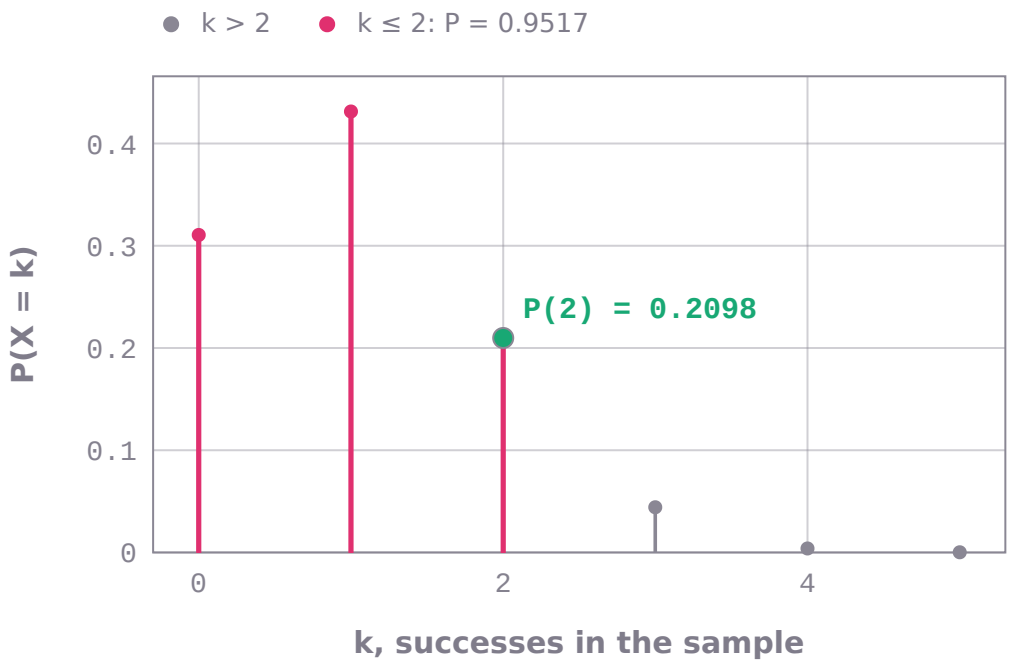
$$P(X = k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$$

input	meaning	condition
`N`	population size	
`K`	number of successes in the population	

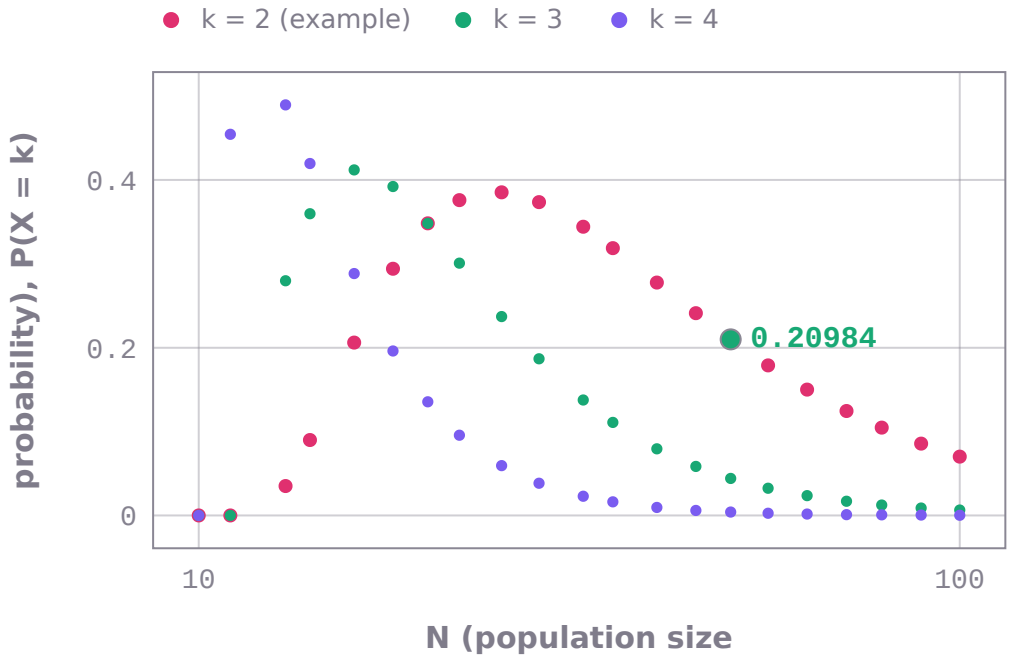
input	meaning	condition
`n`	number of draws	
`k`	number of successes drawn	

output	meaning
`probability`	P(X = k)
`cumulative`	P(X <= k)

- **Method:** exact rational arithmetic: C(K, k) C(N - K, n - k) / C(N, n)
- **Assumes:** draws without replacement; $K \leq N$, $n \leq N$, $k \leq n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The hypergeometric distribution: a sample of n = 10 drawn without replacement from N = 50 items of which K = 5 are successes; P(X = k) for every possible k, each from the package. Marked: k = 2, P = 0.20984; the stems k ≤ 2 carry the returned cumulative 0.95174.



probability (P(X = k)) against N (population size) from 1 to 100 on a logarithmic axis. One curve for each value of k (number of successes drawn): 2, 3, 4. Other inputs as in the worked example: K = 5, n = 10. 21 points per curve computed by statistics-ai999x; marked: the worked example, N = 50, probability = 0.20984.

```
statistics_ai999x.solve('hypergeometric_probability', N=50, K=5, n=10, k=2)
# -> probability = 0.20984, cumulative = 0.95174   (SOLVED)
```

```
statistics_ai999x.solve('hypergeometric_probability', N=20, K=8, n=5, k=3)
# -> probability = 0.23839, cumulative = 0.942208   (SOLVED)
```

```
statistics_ai999x.solve('hypergeometric_probability', N=52, K=4, n=5, k=0)
# -> probability = 0.658842, cumulative = 0.658842   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the hypergeometric probability with population size = 50, population successes = 5, draws = 10 and k = 2.	SOLVED	probability = 0.20984, cumulative = 0.95174
Drawing without replacement: population size = 20, population successes = 8, draws = 5; what is P(X = k) for k = 3?	SOLVED	probability = 0.23839, cumulative = 0.942208
Find the hypergeometric probability with population size = 10, population successes = 12, draws = 3 and k = 1.	INVALID_INPUT	Hypergeometric probability (sampling without replacement): the successes in the population (K) and the draws (n) cannot
Find the hypergeometric probability with population size = 50, draws = 10 and k = 2.	NEEDS_INPUT	hypergeometric_probability needs K (number of successes in the population). Example: solve('hypergeometric_probability',

Negative binomial: r-th success on trial n, `negative_binomial_probability`

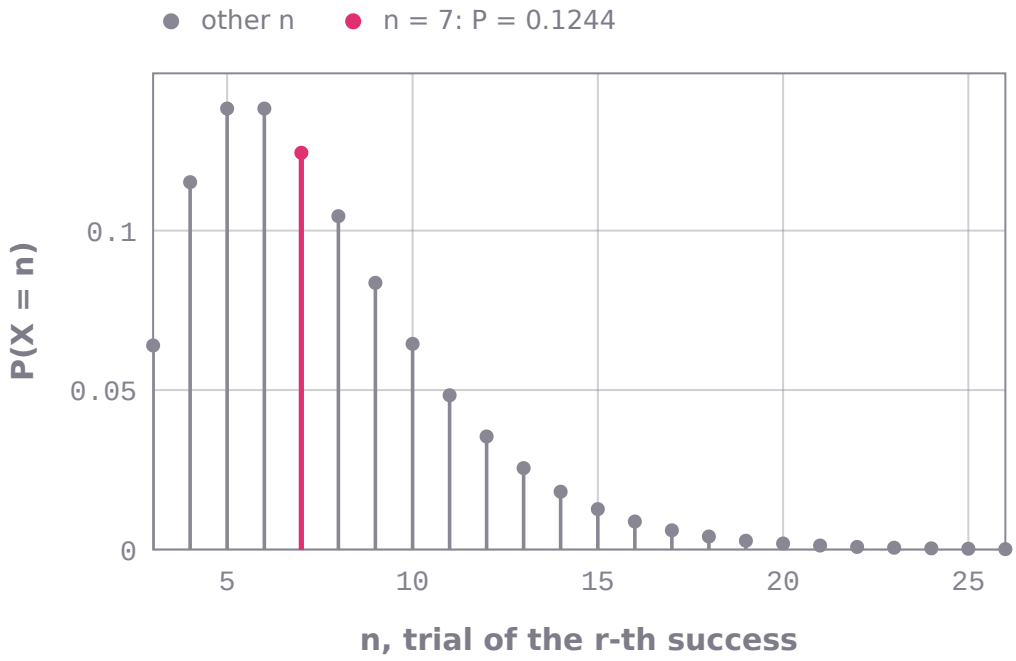
The probability that the r-th success occurs on trial n (independent trials with success probability p).

$$P(N = n) = \binom{n-1}{r-1} p^r (1-p)^{n-r}$$

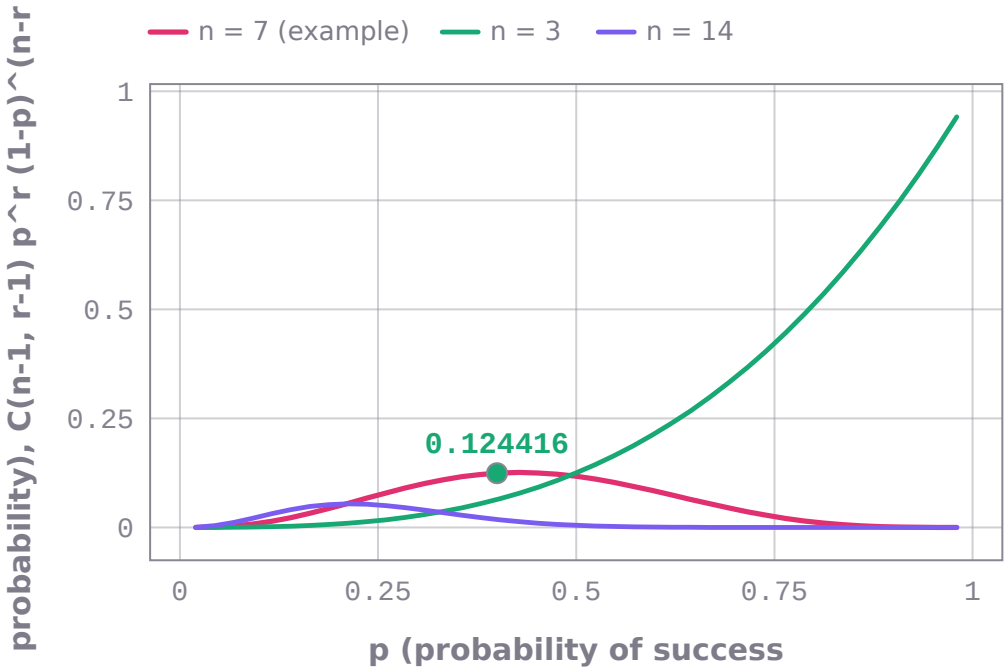
input	meaning	condition
`r`	number of successes required	
`p`	probability of success	
`n`	the trial number	

output	meaning
`probability`	C($n-1$, r-1) $p^r\ (1-p)^{n-r}$

- **Method:** C($n-1$, r - 1) $p^r\ (1-p)^{n-r}$
- **Assumes:** independent trials, constant p; $n \geq r$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The negative binomial distribution of the trial on which success number r = 3 occurs (p = 0.4), for n = 3 … 26, each from the package; the worked example n = 7 has the returned probability 0.124416.



probability (C(n-1, r-1) p^r (1-p)^(n-r)) against p (probability of success) from 0.02 to 0.98. One curve for each value of n (the trial number): 7, 3, 14. Other inputs as in the worked example: r = 3. 41 points per curve computed by statistics-ai999x; marked: the worked example, p = 0.4, probability = 0.124416.

```
statistics_ai999x.solve('negative_binomial_probability', r=3, p=0.4, n=7)
# -> probability = 0.124416   (SOLVED)
```

```
statistics_ai999x.solve('negative_binomial_probability', r=2, p=0.5, n=5)
# -> probability = 0.125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Negative binomial: find the probability that the r = 3 success occurs on trial n = 7 with p = 0.4.	SOLVED	probability = 0.124416
Using the negative binomial distribution with p = 0.5, what is the probability that success number r = 2 comes on trial n = 5?	SOLVED	probability = 0.125
Negative binomial: find the probability that the r = 6 success occurs on trial n = 4 with p = 0.4.	INVALID_INPUT	Negative binomial: r-th success on trial n: the r-th success cannot occur before trial r.

Normal probability below a value, `normal_probability_below`

P(X < x) for a normal distribution with mean mu and standard deviation sigma (the area to the left).

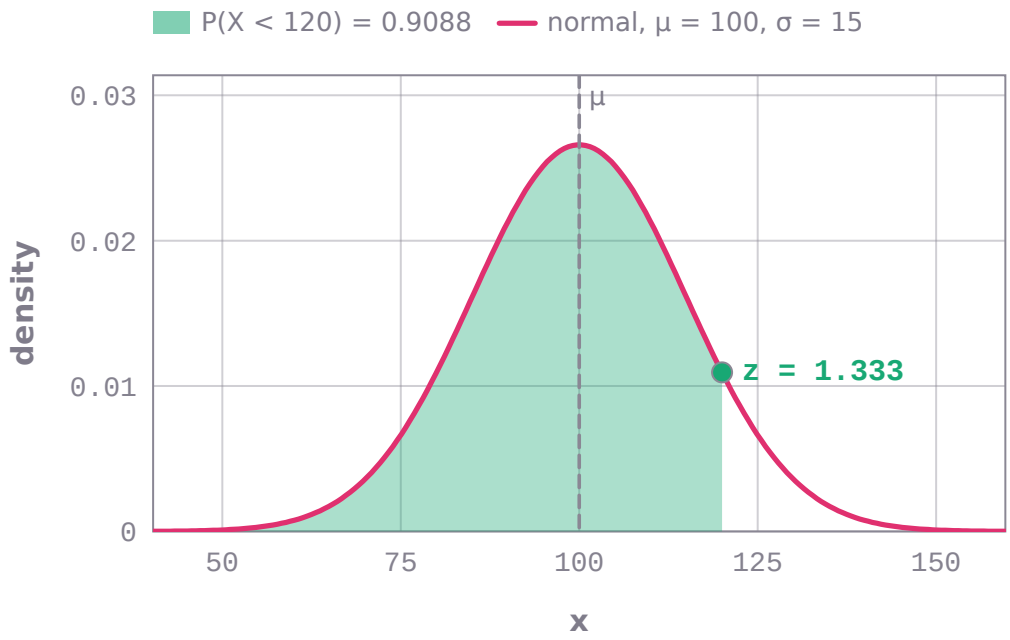
$$P(X < x) = \Phi\left(\frac{x - \mu}{\sigma}\right)$$

input	meaning	condition
`x`	the value	
`mu`	the mean	

input	meaning	condition
`sigma`	the standard deviation (> 0)	

output	meaning
`probability`	P(X < x)
`z`	the z-score of x

- **Method:** $z = (x - mu)/\sigma$; Phi(z) from the complementary error function
- **Assumes:** X normally distributed; $\sigma > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The normal density with μ = 100, σ = 15 (density from the package's cumulative probabilities on a grid, differenced); shaded: P(X < 120) = 0.908789 as returned; marked: x = 120, z = 1.33333.

```
statistics_ai999x.solve('normal_probability_below', x=120, mu=100, sigma=15)
# -> probability = 0.908789, z = 1.33333   (SOLVED)
```

```
statistics_ai999x.solve('normal_probability_below', x=160, mu=170, sigma=8)
# -> probability = 0.10565, z = -1.25   (SOLVED)
```

```
statistics_ai999x.solve('normal_probability_below', x=1.96, mu=0, sigma=1)
# -> probability = 0.975002, z = 1.96   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a normal distribution with mean = 100 and standard deviation = 15, find P(X < 120).	SOLVED	probability = 0.908789, z = 1.33333
Heights are normal with mean = 170 and sd = 8; what is the probability that a height is less than 160?	SOLVED	probability = 0.10565, z = -1.25

question	outcome	result
For the standard normal distribution, find P(Z < 1.96).	SOLVED	probability = 0.975002, z = 1.96
For a normal distribution with mean = 100 and standard deviation = -15, find P(X < 120).	INVALID_INPUT	Normal probability below a value: the standard deviation must be a positive number.
For a normal distribution with mean = 100, find P(X < 120).	NEEDS_INPUT	normal_probability_below needs sigma (the standard deviation (> 0)). Example: solve('normal_probability_below', x=120, m

Normal probability above a value, `normal_probability_above`

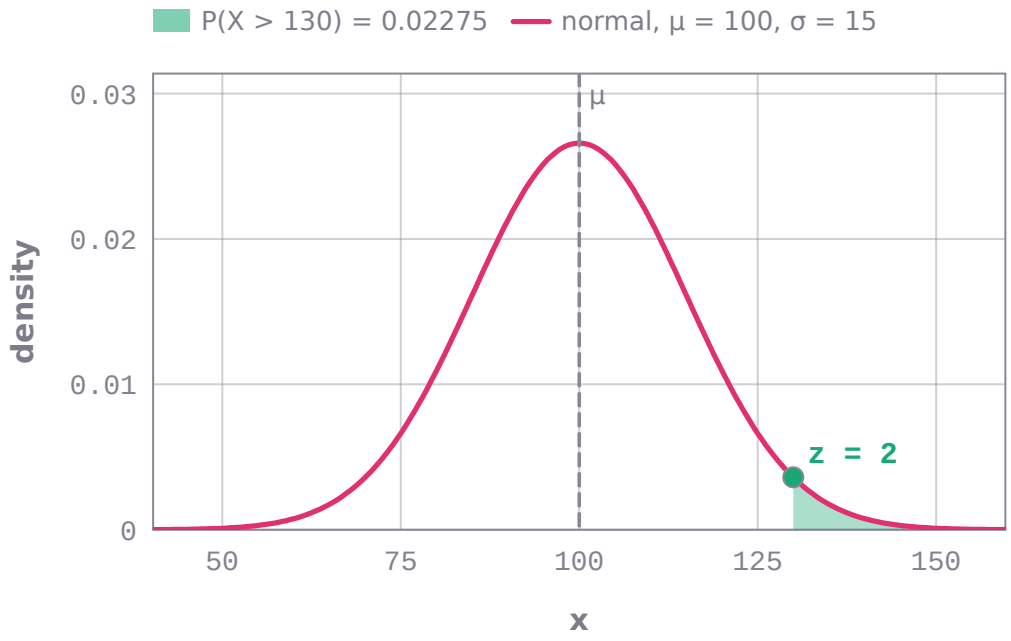
P(X > x) for a normal distribution with mean mu and standard deviation sigma (the area to the right).

$$P(X > x) = 1 - \Phi\left(\frac{x - \mu}{\sigma}\right)$$

input	meaning	condition
`x`	the value	
`mu`	the mean	
`sigma`	the standard deviation (> 0)	

output	meaning
`probability`	P(X > x)
`z`	the z-score of x

- **Method:** $z = (x - mu)/\sigma$; $1 - Phi$(z) computed as erfc(z/sqrt 2)/2
- **Assumes:** X normally distributed; $\sigma > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The normal density with μ = 100, σ = 15 (density from the package's cumulative probabilities on a grid, differenced); shaded: P(X > 130) = 0.0227501 as returned; marked: x = 130, z = 2.

```
statistics_ai999x.solve('normal_probability_above', x=130, mu=100, sigma=15)
# -> probability = 0.0227501, z = 2   (SOLVED)
```

```
statistics_ai999x.solve('normal_probability_above', x=650, mu=500, sigma=100)
# -> probability = 0.0668072, z = 1.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a normal distribution with mean = 100 and standard deviation = 15, find P(X > 130).	SOLVED	probability = 0.0227501, z = 2
Scores are normal with mean = 500 and sd = 100; what is the probability that a score is greater than 650?	SOLVED	probability = 0.0668072, z = 1.5
For the standard normal distribution, find P(Z > 2.5).	SOLVED	probability = 0.00620967, z = 2.5
For a normal distribution with sd = 15, find P(X > 130).	NEEDS_INPUT	normal_probability_above needs mu (the mean). Example: solve('normal_probability_above', x=130, mu=100, sigma=15).

Normal probability between two values, `normal_probability_between`

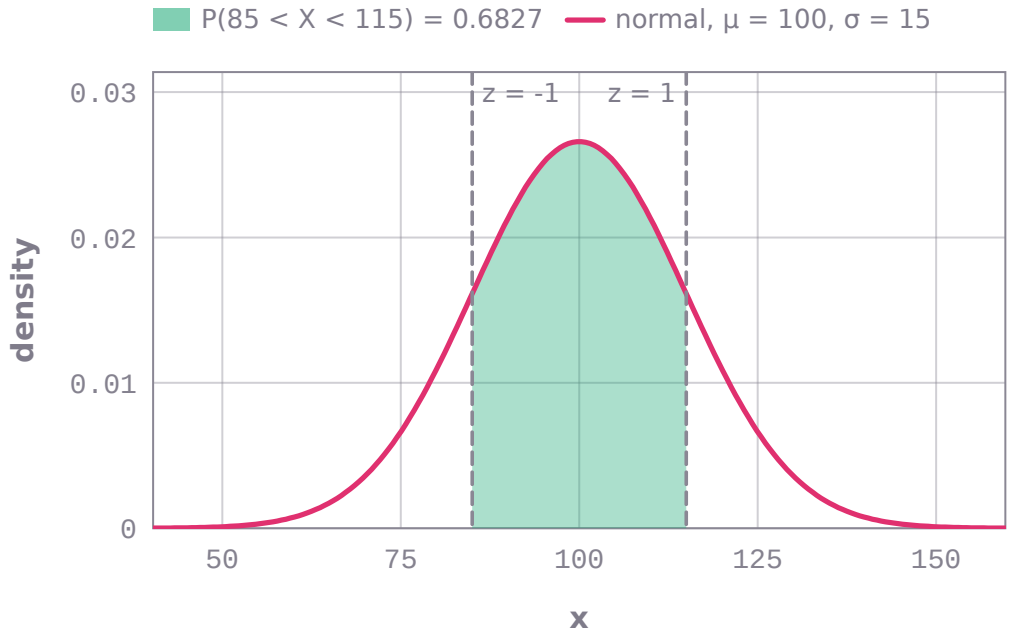
P(a < X < b) for a normal distribution with mean mu and standard deviation sigma.

$$P(a < X < b) = \Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)$$

input	meaning	condition
`a`	lower bound	
`b`	upper bound	
`mu`	the mean	
`sigma`	the standard deviation (> 0)	

output	meaning
`probability`	P(a < X < b)
`z_a`	z-score of a
`z_b`	z-score of b

- **Method:** Phi(z_b) - Phi(z_a), using upper tails when both z-scores are positive (no cancellation)
- **Assumes:** X normally distributed; $\sigma > 0$; $a < b$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The normal density with μ = 100, σ = 15 (density from the package's cumulative probabilities on a grid, differenced); shaded: P(85 < X < 115) = 0.682689 as returned, between z = -1 and z = 1.

```
statistics_ai999x.solve('normal_probability_between', a=85, b=115, mu=100, sigma=15)
# -> probability = 0.682689, z_a = -1, z_b = 1   (SOLVED)
```

```
statistics_ai999x.solve('normal_probability_between', a=90, b=130, mu=100, sigma=15)
# -> probability = 0.724757, z_a = -0.666667, z_b = 2   (SOLVED)
```

```
statistics_ai999x.solve('normal_probability_between', a=-1.96, b=1.96, mu=0, sigma=1)
# -> probability = 0.950004, z_a = -1.96, z_b = 1.96   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a normal distribution with mean = 100 and standard deviation = 15, find P(85 < X < 115).	SOLVED	probability = 0.682689, z_a = -1, z_b = 1
IQ is normal with mean = 100 and sd = 15; what is the probability that IQ is between 90 and 130?	SOLVED	probability = 0.724757, z_a = -0.666667, z_b = 2
For the standard normal distribution, find P(-1 < Z < 2).	SOLVED	probability = 0.818595, z_a = -1, z_b = 2
For a normal distribution with mean = 100 and standard deviation = 15, find the probability between 130 and 90.	INVALID_INPUT	Normal probability between two values: the lower bound a must be below the upper bound b.

Inverse normal (normal percentile), `inverse_normal`

The value x with P(X < x) = p for a normal distribution (percentiles, cut-offs, top and bottom percentages).

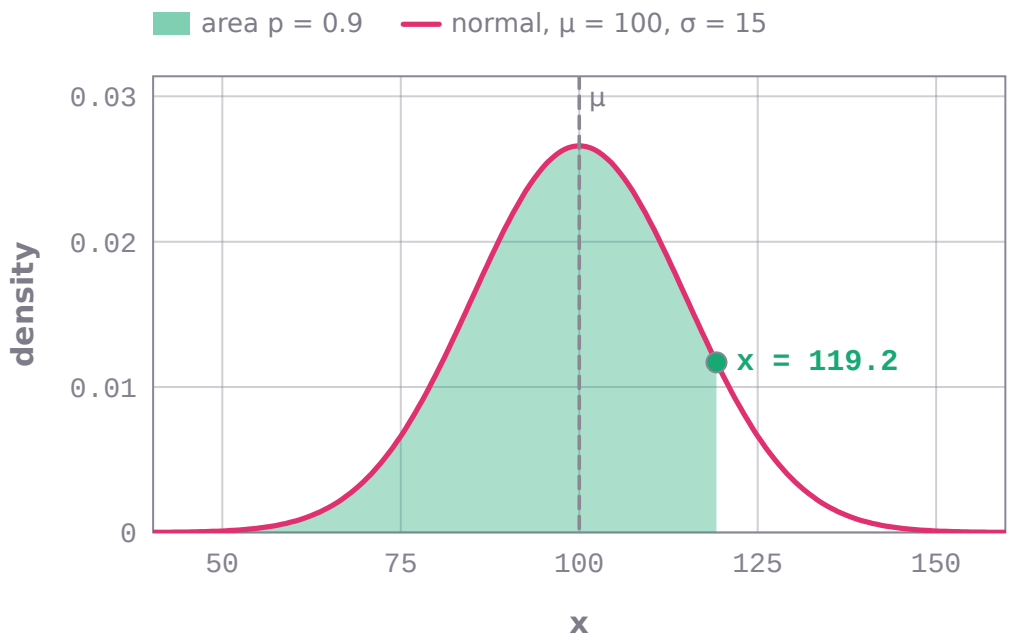
$$x = \mu + \sigma\,\Phi^{-1}(p)$$

input	meaning	condition
p	cumulative probability to the left of x $(0 < p < 1)$	

input	meaning	condition
`mu`	the mean	
`sigma`	the standard deviation (> 0)	

output	meaning
`x`	the value with P(X < x) = p
`z`	the standard normal quantile

- **Method:** standard normal quantile (rational start, Halley refinement on erfc), $x = mu + z$ sigma
- **Assumes:** $0 < p < 1$; $\sigma > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The value below which a normal variable (μ = 100, σ = 15) falls with probability p = 0.9: the returned x = 119.223 (z = 1.28155). Drawn: the density (density from the package's cumulative probabilities on a grid, differenced), the area p shaded.

```
statistics_ai999x.solve('inverse_normal', p=0.9, mu=100, sigma=15)
# -> x = 119.223, z = 1.28155   (SOLVED)
```

```
statistics_ai999x.solve('inverse_normal', p=0.95, mu=50, sigma=10)
# -> x = 66.4485, z = 1.64485   (SOLVED)
```

```
statistics_ai999x.solve('inverse_normal', p=0.975, mu=0, sigma=1)
# -> x = 1.95996, z = 1.95996   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 90th percentile of a normal distribution with mean = 100 and standard deviation = 15.	SOLVED	x = 119.223, z = 1.28155
For a normal distribution with mean = 50 and sd = 10, find x such that P(X < x) = 0.95.	SOLVED	x = 66.4485, z = 1.64485

question	outcome	result
Scores are normal with mean = 500 and sd = 100; what score marks the top 5%?	SOLVED	x = 664.485, z = 1.64485
Find the 100th percentile of a normal distribution with mean = 100 and standard deviation = 15.	INVALID_INPUT	Inverse normal (normal percentile): the cumulative probability must be a probability strictly between 0 and 1.

Critical value of z, `z_critical_value`

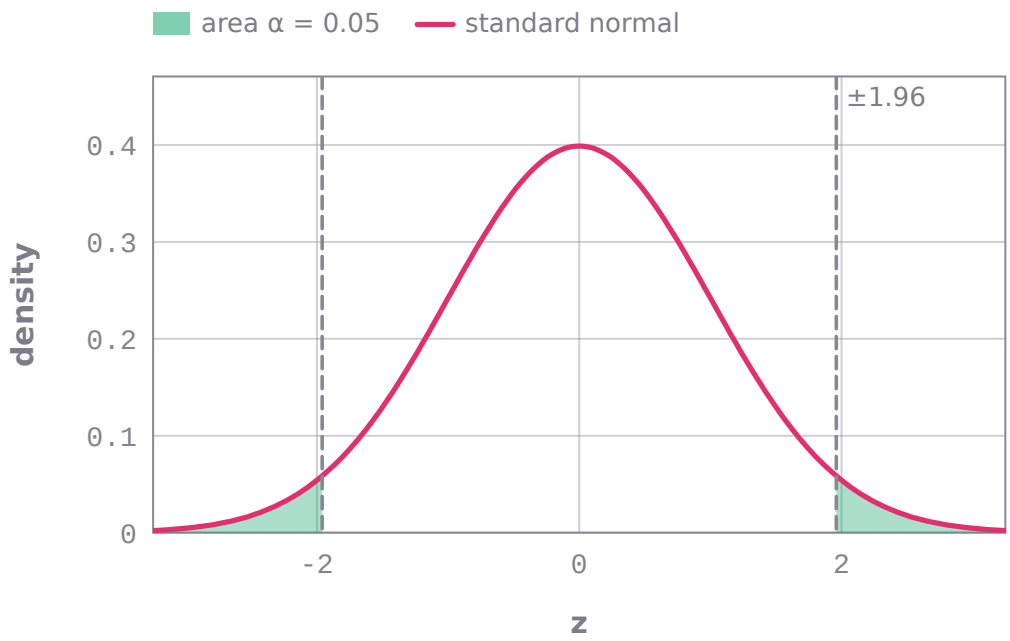
The standard normal critical value for a significance level (or confidence level), two-tailed or one-tailed.

$$z_{\alpha/2} = \Phi^{-1}(1 - \alpha/2)$$

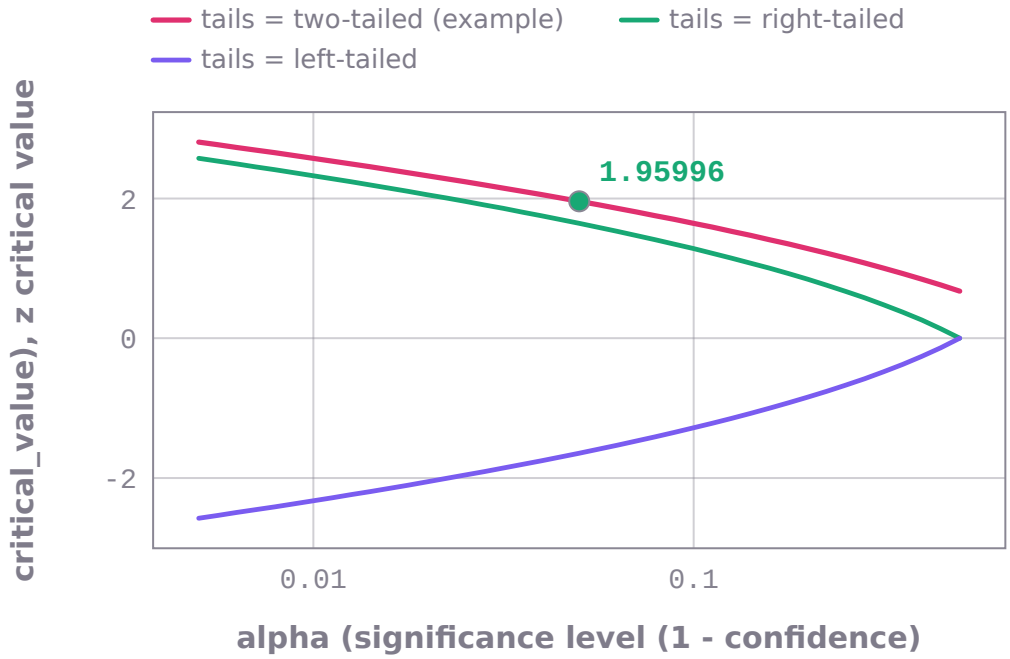
input	meaning	condition
`alpha`	significance level (1 - confidence)	
`tails`	two-tailed, right-tailed or left-tailed	

output	meaning
`critical_value`	z critical value (negative for a left-tailed test)

- **Method:** standard normal quantile at $1 - \alpha/2$ (two-tailed) or $1 - \alpha$ (one-tailed)
- **Assumes:** $0 < \alpha < 1$; a confidence level is read as two-tailed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The standard normal density (density from the package's quantiles at 121 cumulative probabilities, differenced, right-tailed critical values at 121 levels); shaded: the two-tailed area α = 0.05 beyond the returned critical value 1.95996.



critical_value (z critical value (negative for a left-tailed test)) against alpha (significance level (1 - confidence)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of tails (two-tailed, right-tailed or left-tailed): two-tailed, right-tailed, left-tailed. 41 points per curve computed by statistics-ai999x; marked: the worked example, alpha = 0.05, critical_value = 1.95996.

```
statistics_ai999x.solve('z_critical_value', alpha=0.05, tails='two-tailed')
# -> critical_value = 1.95996   (SOLVED)
```

```
statistics_ai999x.solve('z_critical_value', alpha=0.05, tails='right-tailed')
# -> critical_value = 1.64485   (SOLVED)
```

```
statistics_ai999x.solve('z_critical_value', alpha=0.01, tails='left-tailed')
# -> critical_value = -2.32635   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the critical z value for a 95% confidence interval.	SOLVED	critical_value = 1.95996
What is the critical z value for alpha = 0.05, right-tailed?	SOLVED	critical_value = 1.64485
What is the critical z value for alpha = 0.01, two-tailed?	SOLVED	critical_value = 2.57583
What is the critical z value for alpha = 0.05?	NEEDS_INPUT	z_critical_value needs tails (two-tailed, right-tailed or left-tailed). Example: solve('z_critical_value', alpha=0.05, t

Critical value of Student's t, `t_critical_value`

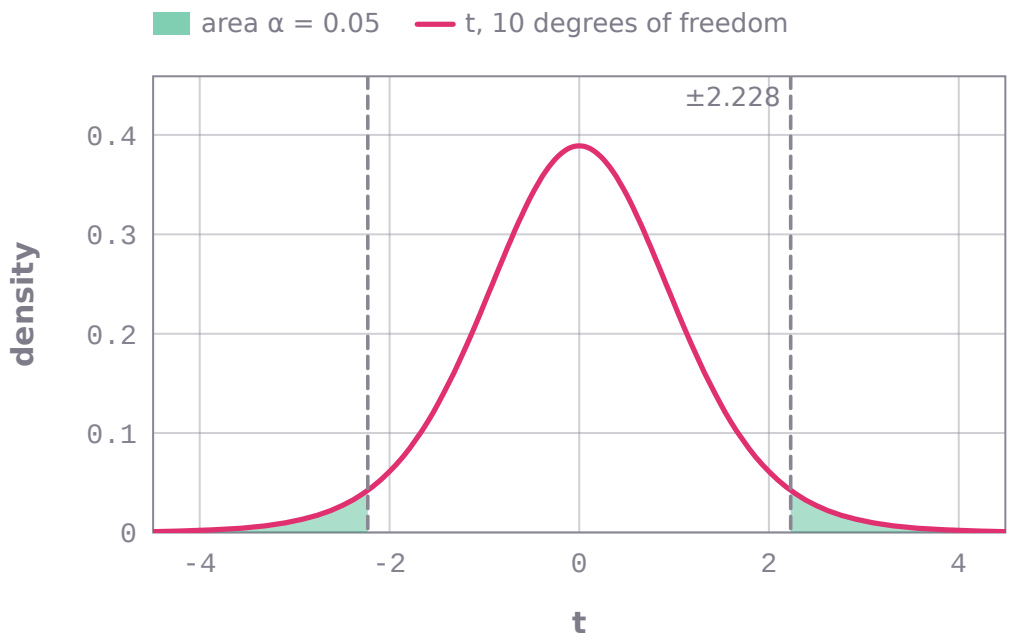
The Student t critical value for given degrees of freedom and significance (or confidence) level.

$$t_{\alpha/2,\,\nu}:\ P(T_\nu > t) = \alpha/2$$

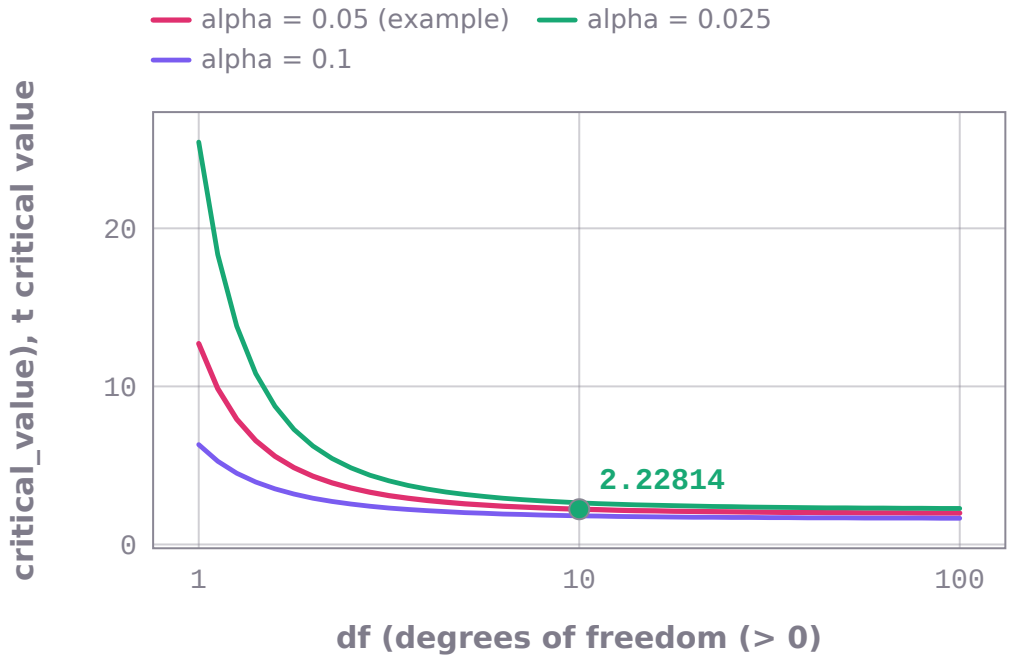
input	meaning	condition
`df`	degrees of freedom (> 0)	
`alpha`	significance level (1 - confidence)	
`tails`	two-tailed, right-tailed or left-tailed	

output	meaning
`critical_value`	t critical value (negative for a left-tailed test)

- **Method:** t quantile by bisection on the t distribution (regularized incomplete beta function)
- **Assumes:** $df > 0$; $0 < \alpha < 1$; a confidence level is read as two-tailed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The t density with df = 10 (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the two-tailed area α = 0.05 beyond the returned critical value 2.22814.



critical_value (t critical value (negative for a left-tailed test)) against df (degrees of freedom (> 0)) from 1 to 100 on a logarithmic axis. One curve for each value of alpha (significance level (1 - confidence)): 0.05, 0.025, 0.1. Other inputs as in the worked example: tails = two-tailed. 41 points per curve computed by statistics-ai999x; marked: the worked example, df = 10, critical_value = 2.22814.

```
statistics_ai999x.solve('t_critical_value', df=10, alpha=0.05, tails='two-tailed')
# -> critical_value = 2.22814   (SOLVED)
```

```
statistics_ai999x.solve('t_critical_value', df=20, alpha=0.05, tails='right-tailed')
# -> critical_value = 1.72472   (SOLVED)
```

```
statistics_ai999x.solve('t_critical_value', df=1, alpha=0.01, tails='two-tailed')
# -> critical_value = 63.6567   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the critical t value for a 95% confidence interval with df = 10.	SOLVED	critical_value = 2.22814
What is the critical t value with df = 20 for alpha = 0.05, right-tailed?	SOLVED	critical_value = 1.72472
Find the critical t value for alpha = 0.01, two-tailed, with df = 1.	SOLVED	critical_value = 63.6567
Find the critical t value for a 95% confidence interval.	NEEDS_INPUT	t_critical_value needs df (degrees of freedom (> 0)). Example: solve('t_critical_value', df=10, alpha=0.05, tails='two-t

Critical value of chi-square, `chi_square_critical_value`

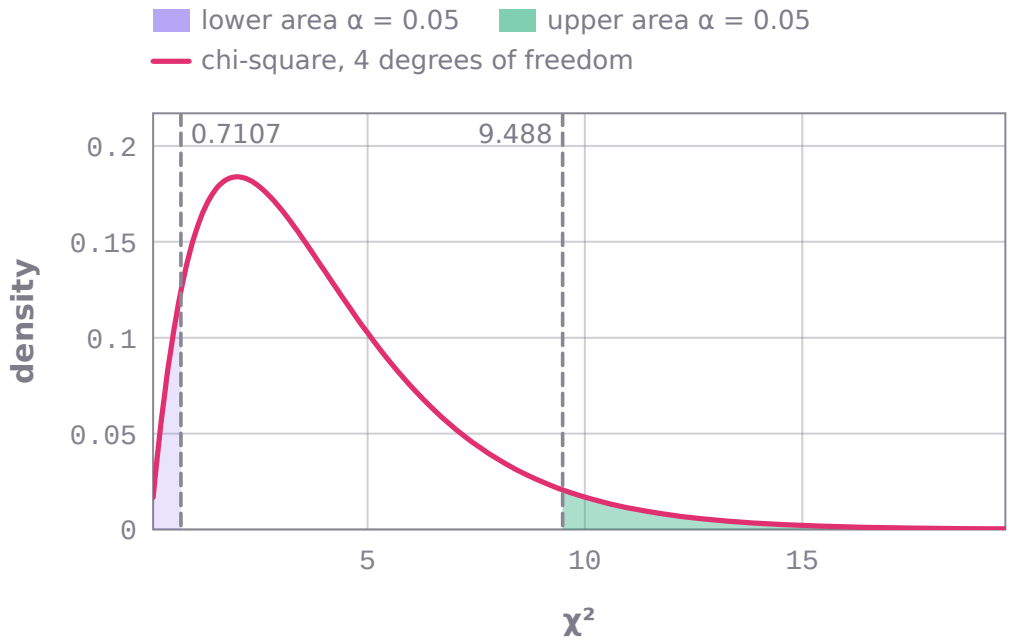
The right-tail chi-square critical value (P(X > c) = alpha) for given degrees of freedom, and the left-tail value (P(X < c) = alpha).

$$\chi^2_{\alpha,\nu}: \; P(\chi^2_\nu > c) = \alpha$$

input	meaning	condition
`df`	degrees of freedom (> 0)	
`alpha`	tail area	

output	meaning
`critical_value`	right-tail critical value
`left_critical_value`	left-tail critical value

- **Method:** chi-square quantiles by bisection on the regularized incomplete gamma function
- **Assumes:** $df > 0$; $0 < \alpha < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The chi-square density with df = 4 (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the upper area α = 0.05 beyond the returned critical value 9.48773 and, paler, the lower area α below the returned left critical value 0.710723.



critical_value (right-tail critical value) against df (degrees of freedom (> 0)) from 2 to 6. One curve for each value of alpha (tail area): 0.05, 0.025, 0.1. 31 points per curve computed by statistics-ai999x; marked: the worked example, df = 4, critical_value = 9.48773.

```
statistics_ai999x.solve('chi_square_critical_value', df=4, alpha=0.05)
# -> critical_value = 9.48773, left_critical_value = 0.710723   (SOLVED)
```

```
statistics_ai999x.solve('chi_square_critical_value', df=10, alpha=0.01)
# -> critical_value = 23.2093, left_critical_value = 2.55821   (SOLVED)
```

```
statistics_ai999x.solve('chi_square_critical_value', df=1, alpha=0.05)
# -> critical_value = 3.84146, left_critical_value = 0.00393214   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the chi-square critical value with df = 4 and alpha = 0.05.	SOLVED	critical_value = 9.48773, left_critical_value = 0.710723
What is the critical value of chi-square for degrees of freedom = 10 at alpha = 0.01?	SOLVED	critical_value = 23.2093, left_critical_value = 2.55821
Find the chi-square critical value with df = 4.	NEEDS_INPUT	chi_square_critical_value needs alpha (tail area). Example: solve('chi_square_critical_value', df=4, alpha=0.05).

Critical value of F, `f_critical_value`

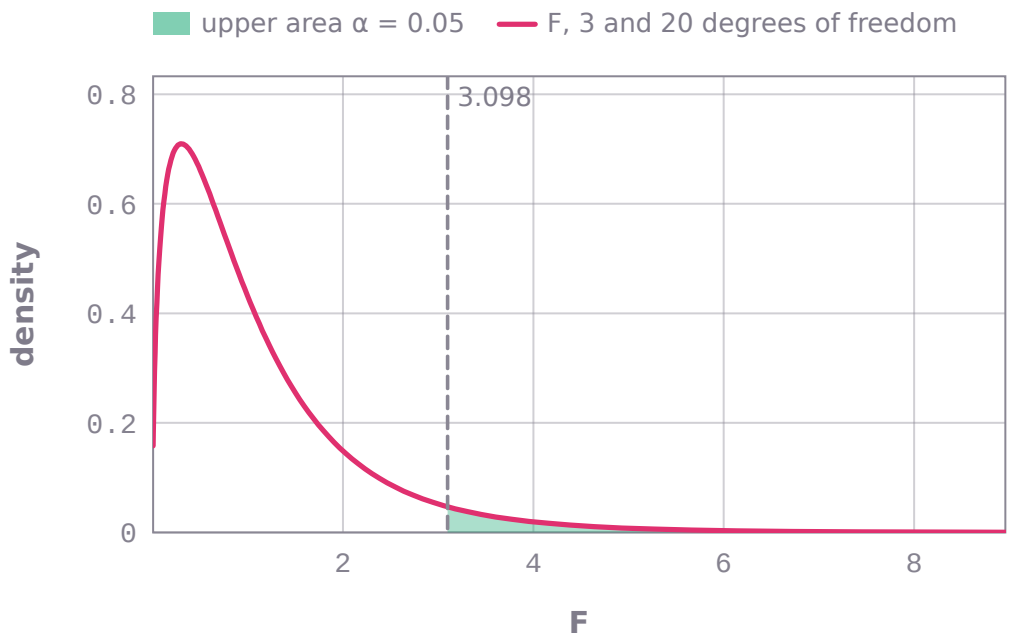
The right-tail F critical value (P(F > c) = alpha) for numerator and denominator degrees of freedom.

$$F_{\alpha;\,\nu_1,\nu_2}:\ P(F_{\nu_1,\nu_2} > c) = \alpha$$

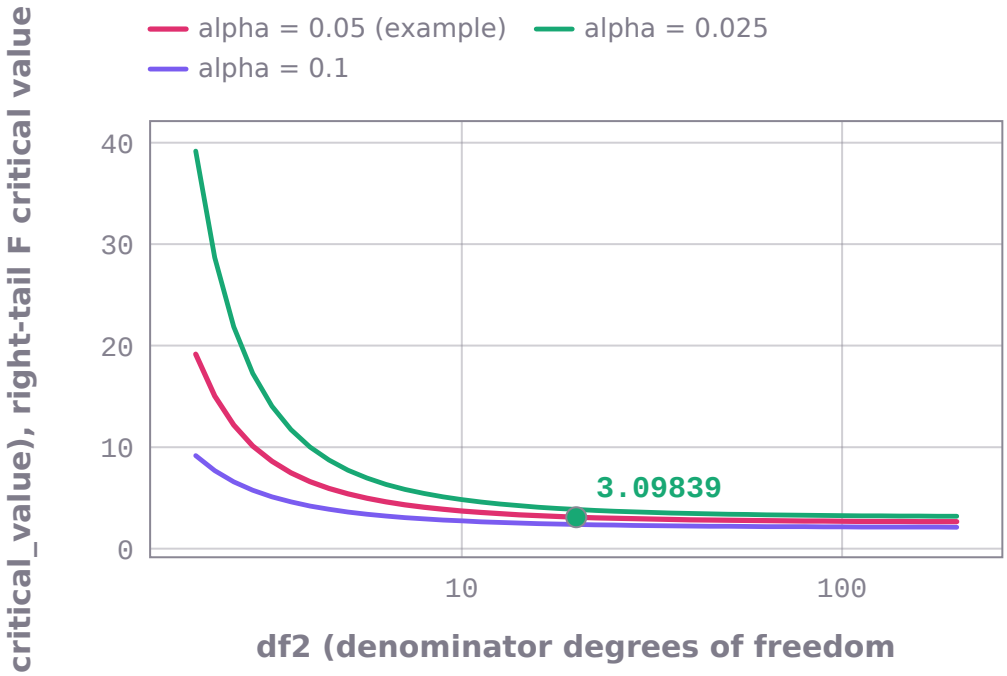
input	meaning	condition
`df1`	numerator degrees of freedom	
`df2`	denominator degrees of freedom	
`alpha`	tail area	

output	meaning
`critical_value`	right-tail F critical value

- **Method:** F quantile by bisection on the regularized incomplete beta function
- **Assumes:** df1, $df2 > 0$; $0 < \alpha < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The F density with 3 and 20 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the upper area α = 0.05 beyond the returned critical value 3.09839.



critical_value (right-tail F critical value) against df2 (denominator degrees of freedom) from 2 to 200 on a logarithmic axis. One curve for each value of alpha (tail area): 0.05, 0.025, 0.1. Other inputs as in the worked example: df1 = 3.0. 41 points per curve computed by statistics-ai999x; marked: the worked example, df2 = 20, critical_value = 3.09839.

```
statistics_ai999x.solve('f_critical_value', df1=3, df2=20, alpha=0.05)
# -> critical_value = 3.09839   (SOLVED)
```

```
statistics_ai999x.solve('f_critical_value', df1=5, df2=10, alpha=0.01)
# -> critical_value = 5.63633   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the F critical value with df1 = 3, df2 = 20 and alpha = 0.05.	SOLVED	critical_value = 3.09839
What is the critical F value for numerator df = 5 and denominator df = 10 at alpha = 0.01?	SOLVED	critical_value = 5.63633
Find the F critical value with df1 = 3 and alpha = 0.05.	NEEDS_INPUT	f_critical_value needs df2 (denominator degrees of freedom). Example: solve('f_critical_value', df1=3, df2=20, alpha=0.0

Exponential probability below a value, `exponential_probability_below`

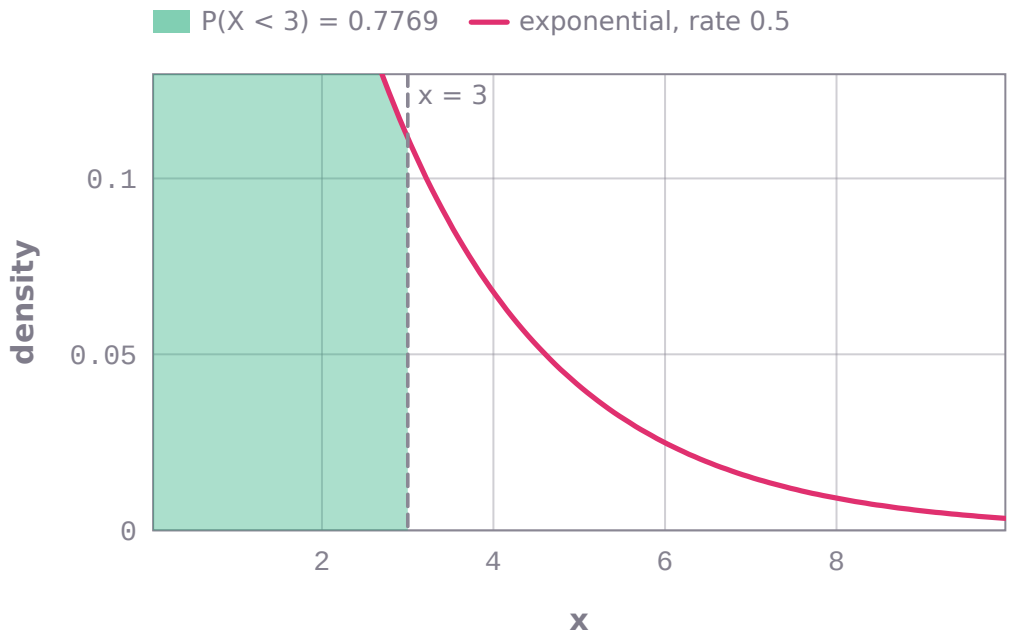
P(X < x) for an exponential distribution given its rate lambda or its mean.

$$P(X < x) = 1 - e^{-\lambda x}$$

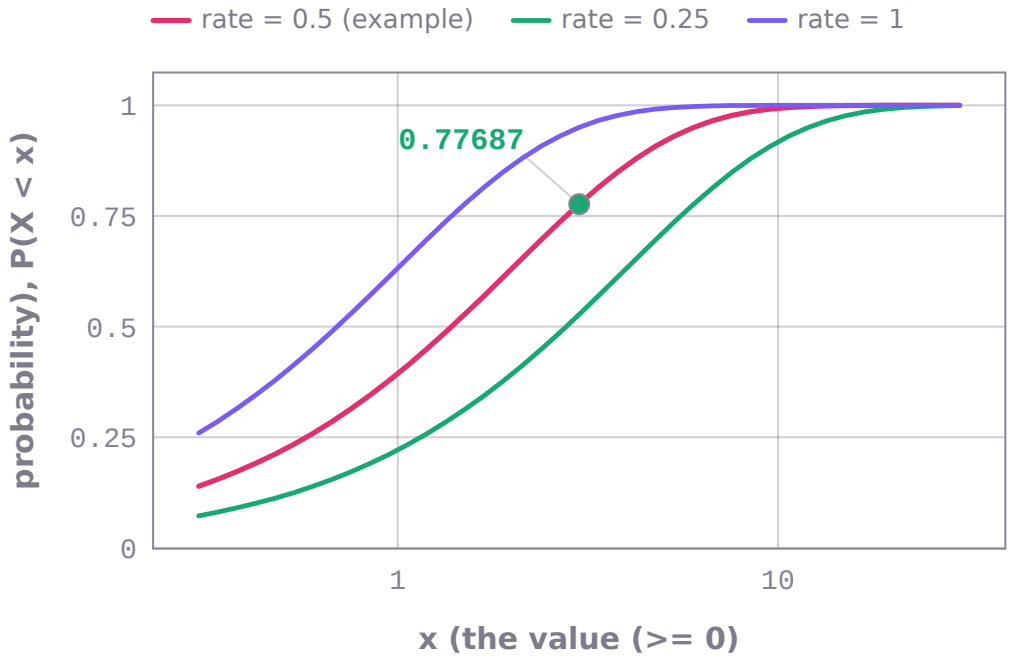
input	meaning	condition
`x`	the value (>= 0)	
`rate`	rate lambda (events per unit)	
`mean`	mean $1/\lambda$ (instead of the rate)	

output	meaning
`probability`	P(X < x)

- **Method:** $1 - exp$(-lambda x) or exp(-lambda x)
- **Assumes:** exponential (memoryless) waiting time; $x \geq 0$; exactly one of rate or mean
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The exponential density with rate 0.5 (density from the package's cumulative probabilities on a grid, differenced); shaded: P(X < 3) = 0.77687 as returned.



probability (P(X < x)) against x (the value (>= 0)) from 0.3 to 30 on a logarithmic axis. One curve for each value of rate (rate lambda (events per unit)): 0.5, 0.25, 1. 41 points per curve computed by statistics-ai999x; marked: the worked example, x = 3, probability = 0.77687.

```
statistics_ai999x.solve('exponential_probability_below', x=3, rate=0.5)
# -> probability = 0.77687   (SOLVED)
```

```
statistics_ai999x.solve('exponential_probability_below', x=5, mean=10)
# -> probability = 0.393469   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For an exponential distribution with rate = 0.5, find P(X < 3).	SOLVED	probability = 0.77687
Waiting times are exponential with mean = 10 minutes; what is the probability of waiting less than 5?	SOLVED	probability = 0.393469
For an exponential distribution with rate = -2, find P(X < 3).	INVALID_INPUT	Exponential probability below a value: the rate must be a positive number.

Exponential probability above a value, `exponential_probability_above`

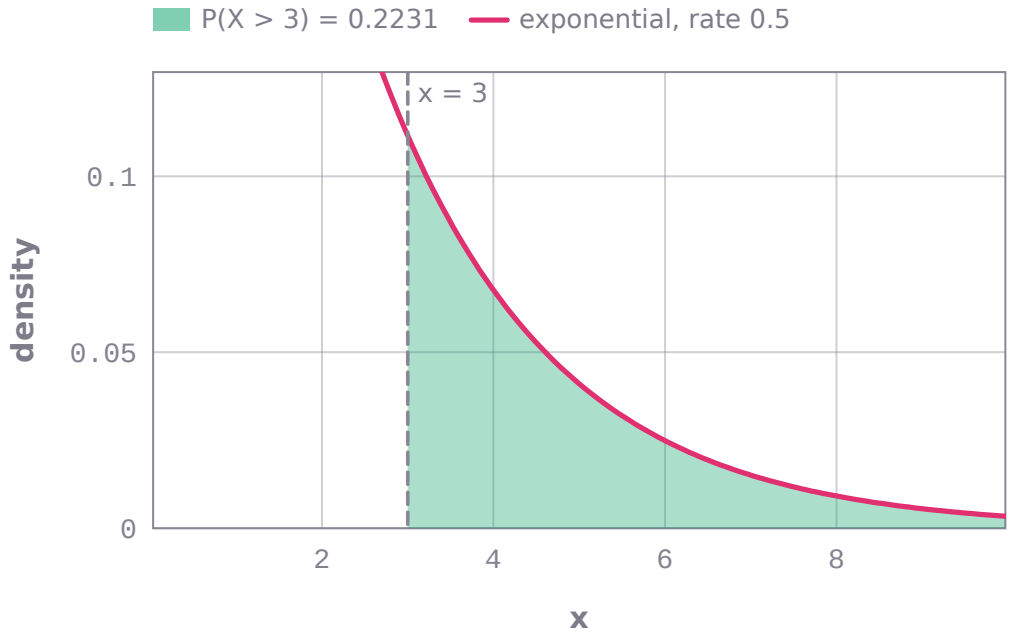
P(X > x) for an exponential distribution given its rate lambda or its mean.

$$P(X > x) = e^{-\lambda x}$$

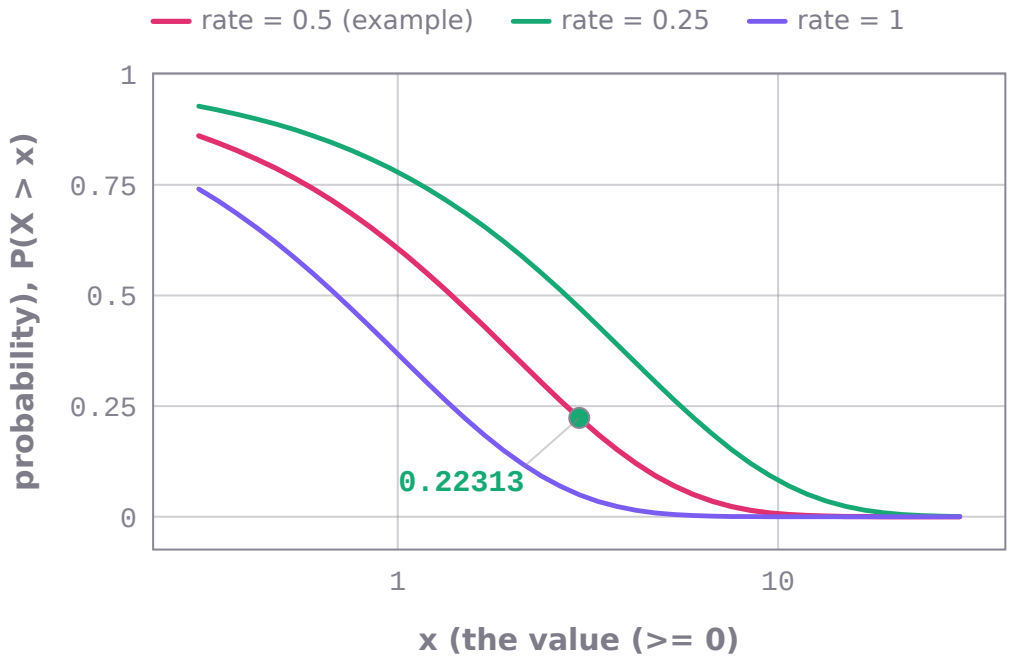
input	meaning	condition
`x`	the value (>= 0)	
`rate`	rate lambda (events per unit)	
`mean`	mean $1/\lambda$ (instead of the rate)	

output	meaning
`probability`	P(X > x)

- **Method:** $1 - exp$(-lambda x) or exp(-lambda x)
- **Assumes:** exponential (memoryless) waiting time; $x \geq 0$; exactly one of rate or mean
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The exponential density with rate 0.5 (density from the package's cumulative probabilities on a grid, differenced); shaded: P(X > 3) = 0.22313 as returned.



probability (P(X > x)) against x (the value (>= 0)) from 0.3 to 30 on a logarithmic axis. One curve for each value of rate (rate lambda (events per unit)): 0.5, 0.25, 1. 41 points per curve computed by statistics-ai999x; marked: the worked example, x = 3, probability = 0.22313.

```
statistics_ai999x.solve('exponential_probability_above', x=3, rate=0.5)
# -> probability = 0.22313   (SOLVED)
```

```
statistics_ai999x.solve('exponential_probability_above', x=5, mean=10)
# -> probability = 0.606531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For an exponential distribution with rate = 0.5, find P(X > 3).	SOLVED	probability = 0.22313
Waiting times are exponential with mean = 10 minutes; what is the probability of waiting more than 5?	SOLVED	probability = 0.606531
For an exponential distribution with rate = -2, find P(X > 3).	INVALID_INPUT	Exponential probability above a value: the rate must be a positive number.

Probability that a sample mean is below a value, `sample_mean_probability_below`

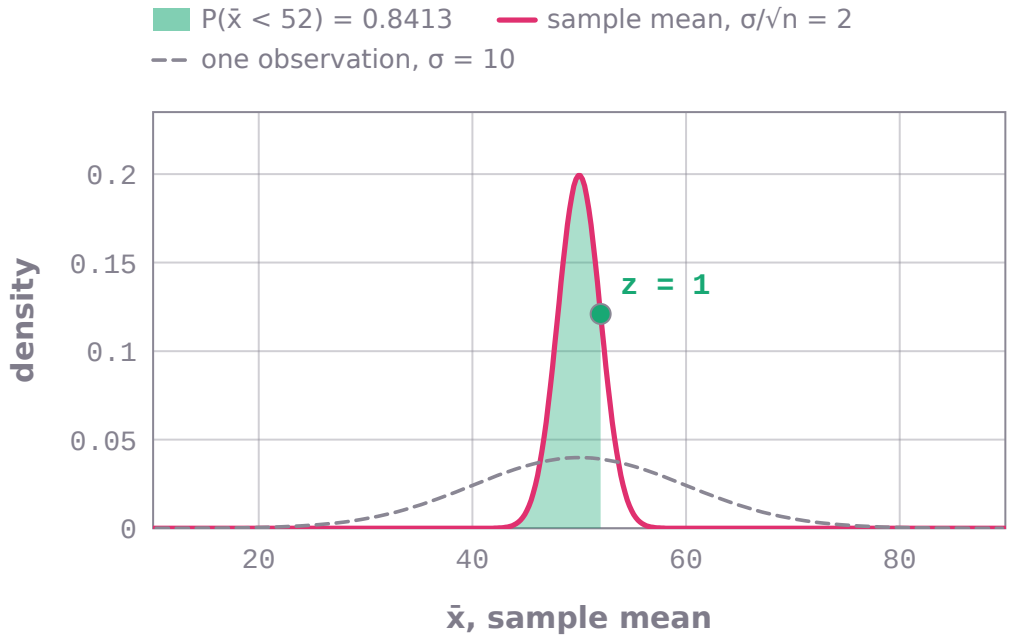
P(sample mean < x) for samples of size n from a population with mean mu and standard deviation sigma (central limit theorem).

$$P(\bar{X} < x) = \Phi\left(\frac{x - \mu}{\sigma/\sqrt{n}}\right)$$

input	meaning	condition
`x`	the value for the sample mean	
`mu`	population mean	
`sigma`	population standard deviation	
`n`	sample size	

output	meaning
`probability`	the probability
`z`	$z = \frac{x-\mu}{\sigma\frac{1}{\sqrt{n}}}$
`standard_error`	sigma/sqrt(n)

- **Method:** standard error sigma/sqrt(n), then the normal probability of the z-score
- **Assumes:** population normal, or n large enough for the central limit theorem; independent observations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The distribution of the mean of n = 25 observations (μ = 50, σ = 10): normal with the returned standard error 2, drawn from the package's normal cumulative probabilities (differenced), beside the wider density of one observation (dashed). Shaded: P(x̄ < 52) = 0.841345; marked: z = 1.

```
statistics_ai999x.solve('sample_mean_probability_below', x=52, mu=50, sigma=10, n=25)
# -> probability = 0.841345, z = 1, standard_error = 2   (SOLVED)
```

```
statistics_ai999x.solve('sample_mean_probability_below', x=97, mu=100, sigma=15, n=36)
# -> probability = 0.11507, z = -1.2, standard_error = 2.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A population has mean = 50 and standard deviation = 10; for a sample of size n = 25, find the probability that the sample mean is less than 52.	SOLVED	probability = 0.841345, z = 1, standard_error = 2
With mu = 100, sigma = 15 and n = 36, what is the probability that the sample mean is below 97?	SOLVED	probability = 0.11507, z = -1.2, standard_error = 2.5
With mu = 100 and sigma = 15, what is the probability that the sample mean is below 97?	NEEDS_INPUT	sample_mean_probability_below needs n (sample size). Example: solve('sample_mean_probability_below', x=52, mu=50, sigma=

Probability that a sample mean is above a value, `sample_mean_probability_above`

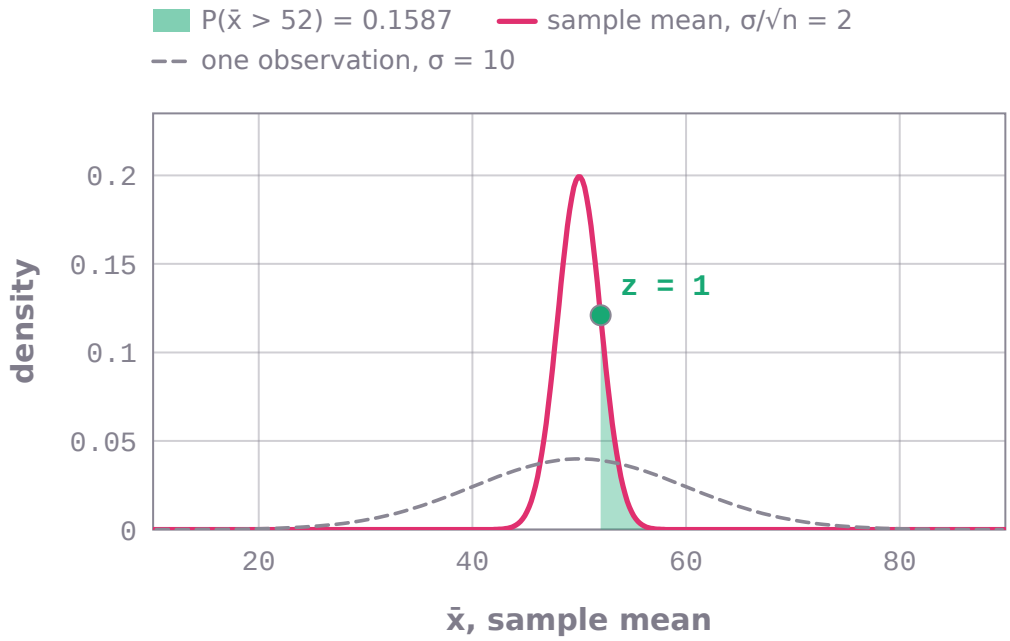
P(sample mean > x) for samples of size n from a population with mean mu and standard deviation sigma (central limit theorem).

$$P(\bar{X} < x) = \Phi\left(\frac{x - \mu}{\sigma/\sqrt{n}}\right)$$

input	meaning	condition
`x`	the value for the sample mean	
`mu`	population mean	
`sigma`	population standard deviation	
`n`	sample size	

output	meaning
`probability`	the probability
`z`	$z = \frac{x-\mu}{\sigma\frac{1}{\sqrt{n}}}$
`standard_error`	sigma/sqrt(n)

- **Method:** standard error sigma/sqrt(n), then the normal probability of the z-score
- **Assumes:** population normal, or n large enough for the central limit theorem; independent observations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The distribution of the mean of n = 25 observations (μ = 50, σ = 10): normal with the returned standard error 2, drawn from the package's normal cumulative probabilities (differenced), beside the wider density of one observation (dashed). Shaded: P(x̄ > 52) = 0.158655; marked: z = 1.

```
statistics_ai999x.solve('sample_mean_probability_above', x=52, mu=50, sigma=10, n=25)
# -> probability = 0.158655, z = 1, standard_error = 2   (SOLVED)
```

```
statistics_ai999x.solve('sample_mean_probability_above', x=97, mu=100, sigma=15, n=36)
# -> probability = 0.88493, z = -1.2, standard_error = 2.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A population has mean = 50 and standard deviation = 10; for a sample of size n = 25, find the probability that the sample mean is greater than 52.	SOLVED	probability = 0.158655, z = 1, standard_error = 2
With mu = 100, sigma = 15 and n = 36, what is the probability that the sample mean is above 97?	SOLVED	probability = 0.88493, z = -1.2, standard_error = 2.5
With mu = 100 and sigma = 15, what is the probability that the sample mean is above 97?	NEEDS_INPUT	sample_mean_probability_above needs n (sample size). Example: solve('sample_mean_probability_above', x=52, mu=50, sigma=

Confidence interval for a mean (sigma known, z-interval), `ci_mean_known_sigma`

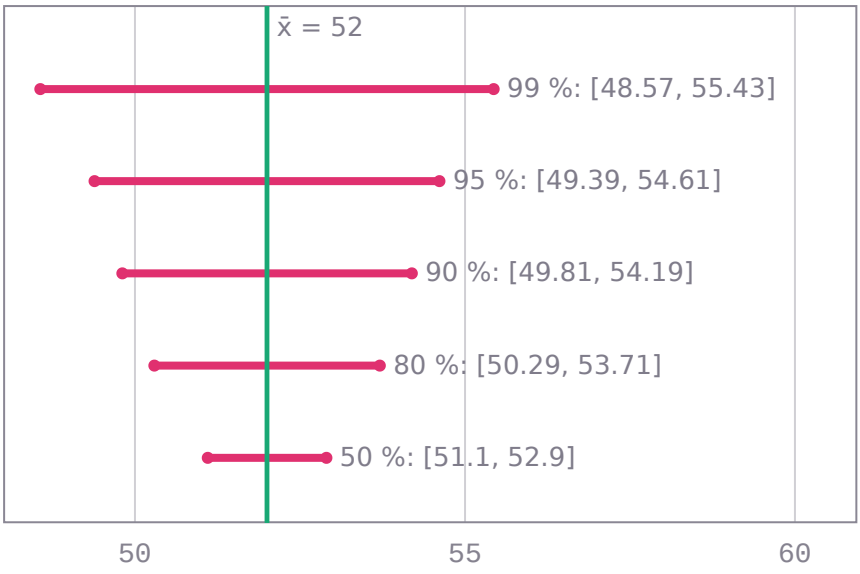
The z confidence interval for a population mean when the population standard deviation sigma is known.

$$\bar{x} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$$

input	meaning	condition
`xbar`	sample mean	
`sigma`	population standard deviation (known)	
`n`	sample size	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`margin_of_error`	z sigma / sqrt(n)
`critical_value`	z critical value

- **Method:** xbar +/- z_(alpha/2) sigma / sqrt(n)
- **Assumes:** sigma known; normal population or large n; random sample
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



z confidence intervals for a mean with known σ = 8 (x̄ = 52, n = 36) at five levels, each from the package: 50 % [51.1007, 52.8993]; 80 % [50.2913, 53.7087]; 90 % [49.8069, 54.1931]; 95 % [49.3867, 54.6133]; 99 % [48.5656, 55.4344]. Higher confidence, wider interval.

```
statistics_ai999x.solve('ci_mean_known_sigma', xbar=52, sigma=8, n=36, confidence=0.95)
# -> lower = 49.3867, upper = 54.6133, margin_of_error = 2.61329, critical_value = 1.95996
(SOLVED)
```

```
statistics_ai999x.solve('ci_mean_known_sigma', xbar=120, sigma=15, n=50, confidence=0.99)
# -> lower = 114.536, upper = 125.464, margin_of_error = 5.46416, critical_value = 2.57583
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for the mean with sample mean = 52, population standard deviation = 8 and n = 36.	SOLVED	lower = 49.3867, upper = 54.6133, margin_of_error = 2.61329, critical_value = 1.95996
Construct a 99% confidence interval for mu with x-bar = 120, sigma = 15, n = 50.	SOLVED	lower = 114.536, upper = 125.464, margin_of_error = 5.46416, critical_value = 2.57583
Find the 95% confidence interval for the mean with sample mean = 52 and sigma = 8.	NEEDS_INPUT	ci_mean_known_sigma needs n (sample size). Example: solve('ci_mean_known_sigma', xbar=52, sigma=8, n=36, confidence=0.95
Find the 150% confidence interval for the mean with sample mean = 52, sigma = 8, n = 36.	INVALID_INPUT	Confidence interval for a mean (sigma known, z-interval): the confidence level must be strictly between 0 and 1 (for exa

Confidence interval for a mean from data (t-interval), `ci_mean_t`

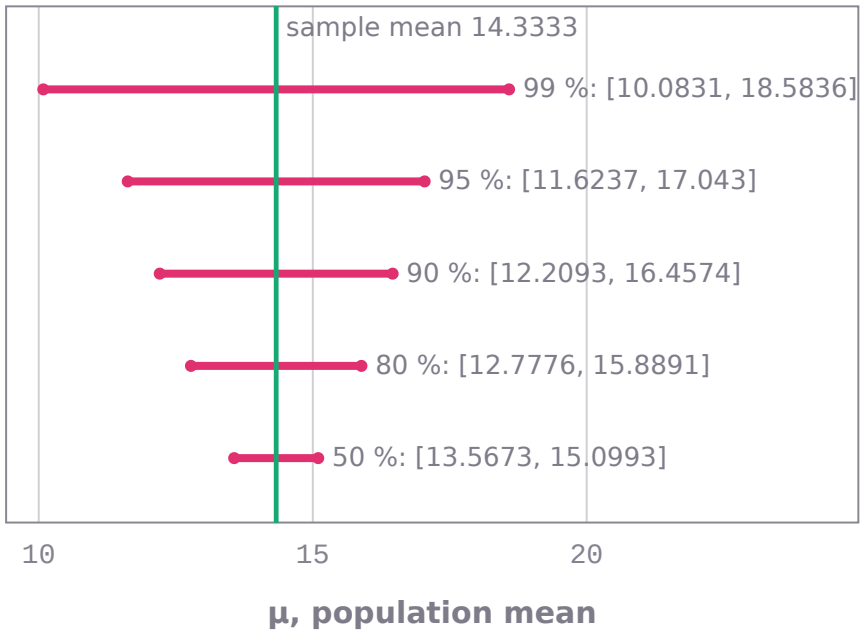
The t confidence interval for a population mean from sample data (sigma unknown).

$$\bar{x} \pm t_{\alpha/2,\,n-1}\frac{s}{\sqrt{n}}$$

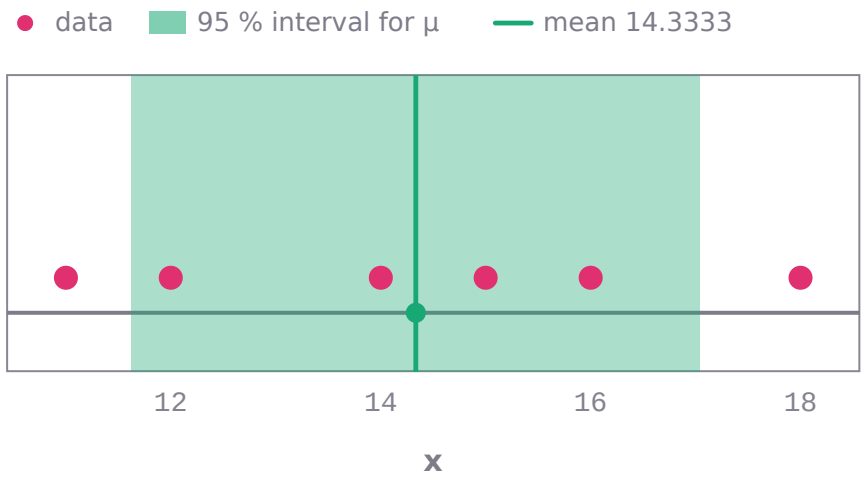
input	meaning	condition
`x`	the data values (at least two)	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`mean`	sample mean
`margin_of_error`	t s / sqrt(n)
`critical_value`	t critical value
`df`	$n - 1$

- **Method:** mean +/- t_($\alpha/2$, n-1) s / sqrt(n)
- **Assumes:** random sample from a roughly normal population (or large n); sigma unknown
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t confidence intervals for the mean of x = 12, 15, 11, 18, 14, 16 at 5 confidence levels, each from the package: 50 % [13.5673, 15.0993]; 80 % [12.7776, 15.8891]; 90 % [12.2093, 16.4574]; 95 % [11.6237, 17.043]; 99 % [10.0831, 18.5836] (sample mean 14.3333, 5 degrees of freedom); higher confidence, wider interval.



The sample x = 12, 15, 11, 18, 14, 16 with the returned 95 % t interval for the population mean shaded: [11.6237, 17.043] around the mean 14.3333 (margin 2.70963, t critical value 2.57058, 5 degrees of freedom). The interval covers where μ plausibly is, not where the data are.

```
statistics_ai999x.solve('ci_mean_t', x='12, 15, 11, 18, 14, 16', confidence=0.95)
# -> lower = 11.6237, upper = 17.043, margin_of_error = 2.70963, critical_value = 2.57058,
df = 5, mean = 14.3333   (SOLVED)
```

```
statistics_ai999x.solve('ci_mean_t', x=[4.8, 5.1, 5.3, 4.9, 5.0], confidence=0.9)
# -> lower = 4.83661, upper = 5.20339, margin_of_error = 0.183388, critical_value =
2.13185, df = 4, mean = 5.02   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for the mean of [12, 15, 11, 18, 14, 16].	SOLVED	lower = 11.6237, upper = 17.043, margin_of_error = 2.70963, critical_value = 2.57058, df = 5, mean = 14.3333
Construct a 90% confidence interval for the population mean from the sample [4.8, 5.1, 5.3, 4.9, 5.0].	SOLVED	lower = 4.83661, upper = 5.20339, margin_of_error = 0.183388, critical_value = 2.13185, df = 4, mean = 5.02
Find the 95% confidence interval for the mean of [12].	INVALID_INPUT	x: needs at least 2 entries, got 1.
Find the confidence interval for the mean of [12, 15, 11, 18].	NEEDS_INPUT	ci_mean_t needs confidence (confidence level as a fraction (0.95 for 95%)). Example: solve('ci_mean_t', x='12, 15, 11, 1

Confidence interval for a mean from summary statistics (t-interval), `ci_mean_t_summary`

The t confidence interval for a mean from the sample mean, sample standard deviation and n.

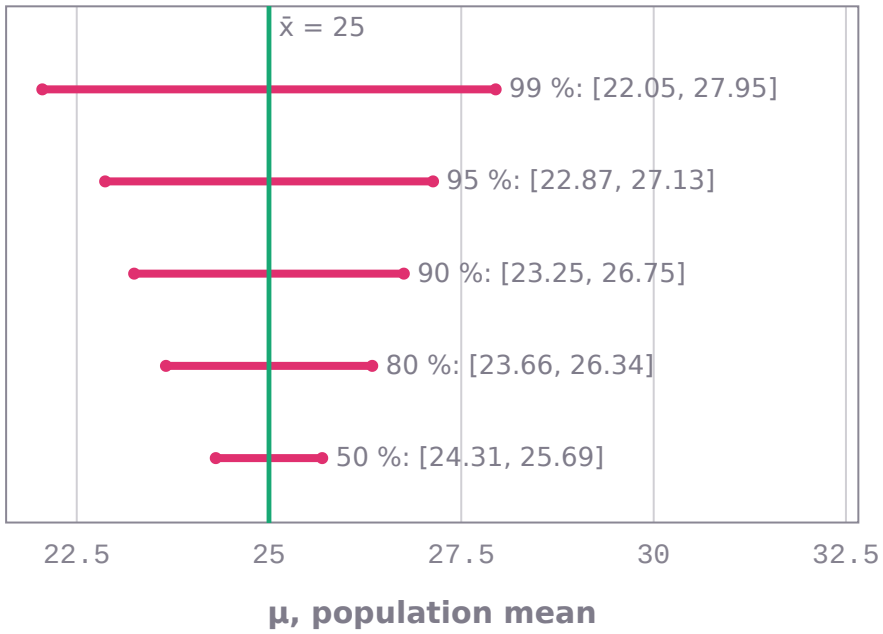
$$\bar{x} \pm t_{\alpha/2,\,n-1}\frac{s}{\sqrt{n}}$$

input	meaning	condition
`xbar`	sample mean	
`s`	sample standard deviation	

input	meaning	condition
`n`	sample size (>= 2)	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`margin_of_error`	t s / sqrt(n)
`critical_value`	t critical value
`df`	$n - 1$

- **Method:** xbar +/- t_($\alpha/2$, n-1) s / sqrt(n)
- **Assumes:** random sample from a roughly normal population (or large n); sigma unknown
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t confidence intervals for a mean from summaries (x̄ = 25, s = 4, n = 16, 15 degrees of freedom) at five levels, each from the package: 50 % [24.3088, 25.6912]; 80 % [23.6594, 26.3406]; 90 % [23.2469, 26.7531]; 95 % [22.8686, 27.1314]; 99 % [22.0533, 27.9467].

```
statistics_ai999x.solve('ci_mean_t_summary', xbar=25, s=4, n=16, confidence=0.95)
# -> lower = 22.8686, upper = 27.1314, margin_of_error = 2.13145, critical_value = 2.13145,
df = 15   (SOLVED)
```

```
statistics_ai999x.solve('ci_mean_t_summary', xbar=3.2, s=0.5, n=10, confidence=0.99)
# -> lower = 2.68616, upper = 3.71384, margin_of_error = 0.513844, critical_value =
3.24984, df = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for the mean with sample mean = 25, sample standard deviation = 4 and n = 16.	SOLVED	lower = 22.8686, upper = 27.1314, margin_of_error = 2.13145, critical_value = 2.13145, df = 15
Construct a 99% confidence interval with x-bar = 3.2, s = 0.5, n = 10.	SOLVED	lower = 2.68616, upper = 3.71384, margin_of_error = 0.513844, critical_value = 3.24984, df = 9
Construct a 99% confidence interval with x-bar = 3.2, s = 0.5, n = 1.	INVALID_INPUT	n: must be at least 2, got 1.

Confidence interval for a proportion (Wald), `ci_proportion`

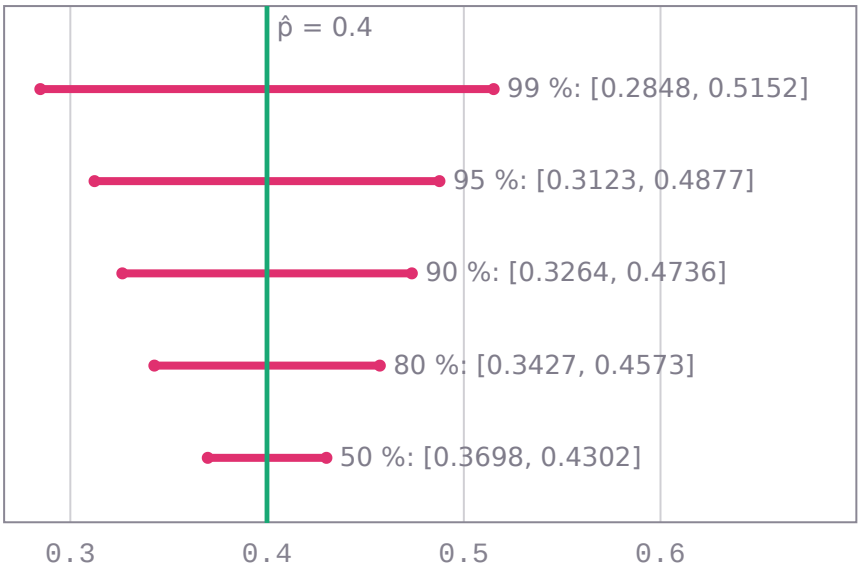
The large-sample (Wald) confidence interval for a population proportion from x successes in n trials.

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

input	meaning	condition
`x`	number of successes	
`n`	sample size	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`proportion`	sample proportion x/n
`lower`	lower limit (clipped at 0)
`upper`	upper limit (clipped at 1)
`margin_of_error`	z sqrt(p(1-p)/n)

- **Method:** p +/- z_(alpha/2) sqrt(p(1 - p)/n)
- **Assumes:** $0 < x < n$; large sample: n p and n(1 - p) at least about 10; limits clipped to [0, 1]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Confidence intervals for a proportion (48 successes in n = 120, p̂ = 0.4) at five levels, each from the package: 50 % [0.369836, 0.430164]; 80 % [0.342687, 0.457313]; 90 % [0.32644, 0.47356]; 95 % [0.312348, 0.487652]; 99 % [0.284805, 0.515195].

```
statistics_ai999x.solve('ci_proportion', x=48, n=120, confidence=0.95)
# -> proportion = 0.4, lower = 0.312348, upper = 0.487652, margin_of_error = 0.0876523
(SOLVED)
```

```
statistics_ai999x.solve('ci_proportion', x=312, n=500, confidence=0.99)
# -> proportion = 0.624, lower = 0.568202, upper = 0.679798, margin_of_error = 0.055798
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for a proportion with x = 48 successes out of n = 120.	SOLVED	proportion = 0.4, lower = 0.312348, upper = 0.487652, margin_of_error = 0.0876523
In a survey, 312 out of 500 people agreed; construct a 99% confidence interval for the proportion.	SOLVED	proportion = 0.624, lower = 0.568202, upper = 0.679798, margin_of_error = 0.055798
Find the 95% confidence interval for a proportion with x = 0 successes out of n = 120.	INVALID_INPUT	Confidence interval for a proportion (Wald): the Wald interval needs 0 < x < n; use the Wilson interval.

Wilson score interval for a proportion, `wilson_interval`

The Wilson score confidence interval for a proportion (good coverage for small n and for p near 0 or 1).

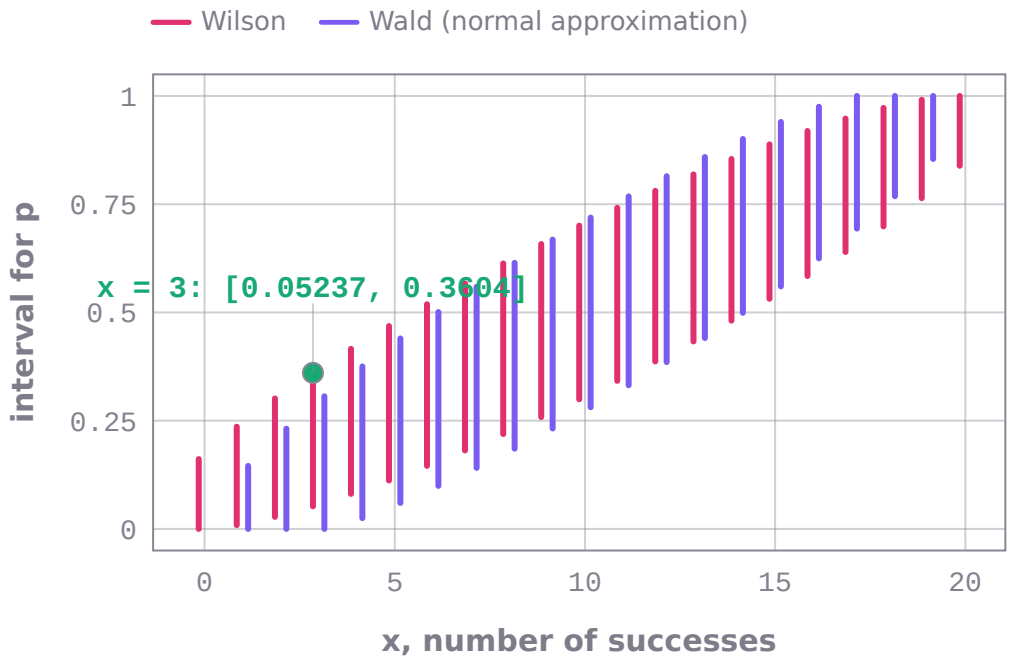
$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

input	meaning	condition
x	number of successes	

input	meaning	condition
`n`	sample size	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`proportion`	x/n
`lower`	lower limit
`upper`	upper limit
`center`	interval centre

- **Method:** invert the score test: centre $(p + z^2/2n)/(1 + z^2/n)$, half-width z sqrt(p(1-p)/n + z^2/4n^2)/(1 + z^2/n)
- **Assumes:** $0 \le x \le n$; independent trials
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



95 % intervals for a proportion for every count x = 0 … 20 out of n = 20: Wilson (this capability) beside the normal-approximation interval of the package's ci_proportion. Near 0 and n the Wald interval collapses or leaves [0, 1]; Wilson does not. Marked: the worked example x = 3, [0.0523687, 0.360419].

```
statistics_ai999x.solve('wilson_interval', x=3, n=20, confidence=0.95)
# -> proportion = 0.15, lower = 0.0523687, upper = 0.360419, center = 0.206394   (SOLVED)
```

```
statistics_ai999x.solve('wilson_interval', x=0, n=15, confidence=0.9)
# -> proportion = 0, lower = 0, upper = 0.152808, center = 0.0764039   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% Wilson interval for x = 3 successes out of n = 20.	SOLVED	proportion = 0.15, lower = 0.0523687, upper = 0.360419, center = 0.206394

question	outcome	result
Construct a 90% Wilson score confidence interval for a proportion with 0 successes out of 15.	SOLVED	proportion = 0, lower = 0, upper = 0.152808, center = 0.0764039
Find the 95% Wilson interval for x = 30 successes out of n = 20.	INVALID_INPUT	Wilson score interval for a proportion: the number of successes x cannot exceed n.

Confidence interval for the difference of two means, `ci_two_means`

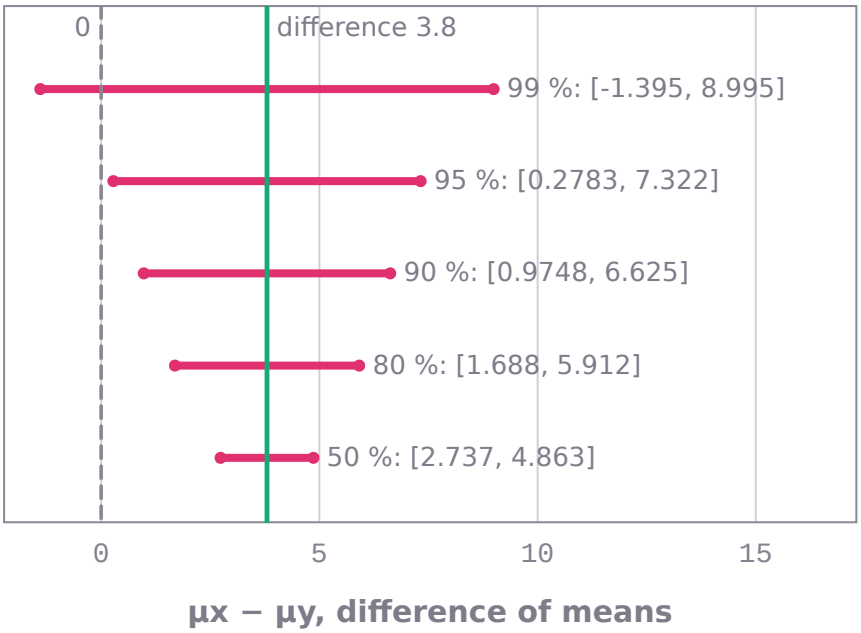
The t confidence interval for $mu1 - mu2$ from two independent samples (Welch or pooled variances).

$$(\bar{x}_1 - \bar{x}_2) \pm t_{\alpha/2,\,\nu}\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

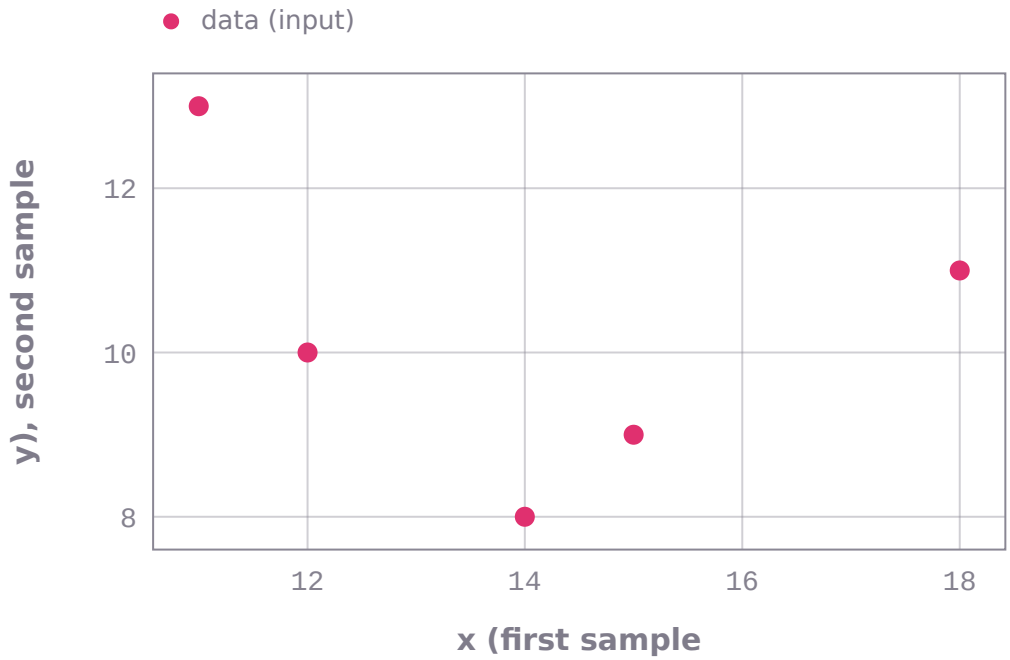
input	meaning	condition
`x`	first sample	
`y`	second sample	
`confidence`	confidence level as a fraction (0.95 for 95%)	
`variance`	welch (unequal variances) or pooled (equal variances)	

output	meaning
`difference`	mean(x) - mean(y)
`lower`	lower limit
`upper`	upper limit
`df`	degrees of freedom (Welch-Satterthwaite or n1 + n2 - 2)
`standard_error`	standard error of the difference

- **Method:** difference +/- t_($\alpha/2$, df) SE, with Welch's SE and df or the pooled variance
- **Assumes:** independent random samples; roughly normal populations or large samples
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Confidence intervals for the difference of the means of x = 12, 15, 11, 18, 14 and y = 10, 9, 13, 11, 8 (variance welch) at five levels, each from the package: 50 % [2.73705, 4.86295]; 80 % [1.68765, 5.91235]; 90 % [0.974776, 6.62522]; 95 % [0.278295, 7.3217]; 99 % [-1.39452, 8.99452]. An interval that excludes 0 (dashed) means a difference at that level.



The data y (second sample) against x (first sample) as given in the worked example (5 points). Also returned: difference = 3.8, lower = 0.278295, upper = 7.3217, df = 7.17415, standard_error = 1.49666. Other inputs: confidence = 0.95, variance = welch.

```
statistics_ai999x.solve('ci_two_means', x='12, 15, 11, 18, 14', y='10, 9, 13, 11, 8',
confidence=0.95, variance='welch')
# -> difference = 3.8, lower = 0.278295, upper = 7.3217, df = 7.17415, standard_error =
1.49666   (SOLVED)
```

```
statistics_ai999x.solve('ci_two_means', x=[5.1, 4.9, 5.6, 5.8], y=[4.2, 4.8, 4.4, 4.0,
4.6], confidence=0.9, variance='pooled')
# -> difference = 0.95, lower = 0.486754, upper = 1.41325, df = 7, standard_error =
0.244511   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for the difference in means of [12, 15, 11, 18, 14] and [10, 9, 13, 11, 8] using Welch.	SOLVED	difference = 3.8, lower = 0.278295, upper = 7.3217, df = 7.17415, standard_error = 1.49666
Construct a 90% confidence interval for mu1 - mu2 with pooled variances for [5.1, 4.9, 5.6, 5.8] and [4.2, 4.8, 4.4, 4.0, 4.6].	SOLVED	difference = 0.95, lower = 0.486754, upper = 1.41325, df = 7, standard_error = 0.244511
Find the 95% confidence interval for the difference in means of [12, 15, 11] and [10, 9, 13].	NEEDS_INPUT	ci_two_means needs variance (welch (unequal variances) or pooled (equal variances)). Example: solve('ci_two_means', x='1

Confidence interval for a variance and standard deviation, `ci_variance`

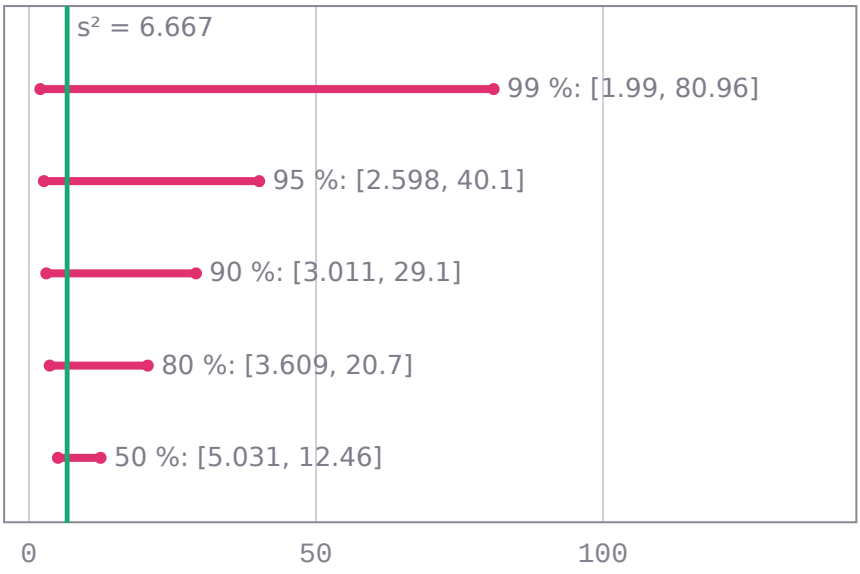
The chi-square confidence interval for a population variance (and standard deviation) from sample data.

$$\left[\frac{(n-1)s^2}{\chi^2_{1-\alpha/2,\,n-1}},\ \frac{(n-1)s^2}{\chi^2_{\alpha/2,\,n-1}}\right]$$

input	meaning	condition
`x`	the data values (at least two)	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`variance`	sample variance
`lower`	lower limit for the variance
`upper`	upper limit for the variance
`sd_lower`	lower limit for the standard deviation
`sd_upper`	upper limit for the standard deviation

- **Method:** [(n-1)Missing superscript or subscript argument(1-alpha/2), (n-1) Missing superscript or subscript argument(alpha/2)]
- **Assumes:** normal population (the interval is sensitive to non-normality)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Chi-square confidence intervals for the variance of x = 12, 15, 11, 18, 14, 16 (sample variance 6.66667) at five levels, each from the package: 50 % [5.03093, 12.4629]; 80 % [3.60893, 20.7]; 90 % [3.01101, 29.1]; 95 % [2.59757, 40.1021]; 99 % [1.9901, 80.9569]. They are not symmetric about s²: the chi-square distribution is skewed.

```
statistics_ai999x.solve('ci_variance', x='12, 15, 11, 18, 14, 16', confidence=0.95)
# -> variance = 6.66667, lower = 2.59757, upper = 40.1021, sd_lower = 1.6117, sd_upper =
6.33262   (SOLVED)
```

```
statistics_ai999x.solve('ci_variance', x=[4.8, 5.1, 5.3, 4.9, 5.0], confidence=0.9)
# -> variance = 0.037, lower = 0.0155991, upper = 0.208239, sd_lower = 0.124896, sd_upper =
0.456332   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for the population variance of [12, 15, 11, 18, 14, 16].	SOLVED	variance = 6.66667, lower = 2.59757, upper = 40.1021, sd_lower = 1.6117, sd_upper = 6.33262
Construct a 90% confidence interval for the standard deviation from [4.8, 5.1, 5.3, 4.9, 5.0].	SOLVED	variance = 0.037, lower = 0.0155991, upper = 0.208239, sd_lower = 0.124896, sd_upper = 0.456332
Find the 95% confidence interval for the population variance of [12].	INVALID_INPUT	x: needs at least 2 entries, got 1.

One-sample z-test for a mean (sigma known), `z_test_mean`

The z statistic and p-value for H0: $mu = mu0$ when the population standard deviation is known.

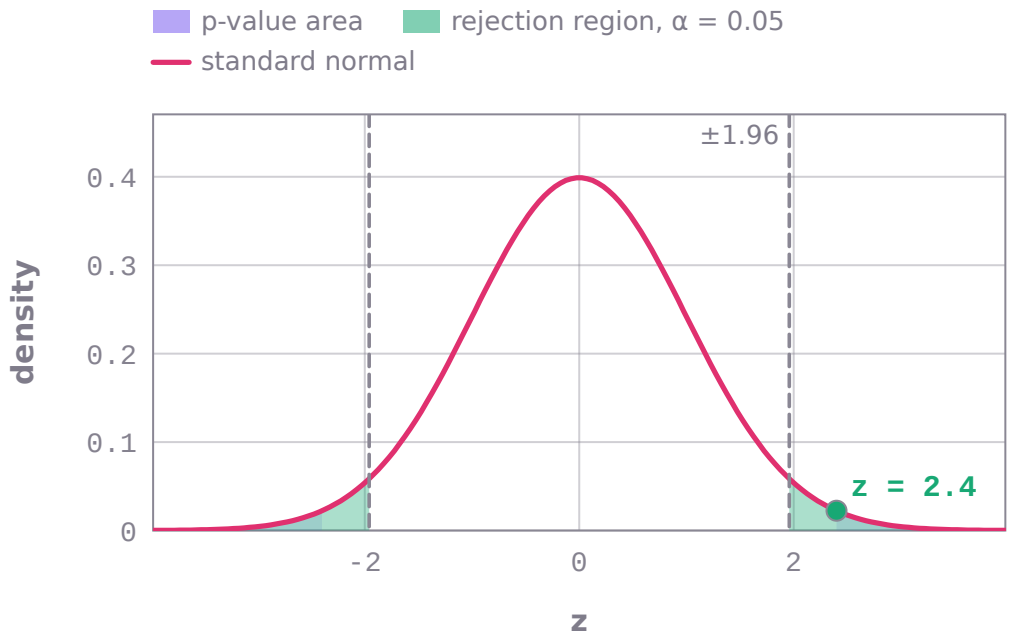
$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

input	meaning	condition
`xbar`	sample mean	

input	meaning	condition
`mu0`	hypothesised mean	
`sigma`	population standard deviation (known)	
`n`	sample size	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`z`	(xbar - mu0) / (sigma / sqrt(n))
`p_value`	p-value for the chosen alternative

- **Method:** z statistic and its normal tail probability
- **Assumes:** sigma known; normal population or large n; the alternative must be stated for a one-sided claim
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The z test of a sample mean x̄ = 52 against μ_0 = 50 (σ = 5, n = 36). Drawn: the standard normal density (density from the package's cumulative probabilities on a grid, differenced); shaded: the rejection region beyond ±1.95996 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: z = 2.4, p-value = 0.0163951.

```
statistics_ai999x.solve('z_test_mean', xbar=52, mu0=50, sigma=5, n=36)
# -> z = 2.4, p_value = 0.0163951   (SOLVED)
```

```
statistics_ai999x.solve('z_test_mean', xbar=103, mu0=100, sigma=12, n=49,
alternative='greater')
# -> z = 1.75, p_value = 0.0400592   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a z-test of H0: mu = 50 with sample mean = 52, sigma = 5 and n = 36, two-sided.	SOLVED	z = 2.4, p_value = 0.0163951
Test whether the mean exceeds 100 using a right-tailed z-test with sample mean = 103, population standard deviation = 12, n = 49, mu0 = 100.	SOLVED	z = 1.75, p_value = 0.0400592
Test whether the mean is greater than 100 with a z-test: sample mean = 103, sigma = 12, n = 49, mu0 = 100.	OUT_OF_SCOPE	Out of scope: a one-sided claim: say right-tailed, left-tailed or two-sided.
Perform a z-test of H0: mu = 50 with sample mean = 52 and sigma = 5, two-sided.	NEEDS_INPUT	z_test_mean needs n (sample size). Example: solve('z_test_mean', xbar=52, mu0=50, sigma=5, n=36).

One-sample t-test, `t_test_one_sample`

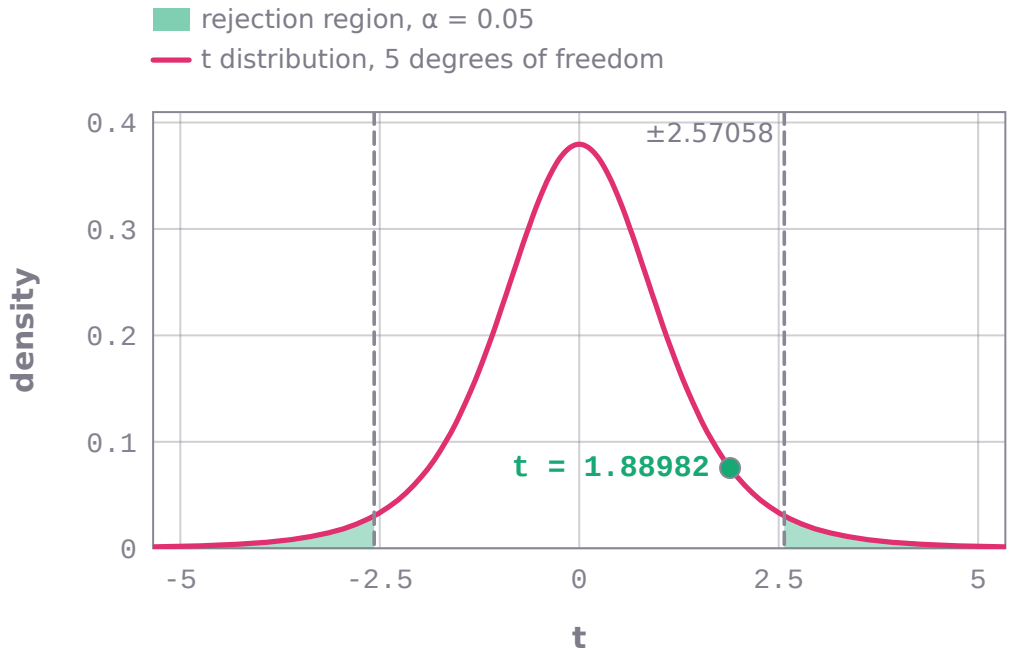
The one-sample t-test of H0: $mu = mu0$ from sample data: t statistic, degrees of freedom and p-value.

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}},\ \nu = n - 1$$

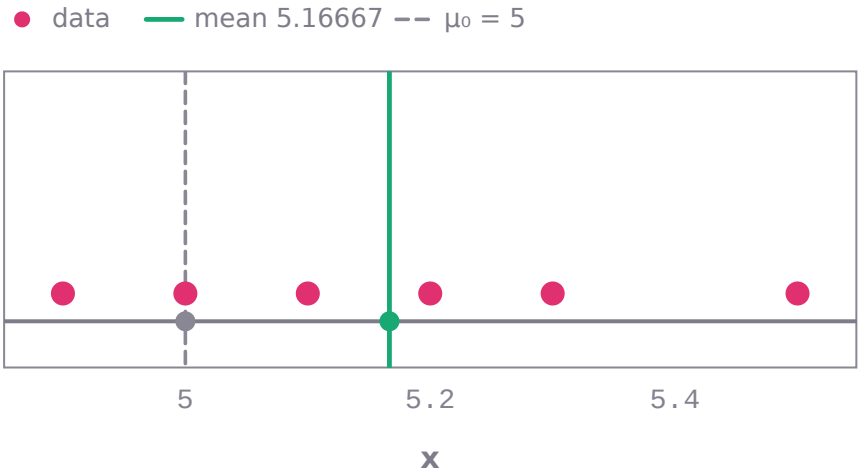
input	meaning	condition
`x`	the data values (at least two)	
`mu0`	hypothesised mean	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`t`	(mean - mu0)/(s/sqrt(n))
`df`	$n - 1$
`p_value`	p-value
`mean`	sample mean
`standard_deviation`	sample standard deviation

- **Method:** t statistic with $n - 1$ degrees of freedom; tail probability from the incomplete beta function
- **Assumes:** random sample from a roughly normal population (or large n); the alternative must be stated for a one-sided claim
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The one-sample t test of x = 5.1, 4.9, 5.3, 5.5, 5.0, 5.2 against μ_0 = 5: the t density with 5 degrees of freedom (from the package's quantiles at 121 cumulative probabilities, differenced), the two-tailed rejection region beyond ±2.57058 for α = 0.05 shaded; marked: t = 1.88982, p-value = 0.117387.



The sample x = 5.1, 4.9, 5.3, 5.5, 5.0, 5.2 against the hypothesised mean μ_0 = 5: returned mean 5.16667, standard deviation 0.216025, t = 1.88982 with 5 degrees of freedom, p-value 0.117387.

```
statistics_ai999x.solve('t_test_one_sample', x='5.1, 4.9, 5.3, 5.5, 5.0, 5.2', mu0=5)
# -> t = 1.88982, df = 5, p_value = 0.117387, mean = 5.16667, standard_deviation = 0.216025
(SOLVED)
```

```
statistics_ai999x.solve('t_test_one_sample', x=[98, 102, 95, 97, 99, 96], mu0=100,
alternative='less')
# -> t = -2.13719, df = 5, p_value = 0.0428108, mean = 97.8333, standard_deviation =
2.48328   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a one-sample t-test of [5.1, 4.9, 5.3, 5.5, 5.0, 5.2] against mu0 = 5, two-sided.	SOLVED	t = 1.88982, df = 5, p_value = 0.117387, mean = 5.16667, standard_deviation = 0.216025

question	outcome	result
Use a left-tailed t-test to check whether the mean of [98, 102, 95, 97, 99, 96] is below the hypothesized mean = 100.	SOLVED	t = -2.13719, df = 5, p_value = 0.0428108, mean = 97.8333, standard_deviation = 2.48328
Use a t-test to check whether the mean of [98, 102, 95, 97] is below the hypothesized mean = 100.	OUT_OF_SCOPE	Out of scope: a one-sided claim: say right-tailed, left-tailed or two-sided.
Perform a one-sample t-test of [5.1, 4.9, 5.3], two-sided.	NEEDS_INPUT	t_test_one_sample needs mu0 (hypothesised mean). Example: solve('t_test_one_sample', x='5.1, 4.9, 5.3, 5.5, 5.0, 5.2', m

Two-sample t-test (pooled variances), `t_test_two_sample_pooled`

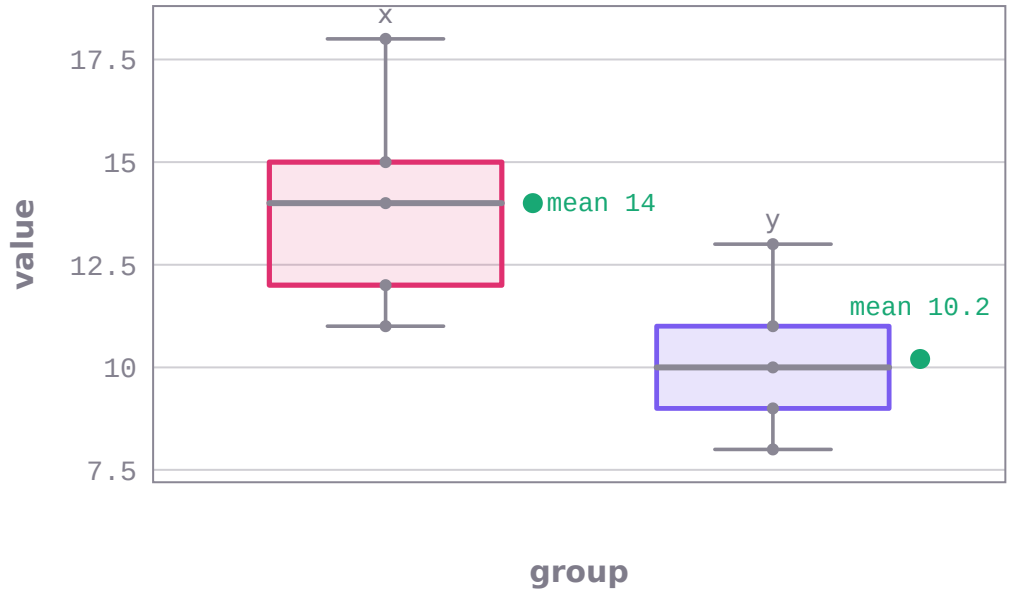
Two-sample t-test (pooled variances) comparing the means of two independent samples: t statistic, degrees of freedom, p-value.

$$t = \frac{\bar{x}_1 - \bar{x}_2}{s_p\sqrt{1/n_1 + 1/n_2}},\ \nu = n_1 + n_2 - 2$$

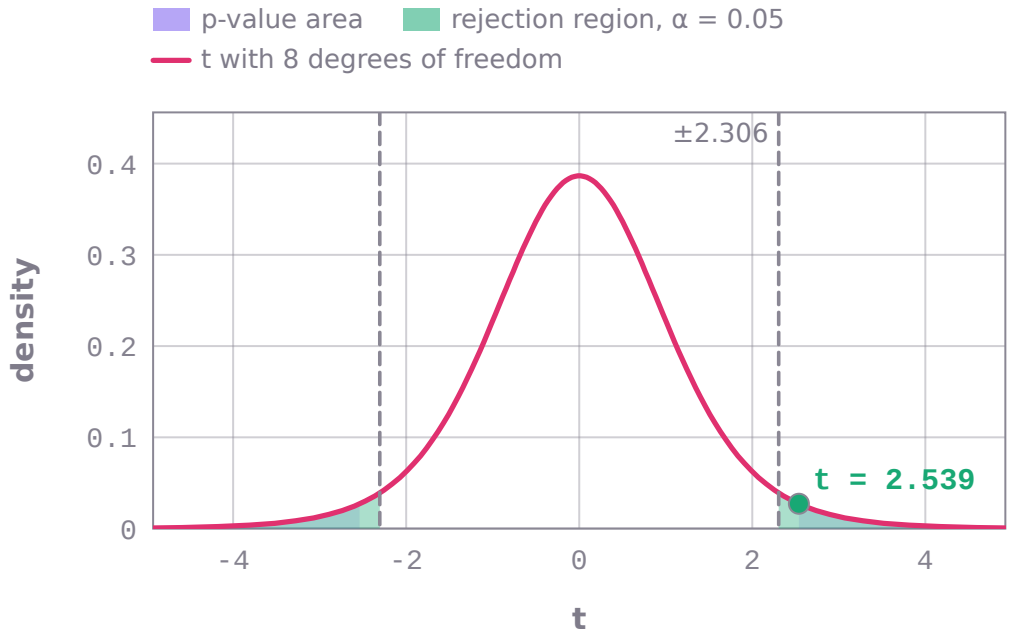
input	meaning	condition
`x`	first sample	
`y`	second sample	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`t`	t statistic
`df`	degrees of freedom
`p_value`	p-value
`difference`	mean(x) - mean(y)

- **Method:** pooled variance with $n_1 + n_2 - 2$ df
- **Assumes:** independent random samples; roughly normal populations or large samples; equal population variances assumed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The two samples x = 12, 15, 11, 18, 14 and y = 10, 9, 13, 11, 8 as box plots (quartiles from the package's percentiles) with their data; marked: the sample means from the package, 14 and 10.2. Pooled t test: difference 3.8, t = 2.53898, 8 degrees of freedom, p-value 0.0347658.



The pooled two-sample t test of x = 12, 15, 11, 18, 14 against y = 10, 9, 13, 11, 8. Drawn: the t density with 8 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region beyond ±2.306 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: t = 2.53898, p-value = 0.0347658.

```
statistics_ai999x.solve('t_test_two_sample_pooled', x='12, 15, 11, 18, 14', y='10, 9, 13, 11, 8')
# -> t = 2.53898, df = 8, p_value = 0.0347658, difference = 3.8   (SOLVED)
```

```
statistics_ai999x.solve('t_test_two_sample_pooled', x=[5.1, 4.9, 5.6, 5.8], y=[4.2, 4.8, 4.4, 4.0, 4.6], alternative='greater')
# -> t = 3.8853, df = 7, p_value = 0.00300585, difference = 0.95   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a two-sample t-test with pooled variances on [12, 15, 11, 18, 14] and [10, 9, 13, 11, 8], two-sided.	SOLVED	t = 2.53898, df = 8, p_value = 0.0347658, difference = 3.8
Is there a difference? Run a pooled t-test for [5.1, 4.9, 5.6, 5.8] versus [4.2, 4.8, 4.4, 4.0, 4.6], two-tailed.	SOLVED	t = 3.8853, df = 7, p_value = 0.0060117, difference = 0.95
Run a pooled t-test for [5.1, 4.9, 5.6] versus [4.2, 4.8, 4.4], right-tailed.	SOLVED	t = 2.68773, df = 4, p_value = 0.0273935, difference = 0.733333
Run a pooled t-test for [5.1] versus [4.2, 4.8, 4.4], two-sided.	INVALID_INPUT	x: needs at least 2 entries, got 1.

Welch's two-sample t-test (unequal variances), `welch_t_test`

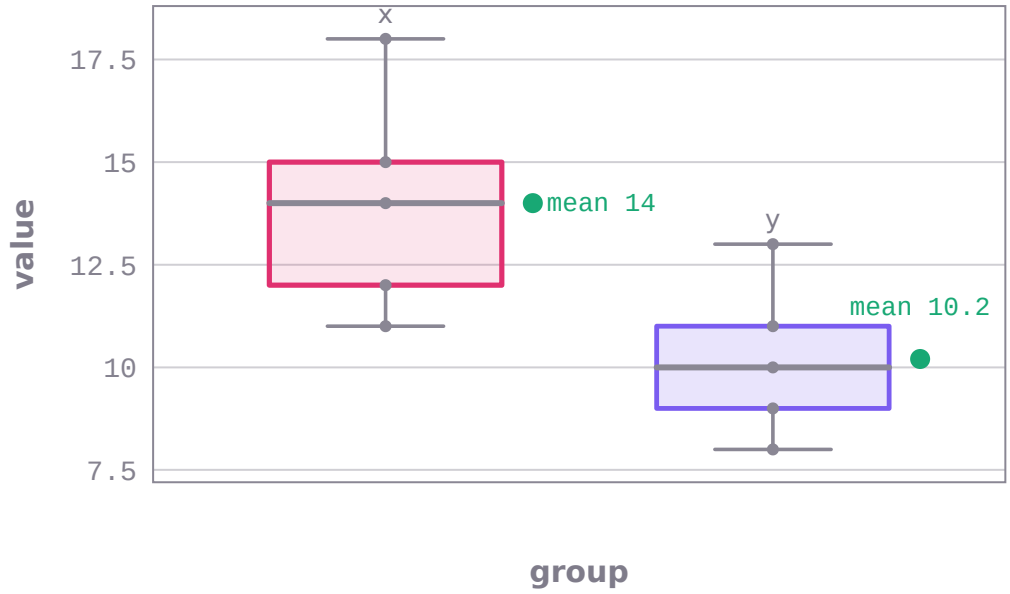
Welch's two-sample t-test (unequal variances) comparing the means of two independent samples: t statistic, degrees of freedom, p-value.

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$$

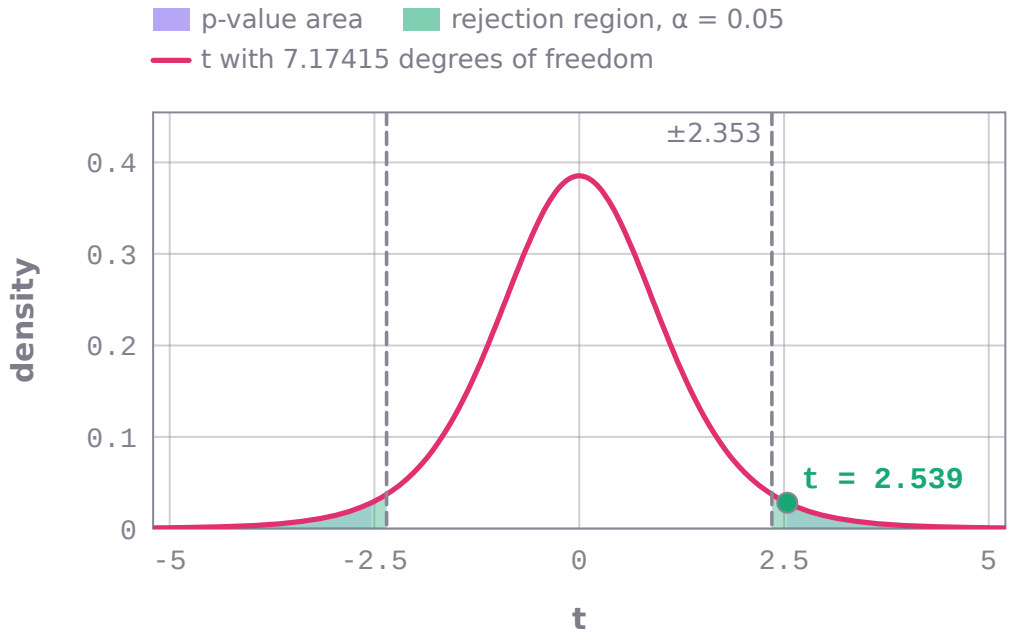
input	meaning	condition
`x`	first sample	
`y`	second sample	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`t`	t statistic
`df`	degrees of freedom
`p_value`	p-value
`difference`	mean(x) - mean(y)

- **Method:** separate variances, Welch-Satterthwaite df
- **Assumes:** independent random samples; roughly normal populations or large samples; Welch-Satterthwaite degrees of freedom
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The two samples x = 12, 15, 11, 18, 14 and y = 10, 9, 13, 11, 8 as box plots (quartiles from the package's percentiles) with their data; marked: the sample means from the package, 14 and 10.2. Welch's t test: difference 3.8, t = 2.53898, 7.17415 degrees of freedom (Welch–Satterthwaite), p-value 0.0379455.



Welch's t test of x = 12, 15, 11, 18, 14 against y = 10, 9, 13, 11, 8. Drawn: the t density with 7.17415 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region beyond ±2.35304 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: t = 2.53898, p-value = 0.0379455.

```
statistics_ai999x.solve('welch_t_test', x='12, 15, 11, 18, 14', y='10, 9, 13, 11, 8')
# -> t = 2.53898, df = 7.17415, p_value = 0.0379455, difference = 3.8   (SOLVED)
```

```
statistics_ai999x.solve('welch_t_test', x=[5.1, 4.9, 5.6, 5.8], y=[4.2, 4.8, 4.4, 4.0,
4.6], alternative='greater')
# -> t = 3.75032, df = 5.48812, p_value = 0.00560003, difference = 0.95   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a two-sample t-test with unequal variances on [12, 15, 11, 18, 14] and [10, 9, 13, 11, 8], two-sided.	SOLVED	t = 2.53898, df = 7.17415, p_value = 0.0379455, difference = 3.8
Is there a difference? Run a Welch t-test for [5.1, 4.9, 5.6, 5.8] versus [4.2, 4.8, 4.4, 4.0, 4.6], two-tailed.	SOLVED	t = 3.75032, df = 5.48812, p_value = 0.0112001, difference = 0.95
Run a Welch t-test for [5.1, 4.9, 5.6] versus [4.2, 4.8, 4.4], right-tailed.	SOLVED	t = 2.68773, df = 3.89501, p_value = 0.0281773, difference = 0.733333
Run a Welch t-test for [5.1] versus [4.2, 4.8, 4.4], two-sided.	INVALID_INPUT	x: needs at least 2 entries, got 1.

Paired t-test, `paired_t_test`

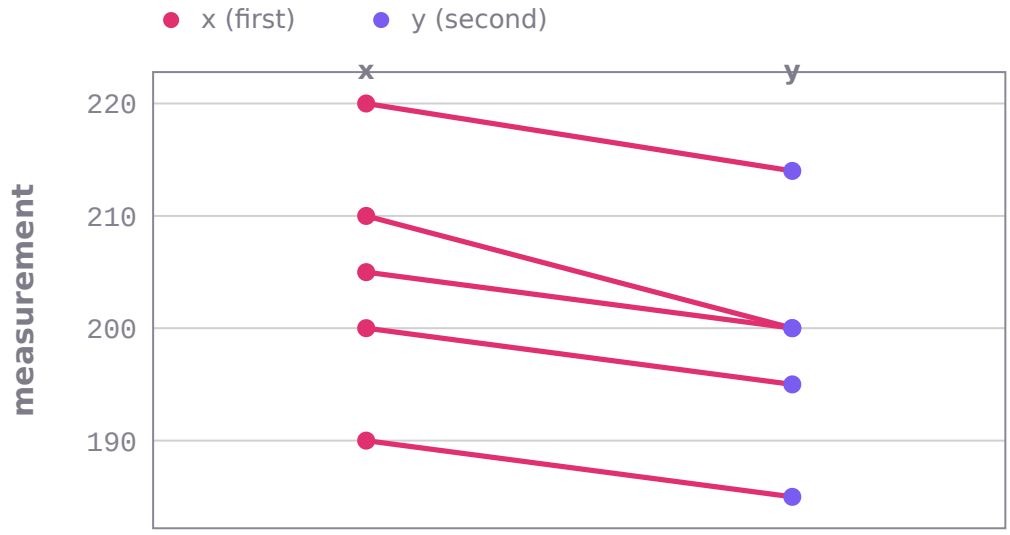
The paired (dependent samples) t-test on the differences x - y: t statistic, df and p-value.

$$t = \frac{\bar{d}}{s_d/\sqrt{n}},\ \nu = n - 1$$

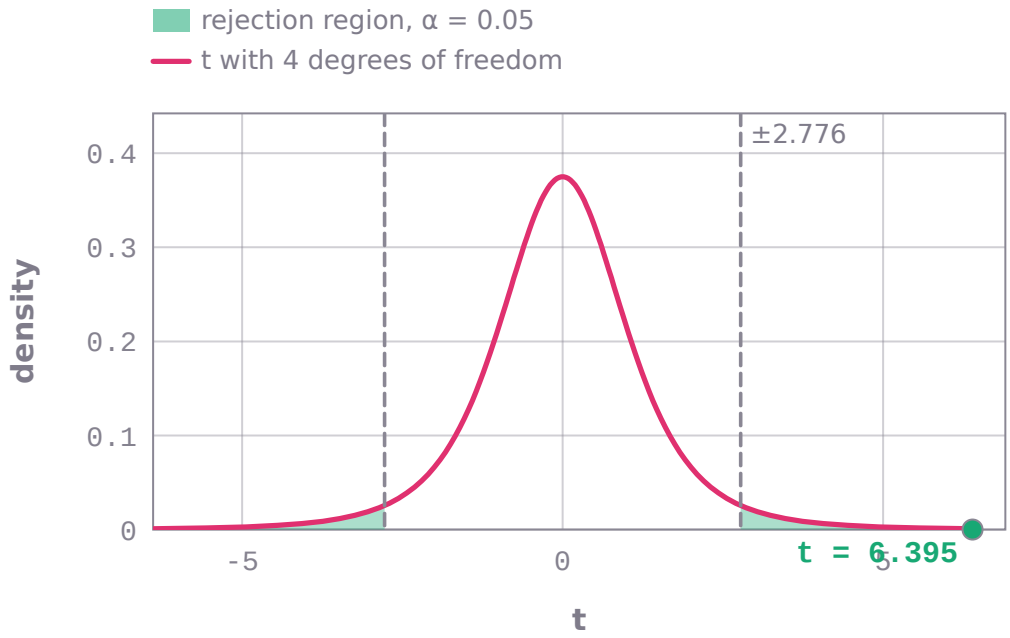
input	meaning	condition
`x`	first measurement of each pair	
`y`	second measurement of each pair	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`t`	mean difference / (s_d / sqrt(n))
`df`	$n - 1$
`p_value`	p-value
`mean_difference`	mean of x - y

- **Method:** one-sample t-test on the differences
- **Assumes:** pairs measured on the same units; differences roughly normal (or many pairs)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each pair (x, y) = (200, 190, 210, 220, 205) → (195, 185, 200, 214, 200) joined by a line, falling lines in the first colour. Returned: mean difference 6.2, t = 6.39481, 4 degrees of freedom, p-value 0.00307015.



The paired t test of x = 200, 190, 210, 220, 205 against y = 195, 185, 200, 214, 200. Drawn: the t density with 4 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region beyond ±2.77645 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: t = 6.39481, p-value = 0.00307015.

```
statistics_ai999x.solve('paired_t_test', x='200, 190, 210, 220, 205', y='195, 185, 200,
214, 200')
# -> t = 6.39481, df = 4, p_value = 0.00307015, mean_difference = 6.2   (SOLVED)
```

```
statistics_ai999x.solve('paired_t_test', x=[5.2, 6.1, 5.8, 6.4], y=[4.9, 5.7, 5.9, 5.8],
alternative='greater')
# -> t = 2.0381, df = 3, p_value = 0.0671461, mean_difference = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a paired t-test on before [200, 190, 210, 220, 205] and after [195, 185, 200, 214, 200], two-sided.	SOLVED	t = 6.39481, df = 4, p_value = 0.00307015, mean_difference = 6.2
Run a right-tailed paired t-test for [5.2, 6.1, 5.8, 6.4] versus [4.9, 5.7, 5.9, 5.8].	SOLVED	t = 2.0381, df = 3, p_value = 0.0671461, mean_difference = 0.3
Run a paired t-test for [5.2, 6.1, 5.8] versus [4.9, 5.7], two-sided.	INVALID_INPUT	Paired t-test: x and y must have the same number of values (paired data).

Two-sample t-test from summary statistics, `t_test_summary_statistics`

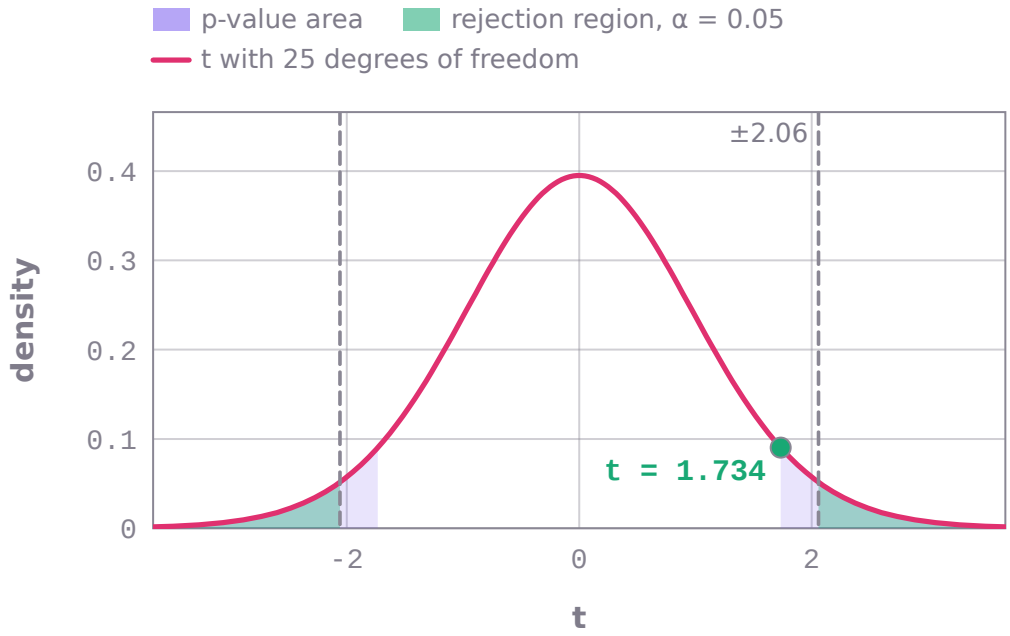
The two-sample t-test from the means, sample standard deviations and sizes of two groups (Welch or pooled).

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$$

input	meaning	condition
`m1`	mean of group 1	
`s1`	sample standard deviation of group 1	
`n1`	size of group 1	
`m2`	mean of group 2	
`s2`	sample standard deviation of group 2	
`n2`	size of group 2	
`variance`	welch (unequal variances) or pooled (equal variances)	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`t`	t statistic
`df`	degrees of freedom
`p_value`	p-value
`difference`	$m_1 - m_2$

- **Method:** pooled or Welch standard error and degrees of freedom from the summaries
- **Assumes:** independent random samples; roughly normal populations or large samples
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The two-sample t test from summaries (group 1: mean 20, sd 4, n 15; group 2: mean 17, sd 5, n 12; variance pooled). Drawn: the t density with 25 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region beyond ±2.05954 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: t = 1.73379, p-value = 0.0952738.

```
statistics_ai999x.solve('t_test_summary_statistics', m1=20, s1=4, n1=15, m2=17, s2=5,
n2=12, variance='pooled')
# -> t = 1.73379, df = 25, p_value = 0.0952738, difference = 3   (SOLVED)
```

```
statistics_ai999x.solve('t_test_summary_statistics', m1=3.4, s1=0.8, n1=30, m2=3.0, s2=1.1,
n2=25, variance='welch', alternative='greater')
# -> t = 1.51475, df = 42.9191, p_value = 0.0685837, difference = 0.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-sample t-test from summary statistics with pooled variances: mean1 = 20, sd1 = 4, n1 = 15, mean2 = 17, sd2 = 5, n2 = 12, two-sided.	SOLVED	t = 1.73379, df = 25, p_value = 0.0952738, difference = 3
Welch t-test from summary statistics: m1 = 3.4, s1 = 0.8, n1 = 30, m2 = 3.0, s2 = 1.1, n2 = 25, right-tailed.	SOLVED	t = 1.51475, df = 42.9191, p_value = 0.0685837, difference = 0.4
Two-sample t-test from summary statistics: mean1 = 20, sd1 = 4, n1 = 15, mean2 = 17, sd2 = 5, n2 = 12, two-sided.	NEEDS_INPUT	t_test_summary_statistics needs variance (welch (unequal variances) or pooled (equal variances)). Example: solve('t_test

One-proportion z-test, `proportion_z_test`

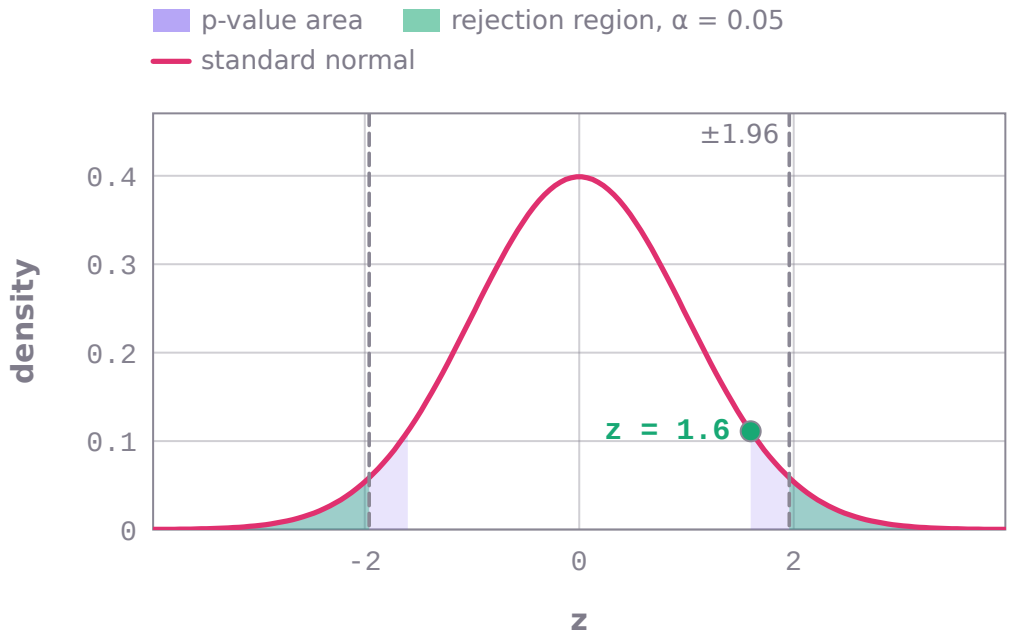
The z-test of H0: $p = p_0$ for a population proportion from x successes in n trials.

$$z = \frac{\hat{p} - p_0}{\sqrt{p_0(1 - p_0)/n}}$$

input	meaning	condition
`x`	number of successes	
`n`	sample size	
`p0`	hypothesised proportion	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`z`	$(x/n - p_0)/sqrt$(p0(1 - p0)/n)
`p_value`	p-value
`proportion`	x/n

- **Method:** z statistic with the null standard error sqrt(p0(1-p0)/n)
- **Assumes:** n p0 and n(1 - p0) at least about 10; independent trials
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The z test of 58 successes in n = 100 (proportion 0.58) against p_0 = 0.5. Drawn: the standard normal density (density from the package's cumulative probabilities on a grid, differenced); shaded: the rejection region beyond ±1.95996 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: z = 1.6, p-value = 0.109599.

```
statistics_ai999x.solve('proportion_z_test', x=58, n=100, p0=0.5)
# -> z = 1.6, p_value = 0.109599, proportion = 0.58   (SOLVED)
```

```
statistics_ai999x.solve('proportion_z_test', x=45, n=120, p0=0.3, alternative='greater')
# -> z = 1.79284, p_value = 0.036499, proportion = 0.375   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Test H0: p = 0.5 for a proportion with x = 58 successes out of n = 100, p0 = 0.5, two-sided.	SOLVED	z = 1.6, p_value = 0.109599, proportion = 0.58

question	outcome	result
A claim says the proportion is 0.3; in a sample 45 out of 120 agreed. Test it with p0 = 0.3, right-tailed.	SOLVED	z = 1.79284, p_value = 0.036499, proportion = 0.375
Test the proportion with x = 58 successes out of n = 100, p0 = 1.2, two-sided.	INVALID_INPUT	One-proportion z-test: p0 must be a probability strictly between 0 and 1.

Two-proportion z-test, `two_proportion_z_test`

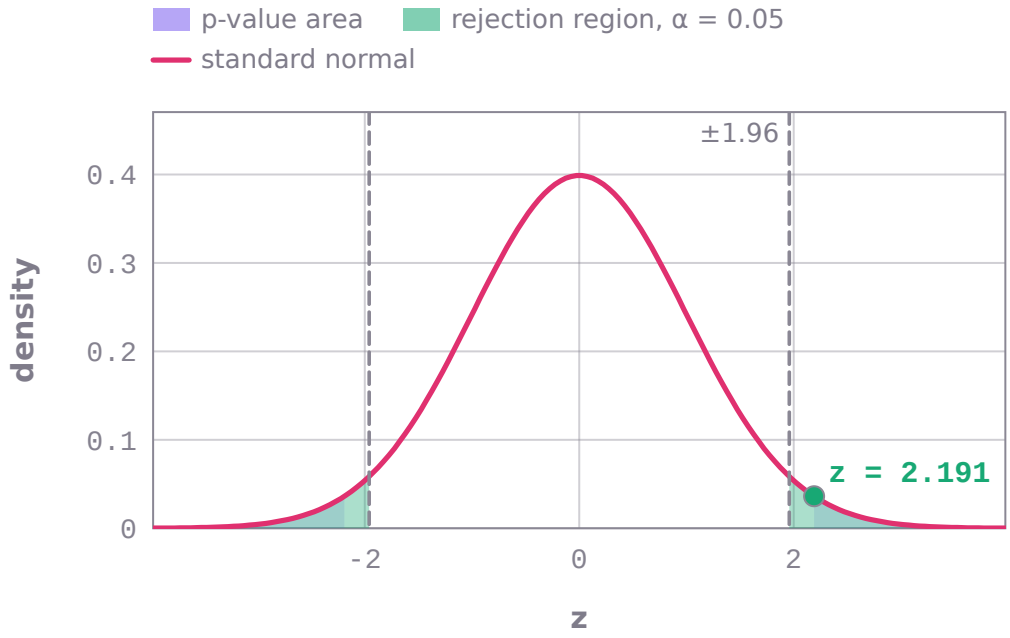
The pooled z-test of H0: $p_1 = p_2$ from x1 of n1 and x2 of n2 successes.

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

input	meaning	condition
`x1`	successes in sample 1	
`n1`	size of sample 1	
`x2`	successes in sample 2	
`n2`	size of sample 2	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`z`	z statistic (pooled standard error)
`p_value`	p-value
`pooled_proportion`	$(x_1 + x_2)/(n_1 + n_2)$
`difference`	$x_1/n_1 - x_2/n_2$

- **Method:** pooled proportion, $z = (p_1 - p_2)/sqrt$(p(1-p)$(1/n_1 + 1/n_2)$)
- **Assumes:** independent samples; at least about 10 successes and failures in each sample
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The z test of 45/100 against 30/100 (difference 0.15, pooled proportion 0.375). Drawn: the standard normal density (density from the package's cumulative probabilities on a grid, differenced); shaded: the rejection region beyond ±1.95996 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: z = 2.19089, p-value = 0.0284597.

```
statistics_ai999x.solve('two_proportion_z_test', x1=45, n1=100, x2=30, n2=100)
# -> z = 2.19089, p_value = 0.0284597, pooled_proportion = 0.375, difference = 0.15
(SOLVED)
```

```
statistics_ai999x.solve('two_proportion_z_test', x1=120, n1=400, x2=90, n2=380,
alternative='greater')
# -> z = 1.98769, p_value = 0.0234227, pooled_proportion = 0.269231, difference = 0.0631579
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-proportion z-test with x1 = 45, n1 = 100, x2 = 30, n2 = 100, two-sided.	SOLVED	z = 2.19089, p_value = 0.0284597, pooled_proportion = 0.375, difference = 0.15
Test the difference between two proportions: x1 = 120, n1 = 400, x2 = 90, n2 = 380, right-tailed.	SOLVED	z = 1.98769, p_value = 0.0234227, pooled_proportion = 0.269231, difference = 0.0631579
Two-proportion z-test with x1 = 145, n1 = 100, x2 = 30, n2 = 100, two-sided.	INVALID_INPUT	Two-proportion z-test: the number of successes x cannot exceed n.
Two-proportion z-test with x1 = 45, n1 = 100, x2 = 30, two-sided.	NEEDS_INPUT	two_proportion_z_test needs n2 (size of sample 2). Example: solve('two_proportion_z_test', x1=45, n1=100, x2=30, n2=100)

Chi-square goodness-of-fit test, `chi_square_goodness_of_fit`

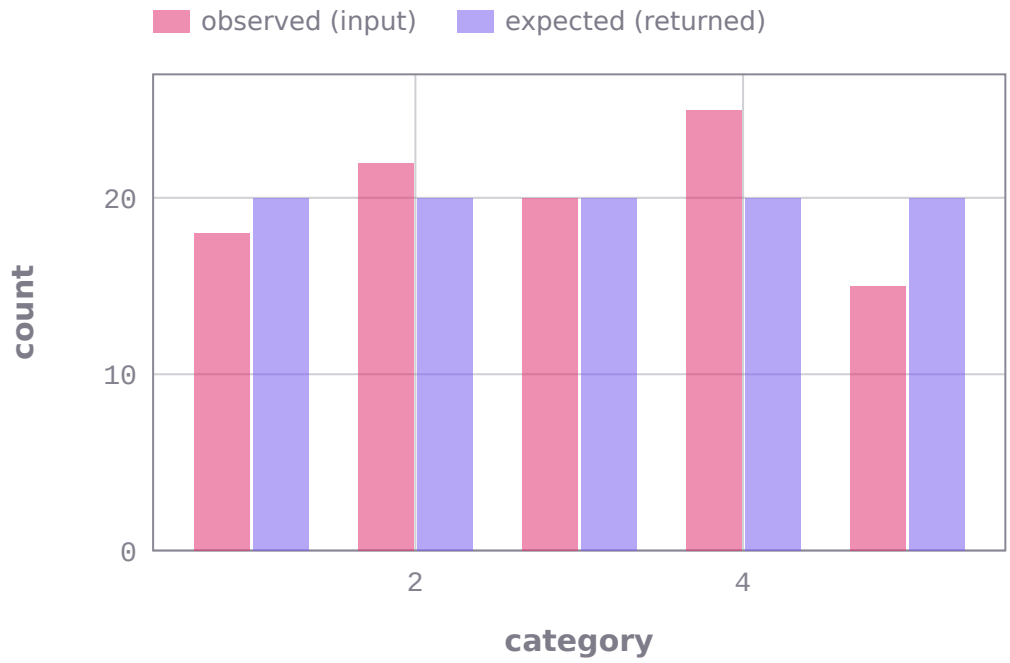
The chi-square goodness-of-fit test of observed counts against expected proportions (equal if omitted).

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i},\ \nu = k - 1$$

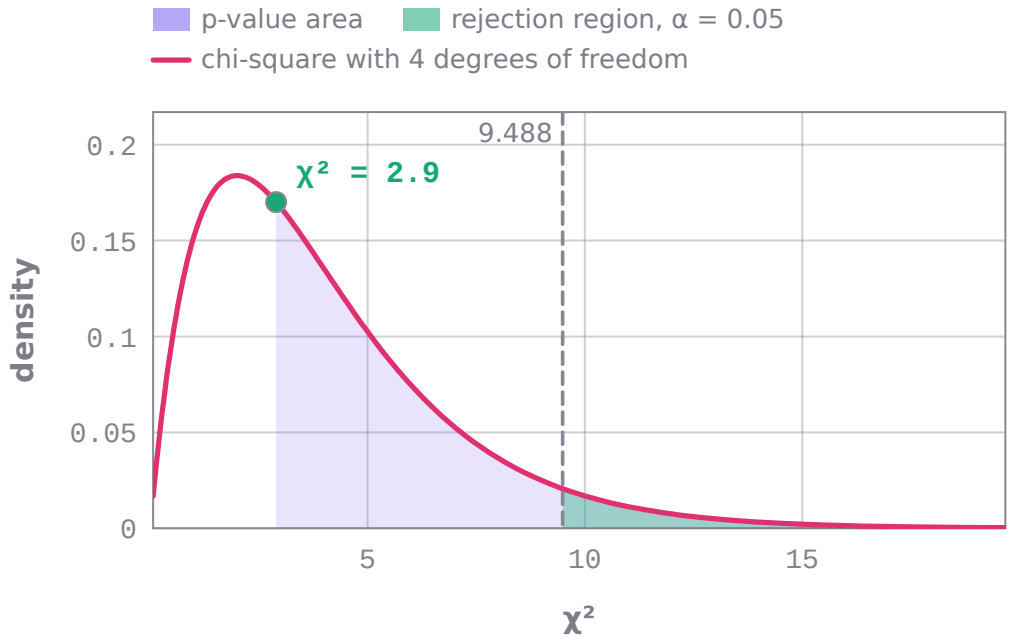
input	meaning	condition
`observed`	observed counts per category	
`proportions`	expected proportions or relative weights per category (equal if omitted)	

output	meaning
`chi_square`	sum $(O-E)^2/E$
`df`	categories - 1
`p_value`	upper-tail p-value
`expected`	expected counts

- **Method:** expected counts n p_i; chi-square statistic with $k-1$ df
- **Assumes:** independent observations; expected counts at least about 5; proportions are rescaled to sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Observed counts 18, 22, 20, 25, 15 against equal proportions, beside the expected counts returned by the package (20, 20, 20, 20, 20). χ² = 2.9 with 4 degrees of freedom, p-value 0.574697.



The goodness-of-fit test of 18, 22, 20, 25, 15. Drawn: the chi-square density with 4 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 9.48773 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: χ² = 2.9, p-value = 0.574697.

```
statistics_ai999x.solve('chi_square_goodness_of_fit', observed='18, 22, 20, 25, 15')
# -> chi_square = 2.9, df = 4, p_value = 0.574697, expected = [20.0, 20.0, 20.0, 20.0,
20.0]   (SOLVED)
```

```
statistics_ai999x.solve('chi_square_goodness_of_fit', observed=[50, 30, 20], proportions=
[0.5, 0.3, 0.2])
# -> chi_square = 0, df = 2, p_value = 1, expected = [50.0, 30.0, 20.0]   (SOLVED)
```

```
statistics_ai999x.solve('chi_square_goodness_of_fit', observed=[30, 50, 20], proportions=
[0.25, 0.5, 0.25])
# -> chi_square = 2, df = 2, p_value = 0.367879, expected = [25.0, 50.0, 25.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chi-square goodness-of-fit test for observed [18, 22, 20, 25, 15] with equal expected proportions.	SOLVED	chi_square = 2.9, df = 4, p_value = 0.574697, expected = [20.0, 20.0, 20.0, 20.0, 20.0]
Goodness-of-fit: observed [50, 30, 20] against expected proportions [0.5, 0.3, 0.2].	SOLVED	chi_square = 0, df = 2, p_value = 1, expected = [50.0, 30.0, 20.0]
Goodness-of-fit: observed [50, 30, 20] against expected proportions [0.5, 0.5].	INVALID_INPUT	Chi-square goodness-of-fit test: give one expected proportion per category.

Chi-square test of independence, `chi_square_independence`

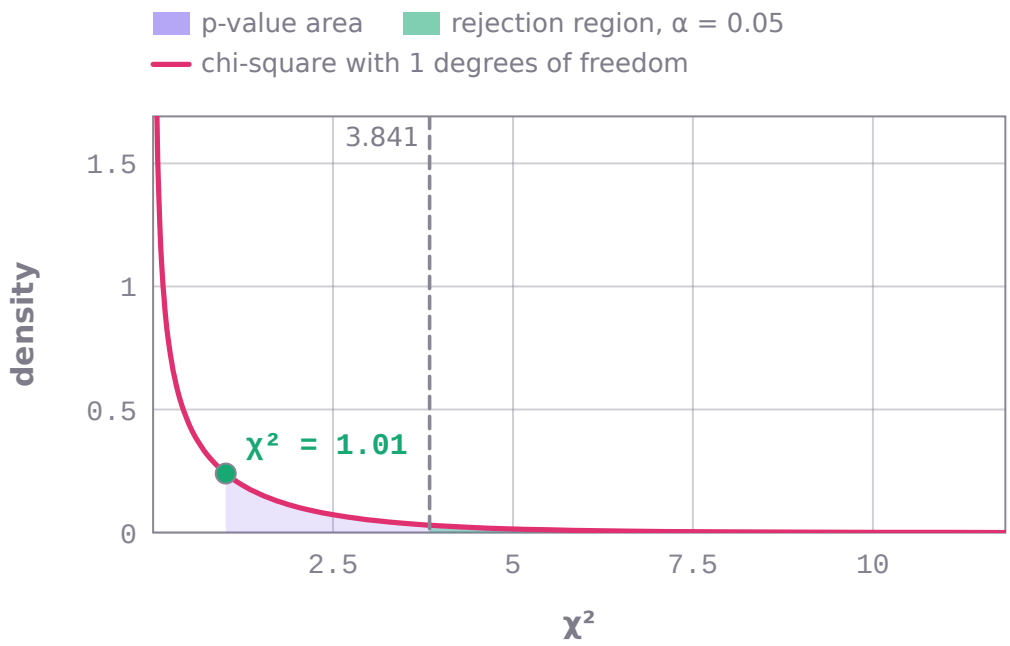
The chi-square test of independence (or homogeneity) for an r x c contingency table of counts.

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}},\ E_{ij} = \frac{R_i C_j}{n}$$

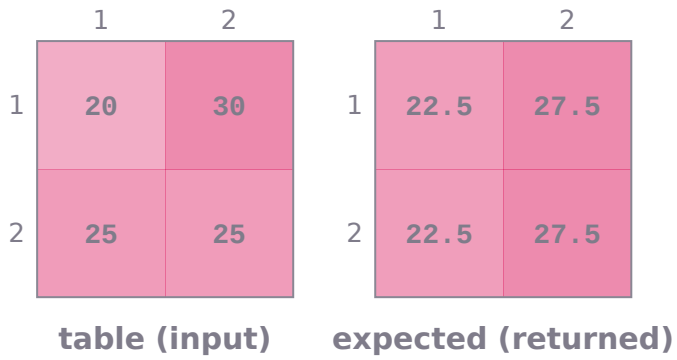
input	meaning	condition
`table`	contingency table of counts (rows x columns)	

output	meaning
`chi_square`	Pearson chi-square statistic
`df`	$(r-1)(c-1)$
`p_value`	upper-tail p-value
`expected`	expected counts under independence

- **Method:** expected counts row total x column total / n; Pearson chi-square with $(r-1)(c-1)$ df
- **Assumes:** independent observations; expected counts at least about 5; no continuity correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The chi-square test of independence for the table [[20, 30], [25, 25]]. Drawn: the chi-square density with 1 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 3.84146 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: χ² = 1.0101, p-value = 0.314879.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): table (input); expected (returned). Returned with them by statistics-ai999x: chi_square = 1.0101, df = 1, p_value = 0.314879.

```
statistics_ai999x.solve('chi_square_independence', table='[[20, 30], [25, 25]]')
# -> chi_square = 1.0101, df = 1, p_value = 0.314879, expected = [[22.5, 27.5], [22.5,
27.5]]   (SOLVED)
```

```
statistics_ai999x.solve('chi_square_independence', table=[[10, 20, 30], [20, 15, 25]])
# -> chi_square = 4.50216, df = 2, p_value = 0.105285, expected = [[15.0, 17.5, 27.5],
[15.0, 17.5, 27.5]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chi-square test of independence for the table [[20, 30], [25, 25]].	SOLVED	chi_square = 1.0101, df = 1, p_value = 0.314879, expected = [[22.5, 27.5], [22.5, 27.5]]
Is there an association? Use a chi-square test on the contingency table [[10, 20, 30], [20, 15, 25]].	SOLVED	chi_square = 4.50216, df = 2, p_value = 0.105285, expected = [[15.0, 17.5, 27.5], [15.0, 17.5, 27.5]]
Chi-square test of independence for the table [[20, 30], [25]].	INVALID_INPUT	table: expected a rectangular matrix (rows of equal length).
Chi-square test of independence for the table [[0, 0], [25, 25]].	INVALID_INPUT	Chi-square test of independence: every row and column needs a positive total.

Chi-square test for a variance, `chi_square_variance_test`

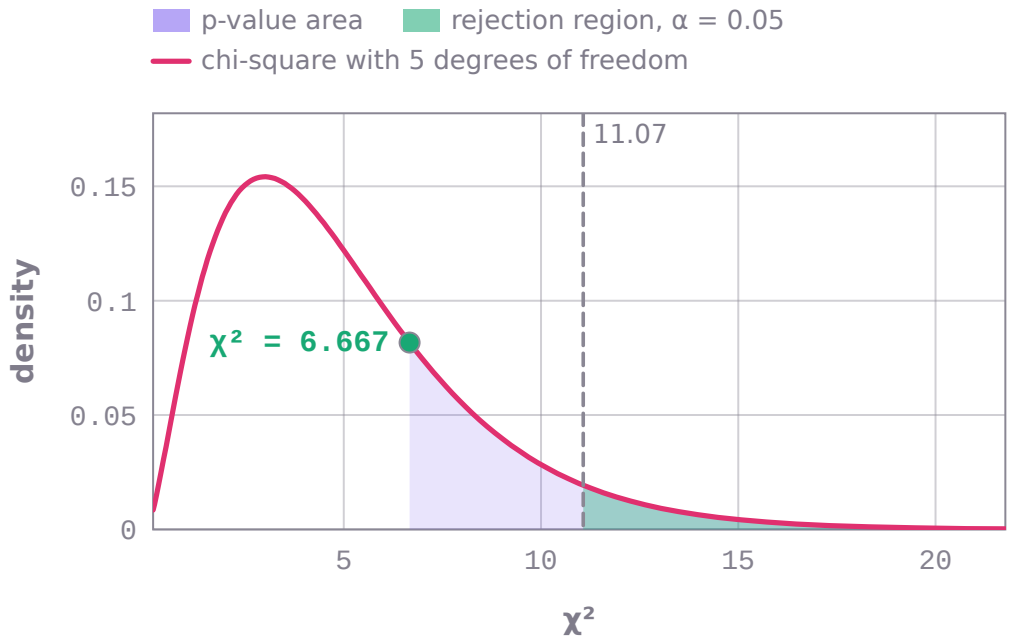
The chi-square test of H0: $\sigma = sigma0$ for a normal population from sample data.

$$\chi^2 = \frac{(n-1)s^2}{\sigma_0^2}$$

input	meaning	condition
`x`	the data values (at least two)	
`sigma0`	hypothesised standard deviation	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`chi_square`	$(n-1)\ s^2/sigma0^2$
`df`	$n-1$
`p_value`	p-value
`sample_variance`	s^2

- **Method:** chi-square statistic with $n-1$ df; two-sided p-value = twice the smaller tail
- **Assumes:** normal population (the test is sensitive to non-normality)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The chi-square test of x = 10.2, 9.8, 10.5, 10.1, 9.6, 10.4 against σ_0 = 0.3 (sample variance 0.12). Drawn: the chi-square density with 5 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 11.0705 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: χ^2 = 6.66667, p-value = 0.493268.

```
statistics_ai999x.solve('chi_square_variance_test', x='10.2, 9.8, 10.5, 10.1, 9.6, 10.4',
sigma0=0.3)
# -> chi_square = 6.66667, df = 5, p_value = 0.493268, sample_variance = 0.12   (SOLVED)
```

```
statistics_ai999x.solve('chi_square_variance_test', x=[52, 48, 55, 45, 50, 58, 41],
sigma0=4, alternative='greater')
# -> chi_square = 12.6786, df = 6, p_value = 0.0484339, sample_variance = 33.8095
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chi-square test for a variance: data [10.2, 9.8, 10.5, 10.1, 9.6, 10.4], sigma0 = 0.3, two-sided.	SOLVED	chi_square = 6.66667, df = 5, p_value = 0.493268, sample_variance = 0.12
Test with a chi-square test whether the standard deviation of [52, 48, 55, 45, 50,	SOLVED	chi_square = 12.6786, df = 6, p_value = 0.0484339, sample_variance = 33.8095

question	outcome	result
58, 41] exceeds sigma0 = 4, right-tailed.		
Chi-square test for a variance: data [10.2, 9.8, 10.5], two-sided.	NEEDS_INPUT	chi_square_variance_test needs sigma0 (hypothesised standard deviation). Example: solve('chi_square_variance_test', x='1

F-test for equality of two variances, `f_test_variances`

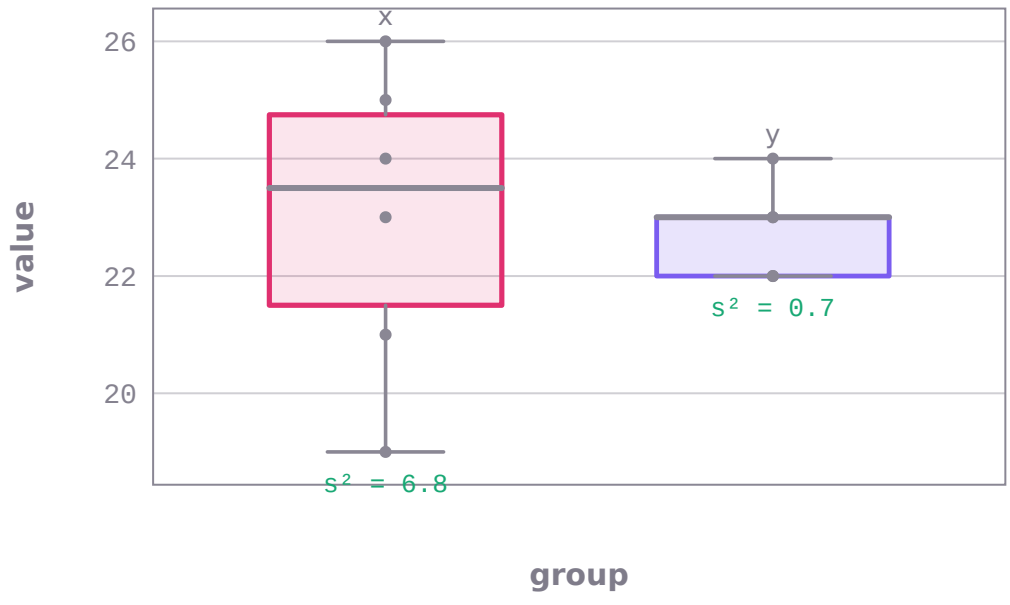
The F-test of H0: $sigma1^2 = sigma2^2$ from two independent normal samples.

$$F = \frac{s_1^2}{s_2^2}$$

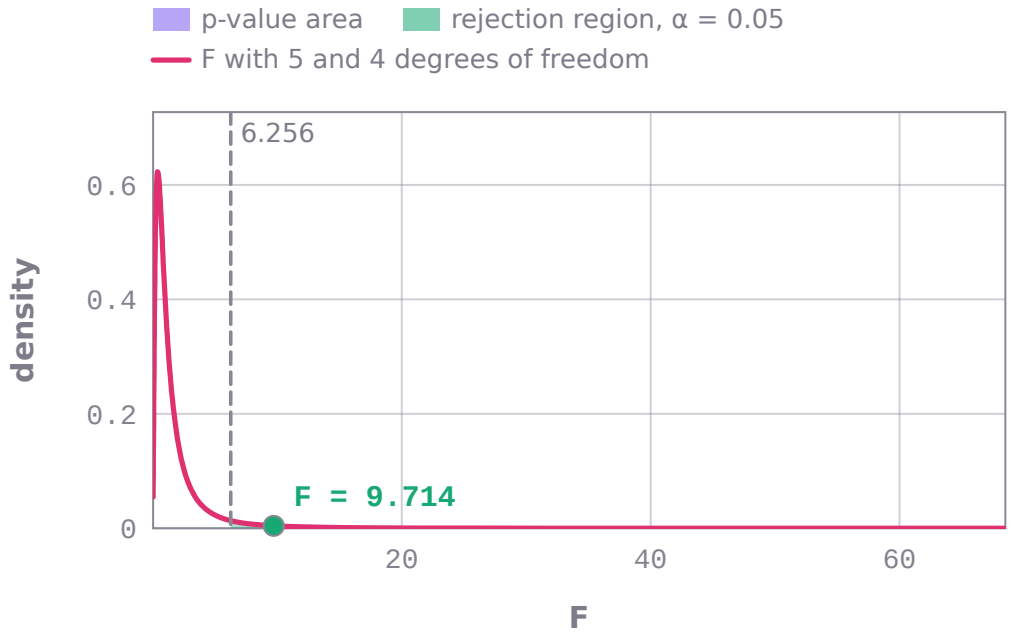
input	meaning	condition
`x`	first sample	
`y`	second sample	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`f`	s_1^2 / s_2^2
`df1`	$n_1 - 1$
`df2`	$n_2 - 1$
`p_value`	p-value

- **Method:** variance ratio with (n1 - 1, n2 - 1) df; two-sided p-value = twice the smaller tail
- **Assumes:** independent samples from normal populations (sensitive to non-normality)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The samples x = 21, 24, 19, 26, 23, 25 and y = 22, 23, 22, 24, 23 as box plots (quartiles from the package's percentiles); labelled with their sample variances from the package (6.8, 0.7). F test: F = 9.71429 with 5 and 4 degrees of freedom, p-value 0.046857.



The F test of equal variances for x = 21, 24, 19, 26, 23, 25 and y = 22, 23, 22, 24, 23 (one-sided region drawn). Drawn: the F density with 5 and 4 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 6.25606 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: F = 9.71429, p-value = 0.046857.

```
statistics_ai999x.solve('f_test_variances', x='21, 24, 19, 26, 23, 25', y='22, 23, 22, 24, 23')
# -> f = 9.71429, df1 = 5, df2 = 4, p_value = 0.046857   (SOLVED)
```

```
statistics_ai999x.solve('f_test_variances', x=[5.1, 6.3, 4.2, 7.0, 5.5], y=[5.0, 5.2, 4.9, 5.1], alternative='greater')
# -> f = 70.02, df1 = 4, df2 = 3, p_value = 0.00271012   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
F-test for equality of variances of [21, 24, 19, 26, 23, 25] and [22, 23, 22, 24, 23], two-sided.	SOLVED	f = 9.71429, df1 = 5, df2 = 4, p_value = 0.046857
Use an F-test to check whether [5.1, 6.3, 4.2, 7.0, 5.5] has a larger variance than [5.0, 5.2, 4.9, 5.1], right-tailed.	SOLVED	f = 70.02, df1 = 4, df2 = 3, p_value = 0.00271012
F-test for equality of variances of [21, 24, 19] and [22, 23, 22], one-sided.	OUT_OF_SCOPE	Out of scope: a one-sided test without a direction: say right-tailed or left-tailed.

Pearson correlation coefficient, `pearson_correlation`

Pearson's correlation coefficient r between paired data, r squared, and the two-sided p-value for H0: $\rho = 0$.

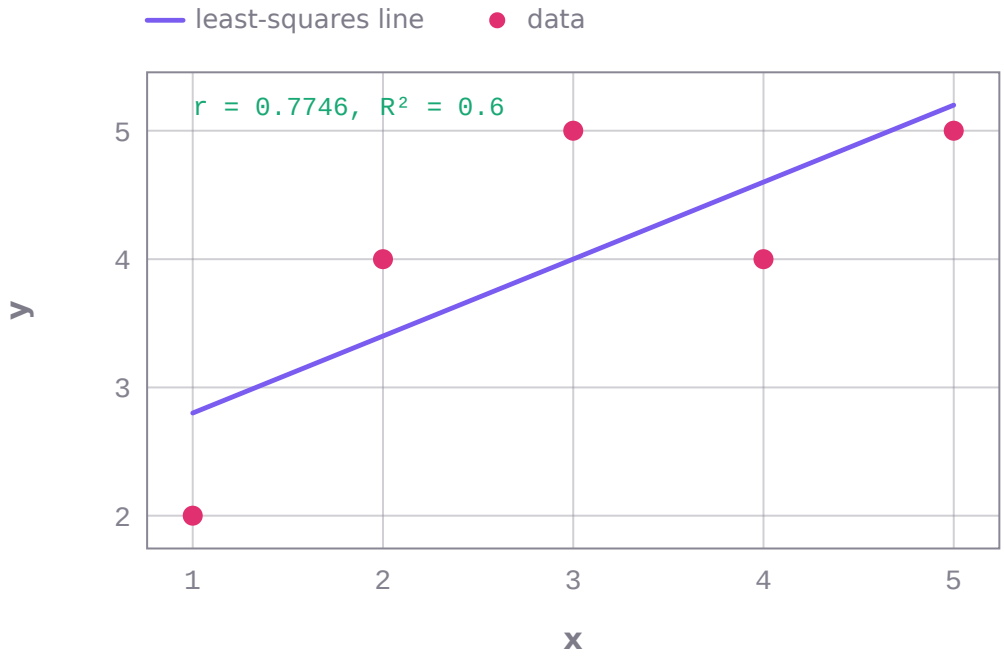
$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

input	meaning	condition
x	values of the first variable (x)	

input	meaning	condition
`y`	paired values of the second variable (y)	

output	meaning
`r`	correlation coefficient
`r_squared`	coefficient of determination
`p_value`	two-sided p-value (t with $n - 2$ df)
`n`	number of pairs

- **Method:** r = Sxy / sqrt(Sxx Syy); $t = r$ sqrt($(n-2)/(1-r^2)$)
- **Assumes:** paired observations; linear relationship; the p-value assumes bivariate normality
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The pairs x = 1, 2, 3, 4, 5, y = 2, 4, 5, 4, 5 with the least-squares line from the package's regression run (y = 2.2 + 0.6x); returned: r = 0.774597, R² = 0.6, p-value 0.124027 (n = 5).

```
statistics_ai999x.solve('pearson_correlation', x='1, 2, 3, 4, 5', y='2, 4, 5, 4, 5')
# -> r = 0.774597, r_squared = 0.6, p_value = 0.124027, n = 5   (SOLVED)
```

```
statistics_ai999x.solve('pearson_correlation', x=[43, 21, 25, 42, 57, 59], y=[99, 65, 79,
75, 87, 81])
# -> r = 0.529809, r_squared = 0.280697, p_value = 0.279645, n = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the correlation coefficient of x = [1, 2, 3, 4, 5] and y = [2, 4, 5, 4, 5].	SOLVED	r = 0.774597, r_squared = 0.6, p_value = 0.124027, n = 5
What is the Pearson correlation between [43, 21, 25, 42, 57, 59] and [99, 65, 79, 75, 87, 81]?	SOLVED	r = 0.529809, r_squared = 0.280697, p_value = 0.279645, n = 6

question	outcome	result
Find the correlation coefficient of x = [1, 2, 3] and y = [2, 4].	INVALID_INPUT	Pearson correlation coefficient: x and y must have the same number of values (paired data).
Find the correlation coefficient of x = [1, 1, 1] and y = [2, 4, 5].	INVALID_INPUT	Pearson correlation coefficient: a variable is constant: the correlation is undefined.

Significance test for a correlation coefficient, `correlation_t_test`

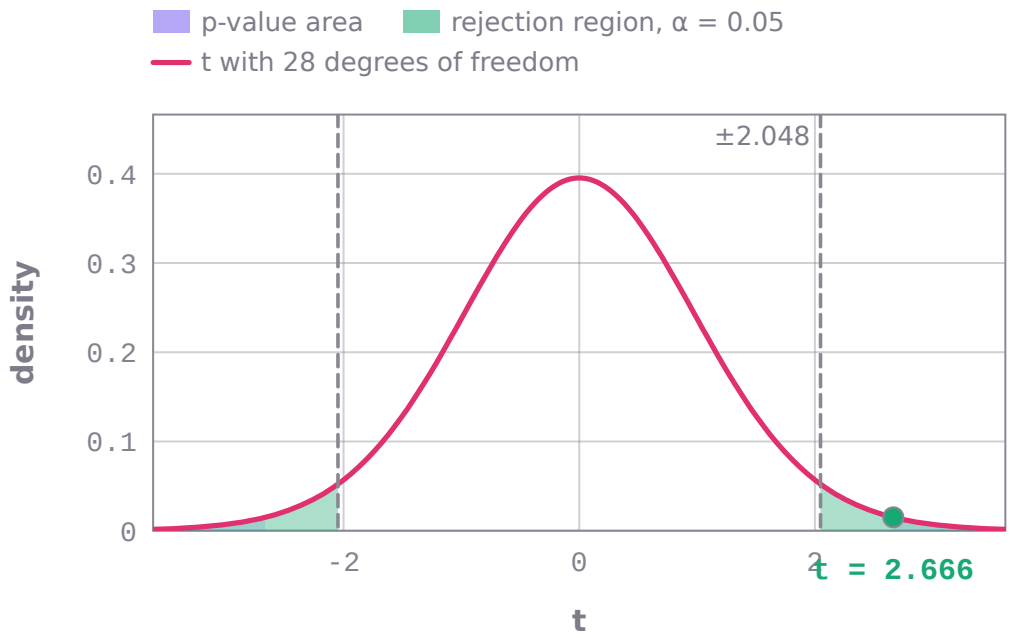
The t-test of H0: $\rho = 0$ from a sample correlation r and the number of pairs n.

$$t = r\sqrt{\frac{n-2}{1-r^2}}$$

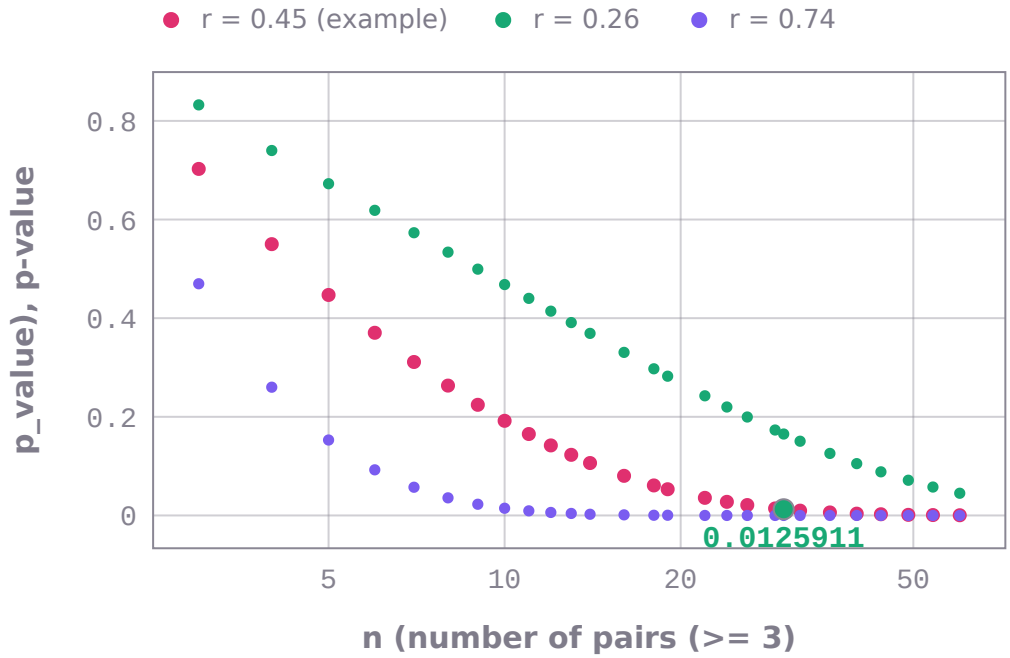
input	meaning	condition
`r`	sample correlation coefficient	
`n`	number of pairs (>= 3)	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`t`	r sqrt($(n-2)/(1-r^2)$)
`df`	$n-2$
`p_value`	p-value

- **Method:** t statistic with $n-2$ degrees of freedom
- **Assumes:** $-1 < r < 1$; bivariate normal data
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The t test of a correlation r = 0.45 from n = 30 pairs. Drawn: the t density with 28 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region beyond ±2.04841 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: t = 2.66641, p-value = 0.0125911.



p_value (p-value) against n (number of pairs (>= 3)) from 1 to 60 on a logarithmic axis. One curve for each value of r (sample correlation coefficient): 0.45, 0.26, 0.74. 27 points per curve computed by statistics-ai999x; marked: the worked example, n = 30, p_value = 0.0125911.

```
statistics_ai999x.solve('correlation_t_test', r=0.45, n=30)
# -> t = 2.66641, df = 28, p_value = 0.0125911   (SOLVED)
```

```
statistics_ai999x.solve('correlation_t_test', r=-0.32, n=50, alternative='less')
# -> t = -2.34007, df = 48, p_value = 0.0117428   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Test the significance of a correlation r = 0.45 with n = 30 pairs, two-sided.	SOLVED	t = 2.66641, df = 28, p_value = 0.0125911
Is a correlation of r = -0.32 from n = 50 significant? Use a left-tailed test.	SOLVED	t = -2.34007, df = 48, p_value = 0.0117428
Test the significance of a correlation r = 1.2 with n = 30 pairs, two-sided.	INVALID_INPUT	Significance test for a correlation coefficient: r must be strictly between -1 and 1.

Confidence interval for a correlation (Fisher z), `correlation_confidence_interval`

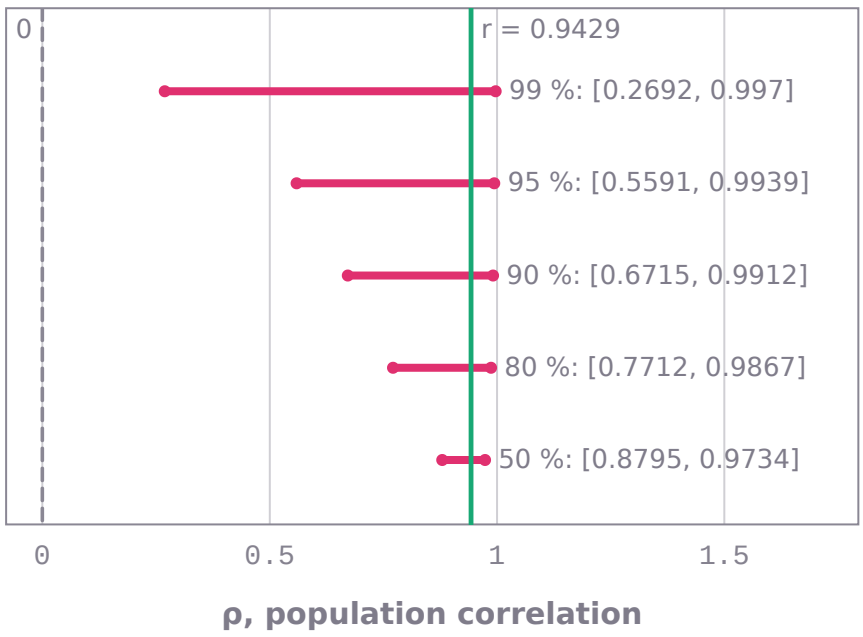
The Fisher z-transformation confidence interval for the population correlation from paired data.

$$\tanh\left(\operatorname{atanh}(r) \pm \frac{z_{\alpha/2}}{\sqrt{n-3}}\right)$$

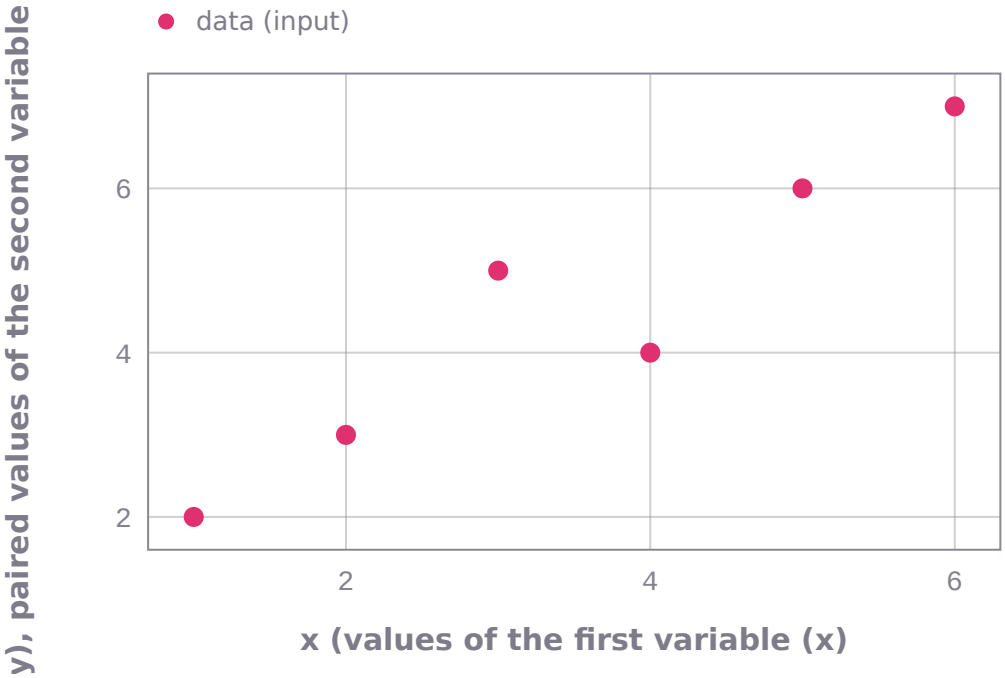
input	meaning	condition
`x`	values of the first variable (x)	
`y`	paired values of the second variable (y)	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`r`	sample correlation
`lower`	lower limit
`upper`	upper limit

- **Method:** tanh(atanh(r) +/- z_(alpha/2) / sqrt(n - 3))
- **Assumes:** bivariate normal data; at least 4 pairs; |r| < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Confidence intervals for the correlation of x = 1, 2, 3, 4, 5, 6 and y = 2, 3, 5, 4, 6, 7 (r = 0.942857) at five levels, each from the package: 50 % [0.879547, 0.973363]; 80 % [0.771198, 0.986696]; 90 % [0.671529, 0.991234]; 95 % [0.559149, 0.9939]; 99 % [0.269222, 0.997]; they are asymmetric about r and stay inside [−1, 1].



The data y (paired values of the second variable (y)) against x (values of the first variable (x)) as given in the worked example (6 points). Also returned: r = 0.942857, lower = 0.559149, upper = 0.9939. Other inputs: confidence = 0.95.

```
statistics_ai999x.solve('correlation_confidence_interval', x='1, 2, 3, 4, 5, 6', y='2, 3, 5, 4, 6, 7', confidence=0.95)
# -> r = 0.942857, lower = 0.559149, upper = 0.9939   (SOLVED)
```

```
statistics_ai999x.solve('correlation_confidence_interval', x=[43, 21, 25, 42, 57, 59], y=[99, 65, 79, 75, 87, 81], confidence=0.9)
# -> r = 0.529809, lower = -0.345018, upper = 0.912042   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 95% confidence interval for the correlation of x = [1, 2, 3, 4, 5, 6] and y = [2, 3, 5, 4, 6, 7].	SOLVED	r = 0.942857, lower = 0.559149, upper = 0.9939
Construct a 90% confidence interval for the correlation between [43, 21, 25, 42, 57, 59] and [99, 65, 79, 75, 87, 81].	SOLVED	r = 0.529809, lower = -0.345018, upper = 0.912042
Find the 95% confidence interval for the correlation of x = [1, 2, 3] and y = [2, 3, 5].	INVALID_INPUT	Confidence interval for a correlation (Fisher z): the paired data needs at least 4 values.

Spearman rank correlation, `spearman_correlation`

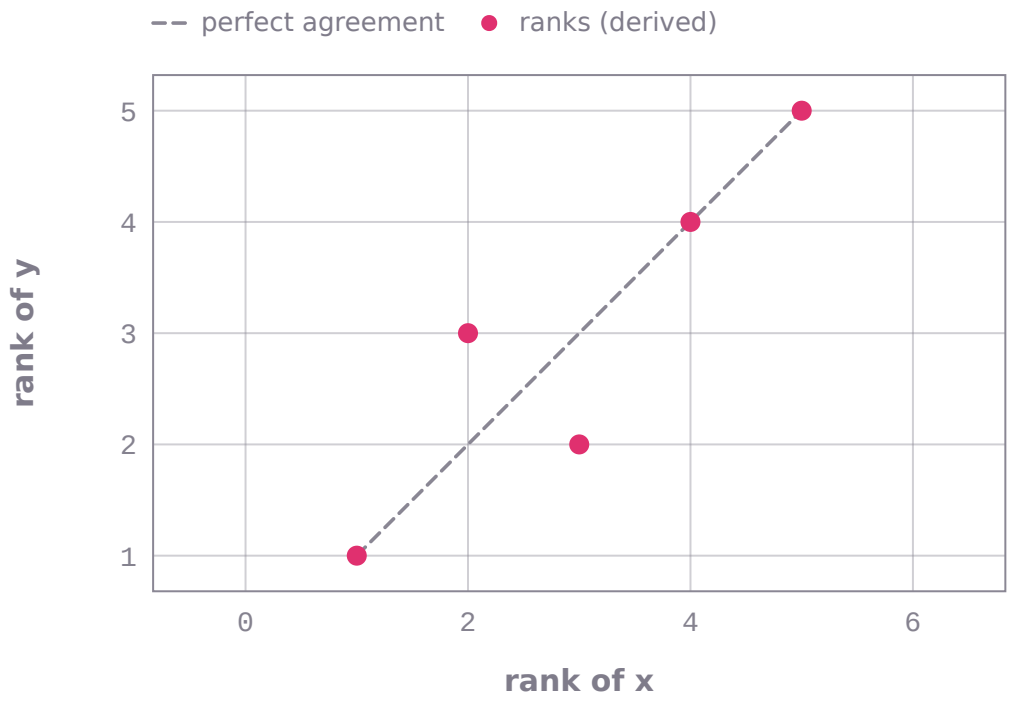
Spearman's rank correlation coefficient (ties given average ranks) with its two-sided p-value.

$$\rho_s = r\left(\mathrm{rank}(x), \mathrm{rank}(y)\right)$$

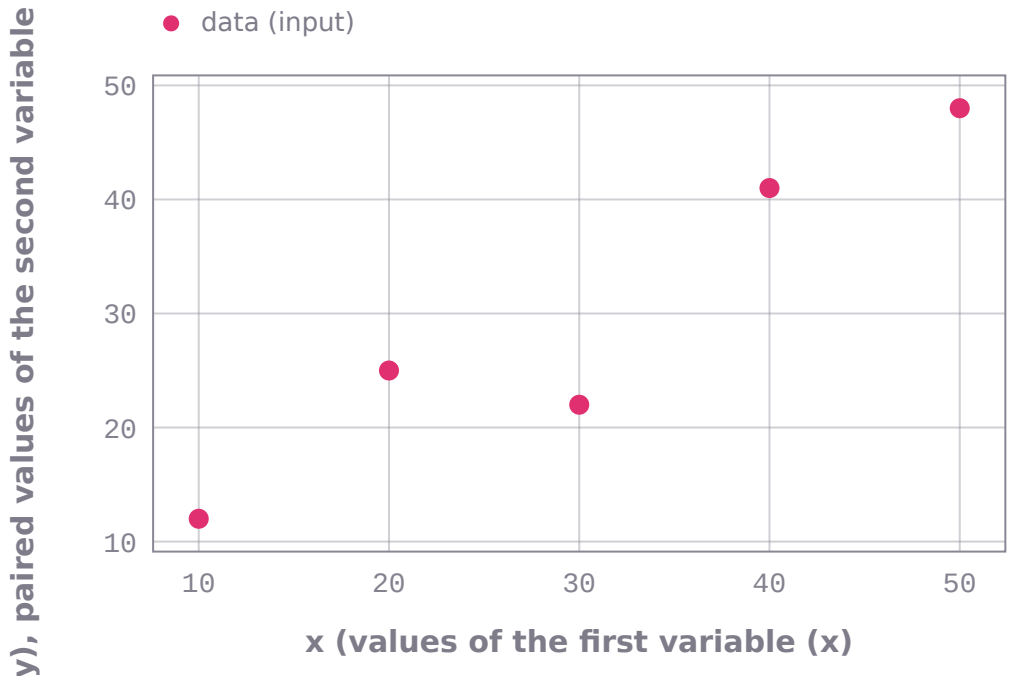
input	meaning	condition
`x`	values of the first variable (x)	
`y`	paired values of the second variable (y)	

output	meaning
`rho`	Spearman's rho
`p_value`	two-sided p-value (t approximation, $n - 2$ df)
`n`	pairs

- **Method:** Pearson correlation of the average ranks
- **Assumes:** paired ordinal or continuous data; p-value by the t approximation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The ranks of x = 10, 20, 30, 40, 50 against the ranks of y = 12, 25, 22, 41, 48 (ranks derived from the inputs); Spearman's ρ is the correlation of these ranks: returned ρ = 0.9, p-value 0.0373861.



The data y (paired values of the second variable (y)) against x (values of the first variable (x)) as given in the worked example (5 points). Also returned: rho = 0.9, p_value = 0.0373861, n = 5.

```
statistics_ai999x.solve('spearman_correlation', x='10, 20, 30, 40, 50', y='12, 25, 22, 41, 48')
# -> rho = 0.9, p_value = 0.0373861, n = 5   (SOLVED)
```

```
statistics_ai999x.solve('spearman_correlation', x=[3, 1, 4, 1, 5, 9], y=[2, 7, 1, 8, 2, 8])
# -> rho = -0.134343, p_value = 0.799697, n = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Spearman rank correlation of x = [10, 20, 30, 40, 50] and y = [12, 25, 22, 41, 48].	SOLVED	rho = 0.9, p_value = 0.0373861, n = 5
What is Spearman's rho between [3, 1, 4, 1, 5, 9] and [2, 7, 1, 8, 2, 8]?	SOLVED	rho = -0.134343, p_value = 0.799697, n = 6
Find the Spearman rank correlation of x = [10, 20] and y = [12, 25].	INVALID_INPUT	Spearman rank correlation: the paired data needs at least 3 values.

Kendall's tau-b rank correlation, `kendall_tau`

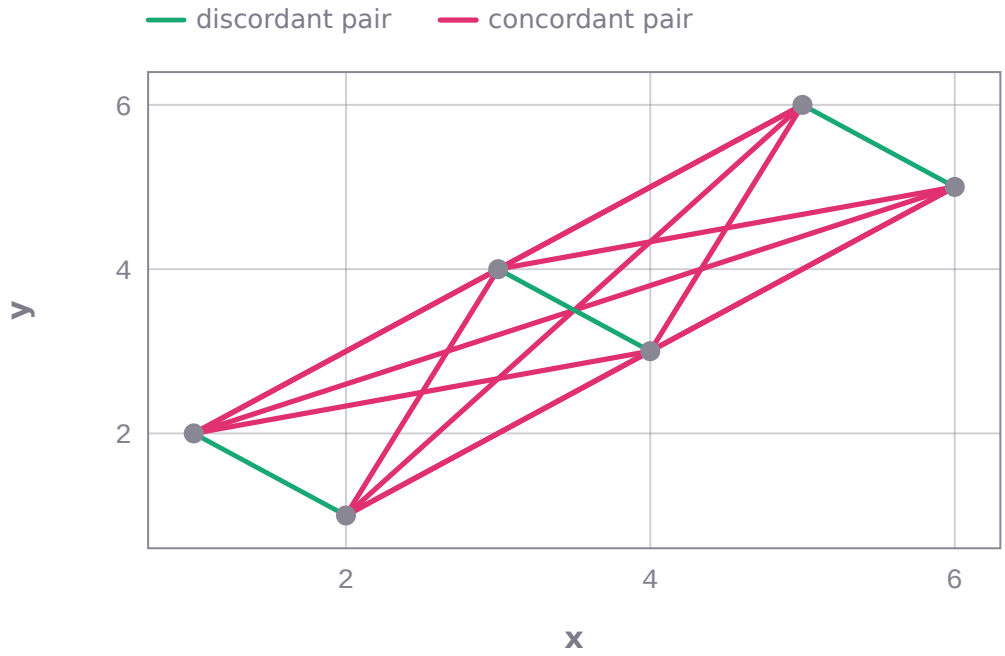
Kendall's tau-b (tie-adjusted) between paired data with its two-sided large-sample p-value.

$$\tau_b = \frac{C - D}{\sqrt{(n_0 - n_1)(n_0 - n_2)}}$$

input	meaning	condition
`x`	values of the first variable (x)	
`y`	paired values of the second variable (y)	

output	meaning
`tau`	Kendall's tau-b
`p_value`	two-sided p-value (normal approximation with tie correction)
`concordant`	concordant pairs
`discordant`	discordant pairs

- **Method:** count concordant and discordant pairs; $\tau - b = (C - D)/sqrt((n_0 - n_1)(n_0 - n_2))$; tie-corrected variance of C - D
- **Assumes:** paired ordinal or continuous data; large-sample normal approximation for the p-value
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Every pair of the points x = 1, 2, 3, 4, 5, 6, y = 2, 1, 4, 3, 6, 5 joined: rising segments (concordant) in the first colour, falling (discordant) in the second (classified from the inputs). Returned: 12 concordant, 3 discordant, τ = 0.6, p-value 0.0908739.

```
statistics_ai999x.solve('kendall_tau', x='1, 2, 3, 4, 5, 6', y='2, 1, 4, 3, 6, 5')
# -> tau = 0.6, p_value = 0.0908739, concordant = 12, discordant = 3   (SOLVED)
```

```
statistics_ai999x.solve('kendall_tau', x=[12, 2, 1, 12, 2], y=[1, 4, 7, 1, 0])
# -> tau = -0.471405, p_value = 0.282745, concordant = 2, discordant = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find Kendall's tau for x = [1, 2, 3, 4, 5, 6] and y = [2, 1, 4, 3, 6, 5].	SOLVED	tau = 0.6, p_value = 0.0908739, concordant = 12, discordant = 3
What is the Kendall rank correlation between [12, 2, 1, 12, 2] and [1, 4, 7, 1, 0]?	SOLVED	tau = -0.471405, p_value = 0.282745, concordant = 2, discordant = 6
Find Kendall's tau for x = [1, 1, 1] and y = [2, 1, 4].	INVALID_INPUT	Kendall's tau-b rank correlation: a variable is constant: Kendall's tau is undefined.

Covariance — `covariance`

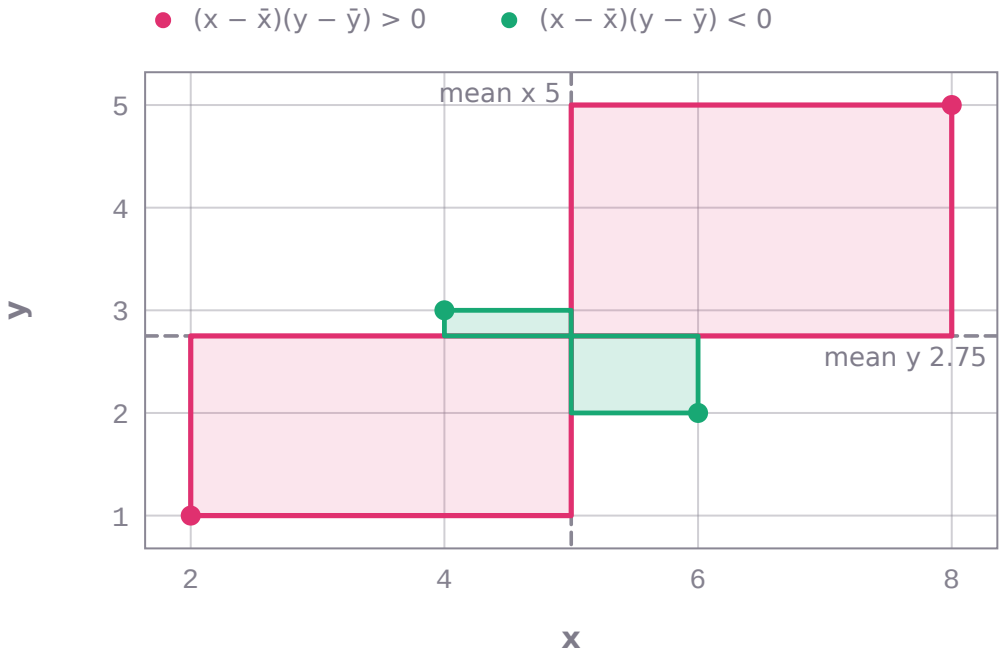
The sample $(n-1)$ and population (n) covariance of paired data.

$$s_{xy} = \frac{1}{n-1}\sum(x_i-\bar{x})(y_i-\bar{y})$$

input	meaning	condition
x	values of the first variable (x)	
y	paired values of the second variable (y)	

output	meaning
`sample_covariance`	divisor $n - 1$
`population_covariance`	divisor n

- **Method:** sum of products of deviations divided by $n - 1$ (sample) or n (population)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The pairs x = 2, 4, 6, 8, y = 1, 3, 2, 5 with the means of x and y (from the package) as a cross; each rectangle is a product (x − x̄)(y − ȳ) (derived), positive in the first colour, negative in the second. Their average is the covariance: sample 3.66667, population 2.75.

```
statistics_ai999x.solve('covariance', x='2, 4, 6, 8', y='1, 3, 2, 5')
# -> sample_covariance = 3.66667, population_covariance = 2.75   (SOLVED)
```

```
statistics_ai999x.solve('covariance', x=[1.5, 2.5, 3.0], y=[10, 12, 15])
# -> sample_covariance = 1.83333, population_covariance = 1.22222   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the covariance of x = [2, 4, 6, 8] and y = [1, 3, 2, 5].	SOLVED	sample_covariance = 3.66667, population_covariance = 2.75
What is the covariance between [1.5, 2.5, 3.0] and [10, 12, 15]?	SOLVED	sample_covariance = 1.83333, population_covariance = 1.22222
Find the covariance of x = [2, 4, 6] and y = [1, 3].	INVALID_INPUT	Covariance: x and y must have the same number of values (paired data).

Simple linear regression (least squares line) — `simple_linear_regression`

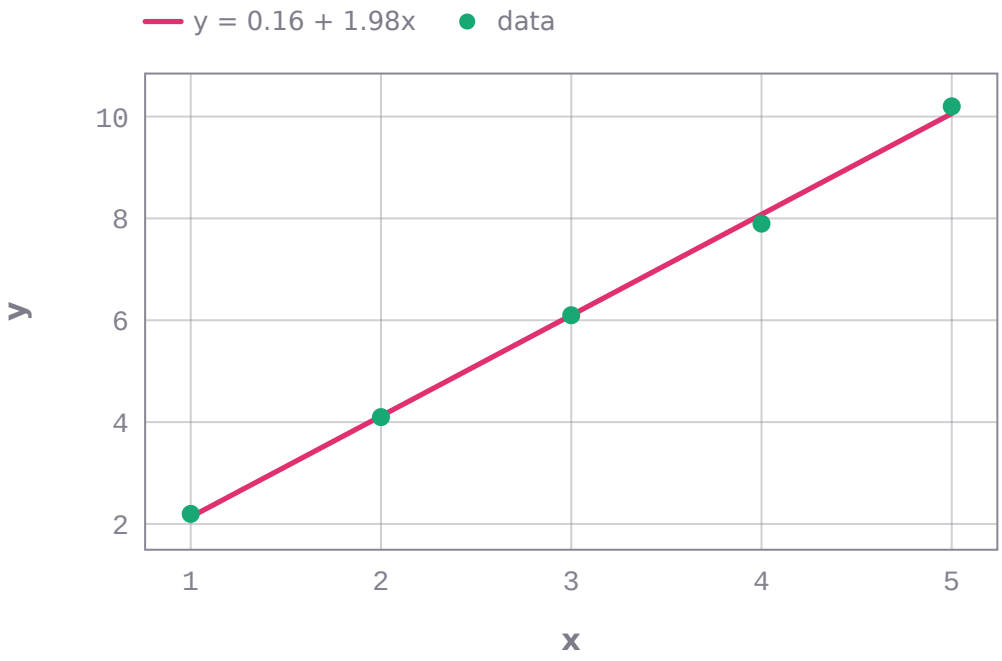
The least-squares line y = intercept + slope x: coefficients, r, r squared, standard errors and the slope's p-value.

$$\hat{y} = b_0 + b_1 x, \quad b_1 = \frac{S_{xy}}{S_{xx}}, \quad b_0 = \bar{y} - b_1 \bar{x}$$

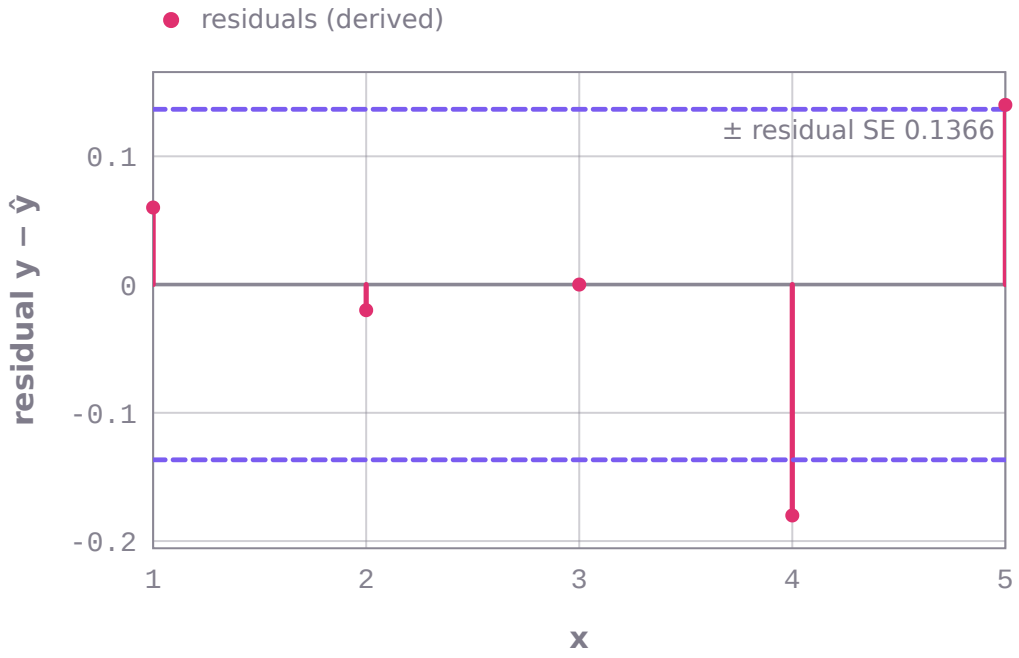
input	meaning	condition
`x`	values of the first variable (x)	
`y`	paired values of the second variable (y)	

output	meaning
`slope`	b1 = Sxy/Sxx
`intercept`	b0 = ybar - b1 xbar
`r`	correlation
`r_squared`	coefficient of determination
`p_value`	two-sided p-value for H0: slope = 0
`slope_se`	standard error of the slope
`intercept_se`	standard error of the intercept
`residual_se`	residual standard error

- **Method:** ordinary least squares; t test on the slope with $n - 2$ df
- **Assumes:** at least 3 pairs, x not all equal; inference assumes independent normal errors with constant variance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The least-squares line through x = 1, 2, 3, 4, 5, y = 2.2, 4.1, 6.1, 7.9, 10.2: intercept 0.16 (standard error 0.143295), slope 1.98 (standard error 0.0432049), r = 0.999287, R² = 0.998574, p-value of the slope 2.2873e-05, as returned by the package.



Residuals of x = 1, 2, 3, 4, 5, y = 2.2, 4.1, 6.1, 7.9, 10.2 from the returned line ŷ = 0.16 + 1.98x (derived from the inputs and the returned coefficients), with the returned residual standard error 0.136626 either side of zero (R^2 = 0.998574): no pattern means the straight line fits.

```
statistics_ai999x.solve('simple_linear_regression', x='1, 2, 3, 4, 5', y='2.2, 4.1, 6.1, 7.9, 10.2')
# -> slope = 1.98, intercept = 0.16, r = 0.999287, r_squared = 0.998574, p_value = 2.28734e-05, slope_se = 0.0432049, intercept_se = 0.143295, residual_se = 0.136626 (SOLVED)
```

```
statistics_ai999x.solve('simple_linear_regression', x=[2, 3, 4, 5, 6], y=[65, 70, 75, 82, 88])
# -> slope = 5.8, intercept = 52.8, r = 0.99763, r_squared = 0.995266, p_value = 0.000138424, slope_se = 0.23094, intercept_se = 0.979796, residual_se = 0.730297   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the least squares regression line for x = [1, 2, 3, 4, 5] and y = [2.2, 4.1, 6.1, 7.9, 10.2].	SOLVED	slope = 1.98, intercept = 0.16, r = 0.999287, r_squared = 0.998574, p_value = 2.28734e-05, slope_se = 0.0432049, intercept_se = 0.143295, residual_se = 0.136626
Find the regression line for x = [2, 3, 4, 5, 6] and y = [65, 70, 75, 82, 88].	SOLVED	slope = 5.8, intercept = 52.8, r = 0.99763, r_squared = 0.995266, p_value = 0.000138424, slope_se = 0.23094, intercept_se = 0.979796, residual_se = 0.730297
Find the regression line for x = [2, 2, 2] and y = [65, 70, 75].	INVALID_INPUT	Simple linear regression (least squares line): all x values are equal: the slope is undefined.

Prediction from a least squares line — `regression_prediction`

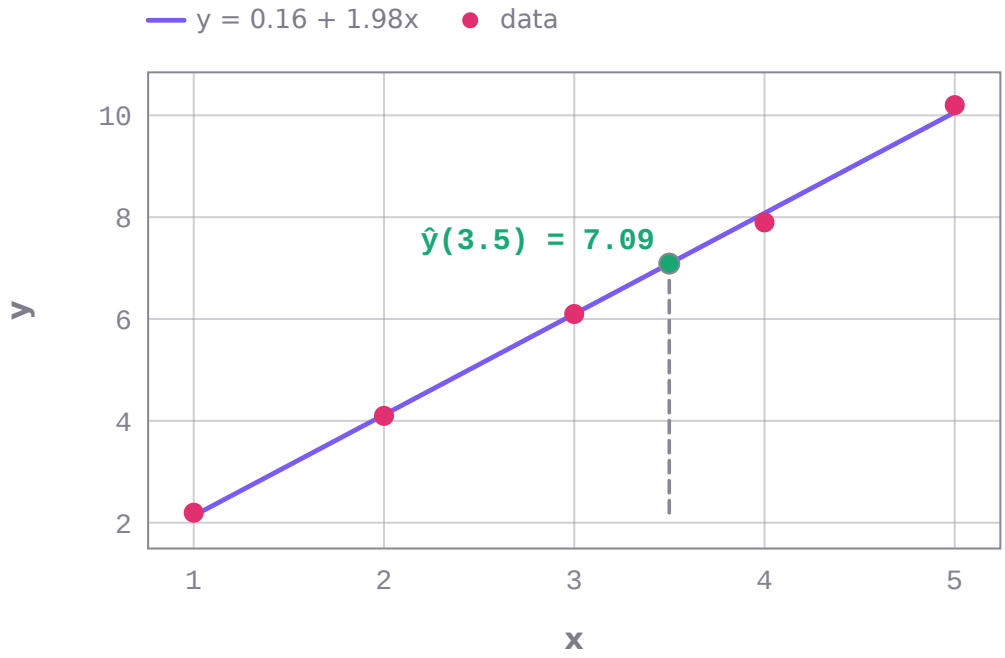
The predicted y at a given x0 from the least-squares line fitted to paired data.

$$\hat{y}_0 = b_0 + b_1 x_0$$

input	meaning	condition
`x`	values of the first variable (x)	
`y`	paired values of the second variable (y)	
`x0`	the x value to predict at	

output	meaning
`y_hat`	$b_0 + b_1$ x0
`slope`	slope
`intercept`	intercept

- **Method:** fit by least squares, evaluate at x0
- **Assumes:** x0 within the range of the data (extrapolation is unreliable)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data x = 1, 2, 3, 4, 5, y = 2.2, 4.1, 6.1, 7.9, 10.2 with the returned least-squares line (intercept 0.16, slope 1.98); marked: the prediction at x_0 = 3.5, $\hat{y}$ = 7.09.

```
statistics_ai999x.solve('regression_prediction', x='1, 2, 3, 4, 5', y='2.2, 4.1, 6.1, 7.9, 10.2', x0=3.5)
# -> y_hat = 7.09, slope = 1.98, intercept = 0.16   (SOLVED)
```

```
statistics_ai999x.solve('regression_prediction', x=[2, 3, 4, 5, 6], y=[65, 70, 75, 82, 88], x0=4.5)
# -> y_hat = 78.9, slope = 5.8, intercept = 52.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Using the regression line for x = [1, 2, 3, 4, 5] and y = [2.2, 4.1, 6.1, 7.9, 10.2], predict y at x0 = 3.5.	SOLVED	y_hat = 7.09, slope = 1.98, intercept = 0.16

question	outcome	result
Fit a least squares line to x = [2, 3, 4, 5, 6] and y = [65, 70, 75, 82, 88] and predict y when x0 = 4.5.	SOLVED	y_hat = 78.9, slope = 5.8, intercept = 52.8
Using the regression line for x = [1, 2, 3, 4, 5] and y = [2.2, 4.1, 6.1, 7.9, 10.2], predict y.	NEEDS_INPUT	regression_prediction needs x0 (the x value to predict at). Example: solve('regression_prediction', x='1, 2, 3, 4, 5', y

Multiple linear regression — `multiple_linear_regression`

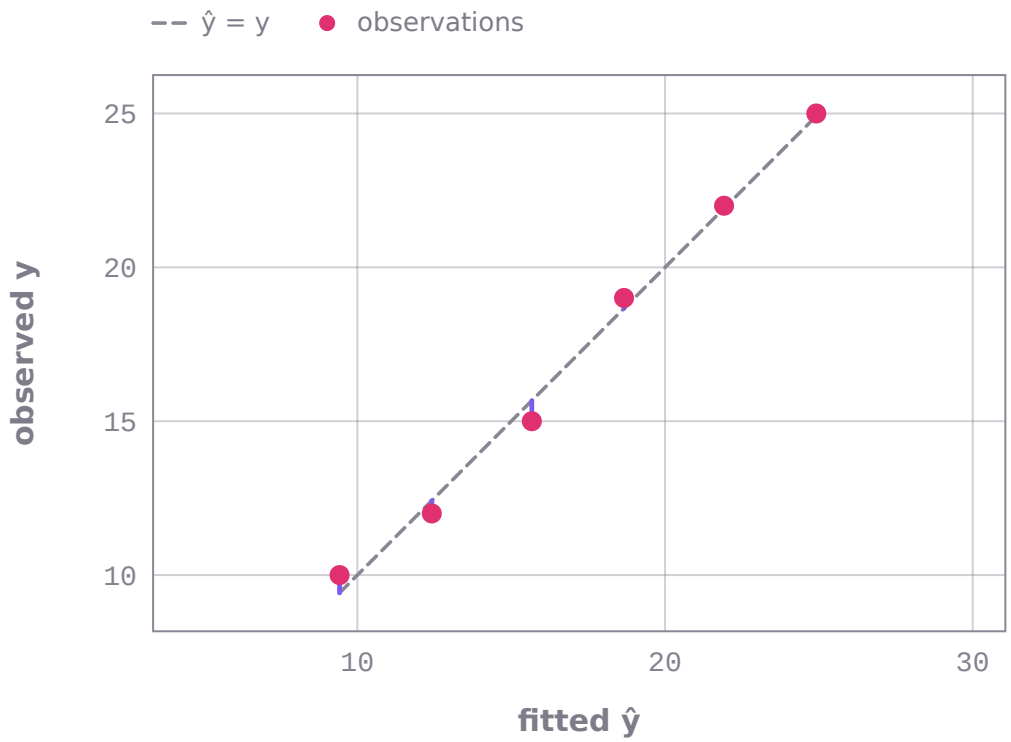
Ordinary least squares with several predictors: coefficients (intercept first), R squared, adjusted R squared and the overall F-test.

$$\hat{\beta} = (X^\top X)^{-1} X^\top y, \quad R^2 = 1 - \frac{SSE}{SST}$$

input	meaning	condition
`X`	predictor matrix: one row per observation, one column per predictor	
`y`	response values	

output	meaning
`coefficients`	[intercept, b1, ..., bk]
`r_squared`	R squared
`adj_r_squared`	adjusted R squared
`f`	overall F statistic
`f_p_value`	p-value of the F test
`residual_se`	residual standard error

- **Method:** centred normal equations solved by Gaussian elimination with partial pivoting; F = (SSR/k)/(SSE/(n-k-1))
- **Assumes:** $n > k + 1$; predictors not collinear; inference assumes independent normal errors
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Observed y = 10, 12, 15, 19, 22, 25 against the fitted values (derived from the returned coefficients 6.22917, 3.0625, 0.0625: intercept first, then one per column of X = [[1, 2], [2, 1], [3, 4], [4, 3], [5, 6], [6, 5]]); vertical segments are the residuals. Returned: R^2 = 0.993659 (adjusted 0.989431), F = 235.038, residual SE 0.600925.

```
statistics_ai999x.solve('multiple_linear_regression', X='[[1, 2], [2, 1], [3, 4], [4, 3], [5, 6], [6, 5]]', y='10, 12, 15, 19, 22, 25')
# -> coefficients = [6.229166666666668, 3.0625, 0.06249999999999978], r_squared = 0.993659, adj_r_squared = 0.989431, f = 235.038, f_p_value = 0.000504992, residual_se = 0.600925 (SOLVED)
```

```
statistics_ai999x.solve('multiple_linear_regression', X=[[2, 50], [3, 60], [5, 55], [7, 70], [8, 65], [9, 80], [11, 75]], y=[20, 25, 28, 35, 36, 42, 44])
# -> coefficients = [3.5407917383821115, 1.937349397590363, 0.25941480206540407], r_squared = 0.998612, adj_r_squared = 0.997919, f = 1439.4, f_p_value = 1.92528e-06, residual_se = 0.405003   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a multiple regression of y = [10, 12, 15, 19, 22, 25] on X = [[1, 2], [2, 1], [3, 4], [4, 3], [5, 6], [6, 5]].	SOLVED	coefficients = [6.229166666666668, 3.0625, 0.06249999999999978], r_squared = 0.993659, adj_r_squared = 0.989431, f = 235.038, f_p_value = 0.000504992, residual_se = 0.600925
Multiple linear regression with predictors X = [[2, 50], [3, 60], [5, 55], [7, 70], [8, 65], [9, 80], [11, 75]] and y = [20, 25, 28, 35, 36, 42, 44].	SOLVED	coefficients = [3.5407917383821115, 1.937349397590363, 0.25941480206540407], r_squared = 0.998612, adj_r_squared = 0.997919, f = 1439.4, f_p_value = 1.92528e-06, residual_se = 0.405003
Fit a multiple regression of y = [1, 2, 3] on X = [[1, 2], [2, 4], [3, 6]].	INVALID_INPUT	Multiple linear regression: more observations than predictors + 1 are required.

Durbin-Watson statistic — `durbin_watson`

The Durbin-Watson statistic of regression residuals (about 2: no first-order autocorrelation; towards 0: positive; towards 4: negative).

$$d = \frac{\sum_{t=2}^{n}(e_t - e_{t-1})^2}{\sum_{t=1}^{n} e_t^2}$$

input	meaning	condition
`residuals`	regression residuals in time order	

output	meaning
`durbin_watson`	sum of squared successive differences / sum of squares

- **Method:** d = sum (e_t - e_(t-1))^2 / sum e_t^2
- **Assumes:** residuals in time order
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Regression residuals 0.5, -0.2, 0.3, -0.4, 0.1, 0.6, -0.5 in time order, consecutive ones joined. The returned Durbin–Watson statistic 2.53448 compares successive differences with the residuals: near 2 no autocorrelation, below 2 positive, above 2 negative (residuals that alternate in sign).

```
statistics_ai999x.solve('durbin_watson', residuals='0.5, -0.2, 0.3, -0.4, 0.1, 0.6, -0.5')
# -> durbin_watson = 2.53448   (SOLVED)
```

```
statistics_ai999x.solve('durbin_watson', residuals=[1.2, 0.8, 0.5, -0.1, -0.6, -0.9])
# -> durbin_watson = 0.270655   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Durbin-Watson statistic for residuals [0.5, -0.2, 0.3, -0.4, 0.1, 0.6, -0.5].	SOLVED	durbin_watson = 2.53448
What is the Durbin-Watson statistic of the residuals [1.2, 0.8, 0.5, -0.1, -0.6, -0.9]?	SOLVED	durbin_watson = 0.270655
Compute the Durbin-Watson statistic for residuals [0, 0, 0].	INVALID_INPUT	Durbin-Watson statistic: all residuals are zero.

One-way ANOVA — `one_way_anova`

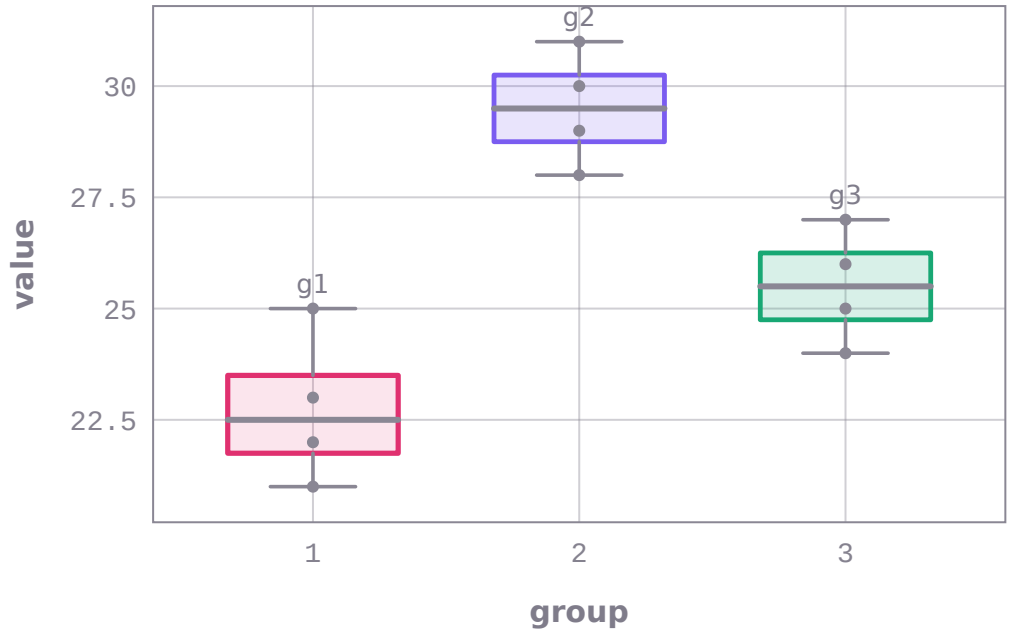
One-way analysis of variance for 2 to 5 independent groups: F statistic, p-value, sums of squares and eta squared.

$$F = \frac{SS_B/(k-1)}{SS_W/(N-k)}$$

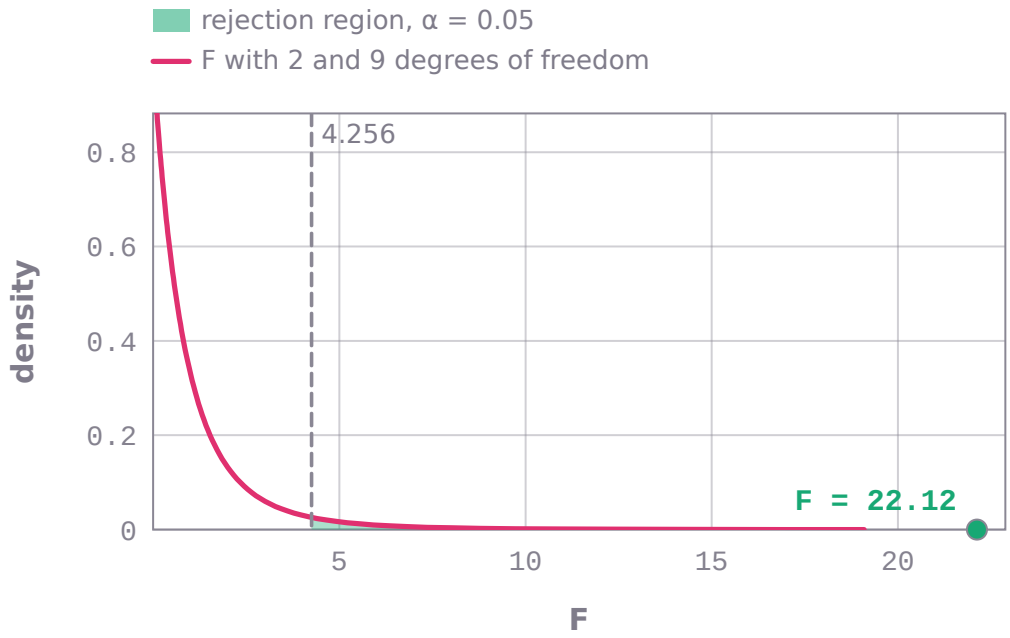
input	meaning	condition
`g1`	group 1	
`g2`	group 2	
`g3`	group 3	
`g4`	group 4	
`g5`	group 5	

output	meaning
`f`	MS between / MS within
`p_value`	upper-tail p-value
`df_between`	$k-1$
`df_within`	N - k
`ss_between`	between-group sum of squares
`ss_within`	within-group sum of squares
`ms_between`	mean square between
`ms_within`	mean square within
`eta_squared`	SSB / SST

- **Method:** partition the total sum of squares into between- and within-group parts; F with (k - 1, N - k) df
- **Assumes:** independent groups; roughly normal populations with equal variances
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Box plots of 3 groups (g1 = 23, 25, 21, 22; g2 = 30, 28, 31, 29; g3 = 26, 27, 24, 25): each box spans the quartiles and the bar is the median, all from the package's percentiles (g1 median 22.5, quartiles 21.75–23.5; g2 median 29.5, quartiles 28.75–30.25; g3 median 25.5, quartiles 24.75–26.25). One-way ANOVA: F = 22.12 with 2 and 9 degrees of freedom, p-value 0.000335753, η^2 = 0.830954.



One-way ANOVA of g1 = 23, 25, 21, 22; g2 = 30, 28, 31, 29; g3 = 26, 27, 24, 25 (SS between 92.1667, SS within 18.75). Drawn: the F density with 2 and 9 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 4.25649 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: F = 22.12, p-value = 0.000335753.

```
statistics_ai999x.solve('one_way_anova', g1='23, 25, 21, 22', g2='30, 28, 31, 29', g3='26, 27, 24, 25')
# -> f = 22.12, p_value = 0.000335753, df_between = 2, df_within = 9, ss_between = 92.1667, ss_within = 18.75, ms_between = 46.0833, ms_within = 2.08333, eta_squared = 0.830954 (SOLVED)
```

```
statistics_ai999x.solve('one_way_anova', g1=[5.1, 4.8, 5.5], g2=[6.2, 6.0, 6.8, 6.4], g3=
[5.9, 5.6, 6.1])
# -> f = 12.2935, p_value = 0.00512341, df_between = 2, df_within = 7, ss_between =
2.54067, ss_within = 0.723333, ms_between = 1.27033, ms_within = 0.103333, eta_squared =
0.778391   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a one-way ANOVA on groups [23, 25, 21, 22], [30, 28, 31, 29] and [26, 27, 24, 25].	SOLVED	f = 22.12, p_value = 0.000335753, df_between = 2, df_within = 9, ss_between = 92.1667, ss_within = 18.75, ms_between = 46.0833, ms_within = 2.08333, eta_squared = 0.830954
One-way analysis of variance for [5.1, 4.8, 5.5], [6.2, 6.0, 6.8, 6.4] and [5.9, 5.6, 6.1].	SOLVED	f = 12.2935, p_value = 0.00512341, df_between = 2, df_within = 7, ss_between = 2.54067, ss_within = 0.723333, ms_between = 1.27033, ms_within = 0.103333, eta_squared = 0.778391
Perform a one-way ANOVA on groups [1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12].	OUT_OF_SCOPE	Out of scope: 6 data lists were given; this capability takes at most 5.
Perform a one-way ANOVA on groups [23, 25, 21, 22].	NEEDS_INPUT	one_way_anova needs g2 (group 2). Example: solve('one_way_anova', g1='23, 25, 21, 22', g2='30, 28, 31, 29', g3='26, 27,

Randomized complete block design (two-way ANOVA without replication) — `randomized_block_anova`

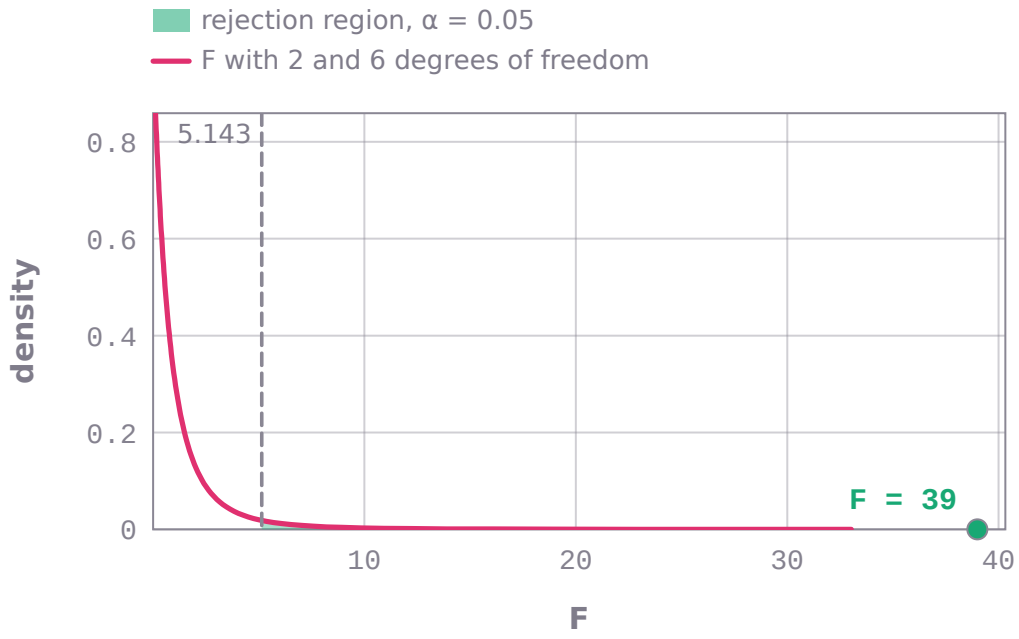
Two-way ANOVA for a randomized complete block design: rows are blocks, columns are treatments, one observation per cell; F-tests for treatments and blocks.

$$F_{\text{trt}} = \frac{SS_{\text{trt}}/(t-1)}{SS_E/((t-1)(b-1))}$$

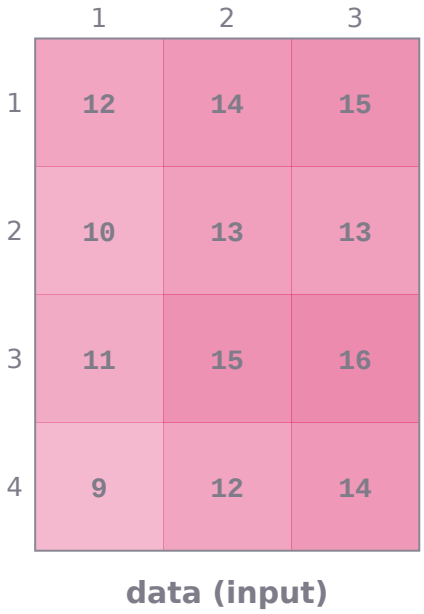
input	meaning	condition
`data`	table with one row per block and one column per treatment	

output	meaning
`f_treatment`	F for treatments
`p_treatment`	p-value for treatments
`f_block`	F for blocks
`p_block`	p-value for blocks
`ss_treatment`	treatment sum of squares
`ss_block`	block sum of squares
`ss_error`	error sum of squares
`df_treatment`	$t-1$
`df_block`	$b-1$
`df_error`	$(t-1)(b-1)$

- **Method:** sums of squares for treatments, blocks and the residual y_ij - row mean - column mean + grand mean
- **Assumes:** one observation per block and treatment; additive model (no block x treatment interaction); normal errors with constant variance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Randomized block ANOVA of [[12, 14, 15], [10, 13, 13], [11, 15, 16], [9, 12, 14]] (rows blocks, columns treatments): the treatment F. The block F is 9.25 (p-value 0.0114399). Drawn: the F density with 2 and 6 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 5.14325 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: F = 39, p-value = 0.000364431.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): data (input). Returned with them by statistics-ai999x: f_treatment = 39, p_treatment = 0.000364431, f_block = 9.25, p_block = 0.0114399, ss_treatment = 34.6667, ss_block = 12.3333, ss_error = 2.66667, df_treatment = 2, df_block = 3, df_error = 6.

```
statistics_ai999x.solve('randomized_block_anova', data='[[12, 14, 15], [10, 13, 13], [11, 15, 16], [9, 12, 14]]')
# -> f_treatment = 39, p_treatment = 0.000364431, f_block = 9.25, p_block = 0.0114399, ss_treatment = 34.6667, ss_block = 12.3333, ss_error = 2.66667, df_treatment = 2, df_block = 3, df_error = 6   (SOLVED)
```

```
statistics_ai999x.solve('randomized_block_anova', data=[[5.1, 5.6, 6.0, 5.8], [4.8, 5.2, 5.9, 5.5], [5.4, 5.9, 6.3, 6.1]])
# -> f_treatment = 95.5789, p_treatment = 1.86898e-05, f_block = 62.6842, p_block = 9.52754e-05, ss_treatment = 1.51333, ss_block = 0.661667, ss_error = 0.0316667, df_treatment = 3, df_block = 2, df_error = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Analyse the randomized block design with data [[12, 14, 15], [10, 13, 13], [11, 15, 16], [9, 12, 14]].	SOLVED	f_treatment = 39, p_treatment = 0.000364431, f_block = 9.25, p_block = 0.0114399, ss_treatment = 34.6667, ss_block = 12.3333, ss_error = 2.66667, df_treatment = 2, df_block = 3, df_error = 6
Two-way ANOVA without replication for [[5.1, 5.6, 6.0, 5.8], [4.8, 5.2, 5.9, 5.5], [5.4, 5.9, 6.3, 6.1]].	SOLVED	f_treatment = 95.5789, p_treatment = 1.86898e-05, f_block = 62.6842, p_block = 9.52754e-05, ss_treatment = 1.51333, ss_block = 0.661667, ss_error = 0.0316667, df_treatment = 3, df_block = 2, df_error = 6
Analyse the randomized block design with data [[12, 14, 15]].	INVALID_INPUT	Randomized complete block design (two-way ANOVA without replication): the data table needs at least 2 rows and 2 columns

Levene's test for equal variances, `levene_test`

Levene's test of equal variances across 2 to 5 groups, centred at the median (Brown-Forsythe) or the mean.

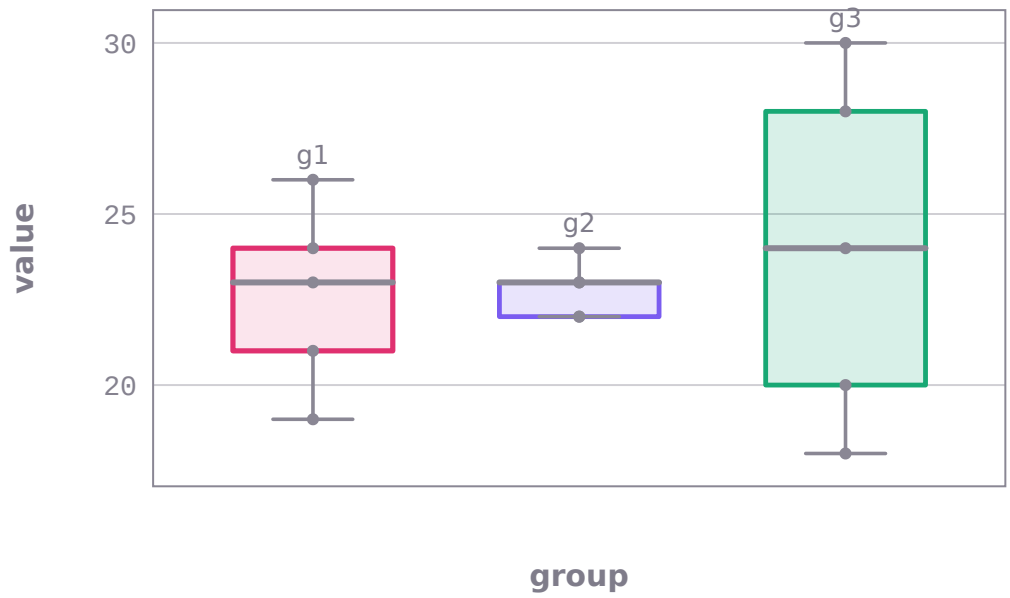
$$W = \frac{(N-k)\sum n_i(\bar{z}_{i\cdot}-\bar{z})^2}{(k-1)\sum\sum(z_{ij}-\bar{z}_{i\cdot})^2},\ z_{ij} = |x_{ij}-\tilde{x}_i|$$

input	meaning	condition
`g1`	group 1	
`g2`	group 2	
`g3`	group 3	
`g4`	group 4	
`g5`	group 5	
`center`	median (Brown-Forsythe) or mean (Levene's original)	

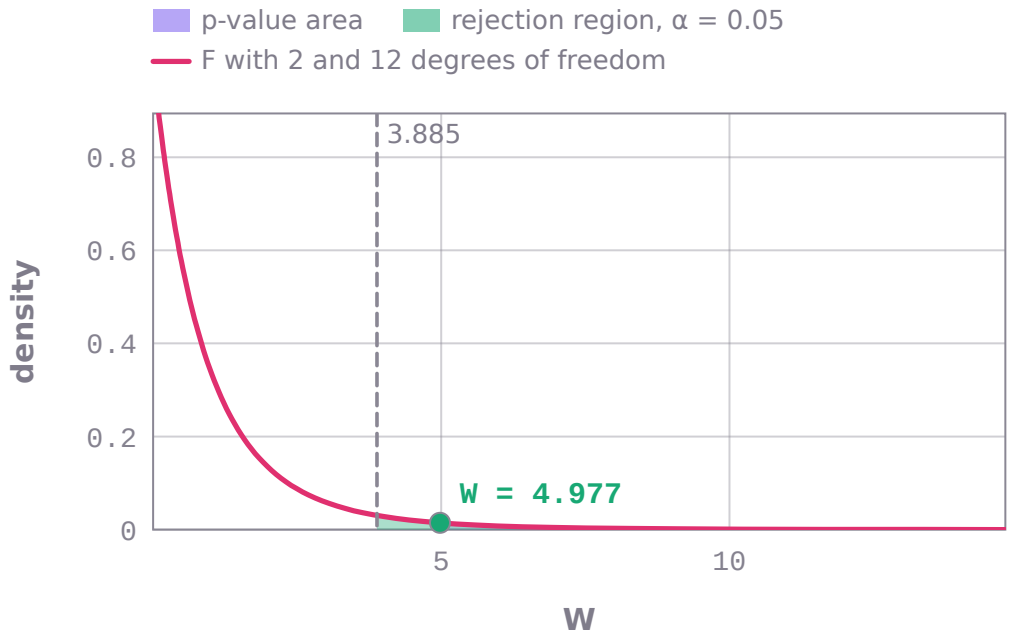
output	meaning
`w`	Levene statistic
`p_value`	p-value
`df1`	$k-1$
`df2`	N - k

- **Method:** one-way ANOVA on the absolute deviations from each group's median (or mean)

- **Assumes:** independent groups, at least 2 values each
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Box plots (quartiles from the package's percentiles) and data of 3 groups (g1 = 21, 24, 19, 26, 23; g2 = 22, 23, 22, 24, 23; g3 = 18, 28, 20, 30, 24); the bar in each box is the median (g1 23, g2 23, g3 24), the centre Levene's test measures spread from (center = median). Returned: W = 4.97727 with 2 and 12 degrees of freedom, p-value 0.026665.



Levene's test (center = median) for g1 = 21, 24, 19, 26, 23; g2 = 22, 23, 22, 24, 23; g3 = 18, 28, 20, 30, 24. Drawn: the F density with 2 and 12 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 3.88529 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: W = 4.97727, p-value = 0.026665.

```
statistics_ai999x.solve('levene_test', g1='21, 24, 19, 26, 23', g2='22, 23, 22, 24, 23',
g3='18, 28, 20, 30, 24', center='median')
# -> w = 4.97727, p_value = 0.026665, df1 = 2, df2 = 12   (SOLVED)
```

```
statistics_ai999x.solve('levene_test', g1=[5.1, 6.3, 4.2, 7.0], g2=[5.0, 5.2, 4.9, 5.1],
center='mean')
# -> w = 14.7273, p_value = 0.00858273, df1 = 1, df2 = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Levene's test (median centred) for [21, 24, 19, 26, 23], [22, 23, 22, 24, 23] and [18, 28, 20, 30, 24].	SOLVED	w = 4.97727, p_value = 0.026665, df1 = 2, df2 = 12
Use Levene's test with the mean to compare variances of [5.1, 6.3, 4.2, 7.0] and [5.0, 5.2, 4.9, 5.1].	SOLVED	w = 14.7273, p_value = 0.00858273, df1 = 1, df2 = 6
Levene's test for [21, 24, 19] and [22, 23, 22].	NEEDS_INPUT	levene_test needs center (median (Brown-Forsythe) or mean (Levene's original)). Example: solve('levene_test', g1='21, 24

Bartlett's test for equal variances, `bartlett_test`

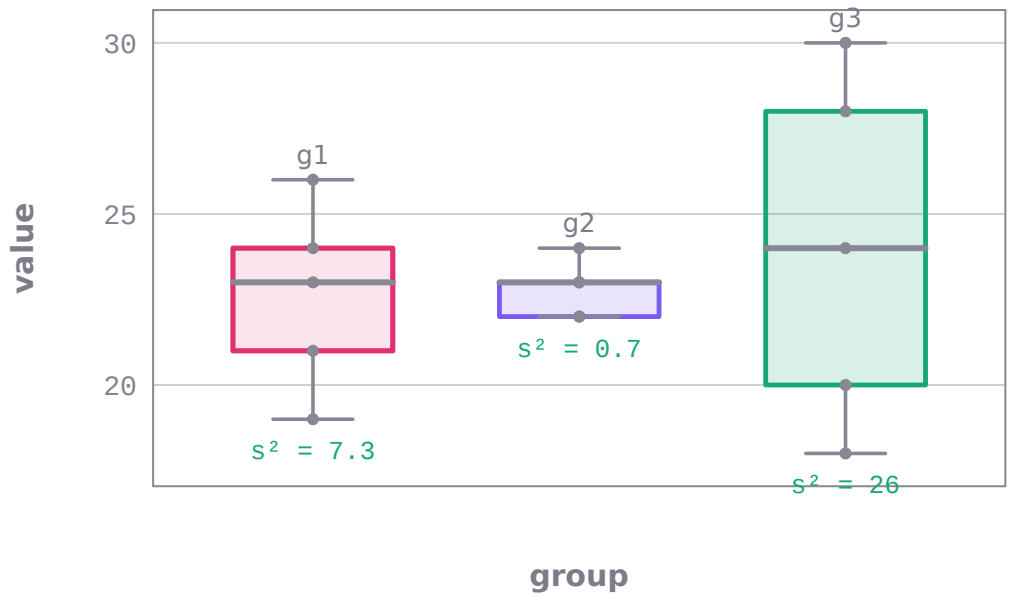
Bartlett's test of equal variances across 2 to 5 groups (normal populations).

$$T = \frac{(N-k)\ln s_p^2 - \sum (n_i - 1)\ln s_i^2}{1 + \frac{1}{3(k-1)}\left(\sum \frac{1}{n_i - 1} - \frac{1}{N-k}\right)}$$

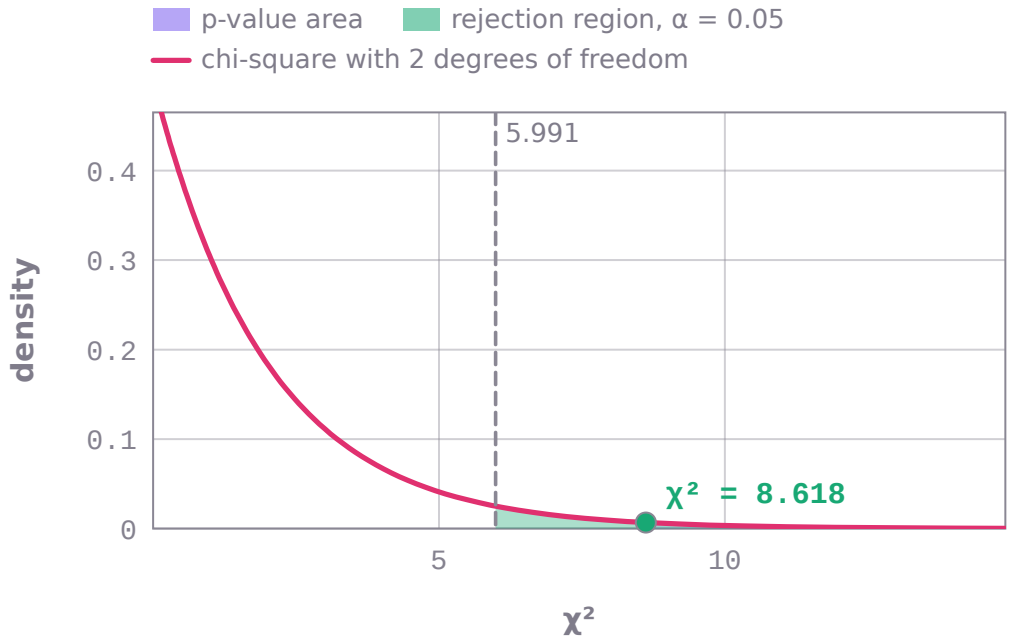
input	meaning	condition
`g1`	group 1	
`g2`	group 2	
`g3`	group 3	
`g4`	group 4	
`g5`	group 5	

output	meaning
`statistic`	Bartlett's chi-square statistic
`df`	$k - 1$
`p_value`	p-value

- **Method:** log of the pooled variance against the group log variances, with Bartlett's correction
- **Assumes:** normal populations (Bartlett's test is sensitive to non-normality); at least 2 values per group
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Box plots (quartiles from the package's percentiles) and data of 3 groups (g1 = 21, 24, 19, 26, 23; g2 = 22, 23, 22, 24, 23; g3 = 18, 28, 20, 30, 24), each labelled with its sample variance from the package: g1 7.3, g2 0.7, g3 26. Bartlett's test of equal variances: statistic 8.61822, 2 degrees of freedom, p-value 0.0134455.



Bartlett's test of equal variances for g1 = 21, 24, 19, 26, 23; g2 = 22, 23, 22, 24, 23; g3 = 18, 28, 20, 30, 24. Drawn: the chi-square density with 2 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 5.99146 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: $\chi^2 = 8.61822$, p-value = 0.0134455.

```
statistics_ai999x.solve('bartlett_test', g1='21, 24, 19, 26, 23', g2='22, 23, 22, 24, 23',
g3='18, 28, 20, 30, 24')
# -> statistic = 8.61822, df = 2, p_value = 0.0134455   (SOLVED)
```

```
statistics_ai999x.solve('bartlett_test', g1=[5.1, 6.3, 4.2, 7.0], g2=[5.0, 5.2, 4.9, 5.1])
# -> statistic = 8.1455, df = 1, p_value = 0.00431681   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bartlett's test for [21, 24, 19, 26, 23], [22, 23, 22, 24, 23] and [18, 28, 20, 30, 24].	SOLVED	statistic = 8.61822, df = 2, p_value = 0.0134455
Use Bartlett's test to compare the variances of [5.1, 6.3, 4.2, 7.0] and [5.0, 5.2, 4.9, 5.1].	SOLVED	statistic = 8.1455, df = 1, p_value = 0.00431681
Bartlett's test for [21, 21, 21] and [22, 23, 22].	INVALID_INPUT	Bartlett's test for equal variances: a group has zero variance.

Mann-Whitney U test (large-sample), `mann_whitney_u`

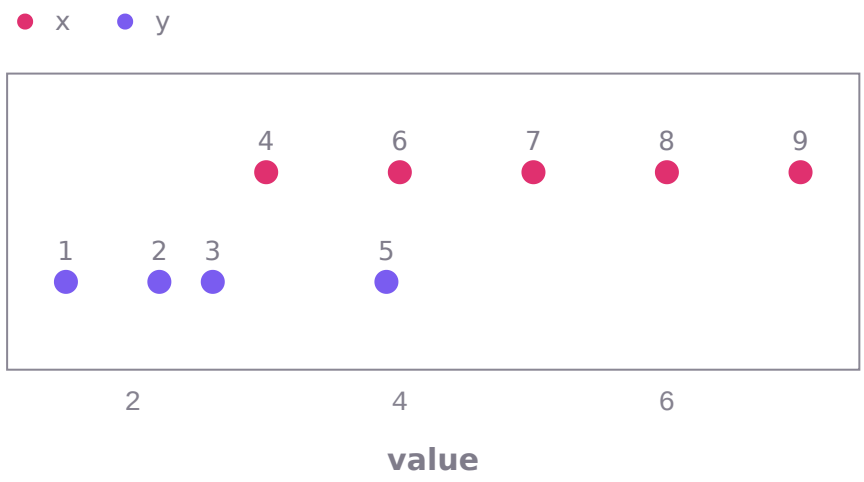
The Mann-Whitney U test (large-sample): compares two independent samples by ranks (statistic U of the first sample and the p-value).

$$U_1 = R_1 - \frac{n_1(n_1+1)}{2}$$

input	meaning	condition
`x`	first sample	
`y`	second sample	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`u`	U statistic of the first sample (R1 - n1(n1+1)/2)
`p_value`	p-value

- **Method:** rank the combined sample; U from the rank sum; normal approximation
- **Assumes:** independent samples, ordinal or continuous data; normal approximation with tie correction and continuity correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The samples x = 3, 5, 4, 6, 7 (upper row) and y = 1.5, 2.6, 2.2, 3.9 (lower row) on one axis, each labelled with its rank in the combined sample (ranks derived from the inputs). Returned: U = 19, p-value 0.0373364.

```
statistics_ai999x.solve('mann_whitney_u', x='3, 5, 4, 6, 7', y='1.5, 2.6, 2.2, 3.9')
# -> u = 19, p_value = 0.0373364   (SOLVED)
```

```
statistics_ai999x.solve('mann_whitney_u', x=[12.1, 15.3, 14.2, 18.4], y=[10.5, 9.6, 13.7,
11.8, 8.9], alternative='greater')
# -> u = 19, p_value = 0.0186682   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a Mann-Whitney U test on [3, 5, 4, 6, 7] and [1.5, 2.6, 2.2, 3.9], two-sided.	SOLVED	u = 19, p_value = 0.0373364
Use the Mann-Whitney test to check whether [12.1, 15.3, 14.2, 18.4] tends to be larger than [10.5, 9.6, 13.7, 11.8, 8.9], right-tailed.	SOLVED	u = 19, p_value = 0.0186682
Perform a Mann-Whitney U test on [1, 2] and [3, 4], one-sided.	OUT_OF_SCOPE	Out of scope: a one-sided test without a direction: say right-tailed or left-tailed.

Mann-Whitney U test (exact), `mann_whitney_exact`

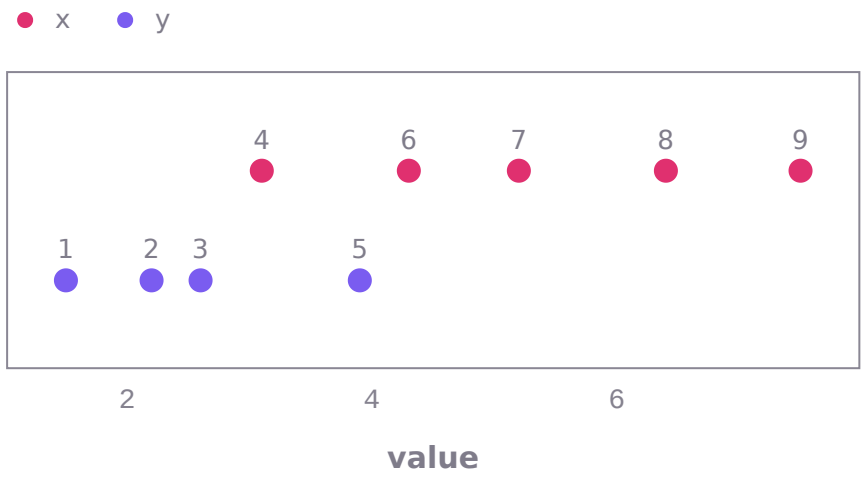
The Mann-Whitney U test (exact): compares two independent samples by ranks (statistic U of the first sample and the p-value).

$$U_1 = R_1 - \frac{n_1(n_1+1)}{2}$$

input	meaning	condition
`x`	first sample	
`y`	second sample	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`u`	U statistic of the first sample (R1 - n1(n1+1)/2)
`p_value`	p-value

- **Method:** rank the combined sample; U from the rank sum; exact null distribution of U (Gaussian binomial coefficients)
- **Assumes:** independent samples, ordinal or continuous data; no ties; samples of at most 30
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The samples x = 3.1, 5.2, 4.3, 6.4, 7.5 (upper row) and y = 1.5, 2.6, 2.2, 3.9 (lower row) on one axis, each labelled with its rank in the combined sample (ranks derived from the inputs). Returned: U = 19, p-value 0.031746.

```
statistics_ai999x.solve('mann_whitney_exact', x='3.1, 5.2, 4.3, 6.4, 7.5', y='1.5, 2.6, 2.2, 3.9')
# -> u = 19, p_value = 0.031746   (SOLVED)
```

```
statistics_ai999x.solve('mann_whitney_exact', x=[12.1, 15.3, 14.2, 18.4], y=[10.5, 9.6, 13.7, 11.8, 8.9], alternative='greater')
# -> u = 19, p_value = 0.015873   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a exact Mann-Whitney U test on [3.1, 5.2, 4.3, 6.4, 7.5] and [1.5, 2.6, 2.2, 3.9], two-sided.	SOLVED	u = 19, p_value = 0.031746
Use the exact Mann-Whitney test to check whether [12.1, 15.3, 14.2, 18.4] tends to be larger than [10.5, 9.6, 13.7, 11.8, 8.9], right-tailed.	SOLVED	u = 19, p_value = 0.015873
Perform a exact Mann-Whitney U test on [1, 2] and [3, 4], one-sided.	OUT_OF_SCOPE	Out of scope: a one-sided test without a direction: say right-tailed or left-tailed.

Wilcoxon signed-rank test (large-sample), `wilcoxon_signed_rank`

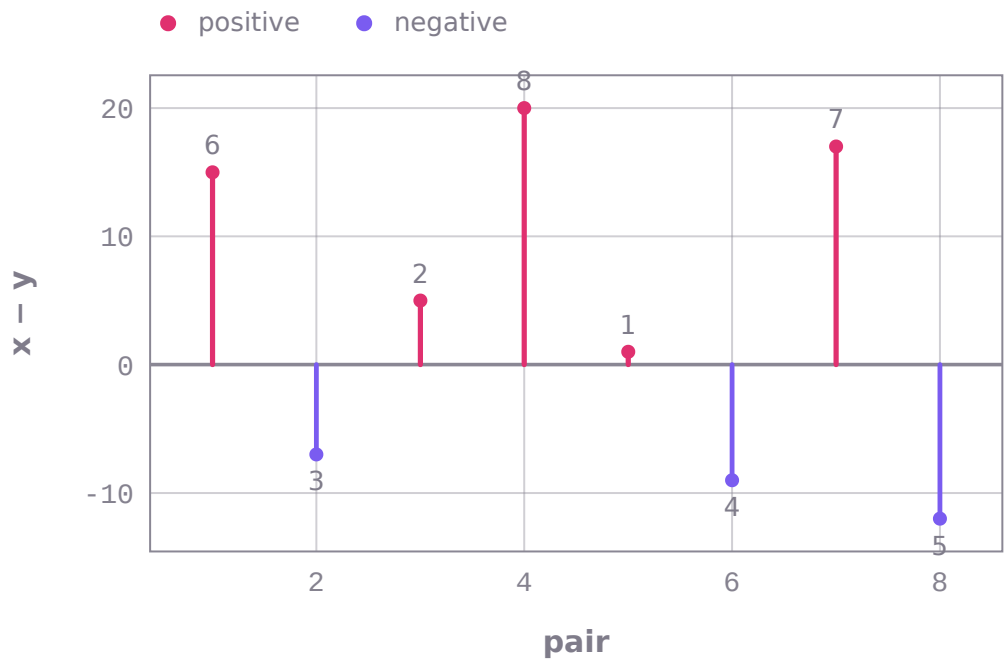
The Wilcoxon signed-rank test (large-sample) for paired data (x - y) or for differences given directly (x alone); zero differences are dropped.

$$R^{+} = \sum_{d_i > 0} \mathbf{rank}(|d_i|)$$

input	meaning	condition
`x`	differences, or the first measurement of each pair	
`y`	second measurement of each pair (optional)	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`statistic`	min(R+, R-) for a two-sided test, R+ otherwise
`p_value`	p-value
`r_plus`	sum of ranks of positive differences
`n`	non-zero differences

- **Method:** rank |d|; R+ = sum of ranks of positive d; normal approximation
- **Assumes:** paired data, symmetric distribution of differences; zero differences dropped; normal approximation with tie correction, no continuity correction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The differences x − y of the pairs x = 125, 115, 130, 140, 141, 115, 140, 125, y = 110, 122, 125, 120, 140, 124, 123, 137 (derived from the inputs), positive in the first colour; each labelled with the rank of its absolute value (derived). Returned: R+ = 24, statistic 12 (n = 8), p-value 0.400814.

```
statistics_ai999x.solve('wilcoxon_signed_rank', x='125, 115, 130, 140, 141, 115, 140, 125',
y='110, 122, 125, 120, 140, 124, 123, 137')
# -> statistic = 12, p_value = 0.400814, r_plus = 24, n = 8   (SOLVED)
```

```
statistics_ai999x.solve('wilcoxon_signed_rank', x=[1.8, -0.7, 2.6, 3.1, -1.2, 4.4, 0.9],
alternative='greater')
# -> statistic = 24, p_value = 0.0454845, r_plus = 24, n = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a Wilcoxon signed-rank test on before [125, 115, 130, 140, 141, 115, 140, 125] and after [110, 122, 125, 120, 140, 124, 123, 137], two-sided.	SOLVED	statistic = 12, p_value = 0.400814, r_plus = 24, n = 8
Use the Wilcoxon signed-rank test on the differences [1.8, -0.7, 2.6, 3.1, -1.2, 4.4, 0.9], right-tailed.	SOLVED	statistic = 24, p_value = 0.0454845, r_plus = 24, n = 7

question	outcome	result
Use the Wilcoxon signed-rank test on the differences [0, 0, 0], two-sided.	INVALID_INPUT	Wilcoxon signed-rank test (large-sample): all differences are zero.

Wilcoxon signed-rank test (exact), `wilcoxon_signed_rank_exact`

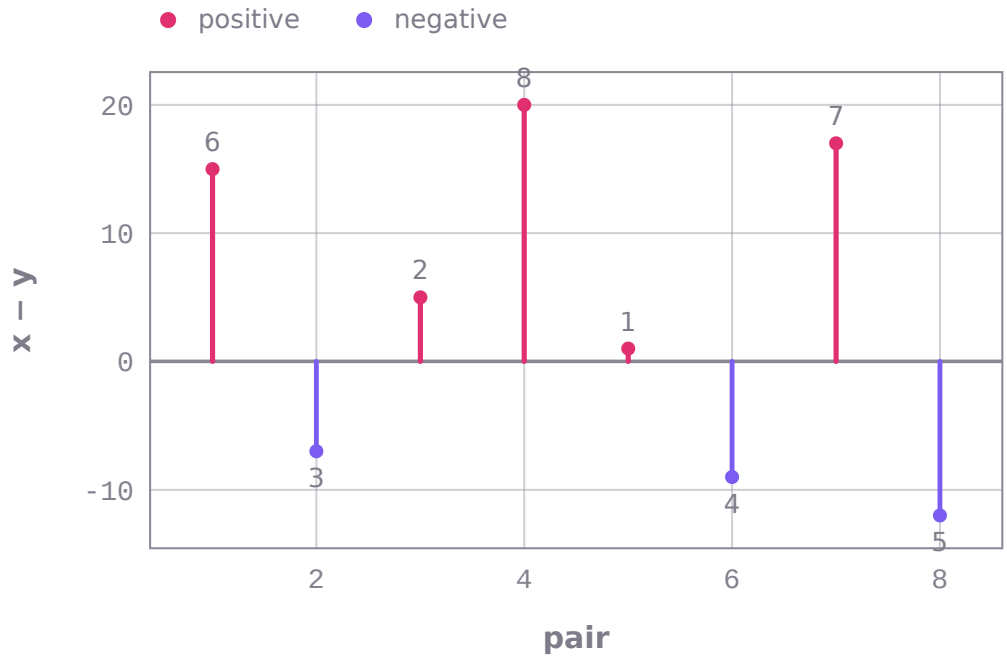
The Wilcoxon signed-rank test (exact) for paired data (x - y) or for differences given directly (x alone); zero differences are dropped.

$$R^+ = \sum_{d_i > 0} \mathbf{rank}(|d_i|)$$

input	meaning	condition
`x`	differences, or the first measurement of each pair	
`y`	second measurement of each pair (optional)	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`statistic`	min(R+, R-) for a two-sided test, R+ otherwise
`p_value`	p-value
`r_plus`	sum of ranks of positive differences
`n`	non-zero differences

- **Method:** rank |d|; R+ = sum of ranks of positive d; exact null distribution by counting subsets
- **Assumes:** paired data, symmetric distribution of differences; zero differences dropped; no zero and no tied absolute differences; at most 60 differences
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The differences x − y of the pairs x = 125, 115, 130, 140, 141, 115, 140, 125, y = 110, 122, 125, 120, 140, 124, 123, 137 (derived from the inputs), positive in the first colour; each labelled with the rank of its absolute value (derived). Returned: R+ = 24, statistic 12 (n = 8), exact p-value 0.460938.

```
statistics_ai999x.solve('wilcoxon_signed_rank_exact', x='125, 115, 130, 140, 141, 115, 140, 125', y='110, 122, 125, 120, 140, 124, 123, 137')
# -> statistic = 12, p_value = 0.460938, r_plus = 24, n = 8   (SOLVED)
```

```
statistics_ai999x.solve('wilcoxon_signed_rank_exact', x=[1.8, -0.7, 2.6, 3.1, -1.2, 4.4, 0.9], alternative='greater')
# -> statistic = 24, p_value = 0.0546875, r_plus = 24, n = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a exact Wilcoxon signed-rank test on before [125, 115, 130, 140, 141, 115, 140, 125] and after [110, 122, 125, 120, 140, 124, 123, 137], two-sided.	SOLVED	statistic = 12, p_value = 0.460938, r_plus = 24, n = 8
Use the exact Wilcoxon signed-rank test on the differences [1.8, -0.7, 2.6, 3.1, -1.2, 4.4, 0.9], right-tailed.	SOLVED	statistic = 24, p_value = 0.0546875, r_plus = 24, n = 7
Use the exact Wilcoxon signed-rank test on the differences [0, 0, 0], two-sided.	INVALID_INPUT	Wilcoxon signed-rank test (exact): all differences are zero.

Kruskal-Wallis H test, `kruskal_wallis`

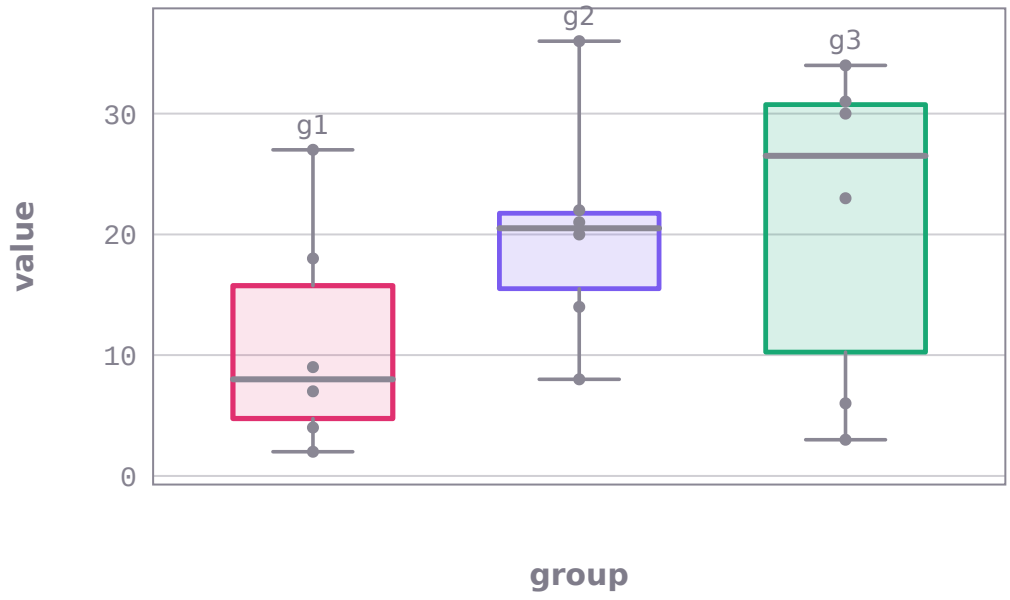
The Kruskal-Wallis rank test comparing 2 to 5 independent groups (nonparametric one-way ANOVA).

$$H = \frac{12}{N(N+1)} \sum \frac{R_i^2}{n_i} - 3(N+1)$$

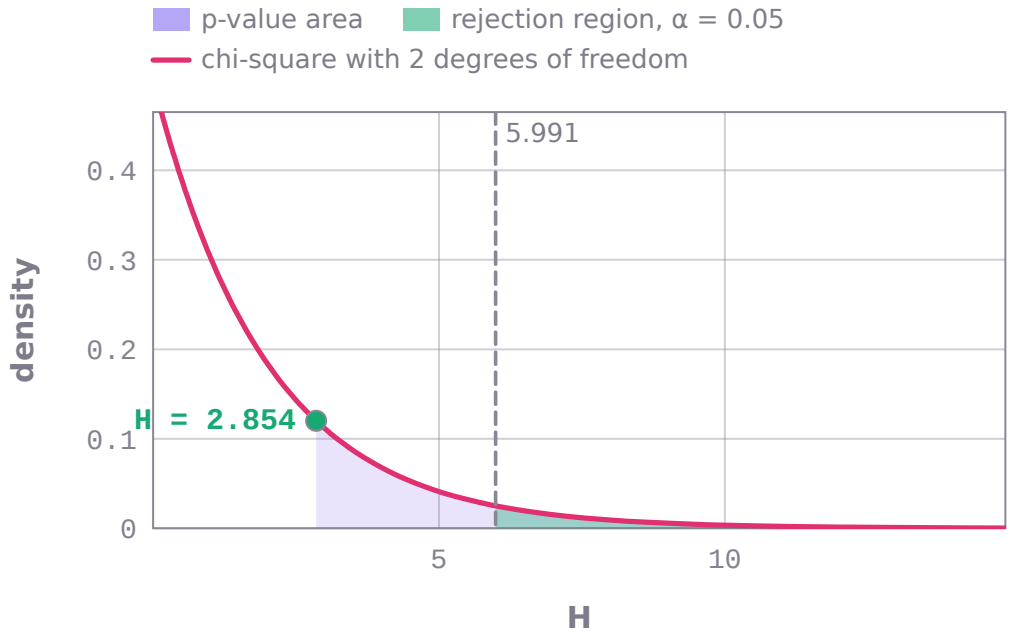
input	meaning	condition
`g1`	group 1	
`g2`	group 2	
`g3`	group 3	
`g4`	group 4	
`g5`	group 5	

output	meaning
`h`	H statistic (tie-corrected)
`df`	$k - 1$
`p_value`	chi-square p-value

- **Method:** rank all observations; H = 12/(N(N+1)) sum $R_i^2/n_i - 3$(N+1), divided by the tie correction
- **Assumes:** independent groups, ordinal or continuous data; chi-square approximation (about 5 or more per group)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Box plots (quartiles from the package's percentiles) and data of 3 groups (g1 = 27, 2, 4, 18, 7, 9; g2 = 20, 8, 14, 36, 21, 22; g3 = 34, 31, 3, 23, 30, 6); medians g1 8, g2 20.5, g3 26.5. Kruskal–Wallis test: H = 2.8538, 2 degrees of freedom, p-value 0.240052.



The Kruskal–Wallis test for g1 = 27, 2, 4, 18, 7, 9; g2 = 20, 8, 14, 36, 21, 22; g3 = 34, 31, 3, 23, 30, 6. Drawn: the chi-square density with 2 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 5.99146 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: H = 2.8538, p-value = 0.240052.

```
statistics_ai999x.solve('kruskal_wallis', g1='27, 2, 4, 18, 7, 9', g2='20, 8, 14, 36, 21,
22', g3='34, 31, 3, 23, 30, 6')
# -> h = 2.8538, df = 2, p_value = 0.240052   (SOLVED)
```

```
statistics_ai999x.solve('kruskal_wallis', g1=[5.1, 4.8, 5.5], g2=[6.2, 6.0, 6.8, 6.4], g3=
[5.9, 5.6, 6.1])
# -> h = 7.31818, df = 2, p_value = 0.0257559   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a Kruskal-Wallis test on [27, 2, 4, 18, 7, 9], [20, 8, 14, 36, 21, 22] and [34, 31, 3, 23, 30, 6].	SOLVED	h = 2.8538, df = 2, p_value = 0.240052
Kruskal-Wallis H test for groups [5.1, 4.8, 5.5], [6.2, 6.0, 6.8, 6.4] and [5.9, 5.6, 6.1].	SOLVED	h = 7.31818, df = 2, p_value = 0.0257559
Perform a Kruskal-Wallis test on [1, 1] and [1, 1].	INVALID_INPUT	Kruskal-Wallis H test: all values are tied.

Friedman test, `friedman_test`

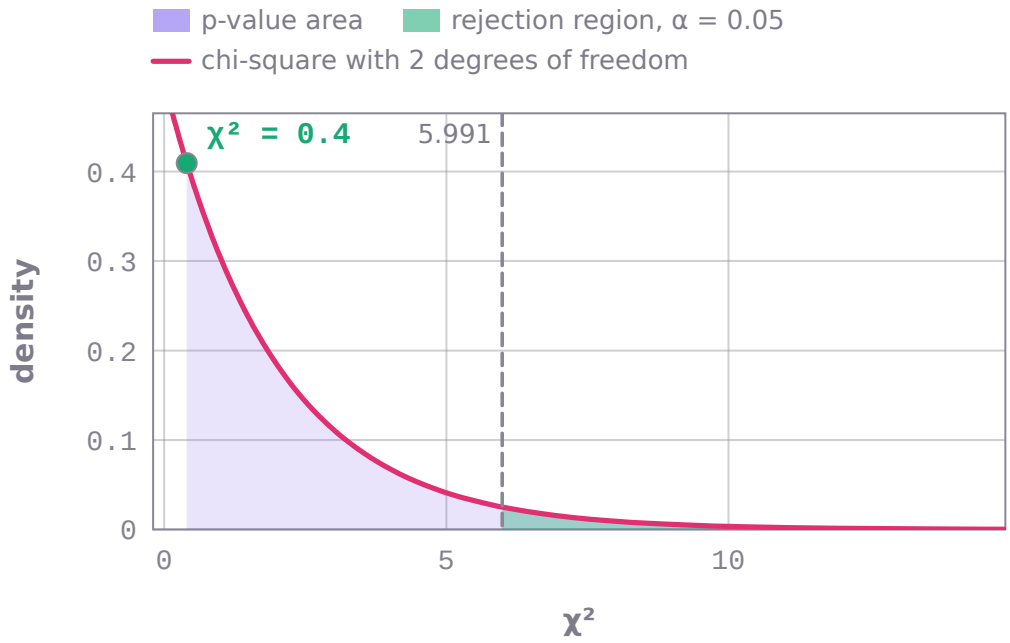
The Friedman rank test for $k \geq 3$ related treatments measured on the same blocks (subjects): rows are blocks, columns are treatments.

$$Q = \frac{12}{nk(k+1)} \sum_j R_j^2 - 3n(k+1)$$

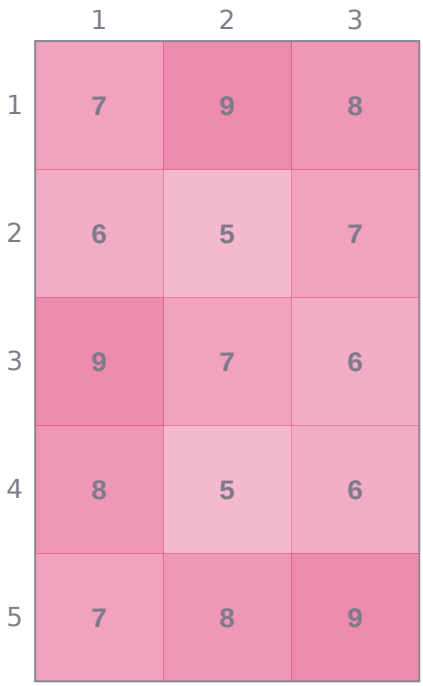
input	meaning	condition
`data`	table with one row per block (subject) and one column per treatment	

output	meaning
`chi_square`	Friedman statistic (tie-corrected)
`df`	$k - 1$
`p_value`	chi-square p-value

- **Method:** rank within each block; Q = 12/(nk(k+1)) sum $R_j^2 - 3n$(k+1), divided by the tie correction
- **Assumes:** at least 3 treatments and 2 blocks; chi-square approximation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Friedman's test for the blocks [[7, 9, 8], [6, 5, 7], [9, 7, 6], [8, 5, 6], [7, 8, 9]]. Drawn: the chi-square density with 2 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 5.99146 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: χ² = 0.4, p-value = 0.818731.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): data (input). Returned with them by statistics-ai999x: chi_square = 0.4, df = 2, p_value = 0.818731.

```
statistics_ai999x.solve('friedman_test', data='[[7, 9, 8], [6, 5, 7], [9, 7, 6], [8, 5, 6], [7, 8, 9]]')
# -> chi_square = 0.4, df = 2, p_value = 0.818731   (SOLVED)
```

```
statistics_ai999x.solve('friedman_test', data=[[1.2, 1.5, 1.9, 1.1], [2.1, 2.4, 2.8, 2.0], [1.7, 1.6, 2.2, 1.4]])
# -> chi_square = 8.2, df = 3, p_value = 0.0420542   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a Friedman test on [[7, 9, 8], [6, 5, 7], [9, 7, 6], [8, 5, 6], [7, 8, 9]].	SOLVED	chi_square = 0.4, df = 2, p_value = 0.818731
Friedman test for 4 treatments on blocks [[1.2, 1.5, 1.9, 1.1], [2.1, 2.4, 2.8, 2.0], [1.7, 1.6, 2.2, 1.4]].	SOLVED	chi_square = 8.2, df = 3, p_value = 0.0420542
Perform a Friedman test on [[7, 9], [6, 5]].	INVALID_INPUT	Friedman test: the data table needs at least 2 rows and 3 columns.

Sign test, `sign_test`

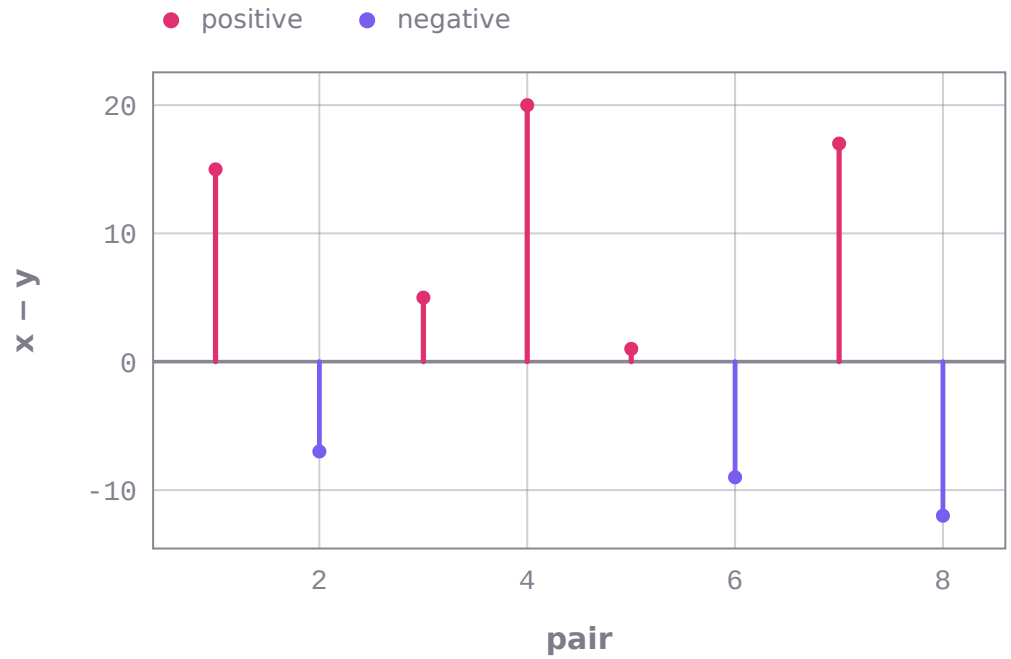
The sign test for the median difference of paired data (or of differences given directly): exact binomial p-value.

$$P = 2\min\left(P(K \le k), P(K \ge k)\right),\ K \sim \mathrm{Bin}(n, \tfrac{1}{2})$$

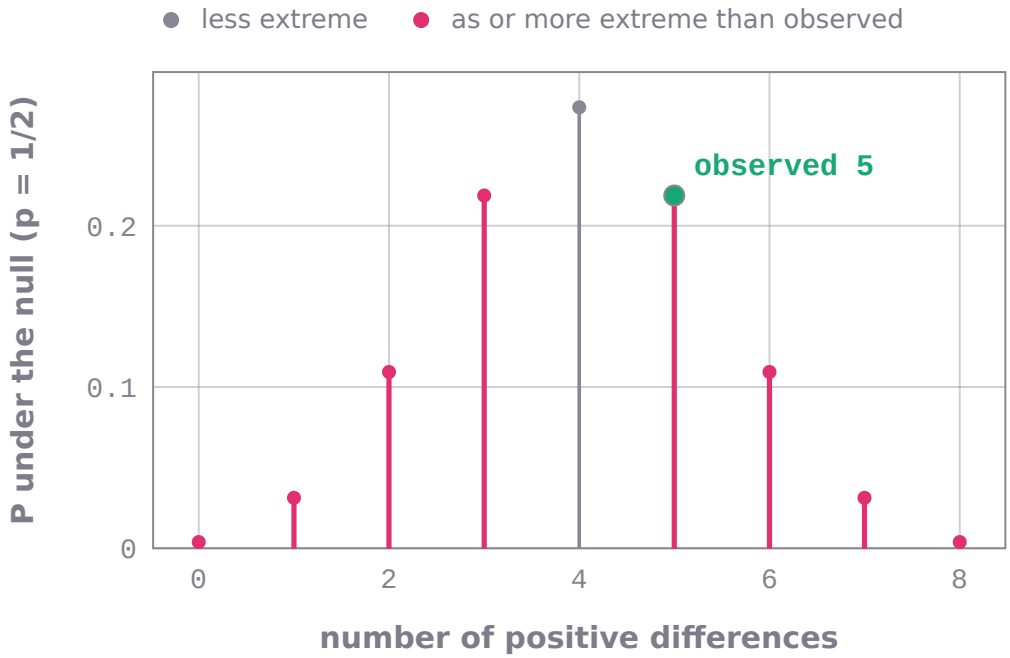
input	meaning	condition
`x`	differences, or the first measurement of each pair	
`y`	second measurement of each pair (optional)	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`positives`	number of positive differences
`n`	non-zero differences
`p_value`	exact binomial p-value

- **Method:** count positive signs; exact Binomial(n, 1/2) tail probabilities
- **Assumes:** paired data; zero differences dropped
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The differences x − y of the pairs x = 125, 115, 130, 140, 141, 115, 140, 125, y = 110, 122, 125, 120, 140, 124, 123, 137 (derived from the inputs), positive in the first colour. Returned: 5 positive of 8 non-zero differences, p-value 0.726562.



Under the null hypothesis the count of positive signs among the 8 non-zero differences is binomial (n = 8, p = 1/2): each probability from the package's binomial run. Counts at least as far from 8/2 as the observed 5 are in the first colour; the returned p-value is 0.726562.

```
statistics_ai999x.solve('sign_test', x='125, 115, 130, 140, 141, 115, 140, 125', y='110,
122, 125, 120, 140, 124, 123, 137')
# -> positives = 5, n = 8, p_value = 0.726562   (SOLVED)
```

```
statistics_ai999x.solve('sign_test', x=[1.8, -0.7, 2.6, 3.1, -1.2, 4.4, 0.9, 2.2],
alternative='greater')
# -> positives = 6, n = 8, p_value = 0.144531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perform a sign test on before [125, 115, 130, 140, 141, 115, 140, 125] and after [110, 122, 125, 120, 140, 124, 123, 137], two-sided.	SOLVED	positives = 5, n = 8, p_value = 0.726562
Use a sign test on the differences [1.8, -0.7, 2.6, 3.1, -1.2, 4.4, 0.9, 2.2], right-tailed.	SOLVED	positives = 6, n = 8, p_value = 0.144531
Use a sign test on the differences [0, 0], two-sided.	INVALID_INPUT	Sign test: all differences are zero.

Fisher's exact test (2 x 2), `fisher_exact_test`

Fisher's exact test of association in a 2 x 2 table, with the sample odds ratio.

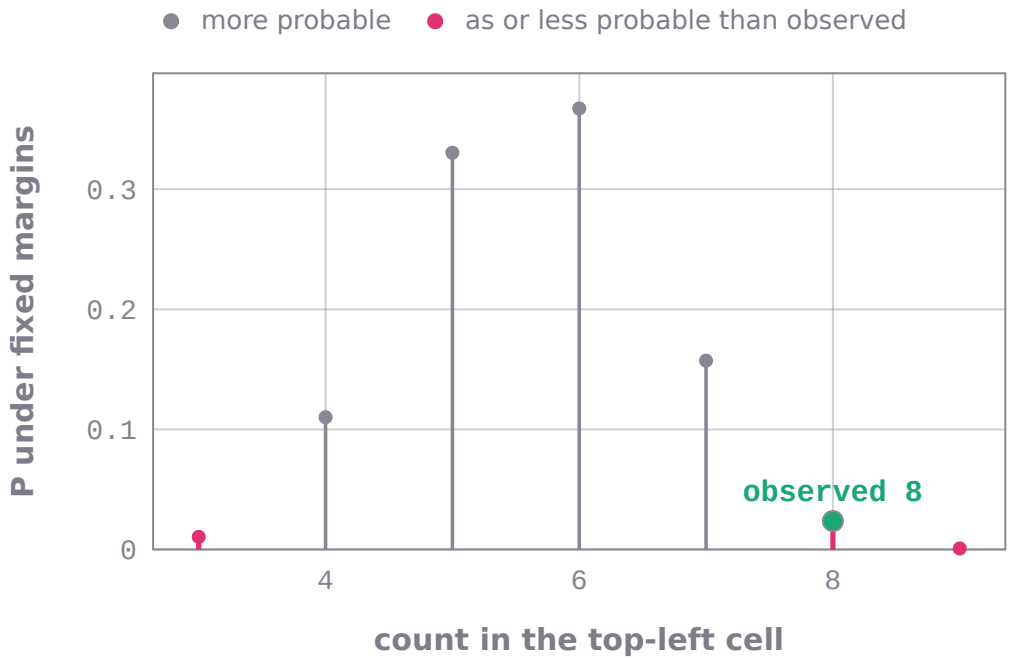
$$P(a) = \frac{\binom{a+b}{a}\binom{c+d}{c}}{\binom{n}{a+c}}$$

input	meaning	condition
`table`	2 x 2 table of counts [[a, b], [c, d]]	

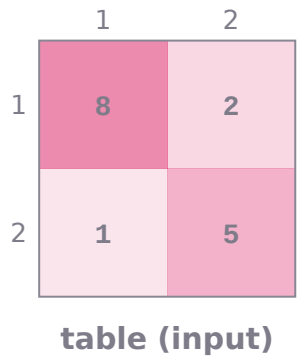
input	meaning	condition
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`p_value`	exact p-value (two-sided: tables no more likely than the observed one)
`odds_ratio`	ad/(bc); empty when b $c = 0$

- **Method:** hypergeometric probabilities of all tables with the observed margins, in exact integer arithmetic
- **Assumes:** fixed margins (conditional test); whole-number counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



With the margins of the table [[8, 2], [1, 5]] fixed (row total 10, column total 9, 16 in all, derived from the inputs), the top-left count is hypergeometric: each probability from the package's hypergeometric run. Tables as or less probable than the observed one are in the first colour; the returned two-sided p-value is 0.034965, odds ratio 20.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): table (input). Returned with them by statistics-ai999x: p_value = 0.034965, odds_ratio = 20.

```
statistics_ai999x.solve('fisher_exact_test', table='[[8, 2], [1, 5]]')
# -> p_value = 0.034965, odds_ratio = 20   (SOLVED)
```

```
statistics_ai999x.solve('fisher_exact_test', table=[[3, 7], [9, 1]], alternative='less')
# -> p_value = 0.00988331, odds_ratio = 0.047619   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fisher's exact test for the table [[8, 2], [1, 5]], two-sided.	SOLVED	p_value = 0.034965, odds_ratio = 20
Use Fisher's exact test on [[3, 7], [9, 1]], left-tailed.	SOLVED	p_value = 0.00988331, odds_ratio = 0.047619
Fisher's exact test for the table [[8, 2, 1], [1, 5, 2]], two-sided.	INVALID_INPUT	Fisher's exact test (2 x 2): a 2 x 2 table is required.

McNemar's test for paired proportions, `mcnemar_test`

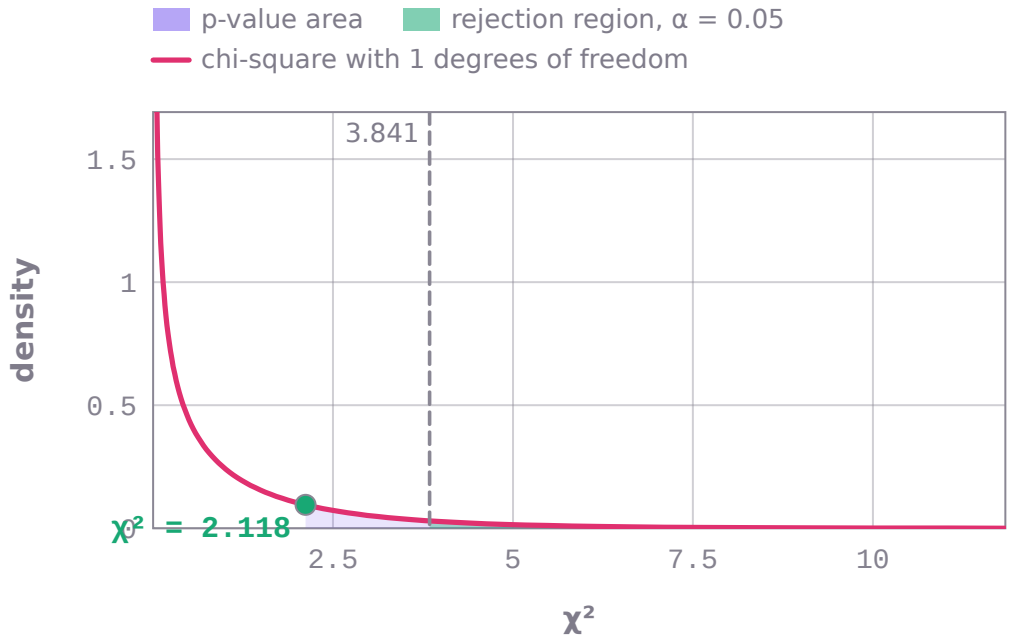
McNemar's test for a 2 x 2 table of paired outcomes: continuity-corrected chi-square and the exact binomial p-value.

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

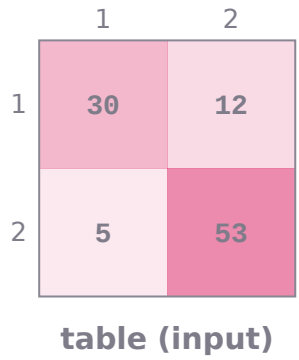
input	meaning	condition
`table`	2 x 2 table of paired counts [[both yes, yes/no], [no/yes, both no]]	

output	meaning
`chi_square`	(|b - c| - 1)^2 / (b + c)
`p_value`	chi-square p-value (1 df)
`exact_p_value`	exact binomial p-value
`discordant`	b + c

- **Method:** only the discordant cells b and c carry information
- **Assumes:** paired (matched) binary outcomes; $b + c > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



McNemar's test for the paired table [[30, 12], [5, 53]] (17 discordant pairs; exact p-value 0.143463). Drawn: the chi-square density with 1 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 3.84146 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: χ² = 2.11765, p-value = 0.14561.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): table (input). Returned with them by statistics-ai999x: chi_square = 2.11765, p_value = 0.14561, exact_p_value = 0.143463, discordant = 17.

```
statistics_ai999x.solve('mcnemar_test', table='[[30, 12], [5, 53]]')
# -> chi_square = 2.11765, p_value = 0.14561, exact_p_value = 0.143463, discordant = 17
(SOLVED)
```

```
statistics_ai999x.solve('mcnemar_test', table=[[45, 10], [20, 25]])
# -> chi_square = 2.7, p_value = 0.100348, exact_p_value = 0.0987371, discordant = 30
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
McNemar's test for the paired table [[30, 12], [5, 53]].	SOLVED	chi_square = 2.11765, p_value = 0.14561, exact_p_value = 0.143463, discordant = 17

question	outcome	result
Use McNemar's test on [[45, 10], [20, 25]].	SOLVED	chi_square = 2.7, p_value = 0.100348, exact_p_value = 0.0987371, discordant = 30
McNemar's test for the paired table [[30, 0], [0, 53]].	INVALID_INPUT	McNemar's test for paired proportions: there are no discordant pairs (b + c = 0).

Cohen's d and Hedges' g, `cohens_d`

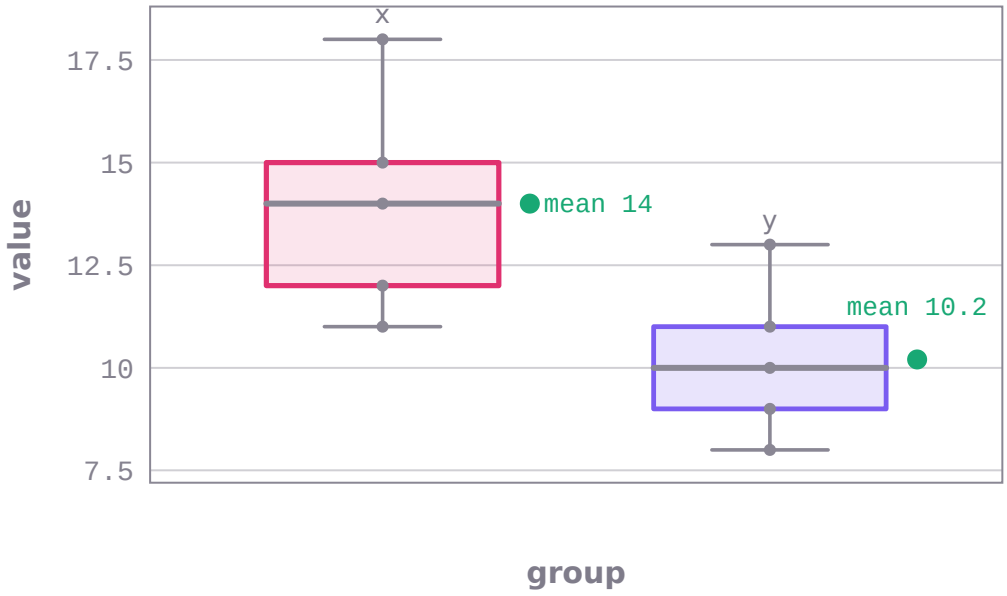
The standardized mean difference of two independent samples: Cohen's d (pooled SD) and Hedges' g (bias-corrected).

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_p},\ s_p = \sqrt{\frac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_1 + n_2 - 2}}$$

input	meaning	condition
`x`	first sample	
`y`	second sample	

output	meaning
`d`	(mean x - mean y) / pooled SD
`hedges_g`	d (1 - 3/(4(n1 + n2) - 9))
`pooled_sd`	pooled standard deviation

- **Method:** difference of means over the pooled sample standard deviation
- **Assumes:** independent samples
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The two samples x = 12, 15, 11, 18, 14 and y = 10, 9, 13, 11, 8 as box plots (quartiles from the package's percentiles) with their data; marked: the sample means from the package, 14 and 10.2. Returned: Cohen's d = 1.60579 (Hedges' g = 1.45039), the difference of the means in units of the pooled standard deviation 2.36643.

```
statistics_ai999x.solve('cohens_d', x='12, 15, 11, 18, 14', y='10, 9, 13, 11, 8')
# -> d = 1.60579, hedges_g = 1.45039, pooled_sd = 2.36643   (SOLVED)
```

```
statistics_ai999x.solve('cohens_d', x=[5.1, 4.9, 5.6, 5.8], y=[4.2, 4.8, 4.4, 4.0, 4.6])
# -> d = 2.60634, hedges_g = 2.31675, pooled_sd = 0.364496   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find Cohen's d for [12, 15, 11, 18, 14] and [10, 9, 13, 11, 8].	SOLVED	d = 1.60579, hedges_g = 1.45039, pooled_sd = 2.36643
What is the effect size (Cohen's d) between [5.1, 4.9, 5.6, 5.8] and [4.2, 4.8, 4.4, 4.0, 4.6]?	SOLVED	d = 2.60634, hedges_g = 2.31675, pooled_sd = 0.364496
Find Cohen's d for [12] and [10, 9, 13].	INVALID_INPUT	x: needs at least 2 entries, got 1.

Cronbach's alpha (internal consistency), `cronbach_alpha`

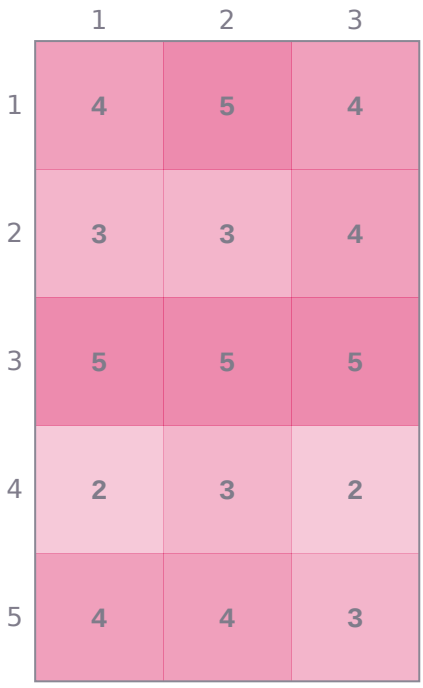
Cronbach's alpha for a questionnaire or test: rows are respondents, columns are items.

$$\alpha = \frac{k}{k-1}\left(1 - \frac{\sum s_i^2}{s_T^2}\right)$$

input	meaning	condition
`data`	scores: one row per respondent, one column per item	

output	meaning
`alpha`	$k/(k-1)$ (1 - sum of item variances / variance of totals)
`items`	k
`respondents`	n

- **Method:** sample variances $(n-1)$ of each item and of the totals
- **Assumes:** at least 2 items and 2 respondents
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): data (input). Returned with them by statistics-ai999x: alpha = 0.913043, items = 3, respondents = 5.

```
statistics_ai999x.solve('cronbach_alpha', data='[[4, 5, 4], [3, 3, 4], [5, 5, 5], [2, 3, 2], [4, 4, 3]]')
# -> alpha = 0.913043, items = 3, respondents = 5   (SOLVED)
```

```
statistics_ai999x.solve('cronbach_alpha', data=[[1, 2, 2, 1], [3, 3, 4, 3], [4, 5, 4, 5], [2, 2, 3, 2]])
# -> alpha = 0.962963, items = 4, respondents = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find Cronbach's alpha for [[4, 5, 4], [3, 3, 4], [5, 5, 5], [2, 3, 2], [4, 4, 3]].	SOLVED	alpha = 0.913043, items = 3, respondents = 5
Compute the internal consistency (Cronbach's alpha) of [[1, 2, 2, 1], [3, 3, 4, 3], [4, 5, 4, 5], [2, 2, 3, 2]].	SOLVED	alpha = 0.962963, items = 4, respondents = 4
Find Cronbach's alpha for [[3, 3], [3, 3], [3, 3]].	INVALID_INPUT	Cronbach's alpha (internal consistency): the total scores do not vary.

2^2 factorial design: main effects and interaction, `factorial_2x2_effects`

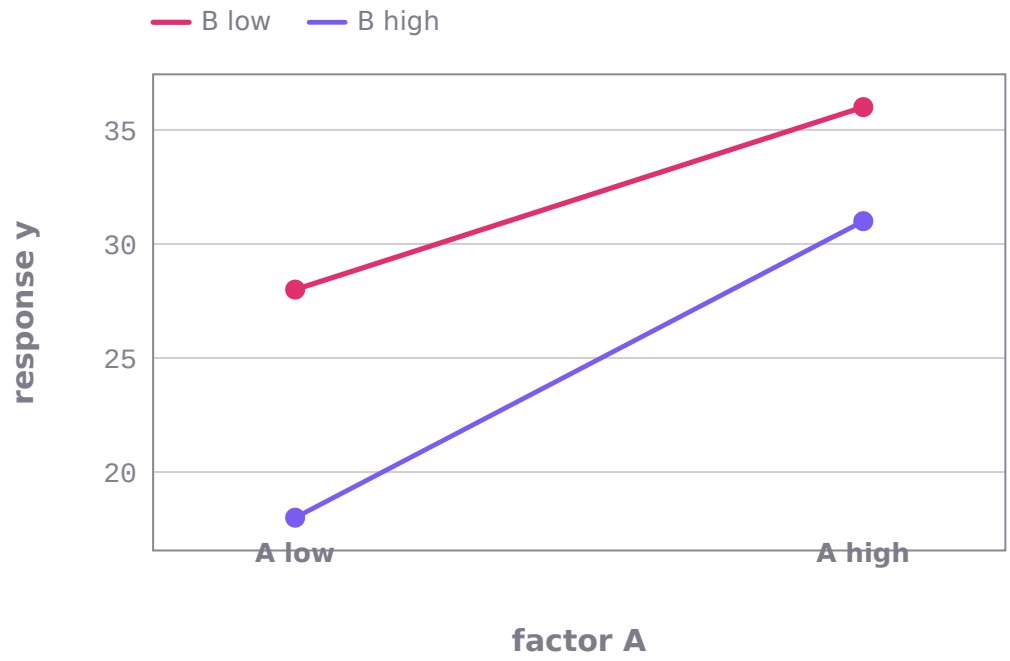
Effects of a two-level 4-run factorial experiment from one response per run in standard (Yates) order ((1), a, b, ab).

$$\text{Effect} = \frac{\text{contrast}}{2^{k-1}}$$

input	meaning	condition
`y`	4 responses in standard order	

output	meaning
`grand_mean`	average response
`A`	effect A
`B`	effect B
`AB`	effect AB

- **Method:** contrast of the +/- signs divided by 2^{k-1}
- **Assumes:** responses in standard order; one response (or cell mean) per run
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Interaction plot of the 2×2 factorial responses y = 28, 36, 18, 31 in standard order ((1), a, b, ab): one line per level of B across the levels of A; non-parallel lines show interaction. Returned: grand mean 28.25, main effects A = 10.5, B = -7.5, interaction AB = 2.5.

```
statistics_ai999x.solve('factorial_2x2_effects', y='28, 36, 18, 31')
# -> grand_mean = 28.25, A = 10.5, B = -7.5, AB = 2.5   (SOLVED)
```

```
statistics_ai999x.solve('factorial_2x2_effects', y=[20, 30, 25, 45])
# -> grand_mean = 30, A = 15, B = 10, AB = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the effects of the 2^2 factorial design with responses in standard order [28, 36, 18, 31].	SOLVED	grand_mean = 28.25, A = 10.5, B = -7.5, AB = 2.5
Compute the main effects and interactions of a 2^2 factorial experiment with y = [20, 30, 25, 45].	SOLVED	grand_mean = 30, A = 15, B = 10, AB = 5

question	outcome	result
Find the effects of the 2^2 factorial design with responses in standard order [1, 2, 3].	INVALID_INPUT	y: needs at least 4 entries, got 3.

2^3 factorial design: main effects and interactions, `factorial_2x2x2_effects`

Effects of a two-level 8-run factorial experiment from one response per run in standard (Yates) order ((1), a, b, ab, c, ac, bc, abc).

$$\text{Effect} = \frac{\text{contrast}}{2^{k-1}}$$

input	meaning	condition
`y`	8 responses in standard order	

output	meaning
`grand_mean`	average response
`A`	effect A
`B`	effect B
`C`	effect C
`AB`	effect AB
`AC`	effect AC
`BC`	effect BC
`ABC`	effect ABC

- **Method:** contrast of the +/- signs divided by 2^{k-1}
- **Assumes:** responses in standard order; one response (or cell mean) per run
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The seven effects returned for the 2×2×2 factorial responses y = 22, 32, 35, 55, 44, 40, 60, 39 (standard order): A = 1.25, B = 12.75, C = 9.75, AB = -1.75, AC = -13.75, BC = -5.25, ABC = -6.75; grand mean 40.875.

```
statistics_ai999x.solve('factorial_2x2x2_effects', y='22, 32, 35, 55, 44, 40, 60, 39')
# -> grand_mean = 40.875, A = 1.25, B = 12.75, C = 9.75, AB = -1.75, AC = -13.75, BC =
-5.25, ABC = -6.75   (SOLVED)
```

```
statistics_ai999x.solve('factorial_2x2x2_effects', y=[550, 669, 604, 650, 633, 642, 601,
635])
# -> grand_mean = 623, A = 52, B = -1, C = 9.5, AB = -12, AC = -30.5, BC = -18.5, ABC =
24.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the effects of the 2^3 factorial design with responses in standard order [22, 32, 35, 55, 44, 40, 60, 39].	SOLVED	grand_mean = 40.875, A = 1.25, B = 12.75, C = 9.75, AB = -1.75, AC = -13.75, BC = -5.25, ABC = -6.75
Compute the main effects and interactions of a 2^3 factorial experiment with y = [550, 669, 604, 650, 633, 642, 601, 635].	SOLVED	grand_mean = 623, A = 52, B = -1, C = 9.5, AB = -12, AC = -30.5, BC = -18.5, ABC = 24.5
Find the effects of the 2^3 factorial design with responses in standard order [1, 2, 3].	INVALID_INPUT	y: needs at least 8 entries, got 3.

Sample size to estimate a proportion, `sample_size_proportion`

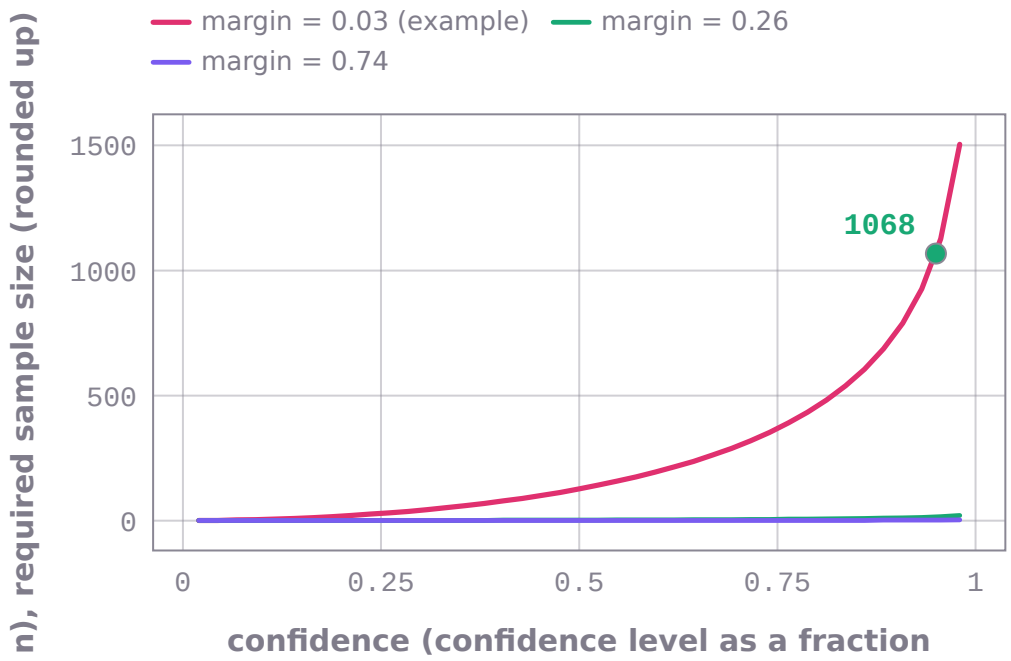
The smallest n giving a confidence interval for a proportion with the required margin of error (p = 0.5 if no estimate).

$$n = \left\lceil \frac{z_{\alpha/2}^2 \, p(1-p)}{E^2} \right\rceil$$

input	meaning	condition
`margin`	margin of error (as a proportion, 0.03 for 3 points)	
`confidence`	confidence level as a fraction (0.95 for 95%)	
`p`	planning estimate of the proportion (default 0.5)	

output	meaning
`n`	required sample size (rounded up)
`n_exact`	z^2 p(1-p)/E^2 before rounding

- **Method:** $n = z^2$ p(1 - p)/E^2, rounded up
- **Assumes:** large-sample (Wald) interval; simple random sample
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n (required sample size (rounded up)) against confidence (confidence level as a fraction (0.95 for 95%)) from 0.02 to 0.98. One curve for each value of margin (margin of error (as a proportion, 0.03 for 3 points)): 0.03, 0.26, 0.74. 41 points per curve computed by statistics-ai999x; marked: the worked example, confidence = 0.95, n = 1068.

```
statistics_ai999x.solve('sample_size_proportion', margin=0.03, confidence=0.95)
# -> n = 1068, n_exact = 1067.07   (SOLVED)
```

```
statistics_ai999x.solve('sample_size_proportion', margin=0.05, confidence=0.99, p=0.3)
# -> n = 558, n_exact = 557.331   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What sample size is needed to estimate a proportion within margin of error = 0.03 at 95% confidence?	SOLVED	n = 1068, n_exact = 1067.07
How large a sample is needed for a 99% confidence interval for a proportion with margin	SOLVED	n = 558, n_exact = 557.331

question	outcome	result
of error = 0.05 and p = 0.3?		
What sample size is needed to estimate a proportion within margin of error = 0.03?	NEEDS_INPUT	sample_size_proportion needs confidence (confidence level as a fraction (0.95 for 95%)). Example: solve('sample_size_pro

Sample size to estimate a mean, `sample_size_mean`

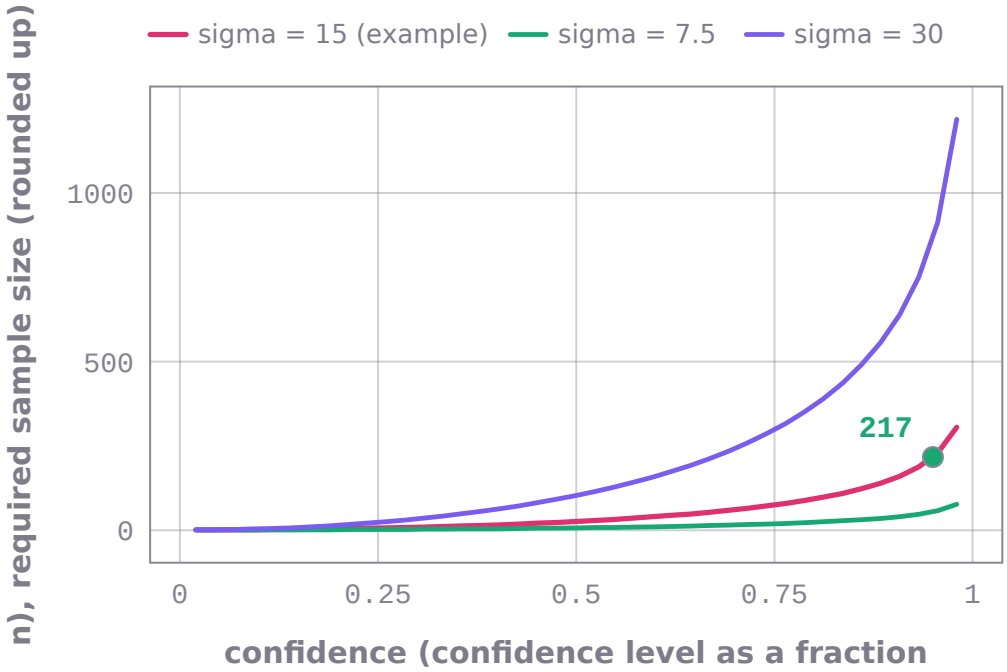
The smallest n giving a z confidence interval for a mean with the required margin of error, sigma known.

$$n = \left\lceil \left(\frac{z_{\alpha/2}\,\sigma}{E} \right)^2 \right\rceil$$

input	meaning	condition
`sigma`	population standard deviation (planning value)	
`margin`	margin of error	
`confidence`	confidence level as a fraction (0.95 for 95%)	

output	meaning
`n`	required sample size (rounded up)
`n_exact`	(z sigma / E)^2

- **Method:** n = (z sigma / E)^2, rounded up
- **Assumes:** sigma known or a planning estimate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n (required sample size (rounded up)) against confidence (confidence level as a fraction (0.95 for 95%)) from 0.02 to 0.98. One curve for each value of sigma (population standard deviation (planning value)): 15, 7.5, 30. Other inputs as in the worked example: margin = 2.0. 41 points per curve computed by statistics-ai999x; marked: the worked example, confidence = 0.95, n = 217.

```
statistics_ai999x.solve('sample_size_mean', sigma=15, margin=2, confidence=0.95)
# -> n = 217, n_exact = 216.082   (SOLVED)
```

```
statistics_ai999x.solve('sample_size_mean', sigma=8, margin=1.5, confidence=0.99)
# -> n = 189, n_exact = 188.726   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What sample size is needed to estimate the mean within margin of error = 2 with sigma = 15 at 95% confidence?	SOLVED	n = 217, n_exact = 216.082
How many observations are needed to estimate a mean with standard deviation = 8 and margin of error = 1.5 at 99% confidence?	SOLVED	n = 189, n_exact = 188.726
What sample size is needed to estimate the mean within margin of error = 2 at 95% confidence?	NEEDS_INPUT	sample_size_mean needs sigma (population standard deviation (planning value)). Example: solve('sample_size_mean', sigma=

Power of a one-sample z-test, `power_one_sample_z`

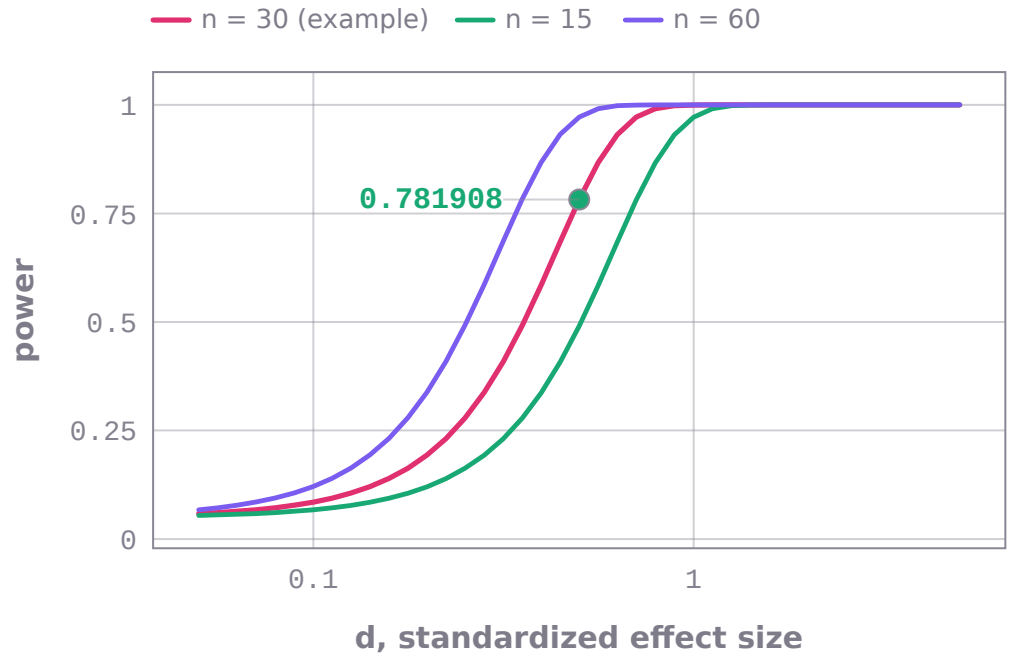
The power of a one-sample z-test for a standardized effect size d (Cohen's d), sample size n and significance level alpha.

$$1 - \beta = \Phi\left(d\sqrt{n_{\text{eff}}} - z_{1-\alpha/2}\right) + \Phi\left(-d\sqrt{n_{\text{eff}}} - z_{1-\alpha/2}\right)$$

input	meaning	condition
`d`	standardized effect size (difference / sigma)	
`n`	sample size	
`alpha`	significance level	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`power`	probability of rejecting H0 when the effect is d

- **Method:** power = P(Z > z_crit - d sqrt(n_eff)) (+ the opposite tail when two-sided), n_eff = n
- **Assumes:** normal approximation (sigma known); two-sided power includes both rejection tails
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



power (probability of rejecting H0 when the effect is d) against d (standardized effect size (difference / sigma)) from 0.05 to 5 on a logarithmic axis. One curve for each value of n (sample size): 30, 15, 60. Other inputs as in the worked example: alpha = 0.05. 41 points per curve computed by statistics-ai999x; marked: the worked example, d = 0.5, power = 0.781908.

```
statistics_ai999x.solve('power_one_sample_z', d=0.5, n=30, alpha=0.05)
# -> power = 0.781908   (SOLVED)
```

```
statistics_ai999x.solve('power_one_sample_z', d=0.3, n=100, alpha=0.01,
alternative='greater')
# -> power = 0.749734   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the power of a one-sample z-test with effect size d = 0.5, n = 30 and alpha = 0.05, two-sided.	SOLVED	power = 0.781908
What is the power of a one-sample z-test for d = 0.3, n = 100, alpha = 0.01, right-tailed?	SOLVED	power = 0.749734
Find the power of a one-sample z-test with effect size d = 0.5 and alpha = 0.05, two-sided.	NEEDS_INPUT	power_one_sample_z needs n (sample size). Example: solve('power_one_sample_z', d=0.5, n=30, alpha=0.05).

Power of a two-sample z-test, `power_two_sample_z`

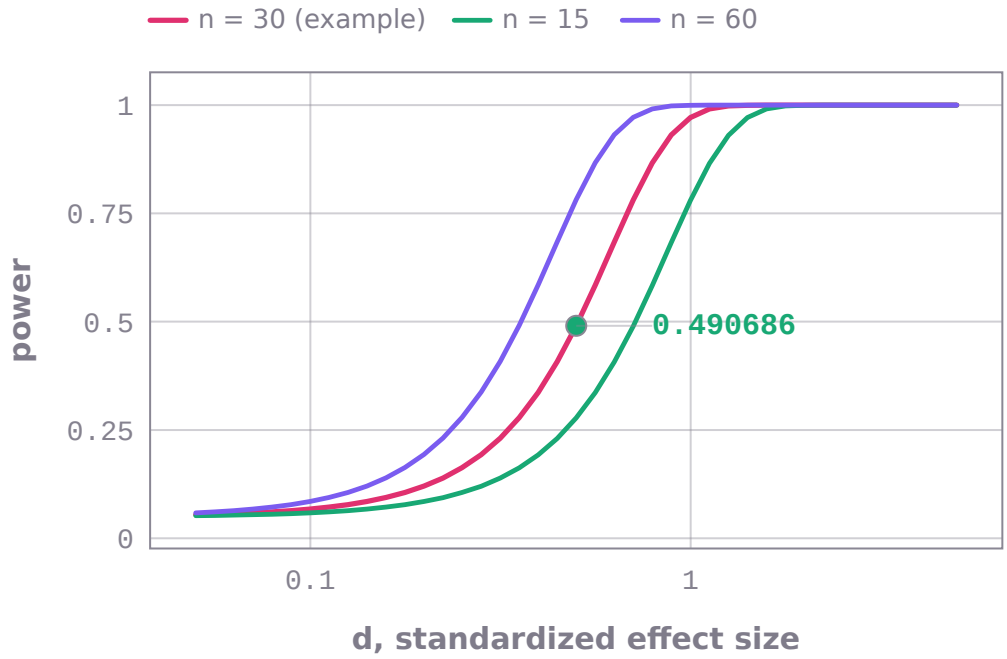
The power of a two-sample z-test for a standardized effect size d (Cohen's d), sample size n per group and significance level alpha.

$$1 - \beta = \Phi\left(d\sqrt{n_{\text{eff}}} - z_{1-\alpha/2}\right) + \Phi\left(-d\sqrt{n_{\text{eff}}} - z_{1-\alpha/2}\right)$$

input	meaning	condition
`d`	standardized effect size (difference / sigma)	
`n`	sample size per group	
`alpha`	significance level	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`power`	probability of rejecting H0 when the effect is d

- **Method:** power = P(Z > z_crit - d sqrt(n_eff)) (+ the opposite tail when two-sided), n_eff = n/2
- **Assumes:** normal approximation (sigma known); two-sided power includes both rejection tails
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



power (probability of rejecting H0 when the effect is d) against d (standardized effect size (difference / sigma)) from 0.05 to 5 on a logarithmic axis. One curve for each value of n (sample size per group): 30, 15, 60. Other inputs as in the worked example: alpha = 0.05. 41 points per curve computed by statistics-ai999x; marked: the worked example, d = 0.5, power = 0.490686.

```
statistics_ai999x.solve('power_two_sample_z', d=0.5, n=30, alpha=0.05)
# -> power = 0.490686   (SOLVED)
```

```
statistics_ai999x.solve('power_two_sample_z', d=0.3, n=100, alpha=0.01,
alternative='greater')
# -> power = 0.418775   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the power of a two-sample z-test with effect size d = 0.5, n = 30 and alpha = 0.05, two-sided.	SOLVED	power = 0.490686

question	outcome	result
What is the power of a two-sample z-test for d = 0.3, n = 100, alpha = 0.01, right-tailed?	SOLVED	power = 0.418775
Find the power of a two-sample z-test with effect size d = 0.5 and alpha = 0.05, two-sided.	NEEDS_INPUT	power_two_sample_z needs n (sample size per group). Example: solve('power_two_sample_z', d=0.5, n=30, alpha=0.05).

Sample size per group to compare two means (z-test power), `sample_size_two_means`

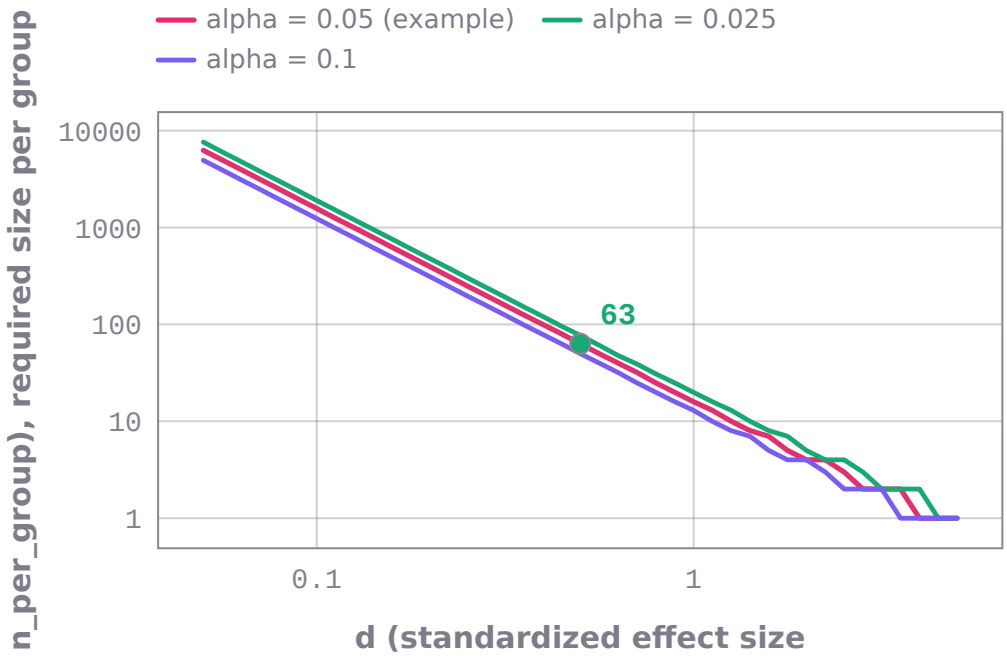
The sample size per group for a two-sample z-test to reach a target power for a standardized effect size d.

$$n \approx 2\left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d}\right)^2$$

input	meaning	condition
`d`	standardized effect size (difference / sigma)	
`alpha`	significance level	
`power`	target power (e.g. 0.8)	
`alternative`	alternative hypothesis: two-sided (default), greater or less	

output	meaning
`n_per_group`	required size per group (rounded up)
`n_exact`	solution before rounding

- **Method:** solve power(n) = target by bisection, round up
- **Assumes:** equal group sizes; normal approximation; two-sided power includes both tails
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n_per_group (required size per group (rounded up)) against d (standardized effect size (difference / sigma)) from 0.05 to 5 on a logarithmic axis, logarithmic output axis. One curve for each value of alpha (significance level): 0.05, 0.025, 0.1. Other inputs as in the worked example: power = 0.8. 41 points per curve computed by statistics-ai999x; marked: the worked example, d = 0.5, n_per_group = 63.

```
statistics_ai999x.solve('sample_size_two_means', d=0.5, alpha=0.05, power=0.8)
# -> n_per_group = 63, n_exact = 62.7909   (SOLVED)
```

```
statistics_ai999x.solve('sample_size_two_means', d=0.3, alpha=0.05, power=0.9,
alternative='greater')
# -> n_per_group = 191, n_exact = 190.308   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What sample size per group is needed for two groups with effect size d = 0.5, alpha = 0.05 and power = 0.8, two-sided?	SOLVED	n_per_group = 63, n_exact = 62.7909
How many subjects per group for two groups to detect d = 0.3 with power = 0.9 at alpha = 0.05, right-tailed?	SOLVED	n_per_group = 191, n_exact = 190.308
What sample size per group is needed for two groups with effect size d = 0.5 and power = 0.8, two-sided?	NEEDS_INPUT	sample_size_two_means needs alpha (significance level). Example: solve('sample_size_two_means', d=0.5, alpha=0.05, power

X-bar and R control chart limits, `xbar_r_chart`

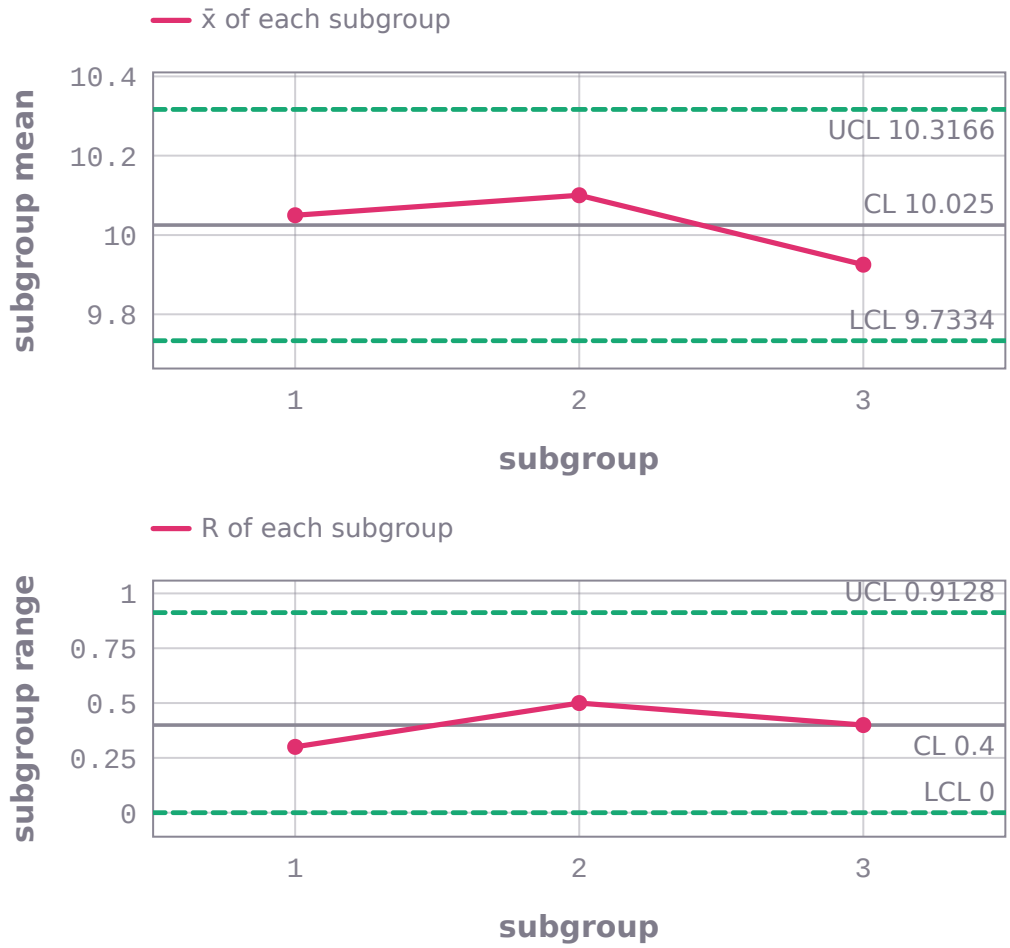
Centre lines and $3-\sigma$ control limits of X-bar and R charts from subgroups (rows) of size 2 to 10.

$$UCL_{\bar{x}} = \bar{\bar{x}} + A_2\bar{R}, \quad UCL_R = D_4\bar{R}$$

input	meaning	condition
`data`	one row per subgroup, one column per observation (subgroup size 2-10)	

output	meaning
`center_xbar`	grand mean
`mean_range`	average range
`xbar_ucl`	X-bar UCL
`xbar_lcl`	X-bar LCL
`r_ucl`	R chart UCL
`r_lcl`	R chart LCL
`A2`	A2
`D3`	D3
`D4`	D4

- **Method:** grand mean +/- A2 R-bar; R chart limits D3 R-bar and D4 R-bar
- **Assumes:** equal subgroup sizes 2 to 10; standard 3-decimal control chart constants (A2, D3, D4)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x̄ chart (top) and R chart (bottom) for the subgroups [[10.2, 9.9, 10.1, 10.0], [10.3, 10.1, 9.8, 10.2], [9.9, 10.0, 10.1, 9.7]]: subgroup means and ranges from the package's mean and range runs; returned centre $\bar{\bar{x}}$ = 10.025, limits 9.7334 – 10.3166 (A2 = 0.729); $\bar{R}$ = 0.4, limits 0 – 0.9128 (D3 = 0, D4 = 2.282).

```
statistics_ai999x.solve('xbar_r_chart', data='[[10.2, 9.9, 10.1, 10.0], [10.3, 10.1, 9.8, 10.2], [9.9, 10.0, 10.1, 9.7]]')
# -> center_xbar = 10.025, mean_range = 0.4, xbar_ucl = 10.3166, xbar_lcl = 9.7334, r_ucl = 0.9128, r_lcl = 0, A2 = 0.729, D3 = 0, D4 = 2.282   (SOLVED)
```

```
statistics_ai999x.solve('xbar_r_chart', data=[[5, 6, 4], [6, 7, 5], [5, 5, 6], [4, 6, 5]])
# -> center_xbar = 5.33333, mean_range = 1.75, xbar_ucl = 7.12358, xbar_lcl = 3.54308, r_ucl = 4.5045, r_lcl = 0, A2 = 1.023, D3 = 0, D4 = 2.574   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the X-bar and R chart limits for subgroups [[10.2, 9.9, 10.1, 10.0], [10.3, 10.1, 9.8, 10.2], [9.9, 10.0, 10.1, 9.7]].	SOLVED	center_xbar = 10.025, mean_range = 0.4, xbar_ucl = 10.3166, xbar_lcl = 9.7334, r_ucl = 0.9128, r_lcl = 0, A2 = 0.729, D3 = 0, D4 = 2.282
Compute X-bar R control limits for [[5, 6, 4], [6, 7, 5], [5, 5, 6], [4, 6, 5]].	SOLVED	center_xbar = 5.33333, mean_range = 1.75, xbar_ucl = 7.12358, xbar_lcl = 3.54308, r_ucl = 4.5045, r_lcl = 0, A2 = 1.023, D3 = 0, D4 = 2.574
Compute X-bar R control limits for [[5, 6, 4, 1, 2, 3, 4, 5, 6, 7, 8], [6, 7, 5, 1, 2, 3, 4, 5, 6, 7, 8]].	INVALID_INPUT	X-bar and R control chart limits: subgroup sizes from 2 to 10 are supported.

X-bar and S control chart limits, `xbar_s_chart`

Centre lines and $3-\sigma$ control limits of X-bar and S charts from subgroups (rows) of size 2 to 10.

$$UCL_{\bar{x}} = \bar{\bar{x}} + A_3\bar{s}, \quad UCL_s = B_4\bar{s}$$

input	meaning	condition
`data`	one row per subgroup, one column per observation (subgroup size 2-10)	

output	meaning
`center_xbar`	grand mean
`mean_sd`	average subgroup standard deviation
`xbar_ucl`	X-bar UCL
`xbar_lcl`	X-bar LCL
`s_ucl`	S chart UCL
`s_lcl`	S chart LCL
A3	A3
B3	B3
B4	B4

- **Method:** grand mean +/- A3 s-bar; S chart limits B3 s-bar and B4 s-bar
- **Assumes:** equal subgroup sizes 2 to 10; standard 3-decimal constants (A3, B3, B4)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x̄ chart (top) and s chart (bottom) for the subgroups [[10.2, 9.9, 10.1, 10.0], [10.3, 10.1, 9.8, 10.2], [9.9, 10.0, 10.1, 9.7]]: subgroup means and standard deviations from the package's runs; returned centre 10.025, limits 9.74503 – 10.305 (A3 = 1.628); s̄ = 0.171969, limits 0 – 0.389681 (B3 = 0, B4 = 2.266).

```
statistics_ai999x.solve('xbar_s_chart', data='[[10.2, 9.9, 10.1, 10.0], [10.3, 10.1, 9.8, 10.2], [9.9, 10.0, 10.1, 9.7]]')
# -> center_xbar = 10.025, mean_sd = 0.171969, xbar_ucl = 10.305, xbar_lcl = 9.74503, s_ucl = 0.389681, s_lcl = 0, A3 = 1.628, B3 = 0, B4 = 2.266   (SOLVED)
```

```
statistics_ai999x.solve('xbar_s_chart', data=[[5, 6, 4], [6, 7, 5], [5, 5, 6], [4, 6, 5]])
# -> center_xbar = 5.33333, mean_sd = 0.894338, xbar_ucl = 7.08087, xbar_lcl = 3.5858, s_ucl = 2.29666, s_lcl = 0, A3 = 1.954, B3 = 0, B4 = 2.568   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the X-bar and S chart limits for subgroups [[10.2, 9.9, 10.1, 10.0], [10.3, 10.1, 9.8, 10.2], [9.9, 10.0, 10.1, 9.7]].	SOLVED	center_xbar = 10.025, mean_sd = 0.171969, xbar_ucl = 10.305, xbar_lcl = 9.74503, s_ucl = 0.389681, s_lcl = 0, A3 = 1.628, B3 = 0, B4 = 2.266
Compute X-bar S control limits for [[5, 6, 4], [6, 7, 5], [5, 5, 6], [4, 6, 5]].	SOLVED	center_xbar = 5.33333, mean_sd = 0.894338, xbar_ucl = 7.08087, xbar_lcl = 3.5858, s_ucl = 2.29666, s_lcl = 0, A3 = 1.954, B3 = 0, B4 = 2.568
Compute X-bar S control limits for [[5, 6, 4]].	INVALID_INPUT	X-bar and S control chart limits: the subgroup table needs at least 2 rows and 2 columns.

Individuals and moving range (I-MR) chart limits, `individuals_moving_range_chart`

Control limits of the individuals (X) and moving range charts from individual measurements in time order.

$$UCL_x = \bar{x} + 2.66\,\overline{MR}$$

input	meaning	condition
`x`	individual measurements in time order	

output	meaning
`center`	mean
`mean_moving_range`	average moving range of span 2
`x_ucl`	mean + 2.66 MR-bar
`x_lcl`	mean - 2.66 MR-bar
`mr_ucl`	3.267 MR-bar
`mr_lcl`	0

- **Method:** moving ranges |x_i - x_(i-1)|; limits mean +/- 2.66 MR-bar
- **Assumes:** measurements in time order; constants $E_2 = 2.66\ (3/d_2)$ and $D_4 = 3.267$ for span 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Individuals chart (top) of x = 10.2, 9.8, 10.5, 10.1, 9.7, 10.3, 10.0 with the returned centre 10.0857 and limits 8.84438 – 11.327; moving-range chart (bottom) of the absolute differences of successive values (derived) with the returned mean moving range 0.466667 and limits 0 – 1.5246.

```
statistics_ai999x.solve('individuals_moving_range_chart', x='10.2, 9.8, 10.5, 10.1, 9.7,
10.3, 10.0')
# -> center = 10.0857, mean_moving_range = 0.466667, x_ucl = 11.327, x_lcl = 8.84438,
mr_ucl = 1.5246, mr_lcl = 0   (SOLVED)
```

```
statistics_ai999x.solve('individuals_moving_range_chart', x=[52, 49, 55, 51, 48, 53, 50,
54])
# -> center = 51.5, mean_moving_range = 4, x_ucl = 62.14, x_lcl = 40.86, mr_ucl = 13.068,
mr_lcl = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the I-MR chart limits for [10.2, 9.8, 10.5, 10.1, 9.7, 10.3, 10.0].	SOLVED	center = 10.0857, mean_moving_range = 0.466667, x_ucl = 11.327, x_lcl = 8.84438, mr_ucl = 1.5246, mr_lcl = 0
Compute individuals and moving range chart limits for [52, 49, 55, 51, 48, 53, 50, 54].	SOLVED	center = 51.5, mean_moving_range = 4, x_ucl = 62.14, x_lcl = 40.86, mr_ucl = 13.068, mr_lcl = 0
Find the I-MR chart limits for [10.2].	INVALID_INPUT	x: needs at least 2 entries, got 1.

p chart limits (fraction defective), `p_chart`

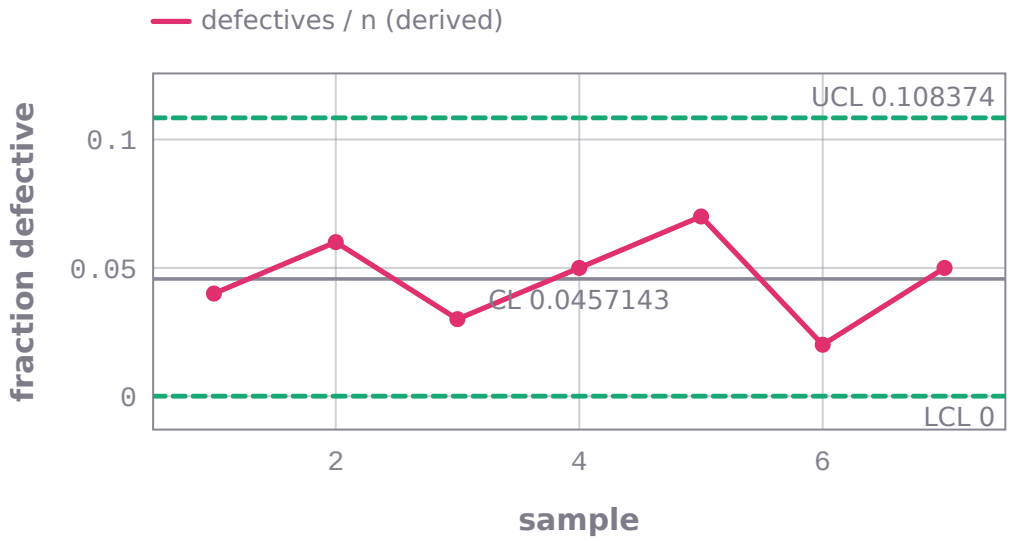
Centre line and $3-\sigma$ limits of a p chart from counts of defective items in samples of constant size n.

$$\bar{p} \pm 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n}}$$

input	meaning	condition
`defectives`	number of defective items in each sample	
`n`	sample size (constant)	

output	meaning
`center`	p-bar
`ucl`	upper control limit (at most 1)
`lcl`	lower control limit (at least 0)
`sigma`	sqrt(p-bar (1 - p-bar)/n)

- **Method:** p-bar +/- 3 sqrt(p-bar(1 - p-bar)/n)
- **Assumes:** constant sample size; LCL truncated at 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A p chart: the fraction defective of each sample of n = 100 (defectives 4, 6, 3, 5, 7, 2, 5, divided by n) with the returned centre p̄ = 0.0457143 and limits UCL = 0.108374, LCL = 0 (σ = 0.0208865).

```
statistics_ai999x.solve('p_chart', defectives='4, 6, 3, 5, 7, 2, 5', n=100)
# -> center = 0.0457143, ucl = 0.108374, lcl = 0, sigma = 0.0208865   (SOLVED)
```

```
statistics_ai999x.solve('p_chart', defectives=[2, 3, 1, 4, 2, 5], n=50)
# -> center = 0.0566667, ucl = 0.154758, lcl = 0, sigma = 0.0326973   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the p chart limits for defectives [4, 6, 3, 5, 7, 2, 5] with sample size n = 100.	SOLVED	center = 0.0457143, ucl = 0.108374, lcl = 0, sigma = 0.0208865
Compute p-chart control limits: n = 50 per sample, defectives [2, 3, 1, 4, 2, 5].	SOLVED	center = 0.0566667, ucl = 0.154758, lcl = 0, sigma = 0.0326973
Find the p chart limits for defectives [4, 6, 3] with sample size n = 5.	INVALID_INPUT	p chart limits (fraction defective): each count of defectives must be a whole number between 0 and n.
Find the p chart limits for defectives [4, 6, 3].	NEEDS_INPUT	p_chart needs n (sample size (constant)). Example: solve('p_chart', defectives='4, 6, 3, 5, 7, 2, 5', n=100).

c chart limits (defects per unit), `c_chart`

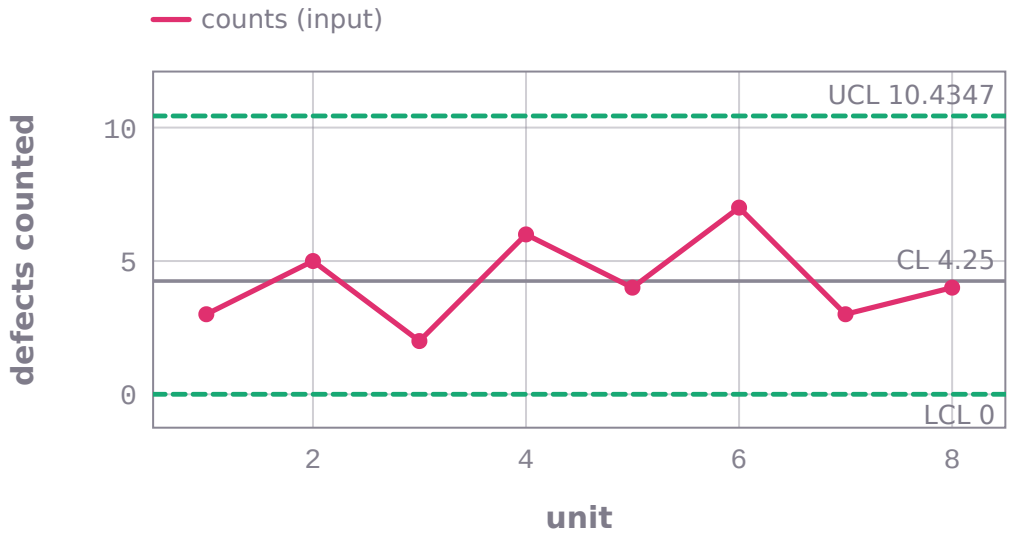
Centre line and $3-\sigma$ limits of a c chart from counts of defects per inspection unit.

$$\bar{c} \pm 3\sqrt{\bar{c}}$$

input	meaning	condition
`counts`	defects counted in each unit	

output	meaning
`center`	c-bar
`ucl`	c-bar + 3 sqrt(c-bar)
`lcl`	max(0, c-bar - 3 sqrt(c-bar))

- **Method:** c-bar +/- 3 sqrt(c-bar)
- **Assumes:** constant inspection unit; Poisson-distributed counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A c chart of the defect counts 3, 5, 2, 6, 4, 7, 3, 4 in time order with the returned centre line c̄ = 4.25 and control limits UCL = 10.4347, LCL = 0.

```
statistics_ai999x.solve('c_chart', counts='3, 5, 2, 6, 4, 7, 3, 4')
# -> center = 4.25, ucl = 10.4347, lcl = 0   (SOLVED)
```

```
statistics_ai999x.solve('c_chart', counts=[12, 9, 15, 11, 8, 10])
# -> center = 10.8333, ucl = 20.7075, lcl = 0.959125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the c chart limits for defect counts [3, 5, 2, 6, 4, 7, 3, 4].	SOLVED	center = 4.25, ucl = 10.4347, lcl = 0
Compute c-chart control limits for [12, 9, 15, 11, 8, 10].	SOLVED	center = 10.8333, ucl = 20.7075, lcl = 0.959125
Find the c chart limits for defect counts [0, 0, 0].	INVALID_INPUT	c chart limits (defects per unit): the average count must be positive.

Process capability (Cp, Cpk) and expected nonconforming, `process_capability`

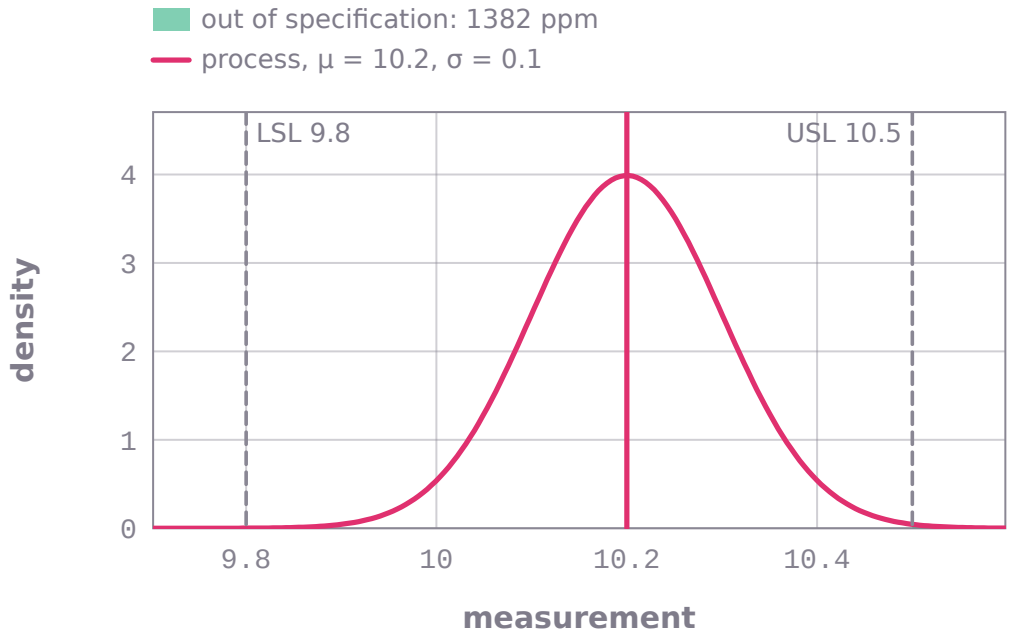
Process capability indices Cp, Cpk (with Cpu, Cpl) and the expected parts per million outside the specification for a normal process with mean mu and standard deviation sigma.

$$C_p = \frac{USL - LSL}{6\sigma}, \quad C_{pk} = \min\left(\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma}\right)$$

input	meaning	condition
`mu`	process mean	
`sigma`	process standard deviation	
`lsl`	lower specification limit	
`usl`	upper specification limit	

output	meaning
`cp`	(USL - LSL)/(6 sigma)
`cpk`	min(Cpu, Cpl)
`cpu`	(USL - mu)/(3 sigma)
`cpl`	(mu - LSL)/(3 sigma)
`ppm_out`	expected nonconforming parts per million

- **Method:** capability ratios; normal tail areas for the ppm
- **Assumes:** normal, stable (in-control) process; USL > LSL
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A process with mean 10.2 and standard deviation 0.1 against the specification limits LSL = 9.8 and USL = 10.5: the density (density from the package's cumulative probabilities on a grid, differenced) with the out-of-specification tails shaded. Returned: Cp = 1.16667, Cpk = 1 (Cpu 1, Cpl 1.33333), 1381.57 parts per million out.

```
statistics_ai999x.solve('process_capability', mu=10.2, sigma=0.1, lsl=9.8, usl=10.5)
# -> cp = 1.16667, cpk = 1, cpu = 1, cpl = 1.33333, ppm_out = 1381.57   (SOLVED)
```

```
statistics_ai999x.solve('process_capability', mu=50, sigma=2, lsl=44, usl=58)
# -> cp = 1.16667, cpk = 1, cpu = 1.33333, cpl = 1, ppm_out = 1381.57   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find Cp and Cpk for a process with mean = 10.2, standard deviation = 0.1, LSL = 9.8 and USL = 10.5.	SOLVED	cp = 1.16667, cpk = 1, cpu = 1, cpl = 1.33333, ppm_out = 1381.57
Process capability: mu = 50, sigma = 2, lower specification limit = 44, upper specification limit = 58.	SOLVED	cp = 1.16667, cpk = 1, cpu = 1.33333, cpl = 1, ppm_out = 1381.57
Find Cp and Cpk for a process with mean = 10.2, standard deviation = 0.1, LSL = 10.8 and USL = 10.5.	INVALID_INPUT	Process capability (Cp, Cpk) and expected nonconforming: the upper specification limit must exceed the lower one.
Find Cp and Cpk for a process with mean = 10.2, LSL = 9.8 and USL = 10.5.	NEEDS_INPUT	process_capability needs sigma (process standard deviation). Example: solve('process_capability', mu=10.2, sigma=0.1, ls

Weibull reliability, hazard rate and MTTF, `weibull_reliability`

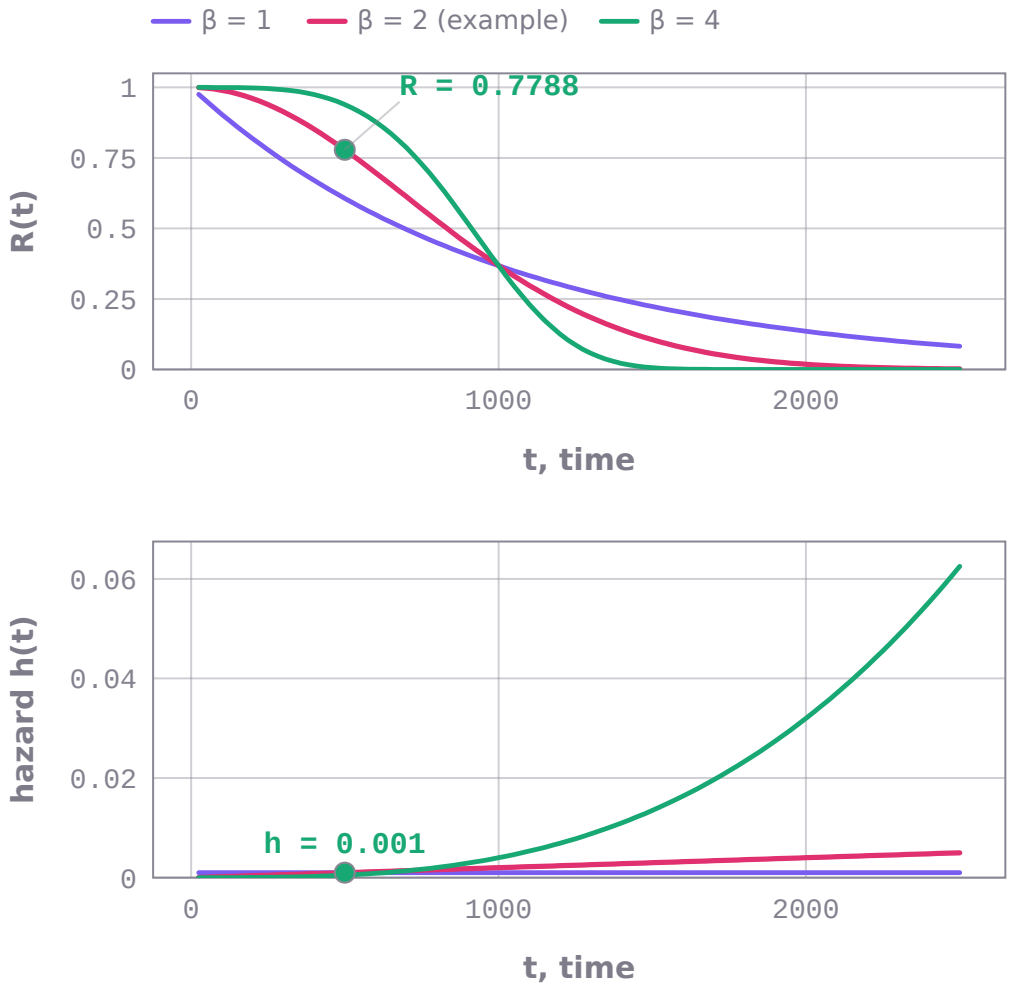
Reliability R(t), unreliability, hazard rate and mean time to failure of a two-parameter Weibull distribution.

$$R(t) = e^{-(t/\eta)^{\beta}}, \quad h(t) = \frac{\beta}{\eta}\left(\frac{t}{\eta}\right)^{\beta-1}$$

input	meaning	condition
`beta`	shape parameter beta	
`eta`	scale parameter eta (characteristic life)	
`t`	time	

output	meaning
`reliability`	exp(-(t/eta)^beta)
`unreliability`	$1 - R$(t)
`hazard_rate`	(beta/eta)$(t/\eta)^{\beta-1}$
`mttf`	eta Gamma(1 + 1/beta)

- **Method:** closed forms of the Weibull survival, hazard and mean
- **Assumes:** two-parameter Weibull (no location shift); $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Weibull reliability (top) and hazard rate (bottom) over time for η = 1000 and shape β = 1, 2 (worked example), 4, one package run per point: β < 1 failures slow down, β > 1 wear-out. Marked at t = 500: R = 0.778801, h = 0.001 (MTTF 886.227).

```
statistics_ai999x.solve('weibull_reliability', beta=2, eta=1000, t=500)
# -> reliability = 0.778801, unreliability = 0.221199, hazard_rate = 0.001, mttf = 886.227
(SOLVED)
```

```
statistics_ai999x.solve('weibull_reliability', beta=1.5, eta=2000, t=1200)
# -> reliability = 0.628287, unreliability = 0.371713, hazard_rate = 0.000580948, mttf =
1805.49   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a Weibull distribution with beta = 2 and eta = 1000 hours, find the reliability at t = 500.	SOLVED	reliability = 0.778801, unreliability = 0.221199, hazard_rate = 0.001, mttf = 886.227
Weibull with shape = 1.5 and scale = 2000: find R(t), the hazard rate and MTTF at time = 1200.	SOLVED	reliability = 0.628287, unreliability = 0.371713, hazard_rate = 0.000580948, mttf = 1805.49
For a Weibull distribution with beta = -2 and eta = 1000 hours, find the reliability at t = 500.	INVALID_INPUT	Weibull reliability, hazard rate and MTTF: the shape beta must be a positive number.
For a Weibull distribution with beta = 2, find the reliability at t = 500.	NEEDS_INPUT	weibull_reliability needs eta (scale parameter eta (characteristic life)). Example: solve('weibull_reliability', beta=2,

Exponential reliability (constant failure rate), `exponential_reliability`

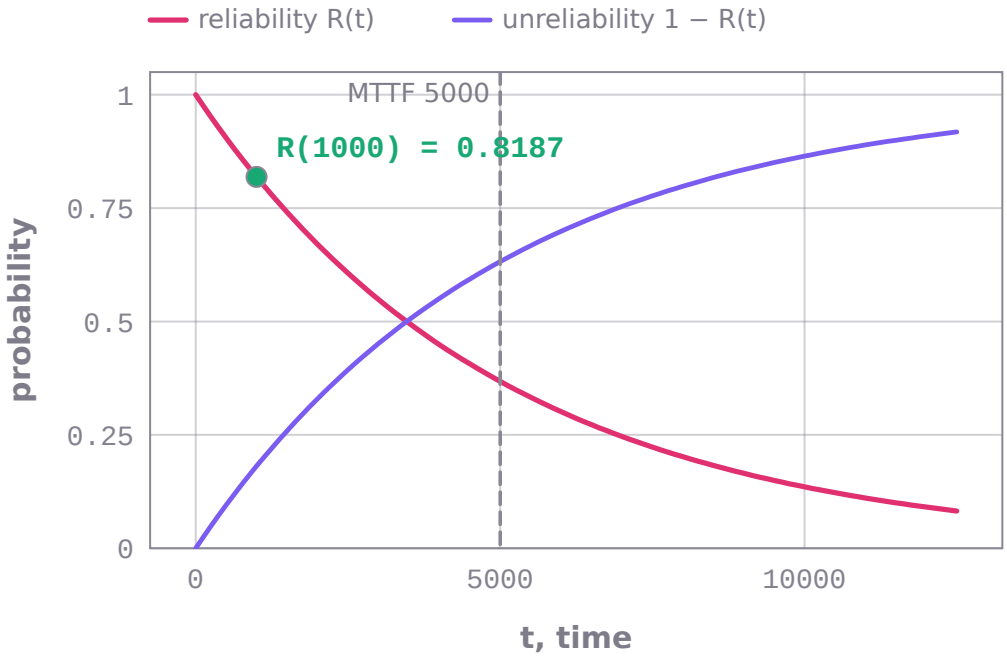
Reliability R(t) = exp(-lambda t), unreliability and MTTF for a constant failure rate (given the rate or the MTBF).

$$R(t) = e^{-\lambda t}, \quad MTTF = 1/\lambda$$

input	meaning	condition
`t`	mission time	
`rate`	failure rate lambda	
`mtbf`	mean time between failures (instead of the rate)	

output	meaning
`reliability`	exp(-lambda t)
`unreliability`	$1 - R$(t)
`mttf`	$1/\lambda$
`failure_rate`	lambda

- **Method:** R(t) = exp(-lambda t)
- **Assumes:** constant failure rate (useful-life period); exactly one of rate or MTBF
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Reliability and unreliability of a component with constant failure rate 0.0002 over time, one package run per point; marked: the mission time t = 1000 with R = 0.818731; dashed: the returned mean time to failure 5000 (where R = e^{-1}).

```
statistics_ai999x.solve('exponential_reliability', t=1000, rate=0.0002)
# -> reliability = 0.818731, unreliability = 0.181269, mttf = 5000, failure_rate = 0.0002
(SOLVED)
```

```
statistics_ai999x.solve('exponential_reliability', t=800, mtbf=5000)
# -> reliability = 0.852144, unreliability = 0.147856, mttf = 5000, failure_rate = 0.0002
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the reliability at t = 1000 hours for a constant failure rate = 0.0002 per hour.	SOLVED	reliability = 0.818731, unreliability = 0.181269, mttf = 5000, failure_rate = 0.0002
A component has MTBF = 5000 hours with exponential failures; what is the reliability for a mission time = 800?	SOLVED	reliability = 0.852144, unreliability = 0.147856, mttf = 5000, failure_rate = 0.0002
Find the reliability at t = 1000 hours for a constant failure rate = -0.0002 per hour.	INVALID_INPUT	Exponential reliability (constant failure rate): the failure rate must be a positive number.

Reliability of a series system, `series_system_reliability`

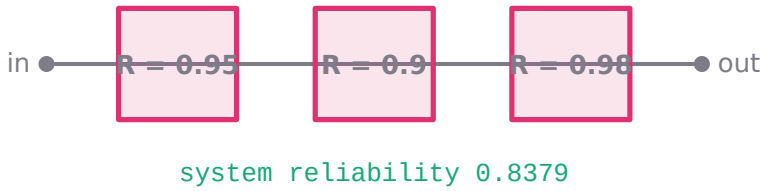
The reliability of independent components in series (all must work).

$$R_s = \prod R_i$$

input	meaning	condition
`r`	component reliabilities (0 to 1)	

output	meaning
`reliability`	system reliability

- **Method:** product rule
- **Assumes:** independent components
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



3 components in series with reliabilities 0.95, 0.9, 0.98: the system works only if every block works, so its returned reliability 0.8379 is below that of the weakest component.

```
statistics_ai999x.solve('series_system_reliability', r='0.95, 0.9, 0.98')
# -> reliability = 0.8379   (SOLVED)
```

```
statistics_ai999x.solve('series_system_reliability', r=[0.8, 0.85])
# -> reliability = 0.68   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the reliability of a series system with component reliabilities [0.95, 0.9, 0.98].	SOLVED	reliability = 0.8379
Components with reliabilities [0.8, 0.85] are connected in series; what is the system reliability?	SOLVED	reliability = 0.68
Find the reliability of a series system with component reliabilities [0.95, 1.9].	INVALID_INPUT	Reliability of a series system: each reliability must be between 0 and 1.

Reliability of a parallel system, `parallel_system_reliability`

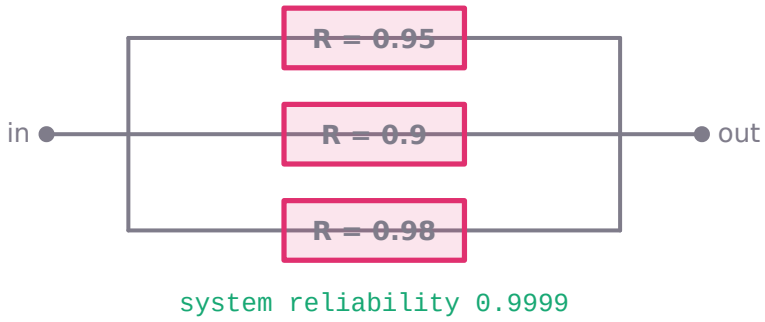
The reliability of independent components in parallel (at least one must work).

$$R_p = 1 - \prod(1 - R_i)$$

input	meaning	condition
`r`	component reliabilities (0 to 1)	

output	meaning
`reliability`	system reliability

- **Method:** product rule on the unreliabilities
- **Assumes:** independent components
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



3 redundant components in parallel with reliabilities 0.95, 0.9, 0.98: the system fails only if every block fails, so its returned reliability 0.9999 exceeds that of the best component.

```
statistics_ai999x.solve('parallel_system_reliability', r='0.95, 0.9, 0.98')
# -> reliability = 0.9999   (SOLVED)
```

```
statistics_ai999x.solve('parallel_system_reliability', r=[0.8, 0.85])
# -> reliability = 0.97   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the reliability of a parallel system with component reliabilities [0.95, 0.9, 0.98].	SOLVED	reliability = 0.9999
Components with reliabilities [0.8, 0.85] are connected in parallel; what is the system reliability?	SOLVED	reliability = 0.97
Find the reliability of a parallel system with component reliabilities [0.95, 1.9].	INVALID_INPUT	Reliability of a parallel system: each reliability must be between 0 and 1.

k-out-of-n system reliability, `k_out_of_n_reliability`

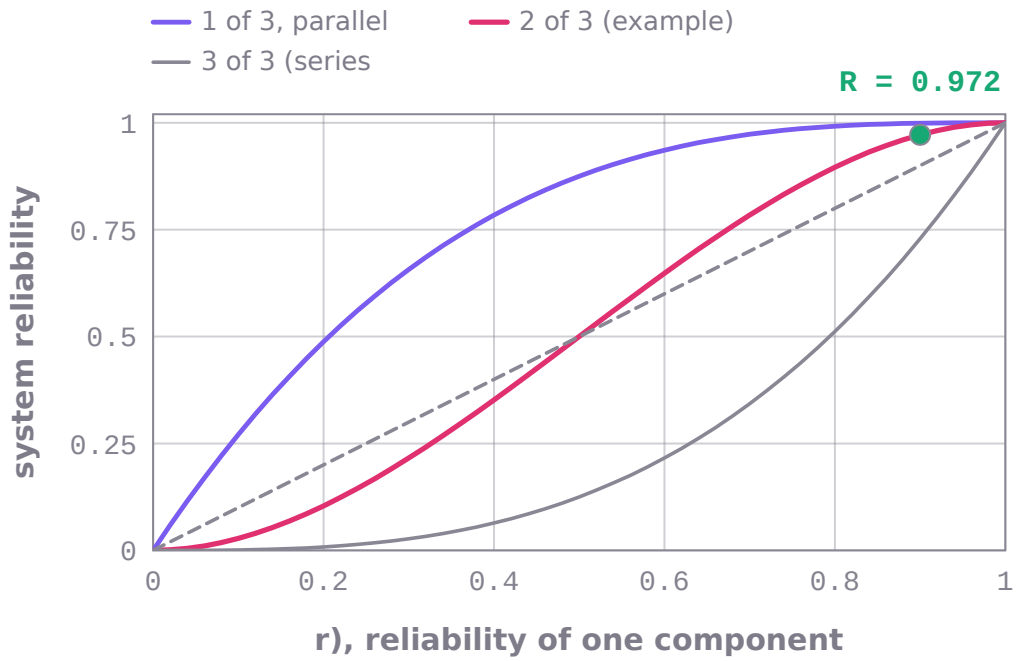
The reliability of a system that works when at least k of n identical independent components work.

$$R = \sum_{j=k}^{n} \binom{n}{j} r^j (1-r)^{n-j}$$

input	meaning	condition
n	number of components	
k	number that must work	
r	reliability of one component	

output	meaning
`reliability`	P(at least k of n work)

- **Method:** binomial upper tail
- **Assumes:** identical, independent components
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



System reliability when at least k of n = 3 identical components must work, against one component's reliability r, for k = 1 … 3 (one package run per point); dashed: a single component. Marked: the worked example k = 2, r = 0.9, R = 0.972.

```
statistics_ai999x.solve('k_out_of_n_reliability', n=3, k=2, r=0.9)
# -> reliability = 0.972   (SOLVED)
```

```
statistics_ai999x.solve('k_out_of_n_reliability', n=5, k=3, r=0.85)
# -> reliability = 0.973388   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the reliability of a k-out-of-n system with k = 2, n = 3 and component reliability = 0.9.	SOLVED	reliability = 0.972
A system needs k = 3 out of n = 5 components working, each with r = 0.85; what is the system reliability?	SOLVED	reliability = 0.973388
Find the reliability of a k-out-of-n system with k = 4, n = 3 and component reliability = 0.9.	INVALID_INPUT	k-out-of-n system reliability: k cannot exceed n.

Kaplan-Meier survival estimate, `kaplan_meier`

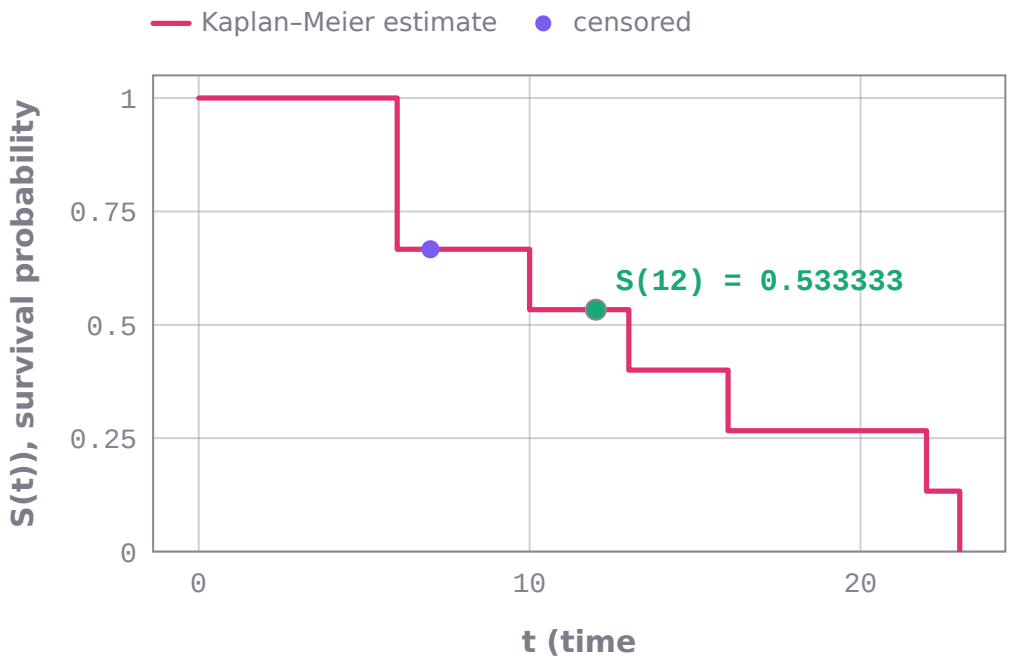
The Kaplan-Meier (product-limit) estimate of the survival function from times with event/censoring indicators, evaluated at time t, and the whole step curve.

$$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right)$$

input	meaning	condition
`times`	follow-up times	
`events`	1 = event observed, 0 = censored	
`t`	time at which to report S(t)	

output	meaning
`survival`	S(t)
`event_times`	distinct event times
`survival_curve`	S just after each event time

- **Method:** product over event times of (1 - deaths / number at risk)
- **Assumes:** non-informative censoring; a censoring at an event time counts as still at risk
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Kaplan–Meier survival curve returned by the package for times 6, 6, 6, 7, 10, 13, 16, 22, 23 with event indicators 1, 1, 1, 0, 1, 1, 1, 1, 1 (0 = censored): it steps down at each event time 6, 10, 13, 16, 22, 23; marked: t = 12, S = 0.533333.



survival (S(t)) against t (time at which to report S(t)) from 1.2 to 120 on a logarithmic axis. Other inputs as in the worked example: times = 6, 6, 6, 7, 10, 13, 16, 22, 23, events = 1, 1, 1, 0, 1, 1, 1, 1, 1. 41 points computed by statistics-ai999x; marked: the worked example, t = 12, survival = 0.533333.

```
statistics_ai999x.solve('kaplan_meier', times='6, 6, 6, 7, 10, 13, 16, 22, 23', events='1, 1, 1, 0, 1, 1, 1, 1, 1', t=12)
# -> survival = 0.533333, event_times = [6.0, 10.0, 13.0, 16.0, 22.0, 23.0], survival_curve = [0.6666666666666667, 0.5333333333333334, 0.4000000000000001, 0.2666666666666668, 0.1333333333333334, 0.0]   (SOLVED)
```

```
statistics_ai999x.solve('kaplan_meier', times=[2, 3, 3, 5, 8, 9, 12], events=[1, 0, 1, 1, 0, 1, 1], t=6)
# -> survival = 0.535714, event_times = [2.0, 3.0, 5.0, 9.0, 12.0], survival_curve = [0.8571428571428572, 0.7142857142857144, 0.5357142857142858, 0.2678571428571429, 0.0]  (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kaplan-Meier: times [6, 6, 6, 7, 10, 13, 16, 22, 23] with events [1, 1, 1, 0, 1, 1, 1, 1, 1]; find the survival at t = 12.	SOLVED	survival = 0.533333, event_times = [6.0, 10.0, 13.0, 16.0, 22.0, 23.0], survival_curve = [0.6666666666666667, 0.5333333333333334, 0.4000000000000001, 0.2666666666666668, 0.1333333333333334, 0.0]
Estimate the survival probability by Kaplan-Meier for times [2, 3, 3, 5, 8, 9, 12] and events [1, 0, 1, 1, 0, 1, 1] at time = 6.	SOLVED	survival = 0.535714, event_times = [2.0, 3.0, 5.0, 9.0, 12.0], survival_curve = [0.8571428571428572, 0.7142857142857144, 0.5357142857142858, 0.2678571428571429, 0.0]
Kaplan-Meier: times [6, 7, 10] with events [1, 2, 1]; find the survival at t = 8.	INVALID_INPUT	Kaplan-Meier survival estimate: event indicators must be 1 (event) or 0 (censored).

Log-rank test comparing two survival curves, `log_rank_test`

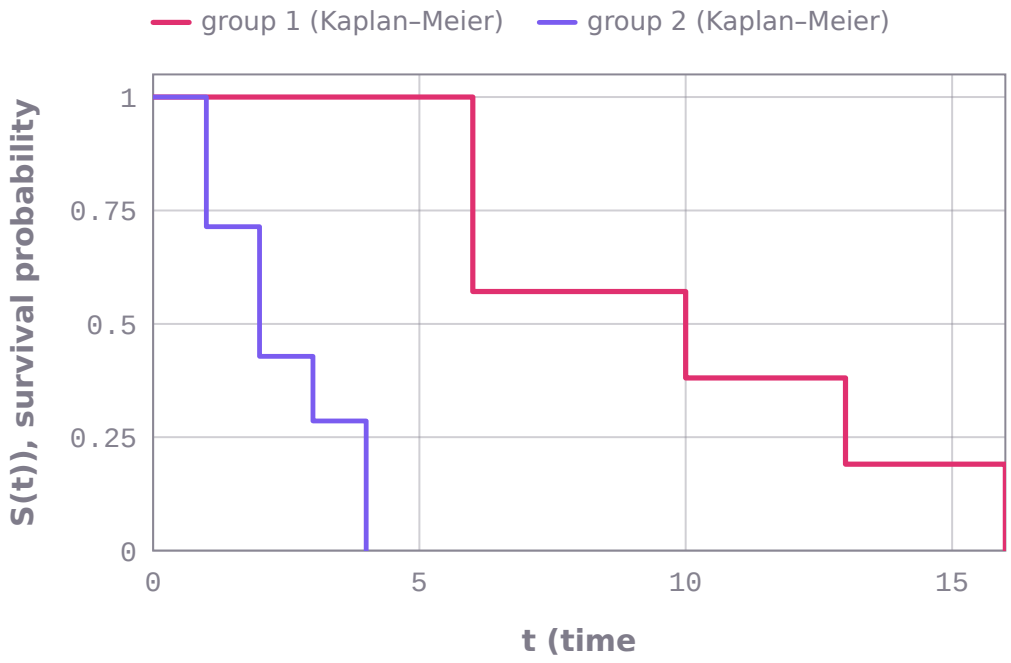
The log-rank (Mantel-Cox) test of equal survival in two groups from times and event indicators.

$$\chi^2 = \frac{(\sum(d_{1i} - e_{1i}))^2}{\sum v_i}$$

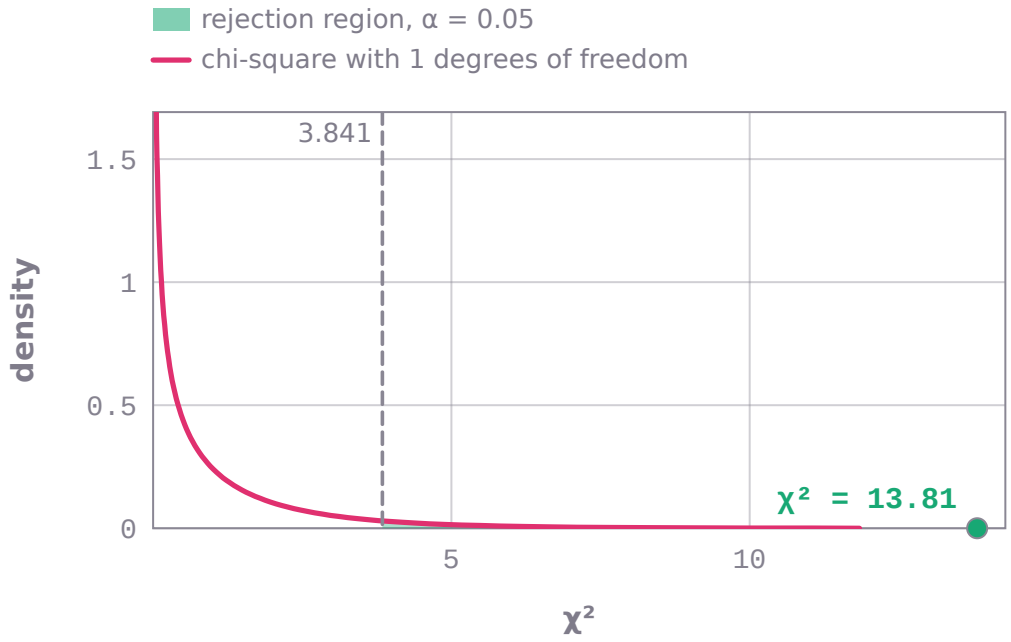
input	meaning	condition
`times1`	times, group 1	
`events1`	events $(1/0)$, group 1	
`times2`	times, group 2	
`events2`	events $(1/0)$, group 2	

output	meaning
`chi_square`	$(O_1 - E_1)^2/V$
`p_value`	chi-square p-value (1 df)
`observed_1`	events in group 1
`expected_1`	expected events in group 1

- **Method:** at each event time: expected events and hypergeometric variance; sum over times
- **Assumes:** non-informative censoring; proportional hazards gives the test its best power
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kaplan–Meier curves returned by the package for group 1 (times 6, 6, 6, 7, 10, 13, 16, events 1, 1, 1, 0, 1, 1, 1) and group 2 (times 1, 1, 2, 2, 3, 4, 4, events 1, 1, 1, 1, 1, 1, 1). Log-rank test: χ² = 13.8115, p-value 0.000202097; group 1 had 6 events against 10.4222 expected.



The log-rank test of the two groups of the worked example. Drawn: the chi-square density with 1 degrees of freedom (density from the package's quantiles at 121 cumulative probabilities, differenced); shaded: the rejection region above 3.84146 for α = 0.05 (critical value from the package) and, paler, the area beyond the statistic that the p-value measures. Marked: χ² = 13.8115, p-value = 0.000202097.

```
statistics_ai999x.solve('log_rank_test', times1='6, 6, 6, 7, 10, 13, 16', events1='1, 1, 1, 0, 1, 1, 1', times2='1, 1, 2, 2, 3, 4, 4', events2='1, 1, 1, 1, 1, 1, 1')
# -> chi_square = 13.8115, p_value = 0.000202097, observed_1 = 6, expected_1 = 10.4222 (SOLVED)
```

```
statistics_ai999x.solve('log_rank_test', times1=[5, 8, 12, 20], events1=[1, 1, 0, 1], times2=[3, 4, 9, 10], events2=[1, 1, 1, 0])
# -> chi_square = 0.581884, p_value = 0.445575, observed_1 = 3, expected_1 = 3.8381 (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Log-rank test: group 1 times [6, 6, 6, 7, 10, 13, 16] events [1, 1, 1, 0, 1, 1, 1], group 2 times [1, 1, 2, 2, 3, 4, 4] events [1, 1, 1, 1, 1, 1, 1].	SOLVED	chi_square = 13.8115, p_value = 0.000202097, observed_1 = 6, expected_1 = 10.4222
Compare survival with a log-rank test for times [5, 8, 12, 20] events [1, 1, 0, 1] versus times [3, 4, 9, 10] events [1, 1, 1, 0].	SOLVED	chi_square = 0.581884, p_value = 0.445575, observed_1 = 3, expected_1 = 3.8381
Log-rank test: group 1 times [6, 7] events [1, 1].	NEEDS_INPUT	log_rank_test needs times2, events2 (times, group 2; events (1/0), group 2). Example: solve('log_rank_test', times1='6,

Probability rules: complement, addition and multiplication, `probability_rules`

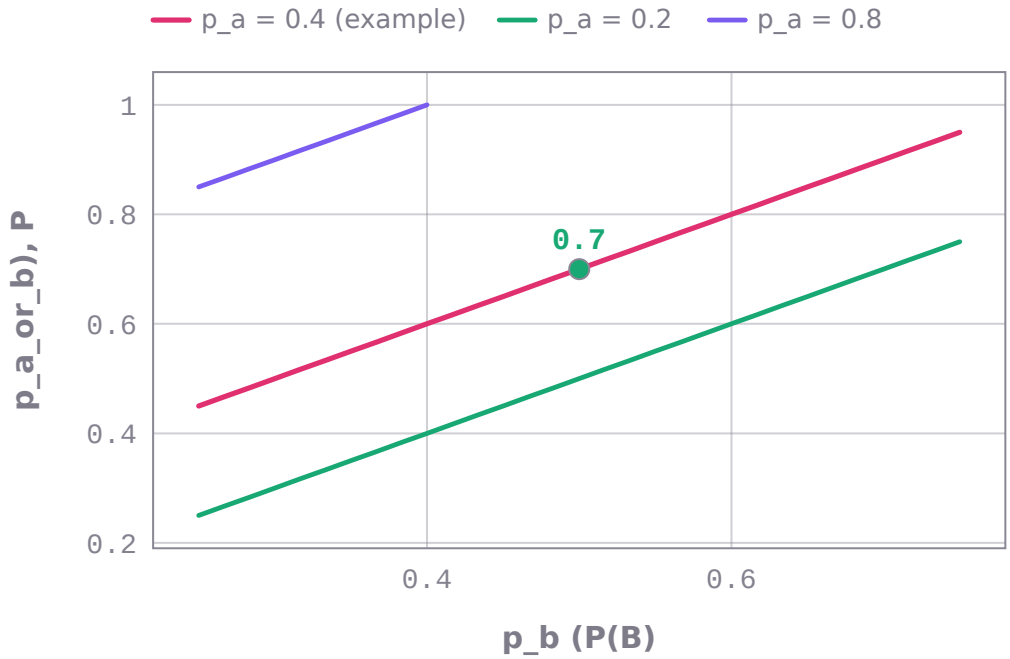
From P(A), P(B) and one statement about how A and B overlap (P(A and B), P(A or B), independent, or mutually exclusive): P(not A), P(not B), P(A and B), P(A or B), P(neither), P(A only), P(B only) and the conditional probabilities P(A | B) and P(B | A).

$$P(A^c) = 1 - P(A), \quad P(A \cup B) = P(A) + P(B) - P(A \cap B), \quad P(A \cap B) \overset{\text{indep.}}{=} P(A)P(B)$$

input	meaning	condition
`p_a`	P(A)	
`p_b`	P(B)	
`p_a_and_b`	P(A and B)	
`p_a_or_b`	P(A or B)	
`relation`	independent or mutually exclusive (instead of P(A and B) or P(A or B))	

output	meaning
`p_not_a`	P(not A) = 1 - P(A)
`p_not_b`	P(not B) = 1 - P(B)
`p_a_and_b`	P(A and B)
`p_a_or_b`	P(A or B) = P(A) + P(B) - P(A and B)
`p_neither`	P(neither) = 1 - P(A or B)
`p_a_only`	P(A and not B)
`p_b_only`	P(B and not A)
`p_a_given_b`	P(A | B) = P(A and B) / P(B)
`p_b_given_a`	P(B | A) = P(A and B) / P(A)

- **Method:** complement rule $1 - P$; multiplication rule P(A and B) = P(A) P(B) for independent events (0 for mutually exclusive ones); addition rule P(A or B) = P(A) + P(B) - P(A and B); conditional probability P(A | B) = P(A and B) / P(B)
- **Assumes:** probabilities between 0 and 1; exactly one statement about the overlap; max(0, P(A) + P(B) - 1) <= P(A and B) <= min(P(A), P(B))
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



p_a_or_b (P(A or B) = P(A) + P(B) - P(A and B)) against p_b (P(B)) from 0.25 to 0.75. One curve for each value of p_a (P(A)): 0.4, 0.2, 0.8. Other inputs as in the worked example: p_a_and_b = 0.2. 31 points per curve computed by statistics-ai999x; marked: the worked example, p_b = 0.5, p_a_or_b = 0.7.

```
statistics_ai999x.solve('probability_rules', p_a=0.4, p_b=0.5, p_a_and_b=0.2)
# -> p_not_a = 0.6, p_not_b = 0.5, p_a_and_b = 0.2, p_a_or_b = 0.7, p_neither = 0.3,
p_a_only = 0.2, p_b_only = 0.3, p_a_given_b = 0.4, p_b_given_a = 0.5   (SOLVED)
```

```
statistics_ai999x.solve('probability_rules', p_a=0.3, p_b=0.6, relation='independent')
# -> p_not_a = 0.7, p_not_b = 0.4, p_a_and_b = 0.18, p_a_or_b = 0.72, p_neither = 0.28,
p_a_only = 0.12, p_b_only = 0.42, p_a_given_b = 0.3, p_b_given_a = 0.6   (SOLVED)
```

```
statistics_ai999x.solve('probability_rules', p_a=0.25, p_b=0.4, relation='mutually
exclusive')
# -> p_not_a = 0.75, p_not_b = 0.6, p_a_and_b = 0, p_a_or_b = 0.65, p_neither = 0.35,
p_a_only = 0.25, p_b_only = 0.4, p_a_given_b = 0, p_b_given_a = 0   (SOLVED)
```

```
statistics_ai999x.solve('probability_rules', p_a=0.5, p_b=0.4, p_a_or_b=0.7)
# -> p_not_a = 0.5, p_not_b = 0.6, p_a_and_b = 0.2, p_a_or_b = 0.7, p_neither = 0.3,
p_a_only = 0.3, p_b_only = 0.2, p_a_given_b = 0.5, p_b_given_a = 0.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
P(A) = 0.4, P(B) = 0.5 and P(A and B) = 0.2. Find P(not A), P(A or B) and P(A | B).	SOLVED	p_not_a = 0.6, p_not_b = 0.5, p_a_and_b = 0.2, p_a_or_b = 0.7, p_neither = 0.3, p_a_only = 0.2, p_b_only = 0.3, p_a_given_b = 0.4, p_b_given_a = 0.5
A and B are independent with P(A) = 0.3 and P(B) = 0.6. Find P(A and B) and P(A or B).	SOLVED	p_not_a = 0.7, p_not_b = 0.4, p_a_and_b = 0.18, p_a_or_b = 0.72, p_neither = 0.28, p_a_only = 0.12, p_b_only = 0.42, p_a_given_b = 0.3, p_b_given_a = 0.6
A and B are mutually exclusive with P(A) = 0.25 and P(B) = 0.4. Find P(A or B).	SOLVED	p_not_a = 0.75, p_not_b = 0.6, p_a_and_b = 0, p_a_or_b = 0.65, p_neither = 0.35, p_a_only = 0.25, p_b_only = 0.4, p_a_given_b = 0, p_b_given_a = 0

question	outcome	result
P(A) = 0.5, P(B) = 0.4 and P(A or B) = 0.7. Find P(A and B).	SOLVED	p_not_a = 0.5, p_not_b = 0.6, p_a_and_b = 0.2, p_a_or_b = 0.7, p_neither = 0.3, p_a_only = 0.3, p_b_only = 0.2, p_a_given_b = 0.5, p_b_given_a = 0.4
P(A) = 0.3, P(B) = 0.4 and P(A and B) = 0.5. Find P(A or B).	INVALID_INPUT	Probability rules: complement, addition and multiplication: these probabilities are inconsistent: P(A and B) must lie be
P(A) = 0.3 and P(B) = 0.4. Find P(A or B).	INVALID_INPUT	Probability rules: complement, addition and multiplication: state exactly one of: P(A and B), P(A or B), independent, mu

Joint, marginal and conditional probabilities from a two-way table, `two_way_table_probabilities`

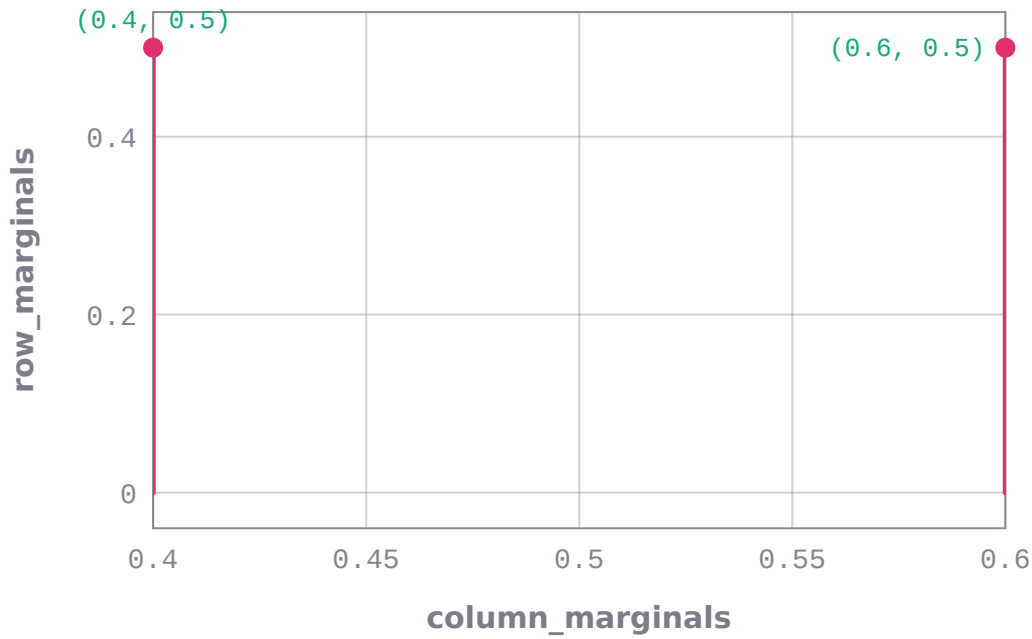
From a two-way table of counts: the joint, marginal and conditional probabilities of every cell, and for a chosen row and column P(row and column), P(row), P(column), P(row | column), P(column | row) and P(row or column).

$$P(R_i \mid C_j) = \frac{n_{ij}}{n_{\cdot j}}, \quad P(C_j \mid R_i) = \frac{n_{ij}}{n_{i\cdot}}, \quad P(R_i \cap C_j) = \frac{n_{ij}}{n}$$

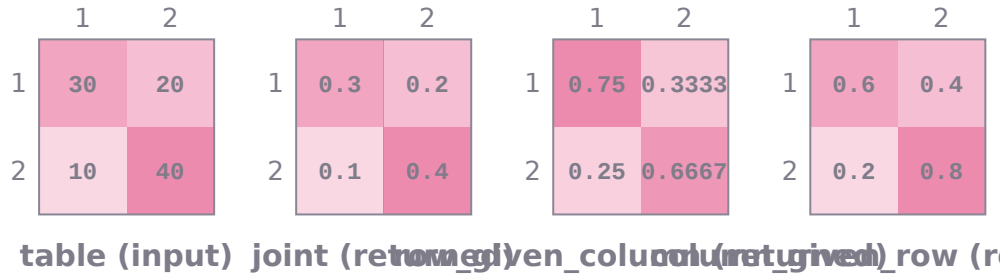
input	meaning	condition
`table`	the two-way table of counts (rows x columns)	
`row`	the row of interest (from 1)	
`column`	the column of interest (from 1)	

output	meaning
`p_row_given_column`	P(row | column) = cell / column total
`p_column_given_row`	P(column | row) = cell / row total
`p_row_and_column`	P(row and column) = cell / grand total
`p_row`	P(row) = row total / grand total
`p_column`	P(column) = column total / grand total
`p_row_or_column`	P(row or column)
`total`	grand total
`joint`	joint probabilities of every cell
`row_marginals`	marginal probabilities of the rows
`column_marginals`	marginal probabilities of the columns
`row_given_column`	P(row i | column j) for every cell
`column_given_row`	P(column j | row i) for every cell

- **Method:** divide a cell by the grand total (joint), by its column total (P(row | column)) or by its row total (P(column | row)); margins are the row and column totals over the grand total
- **Assumes:** counts >= 0; every row and column has a positive total; row and column numbered from 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



marginal probabilities of the rows (row_marginals) at each marginal probabilities of the columns (column_marginals), both returned by statistics-ai999x for the worked example (table = [[30, 20], [10, 40]], row = 1, column = 2): 0.4 → 0.5; 0.6 → 0.5.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): table (input); joint (returned); row_given_column (returned); column_given_row (returned). Returned with them by statistics-ai999x: total = 100, p_row_and_column = 0.2, p_row = 0.5, p_column = 0.6, p_row_given_column = 0.333333, p_column_given_row = 0.4, p_row_or_column = 0.9. Other inputs: row = 1, column = 2.

```
statistics_ai999x.solve('two_way_table_probabilities', table='[[30, 20], [10, 40]]', row=1,
column=2)
# -> total = 100, joint = [[0.3, 0.2], [0.1, 0.4]], row_marginals = [0.5, 0.5],
column_marginals = [0.4, 0.6], row_given_column = [[0.75, 0.3333333333333333], [0.25,
0.6666666666666666]], column_given_row = [[0.6, 0.4], [0.2, 0.8]], p_row_and_column = 0.2,
p_row = 0.5, p_column = 0.6, p_row_given_column = 0.333333, p_column_given_row = 0.4,
p_row_or_column = 0.9   (SOLVED)
```

```
statistics_ai999x.solve('two_way_table_probabilities', table=[[12, 8, 5], [18, 22, 35]],
row=2, column=3)
# -> total = 100, joint = [[0.12, 0.08, 0.05], [0.18, 0.22, 0.35]], row_marginals = [0.25,
0.75], column_marginals = [0.3, 0.3, 0.4], row_given_column = [[0.4, 0.26666666666666666,
0.125], [0.6, 0.7333333333333333, 0.875]], column_given_row = [[0.48, 0.32, 0.2], [0.24,
0.29333333333333333, 0.4666666666666667]], p_row_and_column = 0.35, p_row = 0.75, p_column
= 0.4, p_row_given_column = 0.875, p_column_given_row = 0.466667, p_row_or_column = 0.8
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
From the two-way table [[30, 20], [10, 40]], find the probability of row 1 given column 2.	SOLVED	total = 100, joint = [[0.3, 0.2], [0.1, 0.4]], row_marginals = [0.5, 0.5], column_marginals = [0.4, 0.6], row_given_column = [[0.75, 0.3333333333333333], [0.25, 0.6666666666666666]], column_given_row = [[0.6, 0.4], [0.2, 0.8]], p_row_and_column = 0.2, p_row = 0.5, p_column = 0.6, p_row_given_column = 0.333333, p_column_given_row = 0.4, p_row_or_column = 0.9
Two-way table [[12, 8, 5], [18, 22, 35]]: find the joint, marginal and conditional probabilities for row 2 and column 3.	SOLVED	total = 100, joint = [[0.12, 0.08, 0.05], [0.18, 0.22, 0.35]], row_marginals = [0.25, 0.75], column_marginals = [0.3, 0.3, 0.4], row_given_column = [[0.4, 0.26666666666666666, 0.125], [0.6, 0.7333333333333333, 0.875]], column_given_row = [[0.48, 0.32, 0.2], [0.24, 0.29333333333333333, 0.4666666666666667]], p_row_and_column = 0.35, p_row = 0.75, p_column = 0.4, p_row_given_column = 0.875, p_column_given_row = 0.466667, p_row_or_column = 0.8
From the two-way table [[30, 20], [10, 40]], find the probability of row 3 given column 1.	INVALID_INPUT	Joint, marginal and conditional probabilities from a two-way table: the row and column must be inside the table (numbere
From the two-way table [[30, 20], [10, 40]], find the conditional probability of row 1.	NEEDS_INPUT	two_way_table_probabilities needs column (the column of interest (from 1)). Example: solve('two_way_table_probabilities'